Artificial Intelligence: A Mathematical and Engineering Perspective

March 2026

Contents

1 Learning from Data: Models, Prediction, and Generalization **1**
1.1 What Does It Mean to Learn? . . . 2
1.2 A Mathematical Formulation of Learning . . . 6
1.3 Ideal Prediction, Bayes Rules, and Learnable Models . . . 12
1.4 Generalization: Why Fitting the Data Is Not Enough . . . 18
1.5 Optimization as the Engine of Learning . . . 24
1.6 Main Learning Paradigms . . . 31
1.7 From Classical Prediction to Representation Learning . . . 37

2 Neural Networks and the Rise of Deep Learning **43**
2.1 From Classical Prediction to Learned Representations . . . 44
2.2 Neural Networks as Compositional Function Models . . . 52
2.3 Expressivity, Hierarchy, and the Role of Depth . . . 59
2.4 Computation Graphs, Chain Rule, and Backpropagation . . . 64
2.5 Why Deep Networks Are Hard to Train . . . 70
2.6 Optimization in Practice: SGD, Initialization, and Signal Propagation . . . 78
2.7 Stabilizing Deep Learning: Activations, Normalization, and Regularization . . . 87
2.8 Residual Learning and the Possibility of Very Deep Models . . . 92
2.9 Case Studies and Worked Examples . . . 96
2.10 What We Understand and What We Still Do Not . . . 101
Notes and References . . . 103

3 Architecture and Inductive Bias in Deep Learning **105**
3.1 Why Architecture Matters . . . 106
3.2 Locality, Weight Sharing, and the Logic of Convolution . . . 113
3.3 Multiscale Representation and the Fourier Viewpoint . . . 120
3.4 Sequential Data and Recurrent Computation . . . 126
3.5 Gating and the Control of Memory . . . 133
3.6 From Recurrence to Attention . . . 140
3.7 Transformers as General-Purpose Sequence Architectures . . . 146
3.8 Scaling Transformer Architectures and Multimodal Extensions . . . 153
3.9 Worked Examples and Comparative Case Studies . . . 160
3.10 What Is Understood and What Remains Open . . . 167

4 Representation Learning, Transfer, and Foundation Models **175**
4.1 From Task-Specific Prediction to Reusable Representation . . . 176
4.2 Self-Supervised Learning . . . 181

4.3 Transfer Learning and Adaptation . . . 188
4.4 Foundation Models and Learning at Scale . . . 196
4.5 Representation Learning as a Unifying Principle . . . 203
4.5.1 Overview . . . 203
4.5.2 Core Properties of Representations . . . 203
4.5.3 A Cross-Domain Representation Framework . . . 203
4.5.4 Worked Examples . . . 204
4.5.5 Representation as Geometry . . . 204
4.5.6 Remarks: Limits of the Representation View . . . 205
4.5.7 Synthesis . . . 205
4.5.8 What to Remember . . . 205
4.5.9 Common Confusion . . . 205
4.5.10 Exercises . . . 206
4.5.11 Notes and References . . . 206
4.6 State Representation in Sequential Decision-Making . . . 207
4.6.1 Overview . . . 207
4.6.2 Markov Decision Processes . . . 207
4.6.3 Policies and Returns . . . 207
4.6.4 Bellman Equations . . . 208
4.6.5 Bellman Fixed-Point Theorem . . . 208
4.6.6 Worked Example: A Finite MDP . . . 208
4.6.7 State-Transition View . . . 209
4.6.8 Connection to Representation Learning . . . 209
4.6.9 What to Remember . . . 209
4.6.10 Common Confusion . . . 209
4.6.11 Exercises . . . 209
4.6.12 Notes and References . . . 209
4.7 Deep Reinforcement Learning as Learned Representation Plus Control . . . 210
4.7.1 Overview . . . 210
4.7.2 Function Approximation in Reinforcement Learning . . . 210
4.7.3 Value-Based Methods and Bootstrapping . . . 210
4.7.4 Policy-Based Methods . . . 211
4.7.5 Actor–Critic Methods . . . 211
4.7.6 Model-Free vs Model-Based Reinforcement Learning . . . 211
4.7.7 Worked Example: Learned State Representation . . . 212
4.7.8 Deep RL Training Loop . . . 212
4.7.9 Connection to Representation Learning . . . 212
4.7.10 What to Remember . . . 212
4.7.11 Common Confusion . . . 212
4.7.12 Exercises . . . 213
4.7.13 Notes and References . . . 213
4.8 Case Studies in Modern Representation Learning . . . 214
4.9 What Is Established and What Remains Open . . . 219

5 Generative Modeling and Generative AI 225
5.1 What Does It Mean to Model a Data Distribution? . . . 226
5.2 Latent Variables, Incomplete Data, and the EM Principle . . . 232
5.3 Variational Autoencoders: Deep Latent-Variable Modeling . . . 240
5.4 Adversarial Generative Modeling . . . 248
5.5 Exact Density Modeling Through Invertible Maps . . . 254
5.6 Generation by Denoising and Iterative Refinement . . . 259
5.7 Diffusion Models: Mathematical Formulation . . . 265
5.8 Comparing the Main Generative Paradigms . . . 272
5.9 Worked Examples and Visual Case Studies . . . 277
5.10 What Is Established and What Remains Open . . . 284

6 Understanding Deep Learning: Generalization, Reliability, and Open Questions 287
6.1 Why Deep Learning Still Puzzles Us . . . 288
6.2 Generalization Beyond Classical Expectations . . . 290
6.3 Reliability Under Real-World Conditions . . . 292
6.4 Interpreting Learned Representations . . . 294
6.5 Scaling, Efficiency, and Practical Constraints . . . 296
6.6 Open Problems and Future Directions . . . 298
6.7 Final Synthesis . . . 300

Chapter 1

Learning from Data: Models, Prediction, and Generalization

Chapter roadmap. This chapter develops a mathematical language for learning from data. We begin by asking what it means to learn at all, then formalize learning problems in terms of data, task, and performance. From there we move to Bayes rules and ideal prediction, generalization beyond the training sample, optimization as the engine of fitting, major learning paradigms, and finally the transition from fixed features to learned representations. The goal is to move from a broad conceptual picture to a precise framework that supports the rest of the book.

1.1 What Does It Mean to Learn?

Before introducing probability distributions, loss functions, or optimization algorithms, it is worth fixing the central idea of the subject: learning is *improvement from experience*. This sounds simple, but it already contains the three ingredients that organize the rest of the chapter. There is some source of experience, there is some task to be performed, and there is some criterion by which improved performance is judged.

Definition 1.1 (Learning problem)**.** A learning problem consists of a source of experience E, a task space T, and a performance criterion P. A system learns if, after receiving experience from E, its performance on tasks in T, as measured by P, improves.

This formulation is intentionally broad. In supervised learning, the experience is often a labeled dataset. In generative modeling, it may be a collection of unlabeled observations. In reinforcement learning, it is interaction with an environment. In all cases, the point is not merely to change after seeing data, but to change in a way that improves future behavior.

Example 1.2 (Supervised classification as a triple (E, T, P))**.** Consider binary email classification, where the goal is to predict whether an email is spam or not. A natural formulation is:

$$E = \{(x_i, y_i)\}_{i=1}^n, \qquad T = \{f : \mathcal{X} \to \{0, 1\}\}, \qquad P = \text{classification accuracy on new emails}$$

or, equivalently, the corresponding misclassification rate. Here x_i is an email, $y_i \in \{0, 1\}$ is its label, the task space consists of candidate classifiers, and the performance criterion measures how well the learned classifier predicts labels on previously unseen examples.

Learning as improvement from experience

A model that changes but does not improve has adapted, but it has not learned in the sense relevant to machine learning. Conversely, a model that performs perfectly on examples it has already seen but poorly on new examples has merely memorized. This is why learning is fundamentally future-oriented: a learning system uses finite experience to do better on cases it has not yet encountered.

Proposition 1.3 (Fitting observed data is not enough)**.** *Perfect performance on a finite training sample does not by itself imply learning in the predictive sense.*

Proof. Let

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

be any finite dataset. Define a predictor f by setting $f(x_i) = y_i$ for every training point and assigning arbitrary outputs elsewhere. Then f achieves zero training error on $\mathcal{D}$, yet its behavior on unseen inputs is unconstrained. Hence fitting the observed sample alone does not certify useful predictive learning. □

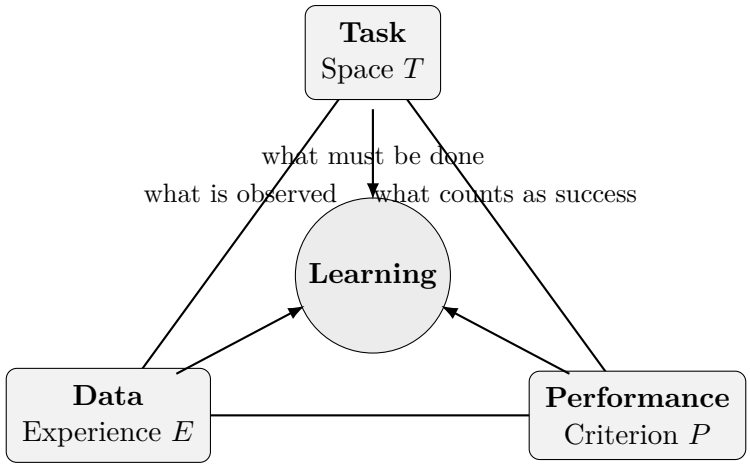


Figure 1.1: A learning problem is organized by three ingredients: experience E, a task space T, and a performance criterion P. Learning sits at their intersection.

The proposition is elementary, but it captures one of the deepest ideas in the subject. Machine learning is not only about describing the observed sample; it is about extracting regularities that persist beyond that sample.

Data, task, and performance criterion

It is helpful to name the components of a learning problem explicitly.

Data. Data provide the experience from which the system learns. In supervised learning we often write

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n,$$

where each x_i is an input and each y_i is a target. In unsupervised settings we may instead have

$$\mathcal{D} = \{x_i\}_{i=1}^n.$$

The data may be images, signals, text, sensor measurements, trajectories, or interaction histories.

Task. The task says what the learner should do. It may predict a label, estimate a real number, generate a sample, compress a signal, or choose an action sequence. The same dataset can support different tasks, so data and task should not be conflated.

Performance criterion. The criterion specifies what it means to do well. Accuracy, squared error, log-likelihood, expected reward, calibration, robustness, and computational efficiency are all possible criteria. A learning problem is incomplete until the performance notion is made explicit.

Prediction, decision, and representation

Introductory presentations often equate learning with prediction, but that is too narrow. Three notions should be distinguished from the beginning.

Prediction. A predictor maps an input to an output,

$$\hat{y} = f(x), \qquad f : \mathcal{X} \to \mathcal{Y}.$$

This is the standard language of classification and regression.

Decision. A decision system maps information to an action. Predictions may inform decisions, but decisions also depend on costs, constraints, and long-term consequences. Predicting that a patient is high-risk is different from deciding which treatment to recommend.

Representation. A representation is an internal encoding of data that makes the relevant task easier. Classical pipelines often used fixed, hand-crafted features; modern deep learning often learns these representations from data.

This distinction matters because later chapters will show that modern AI systems succeed not only by fitting output maps, but also by constructing useful internal coordinate systems for data.

Examples of learning problems

Classification. Given an input x, predict a discrete label $y \in \{1, \ldots, K\}$. Spam filtering, handwriting recognition, and disease diagnosis are standard examples.

Regression. Given an input x, predict a real-valued target $y \in \mathbb{R}$. House-price estimation, demand forecasting, and scientific calibration are familiar examples.

Generation. Given samples from a distribution, learn to model or generate new samples from a similar distribution. Image synthesis, molecule generation, and speech generation fall into this category.

Control. Observe a state, choose an action, and optimize long-term behavior. Robotics, game playing, recommendation under interaction, and navigation are examples of this setting.

What to remember. A learning problem is not defined by data alone. It is defined by a source of experience E, a task space T, and a performance criterion P. Learning is future-oriented improvement from experience: the goal is not merely to fit what has already been observed, but to perform better on new cases, new decisions, or new interactions. From the beginning, it is also important to distinguish prediction, decision, and representation.

Common confusion. A model can have a predictive interface but still solve a decision problem only indirectly. A risk score is a prediction; the treatment chosen after seeing that score is a decision. Likewise, a model can appear to perform well simply because it memorizes a finite sample. Memorization may reproduce the past, but learning aims at useful performance beyond the observed examples.

Exercises

Exercise 1.4. For each of the following scenarios, say whether the central goal is primarily one of *prediction*, *decision*, or *representation*. Briefly justify your answer.

1. Predicting whether an image contains a tumor.

2. Choosing a dosage policy for a patient over time.
3. Learning a low-dimensional embedding of customer behavior.
4. Predicting tomorrow's electricity demand.
5. Selecting moves in a board game to maximize the chance of winning.

Exercise 1.5. Write down plausible triples (E, T, P) for each of the following:

1. a regression problem,
2. a generative modeling problem,
3. a reinforcement learning problem.

Be explicit about what counts as experience, what the task is, and how performance is measured.

Exercise 1.6. Construct a predictor that attains zero error on a finite dataset but is clearly unreasonable on new points. Explain why this illustrates the difference between memorization and learning.

Exercise 1.7. A hospital uses a model to assign a risk score to patients and then uses that score to decide which patients receive immediate follow-up. Identify which part of the pipeline is prediction and which part is decision. Give one reason why optimizing accuracy alone may be insufficient in this setting.

Exercise 1.8. Give an example in which the same dataset could support two different tasks with two different performance criteria. Explain why the learning problem changes when the task or criterion changes, even if the data do not.

Notes and references

This section uses the classical triplet (E, T, P) as a compact way to state what a learning problem is, while emphasizing a modern viewpoint in which learning may concern not only prediction but also decision and representation. The distinction between memorization and future predictive performance foreshadows the later discussion of generalization. The representation viewpoint prepares the transition to neural networks and deep learning in the next chapter.

1.2 A Mathematical Formulation of Learning

The previous section described learning at a conceptual level. We now introduce the mathematical language that makes the subject precise: spaces of inputs and outputs, probability distributions, hypothesis classes, loss functions, risk, and data-dependent learning rules. This section is the mathematical backbone of the chapter. Its goal is to formalize what it means to learn from a finite sample while keeping in view the deeper question that will guide the rest of the book: how does performance on observed data relate to performance on future data?

Input space, output space, and data distribution

Let $\mathcal{X}$ denote the input space and $\mathcal{Y}$ the output space. A supervised example is a pair (X, Y) taking values in $\mathcal{X} \times \mathcal{Y}$. We assume that examples are generated according to an unknown probability distribution P on this product space.

A dataset of size n is then modeled as

$$\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n,$$

where the pairs are typically assumed to be independent draws from P.

Definition 1.9 (Joint, marginal, and conditional laws)**.** The distribution of (X, Y) is the *joint law*. The distributions of X and Y alone are the *marginals*. A *conditional law* describes the distribution of Y given $X = x$ and is denoted by $P(Y \mid X = x)$ or $P(Y \in \cdot \mid X = x)$.

The conditional law is the mathematically precise object behind the idea that the input contains information about the output.

Bayes' formula and conditional inference

When conditional probabilities or densities are well defined, Bayes' formula states that

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}.$$

In density notation this becomes

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

Bayes' formula is not merely a computational identity. It expresses a central structural fact: prediction depends on the relation between prior information, evidence supplied by the observation, and the conditional mechanism connecting them.

Common confusion. Bayes' formula does not mean that every learning algorithm must explicitly compute a posterior distribution. Its role here is conceptual: it shows that learning is fundamentally about conditional structure. Many practical algorithms never evaluate Bayes' formula directly, yet they still aim to approximate the same relation between input and output.

Predictors, hypothesis classes, and inductive bias

A predictive model is usually a function

$$f : \mathcal{X} \to \mathcal{Y}.$$

In parametric families we often write

$$f_\theta : \mathcal{X} \to \mathcal{Y}, \qquad \theta \in \Theta.$$

Definition 1.10 (Hypothesis class)**.** A *hypothesis class* is a set $\mathcal{H}$ of candidate predictors from $\mathcal{X}$ to $\mathcal{Y}$. In parametric models,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

Definition 1.11 (Inductive bias)**.** The *inductive bias* of a learning method is the collection of assumptions that make learning from finite data possible. At a formal level, one source of inductive bias is the restriction of attention to a particular hypothesis class $\mathcal{H}$ rather than to all conceivable functions from $\mathcal{X}$ to $\mathcal{Y}$.

Choosing $\mathcal{H}$ is already a substantive modeling decision. A linear hypothesis class favors linear relations, a convolutional class favors locality and translation structure, and a recurrent or attention-based class favors sequential dependence. Learning never starts from nowhere; it starts from a constrained family of possibilities.

Loss functions

To compare a prediction $\hat{y}$ with a true target y, we introduce a loss function

$$\ell(\hat{y}, y) \geq 0.$$

Equivalently, for a predictor f we write the loss on an example (x, y) as $\ell(f(x), y)$.

Common examples include

$$\ell(\hat{y}, y) = (\hat{y} - y)^2 \qquad \text{(squared loss)},$$

$$\ell(\hat{y}, y) = |\hat{y} - y| \qquad \text{(absolute loss)},$$

$$\ell(\hat{y}, y) = \mathbf{1}\{\hat{y} \neq y\} \qquad \text{(zero–one loss)}.$$

Different losses define different notions of error, and different notions of error lead to different optimal predictors.

Expected risk and empirical risk

The quality of a predictor f under the data-generating distribution is measured by its *expected risk*

$$R(f) = \mathbb{E}[\ell(f(X), Y)].$$

Because the distribution P is unknown, the population risk is usually not directly computable. What we can compute from data is the *empirical risk*

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(X_i), Y_i).$$

Example 1.12 (Empirical risk of a constant predictor)**.** Suppose $\mathcal{Y} = \mathbb{R}$ and the loss is squared error. Consider the constant predictor

$$f_c(x) = c.$$

Its empirical risk on a dataset $\{(x_i, y_i)\}_{i=1}^n$ is

$$\widehat{R}_n(f_c) = \frac{1}{n}\sum_{i=1}^n (c - y_i)^2.$$

Expanding the square gives

$$\widehat{R}_n(f_c) = c^2 - \frac{2c}{n}\sum_{i=1}^n y_i + \frac{1}{n}\sum_{i=1}^n y_i^2.$$

Differentiating with respect to c and setting the derivative equal to zero yields

$$2c - \frac{2}{n}\sum_{i=1}^n y_i = 0,$$

so the minimizing constant is

$$c = \frac{1}{n}\sum_{i=1}^n y_i.$$

Thus, under squared loss, the best constant predictor on the sample is the sample mean.

Proposition 1.13 (Empirical risk is unbiased for fixed f)**.** *Assume $(X_1, Y_1), \ldots, (X_n, Y_n)$ are i.i.d. from P and $\mathbb{E}|\ell(f(X), Y)| < \infty$. Then*

$$\mathbb{E}[\widehat{R}_n(f)] = R(f).$$

Proof. By linearity of expectation and identical distribution of the samples,

$$\mathbb{E}[\widehat{R}_n(f)] = \frac{1}{n}\sum_{i=1}^n \mathbb{E}[\ell(f(X_i), Y_i)] = \frac{1}{n}\sum_{i=1}^n R(f) = R(f).$$

□

This proposition explains why empirical averages are natural surrogates for population quantities. But it does *not* yet solve the learning problem, because learning chooses the predictor using the same data on which the empirical risk is measured.

Learning rules and empirical risk minimization

Definition 1.14 (Learning rule)**.** A *learning rule* is a map

$$A : \mathfrak{D} \to \mathcal{H}, \qquad \mathcal{D} \mapsto \hat{f} = A(\mathcal{D}),$$

where $\mathfrak{D}$ denotes a class of admissible datasets and $\mathcal{H}$ is the hypothesis class. In words, a learning rule takes a dataset as input and returns a predictor.

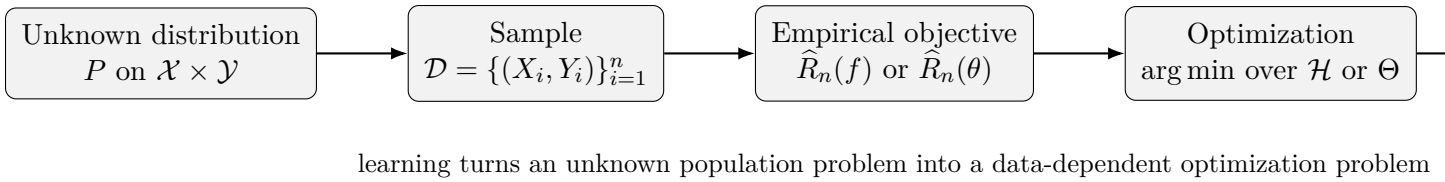


Figure 1.2: A summary of the risk-minimization viewpoint: an unknown distribution generates a sample; the sample defines an empirical objective; optimization over a hypothesis class produces a predictor.

Thus the output of learning is not a single fixed function, but a *data-dependent* function. Before the sample is observed, $\hat{f}$ is a random object because it depends on the random dataset $\mathcal{D}$.

A central principle of machine learning is *empirical risk minimization* (ERM): choose a predictor in $\mathcal{H}$ that minimizes empirical risk,

$$\hat{f}_n \in \arg\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

If the class is parameterized by θ, this becomes

$$\hat{\theta}_n \in \arg\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(X_i), Y_i).$$

Proposition 1.15 (ERM is data-dependent)**.** *Let*

$$\hat{f}_n \in \arg\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

Then $\hat{f}_n$ is a random predictor determined by the sample $\mathcal{D}$. Moreover, for every fixed $f \in \mathcal{H}$,

$$\widehat{R}_n(\hat{f}_n) \leq \widehat{R}_n(f)$$

for every realization of the data, and therefore

$$\mathbb{E}[\widehat{R}_n(\hat{f}_n)] \leq R(f).$$

In particular,

$$\mathbb{E}[\widehat{R}_n(\hat{f}_n)] \leq \inf_{f \in \mathcal{H}} R(f).$$

Proof. By definition of $\hat{f}_n$ as an empirical risk minimizer,

$$\widehat{R}_n(\hat{f}_n) \leq \widehat{R}_n(f)$$

for every fixed $f \in \mathcal{H}$ and every dataset. Taking expectations and using the unbiasedness proposition for fixed f gives

$$\mathbb{E}[\widehat{R}_n(\hat{f}_n)] \leq \mathbb{E}[\widehat{R}_n(f)] = R(f).$$

Since this holds for every fixed $f \in \mathcal{H}$, it follows that

$$\mathbb{E}[\widehat{R}_n(\hat{f}_n)] \leq \inf_{f \in \mathcal{H}} R(f).$$

□

This proposition captures an important subtlety. The empirical risk of a fixed predictor is unbiased for its true risk, but the empirical risk of the predictor *selected from the data* is typically optimistic. This is why training error alone is not a reliable measure of future performance.

Learning as approximate risk minimization

The ideal population problem is

$$\min_{f \in \mathcal{H}} R(f),$$

or more ambitiously,

$$\min_{f} R(f)$$

over a much larger universe of predictors. In practice we only observe finitely many samples and often solve an optimization problem approximately. This gives rise to three distinct sources of error.

Proposition 1.16 (Approximation, estimation, and optimization)**.** *Assume the following predictors exist:*

$$f^\star \in \arg\min_{f} R(f), \qquad f^\star_{\mathcal{H}} \in \arg\min_{f \in \mathcal{H}} R(f), \qquad f^\star_n \in \arg\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

Let $\hat{f}_n$ denote the predictor actually returned by a learning algorithm. Then

$$R(\hat{f}_n) - R(f^\star) = \underbrace{\big(R(f^\star_{\mathcal{H}}) - R(f^\star)\big)}_{\textit{approximation}} + \underbrace{\big(R(f^\star_n) - R(f^\star_{\mathcal{H}})\big)}_{\textit{estimation}} + \underbrace{\big(R(\hat{f}_n) - R(f^\star_n)\big)}_{\textit{optimization}}.$$

Proof. Add and subtract the two intermediate risks $R(f^\star_{\mathcal{H}})$ and $R(f^\star_n)$:

$$R(\hat{f}_n) - R(f^\star) = \big(R(\hat{f}_n) - R(f^\star_n)\big) + \big(R(f^\star_n) - R(f^\star_{\mathcal{H}})\big) + \big(R(f^\star_{\mathcal{H}}) - R(f^\star)\big).$$

Reordering the terms gives the stated decomposition. □

The decomposition should not be overinterpreted as a final theorem of learning, but it is a powerful organizing principle. A model class may be too restrictive, the data may be too limited to identify a good model reliably, or the optimization algorithm may fail to find the empirical minimizer.

What to remember. The mathematical formulation of learning begins with a distribution P on input-output pairs, a hypothesis class $\mathcal{H}$, a loss function ℓ, and a learning rule A that maps a dataset to a predictor. The population objective is expected risk $R(f)$; the observable surrogate is empirical risk $\widehat{R}_n(f)$. Empirical risk is unbiased for a fixed predictor, but once the predictor is chosen from the data, that fixed-function argument no longer suffices. This is why learning theory must study data-dependent selection and generalization, not training error alone.

Common confusion. Because empirical risk is unbiased for a fixed predictor, it is tempting to think that the training error of the learned predictor must also be an unbiased estimate of test error. This is false. The learned predictor is itself chosen using the sample, so it is no longer fixed in advance. The sample has been used twice: once to construct the predictor and again to evaluate it. That is exactly why generalization requires more than the law of large numbers applied naively.

Exercises

Exercise 1.17. Give one example of a learning problem in which the output space $\mathcal{Y}$ is discrete and one in which it is continuous. Suggest a natural loss function in each case.

Exercise 1.18. Let $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Compute the empirical risk of the constant predictor $f_c(x) = c$ on a dataset $\{(x_i, y_i)\}_{i=1}^n$ and verify directly that the minimizing value of c is the sample mean.

Exercise 1.19. Explain in words why the proposition on unbiasedness for fixed f does not imply that the training error of the empirical risk minimizer is an unbiased estimate of its test error.

Exercise 1.20. Describe how inductive bias enters through the choice of hypothesis class $\mathcal{H}$ even before any data are observed.

Exercise 1.21. Consider a classification problem with zero–one loss. Write down the expected risk and empirical risk of a predictor $f : \mathcal{X} \to \{0, 1\}$. What does minimizing each quantity mean in plain language?

Exercise 1.22. In the approximation–estimation–optimization decomposition, describe a concrete situation in which each of the three terms could be large.

Notes and references

The risk-minimization viewpoint is one of the central organizing ideas of modern machine learning. The language of distributions, loss functions, and empirical objectives will reappear throughout the book in different forms: in Bayesian inference, in kernel methods, in large-margin classification, in neural-network training, and in generative modeling. The present section is deliberately foundational. Its main purpose is not to settle the learning problem, but to state it precisely enough that later questions about Bayes rules, generalization, optimization, and representation learning can be asked in mathematical form.

1.3 Ideal Prediction, Bayes Rules, and Learnable Models

The previous section introduced expected risk as the population-level objective of learning. Once that objective is in place, a natural question appears immediately:

> If the true data-generating distribution were known, what prediction rule would be optimal under a chosen loss?

This question leads to one of the central reference objects of statistical learning: the *Bayes-optimal predictor*. It is an ideal object rather than a practically available one, but it gives the benchmark against which all real learning procedures should be compared.

Bayes-optimal prediction as conditional optimization

Let $\ell(\hat{y}, y)$ be a loss function, and let

$$R(f) = \mathbb{E}[\ell(f(X), Y)]$$

denote the expected risk of a predictor f.

Definition 1.23 (Bayes-optimal predictor)**.** A predictor $f^\star$ is called *Bayes-optimal* if it minimizes expected risk over all measurable predictors:

$$f^\star \in \arg\min_f R(f).$$

The corresponding minimum value

$$R^\star = R(f^\star)$$

is called the *Bayes risk*.

The key idea is that expected risk can be analyzed conditionally on the input. For each fixed input value x and each candidate action a, define the conditional risk

$$L_x(a) = \mathbb{E}[\ell(a, Y) \mid X = x].$$

This turns the global optimization problem over functions into a family of local decision problems, one for each input value.

Theorem 1.24 (Pointwise conditional minimization yields global risk minimization)**.** *Assume there exists a measurable predictor $f^\star$ such that for almost every x,*

$$f^\star(x) \in \arg\min_a \mathbb{E}[\ell(a, Y) \mid X = x].$$

Then $f^\star$ is Bayes-optimal. Moreover, for every predictor f,

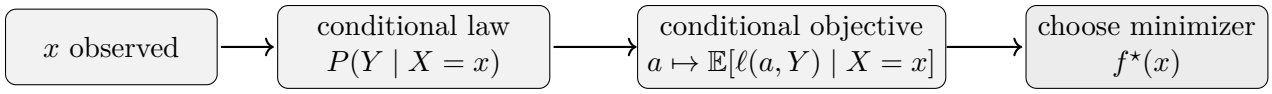
$$R(f) - R(f^\star) = \mathbb{E}[\mathbb{E}[\ell(f(X), Y) \mid X] - \mathbb{E}[\ell(f^\star(X), Y) \mid X]] \geq 0.$$

Proof. For any predictor f, the tower property of conditional expectation gives

$$R(f) = \mathbb{E}[\ell(f(X), Y)] = \mathbb{E}[\mathbb{E}[\ell(f(X), Y) \mid X]]\,.$$

Likewise,

$$R(f^\star) = \mathbb{E}[\mathbb{E}[\ell(f^\star(X), Y) \mid X]]\,.$$



Bayes prediction is obtained by solving a conditional optimization problem at each input x

Figure 1.3: The Bayes decision rule: once x is observed, the conditional distribution $P(Y \mid X = x)$ determines the conditional risk for each candidate action a, and the optimal choice is the minimizer of that conditional risk.

Hence

$$R(f) - R(f^\star) = \mathbb{E}[\mathbb{E}[\ell(f(X), Y) \mid X] - \mathbb{E}[\ell(f^\star(X), Y) \mid X]]\,.$$

Now fix a realized value $X = x$. By assumption, $f^\star(x)$ minimizes

$$a \mapsto \mathbb{E}[\ell(a, Y) \mid X = x].$$

Therefore,

$$\mathbb{E}[\ell(f(x), Y) \mid X = x] \geq \mathbb{E}[\ell(f^\star(x), Y) \mid X = x]$$

for almost every x. Taking expectation over X yields

$$R(f) \geq R(f^\star).$$

Thus $f^\star$ is Bayes-optimal. □

This theorem is one of the fundamental structural results in learning theory. It says that optimal prediction is governed by the conditional law of Y given $X = x$. To understand ideal prediction, one studies

$$P(Y \mid X = x).$$

Squared loss and the conditional mean

Suppose $\mathcal{Y} = \mathbb{R}$ and consider squared loss

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

Proposition 1.25 (Bayes predictor under squared loss)**.** *Under squared loss, any Bayes-optimal predictor is given by the conditional mean:*

$$f^\star(x) = \mathbb{E}[Y \mid X = x].$$

Proof. Fix x and let

$$m(x) = \mathbb{E}[Y \mid X = x].$$

For any candidate prediction a,

$$a - Y = (a - m(x)) + (m(x) - Y),$$

so

$$(a - Y)^2 = (a - m(x))^2 + 2(a - m(x))(m(x) - Y) + (m(x) - Y)^2.$$

Taking conditional expectation given $X = x$,

$$\mathbb{E}[(a - Y)^2 \mid X = x] = (a - m(x))^2 + 2(a - m(x))\mathbb{E}[m(x) - Y \mid X = x] + \mathbb{E}[(m(x) - Y)^2 \mid X = x].$$

Because

$$\mathbb{E}[m(x) - Y \mid X = x] = m(x) - \mathbb{E}[Y \mid X = x] = 0,$$

this becomes

$$\mathbb{E}[(a - Y)^2 \mid X = x] = (a - m(x))^2 + \mathbb{E}[(m(x) - Y)^2 \mid X = x].$$

The second term does not depend on a, so the minimum is attained at

$$a = m(x) = \mathbb{E}[Y \mid X = x].$$

The theorem above then implies that the Bayes-optimal predictor is

$$f^\star(x) = \mathbb{E}[Y \mid X = x].$$

□

Thus regression under squared loss is fundamentally about estimating a conditional expectation.

Absolute loss and the conditional median

Now consider absolute loss

$$\ell(\hat{y}, y) = |\hat{y} - y|.$$

Proposition 1.26 (Bayes predictor under absolute loss)**.** *Under absolute loss, any Bayes-optimal predictor is a conditional median:*

$$f^\star(x) \in \mathrm{Median}(Y \mid X = x).$$

More precisely, a value a is optimal at x if

$$\mathbb{P}(Y \le a \mid X = x) \ge \frac{1}{2} \qquad \textit{and} \qquad \mathbb{P}(Y \ge a \mid X = x) \ge \frac{1}{2}.$$

Proof sketch. Fix x and define

$$\phi_x(a) = \mathbb{E}[|a - Y| \mid X = x].$$

If a is moved slightly to the right, the loss decreases for outcomes with $Y > a$ and increases for outcomes with $Y < a$. The optimum therefore occurs when the conditional mass on the two sides is balanced. More formally, the one-sided derivatives satisfy

$$\phi_x'(a^-) = \mathbb{P}(Y < a \mid X = x) - \mathbb{P}(Y \ge a \mid X = x),$$

$$\phi_x'(a^+) = \mathbb{P}(Y \le a \mid X = x) - \mathbb{P}(Y > a \mid X = x),$$

so a minimizes ϕ_x exactly when

$$\phi_x'(a^-) \le 0 \le \phi_x'(a^+),$$

which is equivalent to the stated median conditions. □

Under absolute loss, the ideal prediction is not the conditional mean in general. Different losses produce different notions of optimal prediction.

Zero–one loss and the Bayes classifier

Now suppose the output space is a finite label set $\mathcal{Y}$ and the loss is zero–one loss,

$$\ell(\hat{y}, y) = \mathbf{1}\{\hat{y} \neq y\}.$$

Proposition 1.27 (Bayes classifier under zero–one loss)**.** *Under zero–one loss, a Bayes-optimal classifier is given by*

$$f^\star(x) \in \arg\max_{y \in \mathcal{Y}} \mathbb{P}(Y = y \mid X = x).$$

Proof. Fix x and predict class $c \in \mathcal{Y}$. The conditional risk is

$$\mathbb{E}[\mathbf{1}\{c \neq Y\} \mid X = x] = \mathbb{P}(Y \neq c \mid X = x) = 1 - \mathbb{P}(Y = c \mid X = x).$$

Minimizing this over c is equivalent to maximizing

$$\mathbb{P}(Y = c \mid X = x).$$

Applying the theorem on pointwise conditional minimization completes the proof. □

This is often called the *maximum a posteriori* rule, or MAP rule.

A worked example: binary posterior and decision threshold

Consider binary classification with labels $\mathcal{Y} = \{0, 1\}$. Suppose that for a given input x, the posterior probabilities are

$$\mathbb{P}(Y = 1 \mid X = x) = 0.73, \qquad \mathbb{P}(Y = 0 \mid X = x) = 0.27.$$

Under zero–one loss, the conditional risks of the two possible predictions are

$$L_x(1) = \mathbb{P}(Y \neq 1 \mid X = x) = 0.27,$$

$$L_x(0) = \mathbb{P}(Y \neq 0 \mid X = x) = 0.73.$$

Hence the Bayes classifier chooses class 1.

More generally, in binary classification under zero–one loss, the Bayes rule is

$$f^\star(x) = \begin{cases} 1, & \text{if } \mathbb{P}(Y = 1 \mid X = x) \geq \frac{1}{2}, \\ 0, & \text{if } \mathbb{P}(Y = 1 \mid X = x) < \frac{1}{2}. \end{cases}$$

Thus the familiar threshold $1/2$ is not arbitrary; it arises from minimizing conditional misclassification probability.

Bayes risk and irreducible uncertainty

Even the Bayes-optimal predictor is generally not perfect. The minimum possible expected risk

$$R^\star = R(f^\star)$$

is the error level forced by the data-generating process itself under the chosen loss.

For squared loss, the proof above shows that the minimum conditional risk at input x is

$$\mathbb{E}\Big[(Y - \mathbb{E}[Y \mid X = x])^2 \mid X = x\Big] = \mathrm{Var}(Y \mid X = x),$$

so the Bayes risk is

$$R^\star = \mathbb{E}[\mathrm{Var}(Y \mid X)].$$

This is the part of the error that remains even when the conditional mean is known exactly.

For classification under zero–one loss, the minimum conditional error at x is

$$1 - \max_{y \in \mathcal{Y}} \mathbb{P}(Y = y \mid X = x).$$

If two labels both have positive conditional probability at the same x, then no deterministic classifier can always be correct there.

So Bayes-optimal does not mean error-free. It means only that no other predictor can do better on average under the specified loss.

Common confusion. Bayes-optimal does not mean computable. The Bayes rule is defined in terms of the true conditional distribution $P(Y \mid X = x)$, which is not known to the learner. It is an ideal benchmark, not usually an algorithm. Real learning methods use finite data to approximate this benchmark within restricted model classes.

From ideal prediction to learnable models

The Bayes rule is the best conceivable predictor relative to the true distribution and the chosen loss. But practical learning takes place under much stronger constraints:

- the distribution P is unknown,
- only a finite sample is available,
- computation is limited,
- the learner works within a restricted hypothesis class $\mathcal{H}$.

Thus real learning usually cannot search over all measurable predictors. Instead, it seeks a predictor

$$\hat{f} \in \mathcal{H}$$

that performs well relative to the Bayes benchmark. This leads naturally to the best-in-class predictor

$$f^\star_{\mathcal{H}} \in \arg\min_{f \in \mathcal{H}} R(f),$$

which need not coincide with the Bayes-optimal predictor $f^\star$.

The central tension is therefore clear:

> The Bayes rule tells us what ideal prediction would be. Learning theory asks when finite data and restricted models allow us to approach that ideal.

That tension drives the rest of the chapter.

What to remember. The Bayes-optimal predictor minimizes expected risk, and it is obtained by minimizing conditional risk pointwise at each input x. Under squared loss, the Bayes predictor is the conditional mean; under absolute loss, it is a conditional median; under zero–one loss, it is the most probable class given x. The Bayes rule is a benchmark determined by the true conditional law $P(Y \mid X = x)$, not something we usually know or can compute directly.

Exercises

Exercise 1.28. Let $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Starting from

$$\mathbb{E}[(a - Y)^2 \mid X = x],$$

show directly that the minimizing value of a is $\mathbb{E}[Y \mid X = x]$.

Exercise 1.29. Suppose $\mathbb{P}(Y = 1 \mid X = x) = p(x)$ in a binary classification problem. Under zero–one loss, write down the Bayes classifier explicitly and explain why the threshold is $1/2$.

Exercise 1.30. Give an example of two different loss functions that would lead to two different notions of ideal prediction for the same conditional distribution of $Y \mid X = x$.

Exercise 1.31. Explain in words why Bayes-optimal prediction is a useful theoretical benchmark even though the learner does not know the true distribution.

Exercise 1.32. Suppose a regression problem satisfies

$$Y = g(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0.$$

Show that under squared loss the Bayes-optimal predictor is $g(X)$.

Exercise 1.33. For a three-class problem, suppose that at some input x,

$$\mathbb{P}(Y = 1 \mid X = x) = 0.4, \qquad \mathbb{P}(Y = 2 \mid X = x) = 0.35, \qquad \mathbb{P}(Y = 3 \mid X = x) = 0.25.$$

What is the Bayes-optimal prediction under zero–one loss, and what is the corresponding conditional Bayes error?

Notes and references

This section introduces one of the most important ideal objects in statistical learning: the Bayes-optimal predictor. Its role is conceptual and mathematical. Conceptual, because it makes clear that learning aims to approximate conditional structure rather than merely fit a sample. Mathematical, because it links prediction to the conditional distribution $P(Y \mid X = x)$ and shows that different losses induce different notions of optimal prediction. The Bayes benchmark will reappear later when we study generalization, approximation, model classes, and the limits of finite-sample learning.

1.4 Generalization: Why Fitting the Data Is Not Enough

The previous sections introduced the population objective of learning and the ideal benchmark given by Bayes-optimal prediction. We now encounter the central difficulty of the subject: the learner does not observe the population distribution directly. It sees only a finite sample. The real question is therefore not whether a model can fit the observed data, but whether it can use those data to perform well on new examples drawn from the same environment.

This distinction between fitting and future performance is the conceptual core of generalization theory.

Training error, test error, and the generalization gap

Let

$$\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n$$

be a training sample, let f be a predictor, and let ℓ be a loss function.

Definition 1.34 (Training error)**.** The *training error* or *training risk* of f on $\mathcal{D}$ is its empirical risk on the sample used for fitting:

$$\widehat{R}_n(f) = \frac{1}{n}\sum_{i=1}^n \ell(f(X_i), Y_i).$$

Definition 1.35 (Test error)**.** The *test error* of f is its performance on fresh data not used in training. At the population level, this is measured by the expected risk

$$R(f) = \mathbb{E}[\ell(f(X), Y)].$$

If one has an independent test sample

$$\mathcal{D}_{\text{test}} = \{(X_i', Y_i')\}_{i=1}^m,$$

then the corresponding empirical test error is

$$\widehat{R}_{\text{test}}(f) = \frac{1}{m}\sum_{i=1}^m \ell(f(X_i'), Y_i').$$

Definition 1.36 (Generalization gap)**.** The *generalization gap* of f is the difference between population risk and training risk:

$$\text{Gen}(f) = R(f) - \widehat{R}_n(f).$$

A model generalizes well when two things happen at once:

1. its training error is a reliable guide to its test error, so the generalization gap is small;
2. its test error itself is small.

These are different requirements. A model may have a tiny generalization gap but still perform badly if both training and test error are large. Conversely, it may have tiny training error and still perform badly if the generalization gap is large.

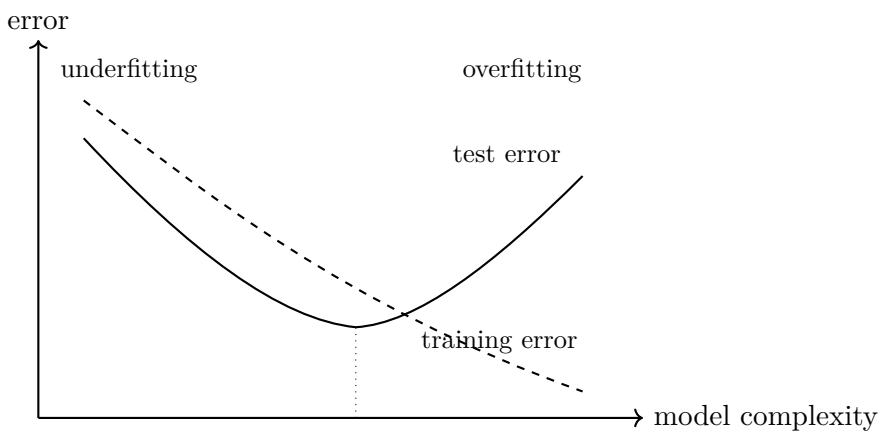


Figure 1.4: A schematic picture of underfitting and overfitting. As model complexity increases, training error usually decreases, but test error may first decrease and then rise if the model begins to fit sample-specific noise rather than stable structure.

Why fitting the training sample is not enough

The following proposition states the basic warning formally.

Proposition 1.37 (Minimizing training error alone does not guarantee low test error)**.** *There exist learning problems and hypothesis classes for which a predictor achieves zero training error while having arbitrarily poor population performance.*

Proof. Take any finite training sample

$$\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}.$$

Consider a hypothesis class rich enough to contain all functions from $\mathcal{X}$ to $\mathcal{Y}$. Then there exists a predictor f_{mem} such that

$$f_{\text{mem}}(x_i) = y_i \qquad \text{for all } i = 1, \ldots, n,$$

while the values of $f_{\text{mem}}(x)$ for inputs outside the training sample are assigned arbitrarily.

By construction,

$$\widehat{R}_n(f_{\text{mem}}) = 0$$

under zero–one loss. However, because the outputs away from the observed points are unconstrained, the expected risk $R(f_{\text{mem}})$ can be large. Thus perfect training performance does not imply good performance on future data. □

The proposition is simple, but its message is profound. Learning cannot be identified with memorization. A predictor that merely reproduces the sample may look successful from the viewpoint of training error while failing completely at the actual task of prediction beyond the sample.

Underfitting and overfitting

Two characteristic failures organize much of the intuition of generalization.

Underfitting. A model may be too rigid to capture the relevant structure in the data. Then it performs poorly even on the training sample and also poorly on new data. This is *underfitting*.

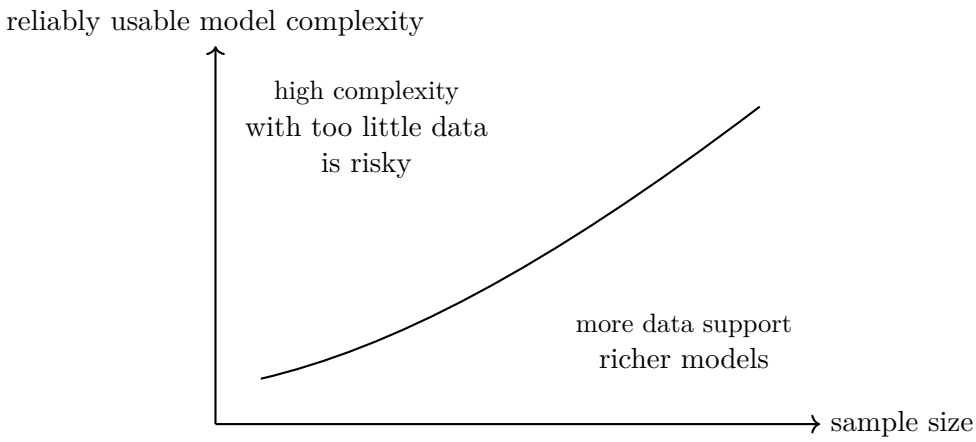


Figure 1.5: A schematic relation between sample size and usable model complexity. As more data become available, richer hypothesis classes can often be estimated more reliably.

Overfitting. A model may be so flexible that it adapts not only to signal but also to accidental idiosyncrasies of the sample. Then it may achieve very low training error while suffering noticeably worse test error. This is *overfitting*.

These are not merely verbal opposites. They reflect two distinct ways in which learning can fail:

- the model class may be too simple to represent the target structure;
- the model class may be so rich that finite data do not constrain it reliably.

Model complexity and sample size

Generalization depends on a balance between the expressive power of the hypothesis class and the amount of information supplied by the data.

A richer hypothesis class can represent more patterns, but it can also fit more accidental fluctuations. A larger sample gives stronger evidence about which patterns are stable and which are merely noise. This leads to a basic qualitative principle:

> Complex models generally require more data to generalize reliably.

This principle is only qualitative at present. Later chapters will make it precise using capacity measures and probability bounds. But even before that theory is introduced, the core idea is already visible: *model complexity and sample size must be matched.*

A uniform-convergence intuition

The previous section showed that empirical risk is unbiased for a fixed predictor. But learning does not select a predictor in advance. It chooses one *after seeing the data.* This is why the fixed-function argument is not enough.

A natural stronger requirement is that empirical risk should approximate population risk *uniformly* over the whole hypothesis class:

$$\sup_{f \in \mathcal{H}} \left| R(f) - \widehat{R}_n(f) \right|.$$

If this quantity is small, then no predictor in the class can look much better on the training sample than it truly is on the population. In that regime, minimizing empirical risk is sensible because training performance is uniformly trustworthy across the class.

At a conceptual level, this is one of the first explanations of why generalization can happen: the sample is large enough, relative to the complexity of the hypothesis class, that empirical comparisons reflect genuine population comparisons.

A high-level decomposition of excess risk

Section 1.2 already introduced the approximation/estimation/optimization trichotomy. It is worth repeating it here because it explains why generalization is not a one-dimensional problem.

Proposition 1.38 (A high-level decomposition of excess risk)**.** *Let $f^\star$ denote the Bayes-optimal predictor, let*

$$f^\star_{\mathcal{H}} \in \arg\min_{f\in\mathcal{H}} R(f)$$

be the best predictor in the chosen hypothesis class, let

$$f^\star_n \in \arg\min_{f\in\mathcal{H}} \widehat{R}_n(f)$$

be an empirical risk minimizer in the class, and let $\hat{f}_n$ be the predictor actually returned by the learning algorithm. Then

$$R(\hat{f}_n) - R(f^\star) = \underbrace{\big(R(f^\star_{\mathcal{H}}) - R(f^\star)\big)}_{\textit{approximation}} + \underbrace{\big(R(f^\star_n) - R(f^\star_{\mathcal{H}})\big)}_{\textit{estimation}} + \underbrace{\big(R(\hat{f}_n) - R(f^\star_n)\big)}_{\textit{optimization}}.$$

Proof. Add and subtract $R(f^\star_{\mathcal{H}})$ and $R(f^\star_n)$:

$$R(\hat{f}_n) - R(f^\star) = \big(R(\hat{f}_n) - R(f^\star_n)\big) + \big(R(f^\star_n) - R(f^\star_{\mathcal{H}})\big) + \big(R(f^\star_{\mathcal{H}}) - R(f^\star)\big).$$

This is exactly the stated decomposition. □

This decomposition is useful because it separates three sources of difficulty:

- **Approximation error:** the class $\mathcal{H}$ may be too limited to represent the ideal predictor well.
- **Estimation error:** finite data may not identify a nearly optimal member of $\mathcal{H}$ reliably.
- **Optimization error:** the training procedure may fail to find the empirical minimizer.

So when a learner performs badly, the reason may be expressive limitation, finite-sample uncertainty, or optimization failure. Generalization sits mainly in the second term, but practical learning depends on all three.

Bias and variance

A classical way to think about generalization is through the distinction between *bias* and *variance*.

Bias. Bias reflects systematic error caused by a restrictive model class or learning rule. A high-bias method tends to miss important structure even if given more data.

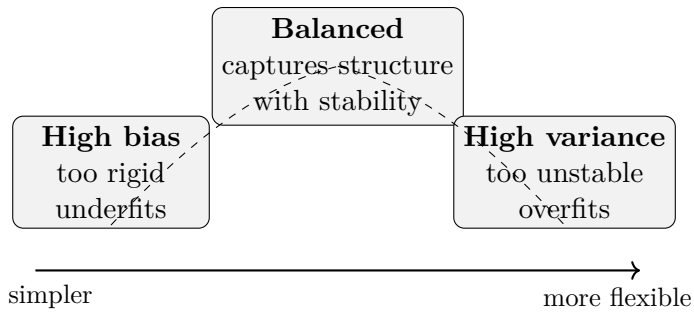


Figure 1.6: A schematic bias–variance picture. Simpler models tend to have higher bias and lower variance; more flexible models tend to have lower bias and higher variance. Good generalization usually requires a balance between the two.

Variance. Variance reflects instability with respect to the training sample. A high-variance method may produce very different predictors when trained on different datasets drawn from the same population.

At a fixed input x, if $\hat{f}(x)$ is the prediction produced by a random training sample and $f^\star(x)$ is the target benchmark, then one common schematic decomposition is

$$\mathbb{E}\big[(\hat{f}(x) - f^\star(x))^2\big] = \big(\mathbb{E}[\hat{f}(x)] - f^\star(x)\big)^2 + \mathbb{E}\Big[\big(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\big)^2\Big].$$

The first term is the squared bias and the second is the variance.

The bias–variance language is especially useful in introductory regression settings. It is not a complete explanation of all modern deep-learning behavior, but it remains a valuable first lens on the tradeoff between expressiveness and stability.

Regularization and cross-validation at a conceptual level

Two practical responses to overfitting appear early in almost every area of machine learning.

Regularization. Instead of minimizing empirical risk alone, one may penalize complexity:

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f) + \lambda\, \Omega(f),$$

where $\Omega(f)$ measures some notion of complexity and $\lambda > 0$ controls the strength of the penalty. The goal is not merely to fit the sample, but to favor predictors that are stable enough to perform well beyond it.

Cross-validation. Because the true population risk is unknown, one often estimates predictive performance by separating fitting from evaluation. Conceptually, cross-validation proceeds by splitting the available data into parts, training on one part, evaluating on another, and repeating this process across several splits. Its role is to imitate the train/test distinction using only the available sample.

Cross-validation does not replace theory, but it encodes a crucial practical lesson: evaluation on the same data used for fitting can be misleading.

Common confusion. Overfitting is not the same thing as "having many parameters" in every regime. A large model can overfit, but whether it does so depends on the data, the optimization procedure, the inductive bias of the architecture, the amount of regularization, and the structure of the task. Parameter count alone is too crude to explain generalization.

What to remember. Generalization is the ability to perform well on new data, not merely on the training sample. Training error, test error, and the generalization gap are different quantities. Minimizing training error alone does not guarantee low test error. Good generalization depends on the interaction among model complexity, sample size, inductive bias, and optimization. A useful high-level guide is the decomposition of excess risk into approximation, estimation, and optimization terms, while the classical bias–variance picture provides an intuitive first explanation of underfitting and overfitting.

Exercises

Exercise 1.39. Define in your own words the difference between training error, test error, and the generalization gap. Why can a model have small training error but large test error?

Exercise 1.40. Construct a simple example of a predictor that memorizes a finite training set. Explain why zero training error does not certify learning in the predictive sense.

Exercise 1.41. Suppose two model classes are available for the same task: one very simple and one highly flexible. Describe a situation in which the simple model underfits and a situation in which the flexible model overfits.

Exercise 1.42. Cross-validation is often used to choose hyperparameters such as model complexity or regularization strength. Explain conceptually why evaluating all candidate models only on the training set is unreliable, and how cross-validation attempts to correct this.

Exercise 1.43. In the decomposition

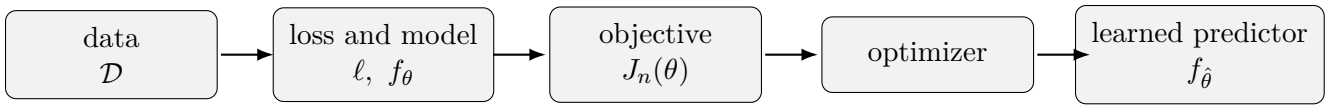
$$R(\hat{f}_n) - R(f^\star) = \text{approximation} + \text{estimation} + \text{optimization},$$

describe one concrete reason each term might be large in practice.

Exercise 1.44. Why is it misleading to define overfitting simply as "using too many parameters"? Give at least two other factors that influence whether a model generalizes well.

Notes and references

This section introduces the central distinction between fitting the observed sample and performing well on future data. The language of training error, test error, and generalization gap is foundational for the rest of learning theory. The underfitting/overfitting picture, the qualitative relation between model complexity and sample size, the approximation/estimation/optimization decomposition, and the bias–variance viewpoint are all first-pass frameworks rather than final explanations. Later chapters will revisit these ideas with more precise tools, including capacity measures, concentration arguments, regularization theory, and modern perspectives on overparameterized learning.



learning becomes computationally concrete once empirical risk is turned into an objective to be minimized

Figure 1.7: A summary of the optimization viewpoint. Data, model, and loss together define an objective, and optimization produces the learned predictor.

1.5 Optimization as the Engine of Learning

The previous sections described learning in statistical terms: a hypothesis class is chosen, a loss function measures predictive quality, and a learner seeks a model with small empirical risk and, ultimately, small population risk. But this still leaves a practical question unanswered:

How is such a model actually found?

The answer is that, once a model class and objective have been specified, learning usually becomes an optimization problem. Statistical ideas tell us what quantity matters; optimization provides the computational procedure that searches for parameters making that quantity small. In this sense, optimization is the engine that turns a learning principle into an actual algorithm.

From empirical risk to an optimization problem

Suppose a model is parameterized by $\theta \in \Theta$, so that predictions are made by a function f_θ. Given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

and a loss function ℓ, the empirical risk is

$$\widehat{R}_n(f_\theta) = \frac{1}{n}\sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

Definition 1.45 (Optimization problem induced by empirical risk)**.** The *optimization problem induced by empirical risk* is the problem of finding parameters θ that minimize a data-dependent objective of the form

$$J_n(\theta) = \widehat{R}_n(f_\theta)$$

or, more generally,

$$J_n(\theta) = \widehat{R}_n(f_\theta) + \lambda\,\Omega(\theta),$$

where $\Omega(\theta)$ is a regularization term and $\lambda \geq 0$ controls its strength.

Thus, after the modeling choices have been made, the learner faces a numerical search problem:

$$\min_{\theta\in\Theta} J_n(\theta).$$

This is the computational side of learning.

Some common objectives

Different learning tasks lead to different objectives.

Squared loss. For regression,

$$\ell(\hat{y}, y) = (\hat{y} - y)^2,$$

so the empirical objective becomes

$$J_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} (f_\theta(x_i) - y_i)^2.$$

Cross-entropy or log loss. If the model outputs probabilities $p_\theta(y \mid x)$, then one often uses

$$\ell(\theta; (x, y)) = -\log p_\theta(y \mid x),$$

which leads to

$$J_n(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \log p_\theta(y_i \mid x_i).$$

Regularized objectives. To discourage unstable or overly complex solutions, one often minimizes

$$J_n(\theta) = \widehat{R}_n(f_\theta) + \lambda\, \Omega(\theta).$$

For example, $\Omega(\theta) = \|\theta\|_2^2$ gives an ℓ_2 penalty.

Different losses and penalties create different optimization landscapes. Part of the art of machine learning is choosing objectives that are statistically meaningful and computationally manageable.

Gradient descent

When the objective is differentiable, the basic local method is gradient descent.

Definition 1.46 (Gradient descent)**.** Let $J : \Theta \to \mathbb{R}$ be differentiable. Starting from an initial point θ_0, *gradient descent* generates iterates

$$\theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t),$$

where $\eta_t > 0$ is the learning rate or step size.

The geometric intuition is that $\nabla J(\theta_t)$ points in the direction of steepest local increase of the objective, so its negative points toward steepest local decrease.

Proposition 1.47 (Negative gradient is the direction of steepest local decrease)**.** *Let $J : \mathbb{R}^d \to \mathbb{R}$ be differentiable at θ, and let u be any unit vector. Then the directional derivative of J at θ in direction u is*

$$D_u J(\theta) = \nabla J(\theta)^\top u.$$

Among all unit directions u, this quantity is minimized when

$$u = -\frac{\nabla J(\theta)}{\|\nabla J(\theta)\|}$$

provided $\nabla J(\theta) \neq 0$. The minimum value is

$$-\|\nabla J(\theta)\|.$$

Proof. For a unit vector u, the directional derivative is

$$D_u J(\theta) = \nabla J(\theta)^\top u.$$

By the Cauchy–Schwarz inequality,

$$\nabla J(\theta)^\top u \geq -\|\nabla J(\theta)\| \, \|u\| = -\|\nabla J(\theta)\|.$$

Equality holds when u points exactly opposite to the gradient, that is,

$$u = -\frac{\nabla J(\theta)}{\|\nabla J(\theta)\|}.$$

Hence the negative gradient gives the direction of steepest local decrease. □

This proposition explains why gradients are so useful in high dimensions. Instead of searching blindly over many possible directions, the gradient gives a principled local direction of improvement.

A worked one-dimensional example

Consider the one-dimensional objective

$$J(\theta) = \frac{1}{2}(\theta - 3)^2.$$

Then

$$J'(\theta) = \theta - 3.$$

Gradient descent with a fixed learning rate η gives

$$\theta_{t+1} = \theta_t - \eta(\theta_t - 3).$$

If we choose $\eta = \frac{1}{2}$ and start from $\theta_0 = 0$, then

$$\theta_1 = 0 - \frac{1}{2}(0 - 3) = \frac{3}{2},$$
$$\theta_2 = \frac{3}{2} - \frac{1}{2}\left(\frac{3}{2} - 3\right) = \frac{9}{4},$$
$$\theta_3 = \frac{9}{4} - \frac{1}{2}\left(\frac{9}{4} - 3\right) = \frac{21}{8}.$$

The iterates move toward $\theta = 3$, which is the minimizer.

This example is simple, but it already shows the key idea: optimization proceeds by repeated local corrections based on derivative information.

Stochastic gradient descent

For large datasets, computing the full gradient of

$$J_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

may be expensive. Indeed,

$$\nabla J_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta \ell(f_\theta(x_i), y_i),$$

which requires a pass through the entire training set at every step.

A cheaper alternative is to approximate the gradient using only a small part of the sample.

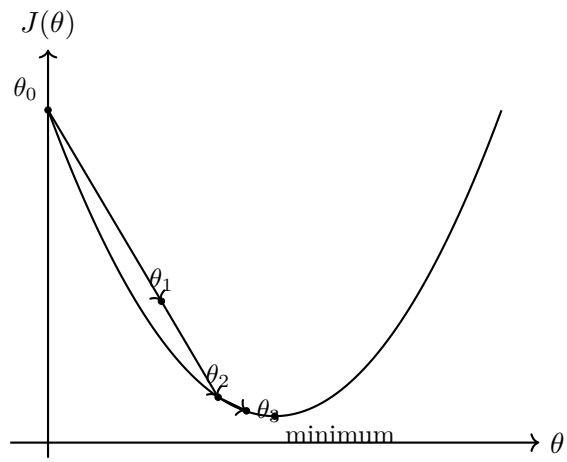


Figure 1.8: A one-dimensional loss landscape. Gradient descent updates move iteratively downhill toward a minimizer.

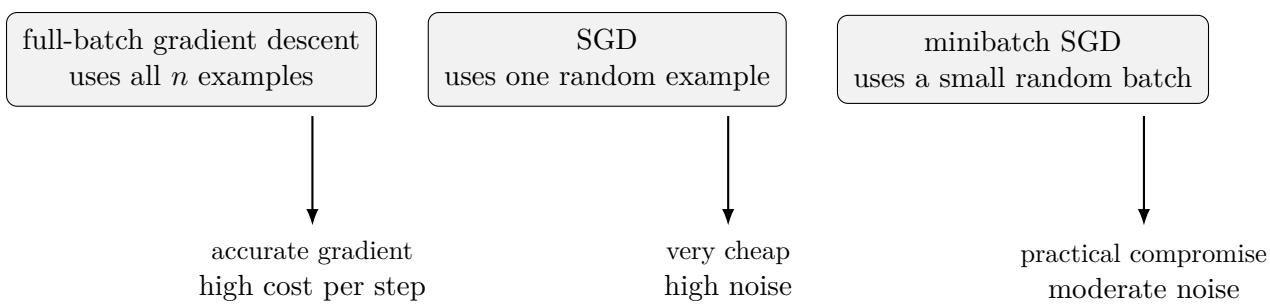


Figure 1.9: Full-batch gradient descent, single-example SGD, and minibatch SGD. Modern training usually uses minibatches because they balance computational cost and gradient quality.

Definition 1.48 (Stochastic gradient descent)**.** *Stochastic gradient descent* (SGD) updates the parameters using a random gradient estimate:

$$\theta_{t+1} = \theta_t - \eta_t g_t,$$

where g_t is computed from a randomly chosen example or minibatch. For a single sampled example (x_i, y_i), one may take

$$g_t = \nabla_\theta \ell(f_{\theta_t}(x_i), y_i).$$

If the example is sampled uniformly at random, then

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J_n(\theta_t),$$

so the stochastic update is noisy but unbiased with respect to the empirical objective.

Minibatches and gradient noise

In practice, one usually uses a minibatch $B_t \subset \{1, \ldots, n\}$ and forms

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell(f_{\theta_t}(x_i), y_i).$$

This gives a compromise between two extremes:

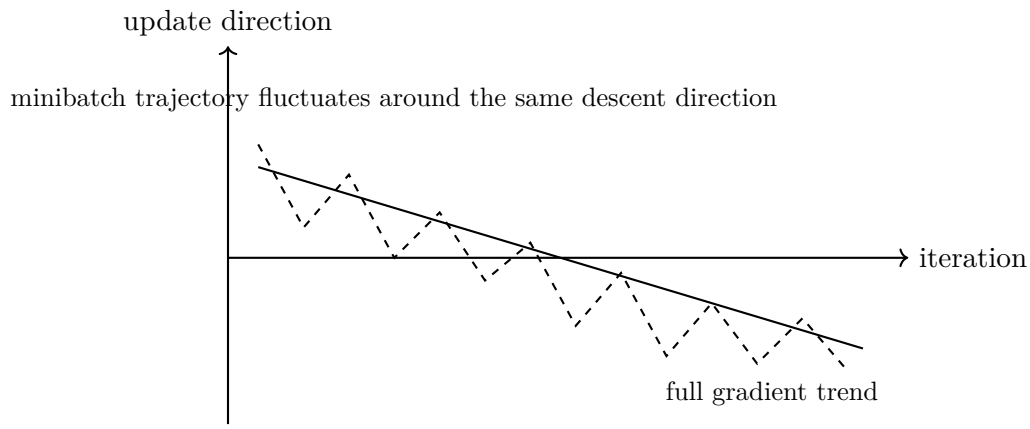


Figure 1.10: Minibatch noise intuition. A stochastic gradient follows the overall downhill trend only approximately, with fluctuations caused by sampling different subsets of data.

- full-batch gradients are accurate but expensive;
- single-example gradients are cheap but noisy.

The noise is not merely a nuisance. It is part of the dynamics of modern optimization. The estimated gradient fluctuates from step to step because different minibatches contain different examples.

Learning rates and the scale of updates

The learning rate η_t determines how far the algorithm moves in the chosen direction. If it is too small, progress may be extremely slow. If it is too large, the algorithm may overshoot, oscillate, or diverge.

Thus optimization is not only about the direction of motion but also about its scale. Later chapters will revisit this when we discuss signal propagation, initialization, and optimization in deep networks.

A convexity intuition

Some classical learning objectives are convex, and convexity gives a particularly clean optimization picture.

Proposition 1.49 (Local minima are global for convex functions)**.** *Let $J : \Theta \to \mathbb{R}$ be convex on a convex set Θ. Then every local minimizer of J is also a global minimizer.*

Proof. Suppose $\theta^\star$ is a local minimizer but not a global minimizer. Then there exists $\theta \in \Theta$ such that

$$J(\theta) < J(\theta^\star).$$

For $0 < \lambda < 1$, convexity gives

$$J\big((1-\lambda)\theta^\star + \lambda\theta\big) \le (1-\lambda)J(\theta^\star) + \lambda J(\theta) < J(\theta^\star).$$

For sufficiently small λ, the point $(1-\lambda)\theta^\star + \lambda\theta$ lies arbitrarily close to $\theta^\star$, contradicting the assumption that $\theta^\star$ is a local minimizer. Therefore every local minimizer must be global. $\square$

This helps explain why convex models such as linear regression and logistic regression are mathematically attractive. Their optimization landscapes are simpler to reason about than the highly nonconvex landscapes of deep networks.

Optimization is not generalization

A central lesson of machine learning is that optimization and generalization are related but distinct.

Optimization question. Can we find parameters that make the training objective small?

Generalization question. Will those parameters perform well on new data?

These are not the same question. A model can be optimized successfully and still generalize poorly if it overfits the training sample. Conversely, a model may be difficult to optimize but still have good predictive potential if a good solution can be found.

Common confusion. Low training loss is not identical to good generalization. Optimization concerns how well we reduce the chosen objective on the training data. Generalization concerns whether the resulting predictor performs well on fresh data. Confusing the two leads to one of the most common misunderstandings in machine learning.

What to remember. Once a model and loss have been chosen, learning usually becomes an optimization problem. Gradient descent follows the negative gradient because it is the direction of steepest local decrease. Stochastic gradient descent replaces the full gradient by a cheaper random estimate, and minibatches provide the practical compromise used in modern training. Optimization determines how low the training objective becomes, but low training loss by itself does not guarantee good prediction on new data.

Exercises

Exercise 1.50. Let

$$J(\theta) = \frac{1}{2}(\theta - 5)^2.$$

Compute $J'(\theta)$ and write the gradient descent update with learning rate η. Starting from $\theta_0 = 1$, compute θ_1 and θ_2 when $\eta = \frac{1}{2}$.

Exercise 1.51. Consider the linear model

$$f_\theta(x) = \theta x$$

with squared loss on a single example (x, y). Show that

$$\ell(\theta; (x, y)) = (\theta x - y)^2$$

has derivative

$$\frac{d}{d\theta}\ell(\theta; (x, y)) = 2x(\theta x - y).$$

Exercise 1.52. Explain in words why the negative gradient is a sensible update direction for a differentiable objective.

Exercise 1.53. What is the practical difference between full-batch gradient descent and stochastic gradient descent? What is the role of a minibatch?

Exercise 1.54. Give an example of a situation in which a model achieves very low training loss but still generalizes poorly. Which concept from the previous section does this illustrate?

Exercise 1.55. Why does convexity make optimization easier to reason about, at least conceptually, than a general nonconvex objective?

Notes and references

This section introduces the optimization viewpoint that turns empirical risk minimization into an algorithmic problem. The role of gradient descent, SGD, minibatches, learning rates, and convexity will recur throughout the book. At this stage the emphasis is conceptual: optimization is the computational mechanism by which learning occurs, but it does not by itself explain why learned models generalize. Chapter 2 will revisit these ideas in the richer setting of neural networks, backpropagation, signal propagation, and the training of deep models.

1.6 Main Learning Paradigms

So far, learning has been described through a common mathematical language: data, hypothesis classes, losses, risk, and optimization. But not all learning problems present information in the same way. In some settings the learner receives explicit labels; in others it must discover structure from raw observations; in still others it must act, observe consequences, and improve through delayed feedback.

These differences give rise to the main paradigms of machine learning. The point of this section is not merely to list them, but to unify them. The paradigms differ in the form of supervision they provide, yet all can be viewed as optimizing objective functionals built from data or interaction.

Four supervision regimes

Definition 1.56 (Supervised learning)**.** In *supervised learning*, the learner receives examples of the form

$$(x_i, y_i) \in \mathcal{X} \times \mathcal{Y},$$

where x_i is an input and y_i is a target or label. The goal is to learn a predictor

$$f : \mathcal{X} \to \mathcal{Y}$$

that performs well on future labeled examples.

The supervision signal here is explicit: the desired output is provided directly by the data source.

Definition 1.57 (Unsupervised learning)**.** In *unsupervised learning*, the learner observes inputs

$$x_1, \ldots, x_n \in \mathcal{X}$$

without externally supplied labels. The goal is to discover structure in the data, such as clustering, latent factors, low-dimensional representations, or a model of the data distribution itself.

The absence of labels does not mean the absence of a task. It means that the task is defined through structure rather than through externally supplied targets.

Definition 1.58 (Self-supervised learning)**.** In *self-supervised learning*, the learner constructs supervision from the data themselves. From raw observations x, it defines auxiliary prediction tasks whose targets are derived automatically from the observed sample.

Examples include predicting a masked token in a sentence, predicting whether two views came from the same image, or reconstructing a withheld part of a signal. The targets are not given by a human annotator, but they are still targets.

Definition 1.59 (Reinforcement learning)**.** In *reinforcement learning*, the learner interacts with an environment over time. At step t, it observes a state s_t, chooses an action a_t, receives a reward r_t, and transitions to a new state. The goal is to learn a policy

$$\pi(a \mid s)$$

that maximizes expected cumulative reward.

Here the supervision signal is evaluative rather than corrective: the learner is not told the right action directly, but is told how good the consequences of its behavior were.

A unified optimization viewpoint

Although these paradigms look different on the surface, they share a common mathematical pattern.

Proposition 1.60 (Learning paradigms as objective-based optimization)**.** *Each of the four paradigms can be viewed as selecting an object h from a hypothesis space $\mathcal{H}$ in order to optimize an objective functional built from observations or interaction:*

$$h^\star \in \arg\min_{h\in\mathcal{H}} \mathcal{J}(h) \qquad \text{or} \qquad h^\star \in \arg\max_{h\in\mathcal{H}} \mathcal{J}(h).$$

More specifically:

1. *in supervised learning, $\mathcal{J}$ is typically an expected prediction loss over labeled pairs (X, Y);*
2. *in unsupervised learning, $\mathcal{J}$ measures a structural criterion over inputs X, such as likelihood, reconstruction quality, or clustering distortion;*
3. *in self-supervised learning, $\mathcal{J}$ measures predictive performance on targets automatically derived from raw data;*
4. *in reinforcement learning, $\mathcal{J}$ is expected cumulative reward over trajectories induced by interaction with an environment.*

Proof. We verify the claim by writing a representative objective for each regime.

In supervised learning, one often minimizes

$$\mathcal{J}(f) = \mathbb{E}[\ell(f(X), Y)].$$

In unsupervised learning, one may maximize log-likelihood

$$\mathcal{J}(g) = \mathbb{E}[\log p_g(X)],$$

or minimize reconstruction error

$$\mathcal{J}(g, \phi) = \mathbb{E}[\|X - g(\phi(X))\|^2].$$

In self-supervised learning, one defines a target transformation $t(X)$ or paired views derived from X and optimizes an objective such as

$$\mathcal{J}(h) = \mathbb{E}[\ell(h(\tilde{X}), t(X))].$$

In reinforcement learning, one optimizes

$$\mathcal{J}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r_t\right],$$

where the expectation is over trajectories generated by policy π and the environment dynamics.

Thus, despite differing forms of supervision, each paradigm is governed by an objective functional over data or interaction. □

This proposition is important because it prevents the chapter from fragmenting into unrelated categories. The paradigms differ in what information is available and what must be predicted or optimized, but they all fit into the broader view of learning as structured objective optimization.

Supervised learning

Supervised learning is the most direct regime. The learner sees examples together with desired outputs and tries to infer a rule that extends beyond the sample.

Typical tasks include classification and regression. A standard objective is empirical risk minimization:

$$\hat{f} \in \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

Its central challenge is generalization: how to use finitely many labeled examples to perform well on new ones.

Unsupervised learning

In unsupervised learning, the learner receives only raw observations. The aim may be to model the distribution of the data, compress them into a lower-dimensional representation, cluster them into groups, or extract latent structure.

Representative objectives include

$$\max_{\theta} \sum_{i=1}^{n} \log p_\theta(x_i)$$

for density modeling, or

$$\min_{\phi, g} \sum_{i=1}^{n} \|x_i - g(\phi(x_i))\|^2$$

for autoencoding.

Common confusion. "Unsupervised" does not mean "no objective." It means "no externally supplied labels." Unsupervised methods still optimize clearly defined criteria such as likelihood, reconstruction error, contrastive structure, or clustering distortion.

Self-supervised learning

Self-supervised learning occupies a middle ground between supervised and unsupervised learning. It uses raw data, like unsupervised learning, but formulates predictive tasks with explicit targets, like supervised learning.

For example, if x is a sentence and one token is masked, the learner may try to predict the missing token:

$$\min_{\theta} \mathbb{E}[-\log p_\theta(x_j \mid x_{\setminus j})].$$

If x is an image, the learner may form two transformed views $v_1(x)$ and $v_2(x)$ and train representations so that matching views are close while mismatched views are separated.

The key point is that supervision is *constructed*, not externally annotated.

Common confusion. Self-supervised learning is not just unsupervised learning with a fashionable name. Its defining feature is that it creates explicit predictive targets from the data themselves. The model is trained to solve auxiliary prediction tasks, often in ways that produce useful transferable representations.

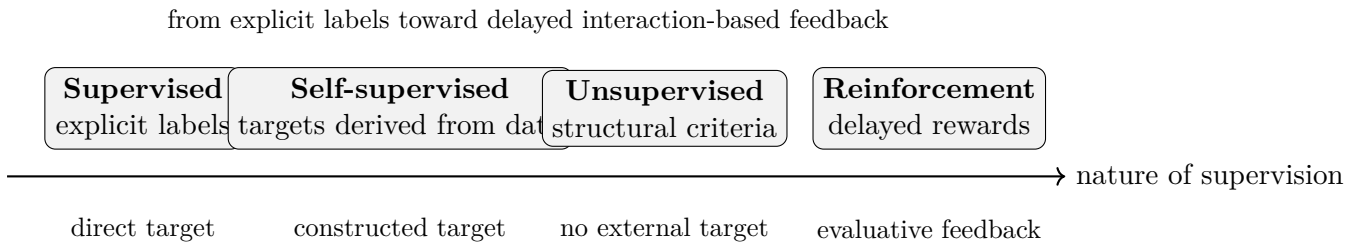


Figure 1.11: The main paradigms can be arranged by the form of supervision they provide: from explicit labels, through internally constructed targets and structural objectives, to delayed rewards from interaction.

Reinforcement learning

Reinforcement learning differs from the previous regimes because learning occurs through sequential interaction. The learner does not merely observe a static sample; it acts in an environment and experiences the consequences of those actions over time.

A policy π is evaluated by expected return

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

The learner seeks

$$\pi^\star \in \arg\max_\pi J(\pi).$$

This introduces new difficulties absent from static prediction:

- actions influence future data,
- rewards may be delayed,
- exploration is necessary,
- feedback is often sparse and noisy.

Thus reinforcement learning is not merely supervised learning on a dataset of actions. The learner must cope with control, interaction, and long-term consequences.

A comparative summary

Why the paradigms matter

The paradigms are not merely administrative categories. They reflect genuinely different ways in which information can constrain learning.

- Supervised learning asks how input-output relationships can be inferred from labeled examples.
- Unsupervised learning asks what structure can be extracted from observations alone.
- Self-supervised learning asks how raw data can be turned into their own supervisory signal.
- Reinforcement learning asks how action can be improved when feedback is delayed and shaped by interaction.

At the same time, the boundaries are not absolute. A modern system may pretrain representations self-supervisedly, fine-tune them with supervision, and then adapt them inside a reinforcement-learning loop. The paradigms are best understood as organizing viewpoints, not sealed compartments.

Paradigm	**Data**	**Supervision signal**	**Objective**	**Output / evaluation**
Supervised	labeled pairs (x, y)	explicit external labels	prediction loss	predictor; accuracy, risk, error
Unsupervised	unlabeled inputs x	structural regularities in data	likelihood, reconstruction, clustering, compression	representation or generative structure; likelihood, distortion, clustering quality
Self-supervised	unlabeled inputs x	targets derived from the data	auxiliary predictive objective	representation or predictor; transfer quality, downstream performance
Reinforcement	trajectories (s_t, a_t, r_t)	delayed reward from interaction	expected cumulative reward	policy; return, success rate, long-term performance

Table 1.1: A comparison of the main learning paradigms by data source, supervision signal, objective, output, and evaluation.

What to remember. The main learning paradigms differ by their supervision regime: supervised learning uses explicit labels, unsupervised learning uses structural criteria on unlabeled data, self-supervised learning creates targets from the data themselves, and reinforcement learning improves behavior through delayed rewards from interaction. Despite these differences, all four can be viewed as optimizing objective functionals over data or trajectories. The key unifying question is not whether there is an objective, but what form of information defines it.

Exercises

Exercise 1.61. For each of the following, identify the most natural learning paradigm and briefly justify your answer:

1. predicting house prices from past sales data,
2. grouping customers into behavioral segments without labels,
3. predicting a masked word in a sentence,
4. learning to control a robot arm from reward signals,
5. reconstructing an image from a compressed latent code.

Exercise 1.62. Write a toy self-supervised objective for one of the following settings:

1. a sentence with one hidden token,
2. an image with one missing patch,
3. a time series with a short future segment to be predicted.

Be explicit about what serves as the input and what serves as the target.

Exercise 1.63. Explain why reinforcement learning is not simply supervised learning on a dataset of trajectories. In your answer, discuss at least two of the following: delayed reward, exploration, action-dependent data, and long-term consequences.

Exercise 1.64. Give one example of an unsupervised objective and one example of a self-supervised objective. Why is it helpful to distinguish these two regimes instead of merging them into a single category?

Exercise 1.65. A modern model is pretrained by predicting missing tokens on a large text corpus and then fine-tuned on labeled sentiment examples. Which paradigms appear in this pipeline, and what role does each one play?

Notes and references

The purpose of this section is to organize the major learning paradigms by the form of supervision they use while keeping a unified mathematical viewpoint. The crucial point is that the absence of explicit labels does not imply the absence of an objective. Unsupervised learning, self-supervised learning, and reinforcement learning all optimize structured criteria, but those criteria arise from different sources: data geometry, automatically constructed prediction tasks, or delayed evaluative feedback from interaction. This broader view will be important later when the book moves from classical supervised prediction to representation learning, generative modeling, and sequential decision-making.

1.7 From Classical Prediction to Representation Learning

The previous sections developed a general mathematical view of learning: data define a task, losses define performance, model classes encode inductive bias, and optimization turns an objective into a learned predictor. We now arrive at a transition that is historically and conceptually decisive.

For a long time, much of machine learning was organized around the following pattern: first design features by hand, then learn a predictor on top of those features. Modern deep learning changed this pattern by making the features themselves learnable.

This shift from *fixed representations* to *learned representations* is one of the central ideas of the book.

Representations as internal coordinate systems

A predictor need not act directly on raw inputs. Often, the input is first transformed into a new space in which the relevant structure is easier to express.

Definition 1.66 (Representation map)**.** A *representation map* is a function

$$\phi : \mathcal{X} \to \mathcal{Z}$$

from the input space $\mathcal{X}$ to a feature or representation space $\mathcal{Z}$. The image $\phi(x)$ is called a *representation* or *feature vector* of the input x.

The point of a representation is not merely to re-encode the data, but to place them in coordinates that make the downstream task simpler. A good representation can separate classes, reveal latent structure, or compress information while retaining what matters for prediction.

Classical learning as prediction on fixed features

Many classical learning pipelines can be written in the compositional form

$$f = g \circ \phi,$$

where ϕ is a feature map and g is a predictor acting on the feature space.

Proposition 1.67 (Classical learning often uses a fixed representation)**.** *In many classical learning methods, prediction takes the form*

$$f(x) = g(\phi(x)),$$

where the representation map $\phi : \mathcal{X} \to \mathcal{Z}$ is fixed before learning, and only the predictor $g : \mathcal{Z} \to \mathcal{Y}$ is learned from data.

Proof. The statement is structural rather than tied to one algorithm. In a classical pipeline, one first defines features

$$z = \phi(x),$$

for example edge counts, frequency statistics, polynomial expansions, or kernel-induced coordinates. One then applies a learned predictor g to those features:

$$\hat{y} = g(z) = g(\phi(x)).$$

Because the feature map is specified in advance and not adapted through training, the learned part of the system is g, not ϕ. □

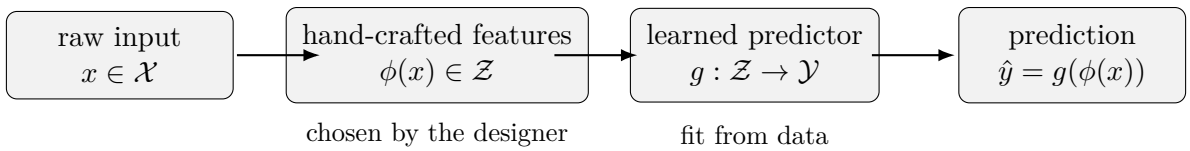


Figure 1.12: A classical pipeline: raw inputs are first converted into hand-crafted features, and a learned predictor is trained on top of those fixed features.

Examples include:

- linear regression on manually engineered input features,
- support vector machines on hand-crafted descriptors,
- classifiers built on signal-processing features such as MFCCs in speech or SIFT-like descriptors in vision,
- kernel methods in which the representation is determined implicitly by the choice of kernel.

The important point is not that these methods are weak; many are extremely powerful. The point is that the representational stage is largely external to the learning process.

Why representations matter

The same predictor class can behave very differently depending on the representation. A linear classifier may fail completely in raw coordinates and succeed immediately in a more suitable feature space.

This is one reason feature design mattered so much in classical machine learning: the choice of representation often determined whether the learning problem became simple or difficult.

At a conceptual level, the representation map ϕ acts as an *internal coordinate system*. It changes the geometry of the problem. Distances, linear separability, smoothness, and cluster structure may look very different in $\mathcal{Z}$ than in $\mathcal{X}$.

Deep learning as representation learning

Modern deep learning changed the balance of the pipeline by making the representation itself adaptive.

Proposition 1.68 (Deep learning makes the representation learnable)**.** *A deep learning model typically has the compositional form*

$$f_\theta = g_{\theta_2} \circ \phi_{\theta_1},$$

or more generally

$$f_\theta = h_L \circ h_{L-1} \circ \cdots \circ h_1,$$

where the representation map is parameterized and learned from data together with the final predictor.

Proof. In a multilayer model, each intermediate map transforms its input into a new representation:

$$z_1 = h_1(x), \qquad z_2 = h_2(z_1), \qquad \ldots, \qquad z_L = h_L(z_{L-1}).$$

The final prediction is obtained by composition. Because the parameters of these layers are updated during training, the intermediate representations are not fixed in advance. They are learned jointly with the output map in order to reduce the training objective. □

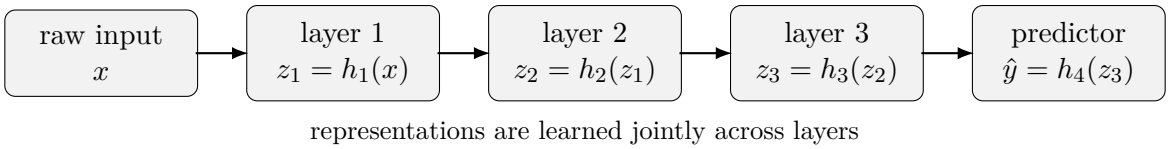


Figure 1.13: A deep learning pipeline: the representation is not fixed in advance but built through multiple learned transformations.

This is the decisive conceptual shift. Instead of asking only, "Which predictor should be fit on top of a feature space?", deep learning asks, "Which feature space should itself be learned so that prediction becomes easier?"

A worked example: raw inputs, hand-crafted features, and learned hidden representations

Consider a binary classification problem in $\mathbb{R}^2$ where the positive class lies near a ring around the origin and the negative class lies near the center. In raw coordinates, these classes are not linearly separable.

Raw-input linear classifier. A linear classifier has the form

$$f(x) = \operatorname{sign}(w^\top x + b).$$

Because its decision boundary is a line, it cannot separate a central cluster from a surrounding ring.

Hand-crafted feature representation. Now define a feature map

$$\phi(x) = \|x\|^2 = x_1^2 + x_2^2.$$

A linear rule in feature space becomes

$$g(\phi(x)) = \operatorname{sign}(a\,\phi(x) + c),$$

which is equivalent to thresholding the squared radius. This can separate the inner cluster from the outer ring.

Learned hidden representation. A multilayer neural network may instead learn an internal map

$$\phi_\theta : \mathbb{R}^2 \to \mathbb{R}^m$$

such that the transformed points become approximately linearly separable in representation space. The final layer may still be linear,

$$\hat{y} = \operatorname{sign}(w^\top \phi_\theta(x) + b),$$

but now the coordinates have been learned rather than manually designed.

The lesson is not merely that neural networks are more flexible. It is that deep learning moves feature construction inside the learning process.

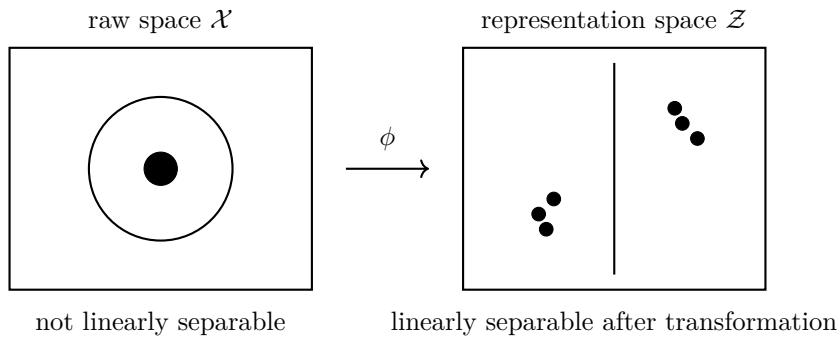


Figure 1.14: A schematic feature-space view. In raw coordinates, the relevant classes may be entangled, while an appropriate representation can transform them into a space where a simple decision rule becomes effective.

Representation learning as the deep-learning successor to feature engineering

This perspective helps connect classical and modern machine learning.

In classical methods, the designer often supplied the structure:

$$x \longmapsto \phi(x) \longmapsto g(\phi(x)).$$

In deep learning, the model seeks that structure from data:

$$x \longmapsto \phi_{\theta_1}(x) \longmapsto g_{\theta_2}(\phi_{\theta_1}(x)).$$

Thus architecture takes over part of the role once played by manual feature engineering. Convolutions encode locality and translation structure. Sequence models encode temporal dependence. Attention mechanisms encode content-based interaction. The model class does not only specify how to predict; it also specifies how useful representations may be constructed.

Common confusion. Representation learning does not mean that raw inputs become irrelevant or that hand-crafted structure disappears entirely. Deep models still rely on inductive bias: architecture, normalization, optimization, and data design all matter. The difference is that many useful intermediate features are now learned jointly with the task, rather than fixed completely in advance.

What to remember. A representation map $\phi : \mathcal{X} \to \mathcal{Z}$ transforms raw inputs into coordinates better suited to a task. Classical learning often has the form $f = g \circ \phi$ with a fixed feature map ϕ and a learned predictor g. Deep learning changes this by making ϕ itself parameterized and learnable, often through many layers of composition. This shift from fixed features to learned representations is one of the defining ideas of modern machine learning.

Exercises

Exercise 1.69. Give an example of a learning problem in which a simple predictor becomes much more effective after a suitable feature transformation. Describe both the raw input space and the transformed feature space.

Exercise 1.70. Consider the feature map

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2).$$

Explain why a linear classifier in the feature space can represent decision boundaries that are not linear in the original input space.

Exercise 1.71. In your own words, explain the difference between

$$f(x) = g(\phi(x))$$

with fixed ϕ and

$$f_\theta(x) = g_{\theta_2}(\phi_{\theta_1}(x))$$

with learned ϕ_{θ_1}.

Exercise 1.72. Why is it reasonable to say that representation learning changes the geometry of the problem? Give one example of a property that may become simpler in representation space than in raw space.

Exercise 1.73. A neural network ends with a linear output layer. Why is it still not "just a linear model" in general?

Notes and references

This section marks the conceptual transition from classical prediction to deep learning. The central idea is that many learning systems can be written compositionally, but classical pipelines usually fix the representation map in advance, whereas deep learning makes that representation itself learnable. This is why the rise of neural networks was not only a change of model family but also a change in where feature construction occurs: it moved from external design into the learned system. The next chapter begins from exactly this question: if learning the representation is the key idea, what mathematical object is a neural network?

Chapter 2

Neural Networks and the Rise of Deep Learning

2.1 From Classical Prediction to Learned Representations

Chapter 1 ended with a structural observation that now becomes the starting point of this chapter. Many learning systems can be written in compositional form:

$$f = g \circ \phi.$$

The decisive question is therefore not only which predictor g should be fit, but also whether the representation map ϕ is fixed in advance or learned from data.

This section makes that contrast concrete by following a classical path:

perceptron → logistic regression → regularization → margin → SVM → kernels → limits of fixed representations → neural networks.

The goal is not to turn the chapter into a full course on support vector machines, but to show why large-margin methods are the high point of the classical fixed-feature pipeline and why deep learning becomes attractive when useful representations are not available in advance.

A formal decomposition of classical prediction

Let $\mathcal{X}$ be the input space, $\mathcal{Z}$ a feature space, and $\mathcal{Y}$ the output space. A representation map

$$\phi : \mathcal{X} \to \mathcal{Z}$$

transforms raw inputs into features, while a predictor

$$g : \mathcal{Z} \to \mathcal{Y}$$

acts on those features.

Definition 2.1 (Fixed-feature predictive pipeline)**.** A *fixed-feature predictive pipeline* is a predictor of the form

$$f = g \circ \phi,$$

where the representation map ϕ is specified in advance and only the predictor g is learned from data.

This description covers a large fraction of classical machine learning. The features may be polynomial expansions, hand-crafted signal descriptors, bag-of-words vectors, random features, or kernel-induced coordinates. The learning algorithm then operates on those features rather than on the raw input directly.

Proposition 2.2 (Fixed representations induce hypothesis classes)**.** *Let $\phi : \mathcal{X} \to \mathcal{Z}$ be a fixed representation map, and let $\mathcal{G}$ be a class of predictors from $\mathcal{Z}$ to $\mathcal{Y}$. Then ϕ induces a hypothesis class on $\mathcal{X}$ given by*

$$\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}.$$

In contrast, when both the representation map and the predictor are learned, the effective hypothesis class takes the form

$$\mathcal{H} = \{g \circ \phi : \phi \in \Phi,\ g \in \mathcal{G}\},$$

which is generally much richer.

Proof. If ϕ is fixed, then every candidate predictor on $\mathcal{X}$ is obtained by composing ϕ with some $g \in \mathcal{G}$. Hence the available functions on the original input space are exactly

$$\{g \circ \phi : g \in \mathcal{G}\}.$$

If, on the other hand, the representation itself can vary over a family Φ, then the learner may choose both a representation and a predictor, producing the larger composed class

$$\{g \circ \phi : \phi \in \Phi,\ g \in \mathcal{G}\}.$$

□

This proposition is simple but fundamental. It makes clear that feature design is not a cosmetic preprocessing step. It directly determines the hypothesis class the learner can realize.

The classical path: from separability to margin

The perceptron and logistic regression are natural starting points because both use an affine score

$$s(x) = w^\top \phi(x) + b,$$

but interpret and train that score differently.

Perceptron. The perceptron predicts by thresholding the sign of the score. Its geometry is crisp: it asks whether a hyperplane can separate the two classes.

Logistic regression. Logistic regression uses the same affine score but passes it through a sigmoid or softmax and optimizes a smooth likelihood-based loss. The output is naturally interpreted in probabilistic terms.

Regularization. As Chapter 1 already emphasized, fitting the training data is not the whole story. A penalty such as $\lambda\|w\|^2$ controls the scale of the classifier and influences which separator is preferred among many that fit the sample.

These three ideas lead directly to the margin viewpoint. If many separators classify the data correctly, it is natural to ask whether some are geometrically better than others.

Why margin matters

Suppose binary labels satisfy $y_i \in \{-1, 1\}$ and the classifier is

$$f(x) = \operatorname{sign}(w^\top x + b).$$

A point is correctly classified when $y_i(w^\top x_i + b) > 0$. But correct classification alone does not distinguish a fragile separator from a robust one.

Definition 2.3 (Functional margin)**.** For a labeled example (x_i, y_i) and affine score $s(x) = w^\top x + b$, the *functional margin* is

$$y_i(w^\top x_i + b).$$

For a dataset, the functional margin is the minimum of this quantity over all training examples.

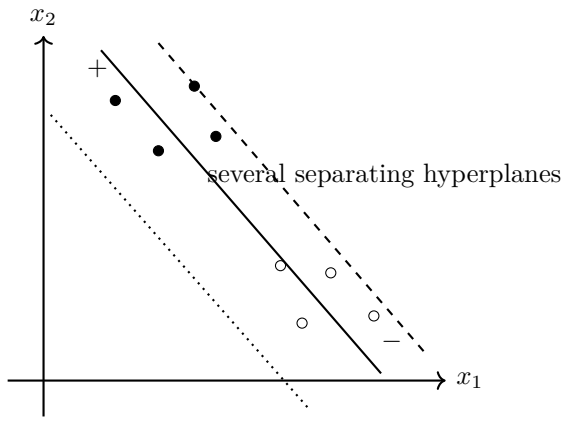


Figure 2.1: Linearly separable data can admit many separating hyperplanes. Margin-based methods ask not only for separation, but for a separator that stays as far as possible from the closest training points.

The functional margin depends on scale. If we multiply (w, b) by a positive constant, the classifier does not change, but the functional margin does. The geometric margin corrects for this.

Definition 2.4 (Geometric margin)**.** The *geometric margin* of (x_i, y_i) with respect to (w, b) is

$$\frac{y_i(w^\top x_i + b)}{\|w\|}.$$

For a dataset, the geometric margin is the minimum of this quantity over the sample.

Proposition 2.5 (Margin is scale-invariant after normalization)**.** *For any $c > 0$, the classifiers (w, b) and (cw, cb) define the same decision boundary. Their functional margins differ by a factor of c, but their geometric margins are equal.*

Proof. The sign of $cw^\top x + cb$ is the same as the sign of $w^\top x + b$ for every x because $c > 0$. Thus the decision boundary is unchanged. The functional margin becomes

$$y_i(cw^\top x_i + cb) = c\, y_i(w^\top x_i + b),$$

so it scales by c. The geometric margin becomes

$$\frac{y_i(cw^\top x_i + cb)}{\|cw\|} = \frac{c\, y_i(w^\top x_i + b)}{c\|w\|} = \frac{y_i(w^\top x_i + b)}{\|w\|},$$

which is unchanged. □

This is why the geometric margin is the meaningful object. It measures distance to the decision boundary rather than arbitrary parameter scale.

Hard-margin SVM

If the data are linearly separable, one can ask for the separator of largest geometric margin. By scaling (w, b) so that the closest points satisfy

$$y_i(w^\top x_i + b) \geq 1,$$

maximizing geometric margin becomes equivalent to minimizing $\|w\|$.

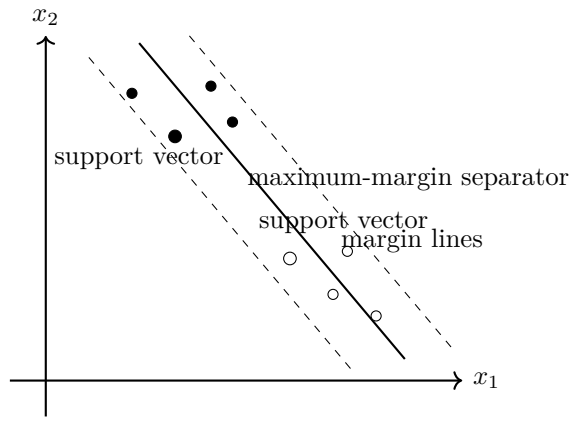


Figure 2.2: Hard-margin geometry. The central line is the decision boundary, the dashed lines mark equal-distance margin boundaries, and the nearest points are the support vectors. Only those closest points determine the optimal separator.

Definition 2.6 (Hard-margin support vector machine)**.** For linearly separable data, the *hard-margin SVM* solves

$$\min_{w,b} \frac{1}{2}\|w\|^2 \qquad \text{subject to} \qquad y_i(w^\top x_i + b) \geq 1 \quad \text{for all } i.$$

The geometry now becomes very clean. A large-margin separator is not merely one that classifies correctly; it is one that keeps the boundary away from the closest training examples.

Soft margin, slack, and regularization

Real data are rarely perfectly separable. Noise, overlap, or label ambiguity may force some violations. The soft-margin SVM introduces slack variables $\xi_i \geq 0$:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

The optimization problem becomes

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \qquad \text{subject to} \qquad \xi_i \geq 0, \quad y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

The parameter C trades off two goals:

- keeping the separator simple and large-margin through $\|w\|^2$;
- penalizing violations through the slack terms.

This is regularization in action. The model does not merely search for any classifier with low training error; it balances data fit against separator scale.

Why SVM is not just another linear classifier

At first glance, perceptron, logistic regression, and a linear SVM all use the score $w^\top x + b$. But they embody different learning principles:

- the perceptron focuses on correcting mistakes;
- logistic regression models class probabilities through a smooth loss;
- SVM focuses on the geometry of margin and the scale of the separator.

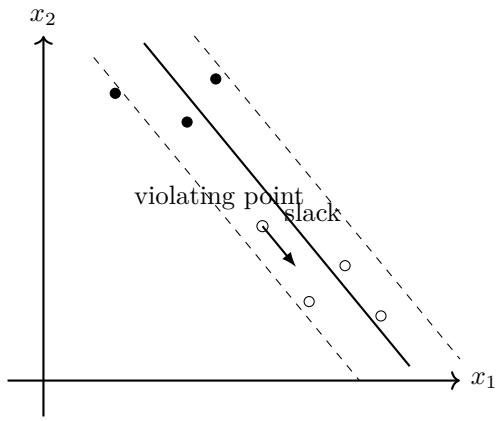


Figure 2.3: Soft-margin SVM allows some points to lie inside the margin or even on the wrong side of the boundary. Slack variables measure the size of these violations, and the objective penalizes them while still preferring a large-margin separator.

Common confusion. An SVM is not "just another linear classifier." In a linear SVM, the decision boundary is linear in the chosen feature space, but the method is distinguished by the margin principle, the role of support vectors, and the explicit control of separator scale through regularization. With kernels, the boundary can also become highly nonlinear in the original input space even though the classifier remains linear in feature space.

The kernel viewpoint

A key classical insight is that linear prediction in a feature space can look nonlinear in the original coordinates. If one chooses a nonlinear map

$$\phi(x),$$

and then applies a linear classifier in the ϕ-coordinates, the resulting boundary in input space can be curved and expressive.

The kernel trick pushes this idea further. Rather than write $\phi(x)$ explicitly, one works with inner products

$$K(x, x') = \langle \phi(x), \phi(x') \rangle.$$

The predictor can then be written in terms of similarities to selected training points.

Kernels are therefore a powerful extension of the fixed-feature pipeline. They allow nonlinear decision boundaries while preserving the basic classical structure:

> choose a representation geometry first, then fit a shallow predictor inside it.

A comparative summary

The limit of fixed representations

Classical methods can be extremely powerful when the right representation is available. A hand-crafted signal descriptor, a good polynomial basis, or a well-chosen kernel may transform a hard problem into one that a shallow predictor solves elegantly.

But that strategy has a ceiling. If the task requires discovering many levels of abstraction from raw data—edges to parts to objects, phonemes to words to meanings, local patterns to global

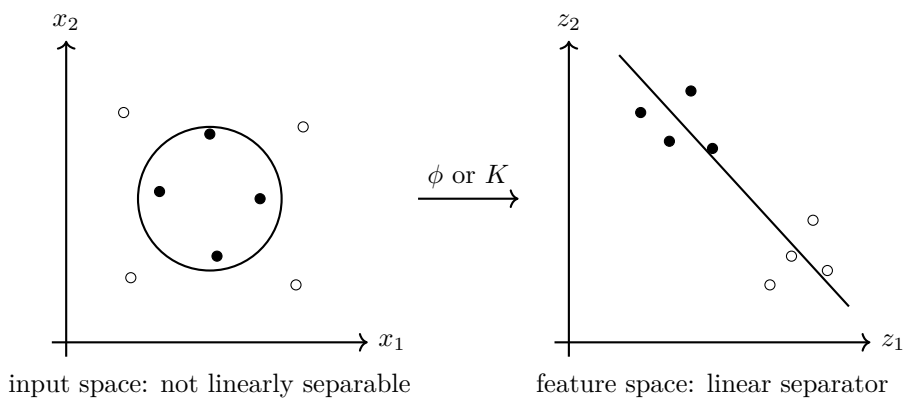


Figure 2.4: A toy nonlinear dataset and its kernelized viewpoint. In the original coordinates the classes may not be linearly separable, but after a suitable feature lifting, a linear separator can become available. Kernel methods realize this idea without always writing the feature map explicitly.

Method	**Representation**	**Learning principle**	**Typical output view**	**Main limitation**
Perceptron	raw inputs or fixed features	update on mistakes for a separating hyperplane	hard classification	requires linear separability for its clean guarantee
Logistic regression	raw inputs or fixed features	smooth likelihood / cross-entropy minimization	probabilistic score	still shallow unless features are enriched
SVM	fixed explicit or kernel features	maximize margin while controlling scale	large-margin classifier	representation is still fixed in advance
Neural network	learned intermediate representations	joint optimization of many composed maps	flexible task-dependent predictor	optimization and generalization are more complex

Table 2.1: A comparison that clarifies the conceptual transition. Perceptron, logistic regression, and SVM differ in training principle, but all remain shallow methods on a chosen representation. Neural networks change the game by learning the representation itself.

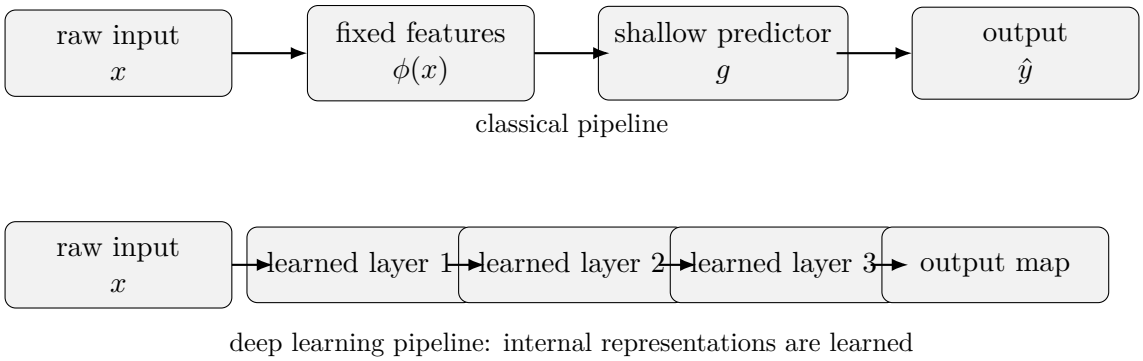


Figure 2.5: The shift from classical prediction to deep learning. Classical methods choose the representation first and then fit a shallow rule on top of it. Deep learning makes the intermediate representation itself trainable.

structure—then a fixed representation may be too rigid. The learner can only optimize within the geometry already chosen for it.

This is the key limitation of the classical pipeline. Its strength is clean geometry and often cleaner optimization. Its weakness is dependence on representation choices made before training begins.

What to remember. The classical fixed-feature pipeline can be summarized as $f = g \circ \phi$: choose a representation, then fit a shallow rule. Perceptron, logistic regression, and SVM all fit this picture, even when kernels make the induced boundary nonlinear in the original input space. Margin turns linear separation into a geometric optimization problem, and soft-margin SVM shows how regularization controls the tradeoff between separator scale and training violations. The decisive limitation is that the representation is fixed in advance. Neural networks will matter in the rest of the chapter because they learn the representation itself.

The transition to learned representations

Deep learning changes the pipeline structurally:

$$x \longmapsto \phi_{\theta_1}(x) \longmapsto g_{\theta_2}(\phi_{\theta_1}(x)).$$

The representation map is no longer fixed by external design. It is parameterized, optimized, and adapted jointly with the final predictor.

This change matters for at least three reasons.

1. It allows the model to discover task-relevant features rather than rely entirely on manual design.
2. It permits multiple layers of transformation, so representations themselves can be hierarchical.
3. It couples representation learning to the final task, allowing internal features to be shaped by predictive success.

So the rise of neural networks was not only a shift in model family. It was a shift in where structure is built: from external preprocessing into the learned system itself.

Exercises

Exercise 2.7. A classifier has score $s(x) = w^\top x + b$. Explain in words how the same score can underlie a perceptron, logistic regression, and a linear SVM while still leading to different learning behavior.

Exercise 2.8. Suppose a separator correctly classifies all training points. Why does that not settle the learning problem from a margin viewpoint?

Exercise 2.9. Show directly that replacing (w, b) by (cw, cb) for $c > 0$ leaves the decision boundary unchanged. Then explain why this makes the geometric margin more meaningful than the functional margin.

Exercise 2.10. In the soft-margin objective

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i,$$

explain informally what happens when C is chosen extremely large and when it is chosen very small.

Exercise 2.11. Consider the XOR pattern in $\mathbb{R}^2$. Why can a linear classifier fail in the original coordinates while a kernel method or a learned hidden representation can succeed?

Exercise 2.12. Using the comparison table, explain in your own words why SVM is best understood as the culmination of the classical fixed-feature pipeline rather than as a first deep-learning model.

Notes and references

The purpose of this section is to create a deliberate bridge from the late foundations material of Chapter 1 into the neural-network machinery that follows. The perceptron will be studied formally in the next section, but the larger conceptual point should already be clear here: classical methods can become quite sophisticated through margin, regularization, and kernels, yet they still depend on a representation that is externally fixed or implicitly chosen in advance. The rest of the chapter studies what changes once that representation itself becomes trainable.

2.2 Neural Networks as Compositional Function Models

The previous section argued that the central shift from classical machine learning to deep learning is the move from fixed representations to learned representations. We now introduce the mathematical object that makes this possible: the neural network.

A neural network is best understood not first as a mysterious biological metaphor, but as a *compositional function model*. It is built from simple maps arranged in layers, with intermediate transformations that act as learned representations. The classical starting point of this story is the perceptron, which already shows both the power and the limitations of single-layer linear prediction.

Affine maps as basic building blocks

A basic component of modern learning models is the affine map

$$x \mapsto Wx + b,$$

where $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{m \times d}$, and $b \in \mathbb{R}^m$.

Definition 2.13 (Affine map)**.** A map $A : \mathbb{R}^d \to \mathbb{R}^m$ is called *affine* if it has the form

$$A(x) = Wx + b$$

for some matrix W and vector b.

Affine maps generalize linear maps by allowing a translation term. They are simple, mathematically tractable, and expressive enough to define hyperplanes and linear decision boundaries. But by themselves they are limited: without nonlinearities, repeated layering does not create genuinely new expressive power.

Perceptrons as linear threshold classifiers

The perceptron is the simplest neural-network-style classifier. For binary labels in $\{-1, 1\}$, it predicts by thresholding an affine score.

Definition 2.14 (Perceptron)**.** A *perceptron* is a classifier of the form

$$f(x) = \operatorname{sign}(w^\top x + b),$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

The equation

$$w^\top x + b = 0$$

defines a hyperplane in $\mathbb{R}^d$. This hyperplane separates the space into two halfspaces,

$$\{x : w^\top x + b > 0\}, \qquad \{x : w^\top x + b < 0\},$$

so a perceptron is a *halfspace classifier*.

Geometrically, the vector w is normal to the separating hyperplane, and the bias b shifts the hyperplane away from the origin.

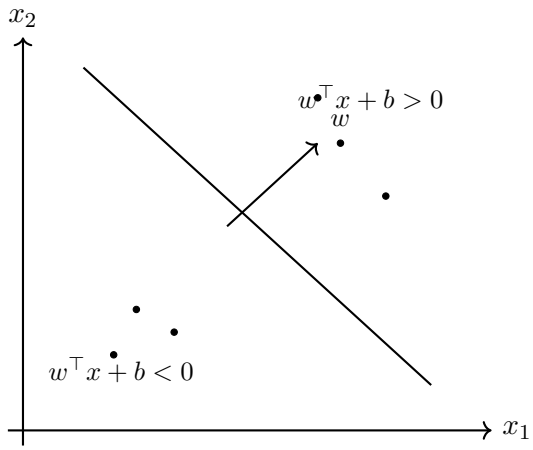


Figure 2.6: A perceptron defines a separating hyperplane and classifies by which side of the hyperplane the input lies on. It is therefore a halfspace classifier.

The perceptron learning algorithm

The perceptron is not only a model class; it also comes with a natural online learning rule. Suppose we have labeled examples

$$(x_i, y_i), \qquad y_i \in \{-1, 1\}.$$

Starting from some initial parameters, one scans through the examples and updates only when a mistake occurs.

Perceptron learning algorithm. Initialize

$$w_0 = 0, \qquad b_0 = 0.$$

Whenever the current classifier misclassifies an example (x_i, y_i), update by

$$w \leftarrow w + y_i x_i, \qquad b \leftarrow b + y_i.$$

Equivalently, if at step t the algorithm makes a mistake on (x_i, y_i), then

$$w_{t+1} = w_t + y_i x_i, \qquad b_{t+1} = b_t + y_i.$$

The update has a simple geometric meaning. If a positive example is classified incorrectly, the weight vector is nudged toward that example. If a negative example is classified incorrectly, the weight vector is nudged away from it.

The perceptron convergence theorem

The perceptron is one of the first examples in machine learning where a simple algorithm admits a clean mathematical guarantee.

Theorem 2.15 (Perceptron convergence)**.** *Let $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, 1\}$ be linearly separable. Assume there exist a unit vector u and a margin $\gamma > 0$ such that*

$$y_i \, u^\top x_i \geq \gamma \quad \textit{for all } i,$$

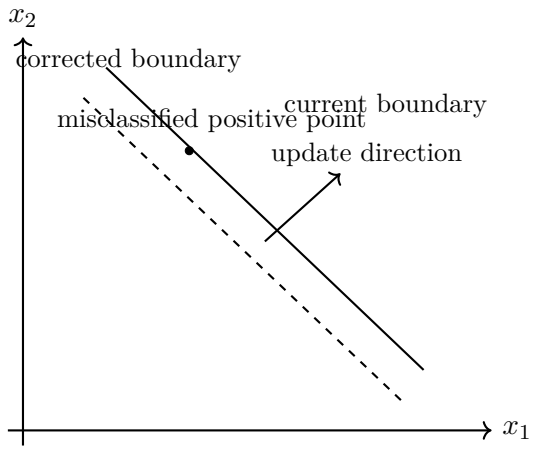


Figure 2.7: A perceptron update after a misclassified positive example. The update moves the separator so that the mistaken example becomes more likely to lie on the correct side.

and assume $\|x_i\| \le R$ for all i. Then the perceptron algorithm, initialized at $w_0 = 0$, makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

mistakes before it finds a separating hyperplane.

Proof. Suppose the algorithm makes M mistakes. Let w_t denote the weight vector after the tth mistake, with $w_0 = 0$.

At a mistake on example (x_i, y_i), the update is

$$w_{t+1} = w_t + y_i x_i.$$

Taking inner product with u gives

$$u^\top w_{t+1} = u^\top w_t + y_i u^\top x_i \ge u^\top w_t + \gamma,$$

because $y_i u^\top x_i \ge \gamma$ by assumption. Iterating this inequality over M mistakes yields

$$u^\top w_M \ge M\gamma.$$

Now consider the squared norm. Since the update occurs only on a mistake, we have

$$y_i(w_t^\top x_i + b_t) \le 0.$$

Ignoring the bias for the moment and focusing on the weight vector, this implies in particular

$$y_i w_t^\top x_i \le 0.$$

Hence

$$\|w_{t+1}\|^2 = \|w_t + y_i x_i\|^2 = \|w_t\|^2 + 2y_i w_t^\top x_i + \|x_i\|^2 \le \|w_t\|^2 + R^2.$$

Iterating over M mistakes gives

$$\|w_M\|^2 \le MR^2.$$

By the Cauchy–Schwarz inequality,

$$M\gamma \le u^\top w_M \le \|u\|\,\|w_M\| = \|w_M\| \le \sqrt{M}\,R,$$

because $\|u\| = 1$. Therefore

$$M\gamma \le \sqrt{M}\,R.$$

If $M > 0$, dividing by $\sqrt{M}\gamma$ gives

$$\sqrt{M} \le \frac{R}{\gamma},$$

so

$$M \le \left(\frac{R}{\gamma}\right)^2.$$

This proves the claim. □

The theorem is important for two reasons. First, it shows that learning can be proved to succeed under a geometric condition, namely linear separability with margin. Second, it clarifies the limitation of the perceptron: the guarantee depends on a setting in which a linear separator already exists.

Why nonlinear activations are essential

The perceptron is already a composition of an affine map and a thresholding operation. But if we stack affine maps without nonlinearities, nothing fundamentally new happens.

Proposition 2.16 (Composition of affine maps is affine)**.** *Let*

$$A_1(x) = W_1 x + b_1, \qquad A_2(z) = W_2 z + b_2$$

be affine maps. Then the composition $A_2 \circ A_1$ is also affine:

$$(A_2 \circ A_1)(x) = W_2 W_1 x + (W_2 b_1 + b_2).$$

Consequently, a multilayer network with only affine layers and no nonlinear activations is equivalent to a single affine map.

Proof. By direct substitution,

$$(A_2 \circ A_1)(x) = A_2(W_1 x + b_1) = W_2(W_1 x + b_1) + b_2 = W_2 W_1 x + W_2 b_1 + b_2,$$

which is again affine. Repeating the same argument across more layers shows that any finite composition of affine maps is affine. □

This proposition explains why nonlinearity is essential. Without it, depth collapses algebraically, and a deep stack of affine maps has no more expressive power than a single affine transformation.

Common activation functions

A neural network inserts nonlinear activation functions between affine layers. If $z = Wx + b$, a common layer has the form

$$h(x) = \sigma(z) = \sigma(Wx + b),$$

where σ is applied componentwise.

Some standard choices are:

Threshold activation. The perceptron uses

$$\sigma(t) = \operatorname{sign}(t).$$

This is conceptually simple but not differentiable in a way convenient for gradient-based training.

Sigmoid.

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

This maps real numbers into $(0, 1)$ and is historically important.

Hyperbolic tangent.

$$\sigma(t) = \tanh(t).$$

This maps into $(-1, 1)$ and is zero-centered.

ReLU.

$$\sigma(t) = \max\{0, t\}.$$

This is simple, piecewise linear, and widely used in modern deep learning.

Leaky ReLU and related variants. These modify ReLU to allow a small negative slope when $t < 0$.

The choice of activation influences both expressive power and trainability. Later sections will discuss this in more detail.

Multilayer feedforward networks

With nonlinear activations in place, one can build genuine multilayer models.

Definition 2.17 (Multilayer feedforward network)**.** A *multilayer feedforward network* is a function obtained by composing affine maps and nonlinear activations in successive layers:

$$h_1(x) = \sigma_1(W_1 x + b_1),$$

$$h_2(x) = \sigma_2(W_2 h_1(x) + b_2),$$

and so on, until the final output

$$f_\theta(x) = h_L(x).$$

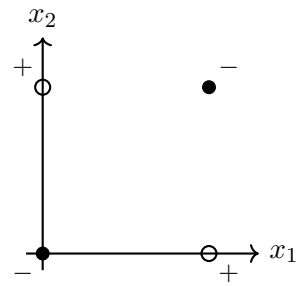
The word *feedforward* means that information flows from input to output without cycles. Each layer takes the previous layer's representation and transforms it into a new one.

Hidden layers as intermediate representations

The intermediate variables

$$z_1 = h_1(x), \qquad z_2 = h_2(x), \qquad \dots$$

are called *hidden representations*. They are "hidden" not in a mysterious sense, but simply because they are internal to the model rather than directly observed in the data.



XOR is not linearly separable

Figure 2.8: The XOR pattern cannot be separated by a single hyperplane, so it cannot be represented by a single-layer perceptron. This motivates multilayer networks and nonlinear hidden representations.

This is where the link to Chapter 1 becomes explicit. A neural network is not just an output predictor. It is a machine for constructing a sequence of learned representations:

$$x \mapsto z_1 \mapsto z_2 \mapsto \cdots \mapsto \hat{y}.$$

One can therefore interpret hidden layers as learned feature maps,

$$\phi_{\theta_1}(x) = z_k,$$

followed by a predictor acting on those features.

A failure case: XOR

The limitations of single-layer perceptrons are visible in the classical XOR example. Consider the four points

$$(0,0),\ (0,1),\ (1,0),\ (1,1),$$

with labels

$$0,\ 1,\ 1,\ 0$$

respectively. No single hyperplane can separate the positive and negative points correctly.

The XOR example matters pedagogically because it shows that the limitation of the perceptron is not only quantitative but structural. Some simple logical patterns require hidden layers.

Composition as the central structural idea

We can now state the main lesson of the section.

A neural network is a compositional model:

$$f_\theta = h_L \circ h_{L-1} \circ \cdots \circ h_1.$$

Each layer transforms one representation into another. The perceptron is the simplest instance of this idea. Multilayer networks generalize it by allowing repeated learned transformations, and nonlinear activations ensure that these transformations do not collapse into a single affine map.

So the real content of neural networks is not just "many parameters." It is the structured composition of simple maps into increasingly expressive representation-building systems.

Common confusion. The perceptron is not just a historical curiosity. It is the first mathematically clean example of a learned classifier, and its convergence theorem is an important anchor result. At the same time, the success of modern deep learning does not come from stacking linear predictors alone. Without nonlinear activations, depth collapses to a single affine map, so the key issue is learned nonlinear composition, not mere repetition of matrix multiplication.

What to remember. A perceptron is a halfspace classifier of the form $\operatorname{sign}(w^\top x + b)$, and the perceptron algorithm updates the weights only on mistakes. If the data are linearly separable with margin γ and bounded norm R, the perceptron makes at most $(R/\gamma)^2$ mistakes. This gives Chapter 2 its first theorem-level anchor. The theorem also clarifies the limits of single-layer linear threshold models. To move beyond those limits, neural networks introduce nonlinear activations and hidden layers, turning learning into the composition of multiple trainable representation maps.

Exercises

Exercise 2.18. Run the perceptron algorithm by hand on the following dataset in $\mathbb{R}^2$:

$$x_1 = (1,1),\ y_1 = 1, \qquad x_2 = (-1,-1),\ y_2 = -1.$$

Start from $w_0 = 0$, $b_0 = 0$, process the examples cyclically, and compute the first few updates until the dataset is classified correctly.

Exercise 2.19. Show that the bias term can be absorbed into the weight vector by augmenting the input with a constant 1. More precisely, prove that

$$w^\top x + b = \tilde{w}^\top \tilde{x}$$

for suitable augmented vectors $\tilde{w}$ and $\tilde{x}$.

Exercise 2.20. Explain why the XOR labeling of the four points

$$(0,0),\ (0,1),\ (1,0),\ (1,1)$$

is not linearly separable.

Exercise 2.21. Prove directly that the composition of three affine maps is affine.

Exercise 2.22. Why does the proposition on affine composition show that nonlinear activations are essential in deep learning?

Exercise 2.23. Consider a perceptron $f(x) = \operatorname{sign}(w^\top x + b)$. What geometric object is defined by the equation

$$w^\top x + b = 0?$$

How does the sign of $w^\top x + b$ determine the prediction?

Notes and references

This section introduces the perceptron as the simplest neural-network-style classifier and uses it to establish the chapter's first major mathematical result: the perceptron convergence theorem. The perceptron is valuable not only historically but conceptually, because it shows how geometry, optimization, and classification already interact in a simple model. The section then explains why single-layer linear threshold models are not enough and why nonlinear activations and hidden layers are necessary. This opens the path to viewing neural networks as compositional systems for learned representation building rather than as mere collections of parameters.

2.3 Expressivity, Hierarchy, and the Role of Depth

The previous section introduced neural networks as compositional function models. We now ask the next natural question:

What kinds of functions can such models represent, and why might depth matter?

This is the section in which neural networks first receive a mathematical defense beyond historical success. The central issue is *expressive power*: how the architecture of a model constrains the class of functions it can approximate. Width matters, but so does depth. A shallow model can already be remarkably expressive; a deep model can express some structures more naturally by reusing intermediate computations hierarchically.

Expressive power and approximation classes

To speak precisely, we need a language for what it means for a model family to be expressive.

Definition 2.24 (Expressive power)**.** The *expressive power* of a model class is its ability to represent or approximate functions of interest on a given domain. A class is more expressive if it can realize a broader or more structurally rich set of input-output mappings.

Definition 2.25 (Approximation class)**.** Let $K \subset \mathbb{R}^d$ be compact. A family of functions $\mathcal{H}$ is called an *approximation class* for a target class $\mathcal{F}$ on K if, for every $f \in \mathcal{F}$ and every $\varepsilon > 0$, there exists $h \in \mathcal{H}$ such that

$$\sup_{x \in K} |f(x) - h(x)| < \varepsilon.$$

In that case we say that $\mathcal{H}$ can approximate $\mathcal{F}$ uniformly on K.

This is an approximation-based notion of expressivity. It does not require exact representation. It asks whether the model class can get arbitrarily close to the desired target functions.

A carefully stated universal approximation theorem

One of the best-known theorems in neural-network theory says that even shallow networks can be very expressive.

Theorem 2.26 (Universal approximation, one-hidden-layer form)**.** *Let $K \subset \mathbb{R}^d$ be compact, and let*

$$f : K \to \mathbb{R}$$

be continuous. Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous activation function that is not a polynomial. Then for every $\varepsilon > 0$, there exist an integer m, coefficients $a_1, \dots, a_m \in \mathbb{R}$, vectors $w_1, \dots, w_m \in \mathbb{R}^d$, biases $b_1, \dots, b_m \in \mathbb{R}$, and a constant $c \in \mathbb{R}$ such that

$$h(x) = c + \sum_{j=1}^{m} a_j \, \sigma(w_j^\top x + b_j)$$

satisfies

$$\sup_{x \in K} |f(x) - h(x)| < \varepsilon.$$

In particular, a sufficiently wide network with one hidden layer can approximate any continuous function on a compact domain arbitrarily well.

Proof idea. A full proof is beyond the scope of this chapter, but the structure is worth understanding. The theorem is an approximation statement on compact sets. The key point is that nonpolynomial activations generate a rich enough family of ridge functions

$$x \mapsto \sigma(w^\top x + b)$$

that finite linear combinations of them are dense in $C(K)$, the space of continuous functions on K with the uniform norm. Different proofs establish this density through different functional-analytic arguments. The conclusion is that one-hidden-layer networks are universal approximators in the approximation sense stated above. □

This theorem is mathematically important because it shows that shallow neural networks are not inherently too weak to approximate complicated continuous functions. But it does *not* say that depth is irrelevant.

What universal approximation does not say. The theorem is often quoted too loosely. It does *not* say:

- that every function can be represented exactly by a small shallow network;
- that approximation is efficient in width, parameters, or sample size;
- that a network which can approximate a function can also be trained easily to find that approximation;
- that depth provides no advantage.

A model family may be universal in principle and still be impractical in size or difficult to optimize in practice. Universal approximation is an existence theorem, not a trainability theorem.

Depth as hierarchical reuse

If shallow networks are already universal approximators, why does depth matter?

The answer is that expressivity is not only about *whether* approximation is possible. It is also about *how naturally* and *how efficiently* a function can be represented. Many functions have internal compositional structure. When that structure is exploited, a deep architecture can reuse intermediate computations instead of recomputing them from scratch.

For example, consider a function of the form

$$f(x) = q\big(r_1(x), r_2(x), r_3(x)\big),$$

where each r_j itself has internal structure. A deep network can assign different layers to different stages of this computation. A shallow network can still approximate the same overall mapping, but may need to encode all interactions more directly in one large layer.

This is the main mathematical intuition behind the role of depth: depth is a mechanism for hierarchical reuse.

ReLU networks and piecewise linear structure

A particularly important modern activation is the rectified linear unit,

$$\mathrm{ReLU}(t) = \max\{0, t\}.$$

Because ReLU is piecewise linear, networks built from it have a distinctive geometric structure.

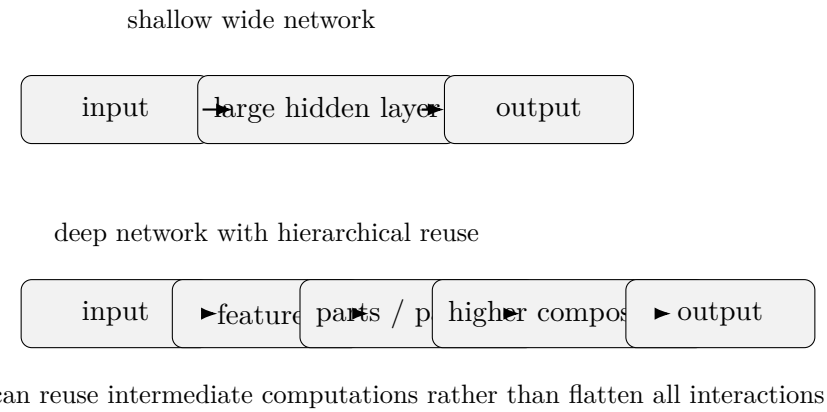


Figure 2.9: A schematic contrast. A shallow model may approximate a complex mapping in one large hidden layer, while a deep model can exploit hierarchical structure by building and reusing intermediate representations across layers.

Proposition 2.27 (ReLU networks represent piecewise linear functions)**.** *Any feedforward network with ReLU activations and a linear output layer represents a continuous piecewise linear function of its input. More precisely, the input space can be partitioned into finitely many polyhedral regions such that on each region the network acts as an affine map.*

Proof. For a single ReLU unit,

$$x \mapsto \max\{0, w^\top x + b\},$$

the input space is divided by the hyperplane

$$w^\top x + b = 0$$

into two regions. On one region the unit outputs 0; on the other it outputs the affine function $w^\top x + b$. So a single ReLU unit is piecewise affine.

Now consider one layer of ReLU units. The activation pattern of the layer is determined by which pre-activations are positive and which are nonpositive. Each fixed activation pattern defines a polyhedral region in input space, and on that region the layer acts affinely.

Proceeding layer by layer, each new layer refines the partition into finitely many polyhedral regions. On each such region, every ReLU unit has a fixed linear branch, so the entire network reduces to a composition of affine maps, hence is affine on that region. Therefore the network as a whole is continuous and piecewise linear. □

This proposition gives a very useful geometric picture of ReLU networks. They build complicated decision surfaces by partitioning the input space into many linear pieces and arranging those pieces hierarchically across layers.

A toy example of hierarchical composition

Consider the function

$$f(x_1, x_2, x_3, x_4) = (x_1 x_2) + (x_3 x_4).$$

A shallow approximation strategy might attempt to learn the entire interaction pattern at once. A hierarchical strategy would first form two intermediate quantities,

$$z_1 = x_1 x_2, \qquad z_2 = x_3 x_4,$$

and then combine them as

$$f = z_1 + z_2.$$

The point is not that this example requires deep networks in a strict theorem-level sense. The point is that it exhibits *reuse of subcomputations*. If many related outputs depend on the same intermediate structures, depth provides a natural way to represent that fact.

This perspective becomes even more compelling in vision, language, and structured reasoning: local patterns combine into motifs, motifs combine into larger structures, and higher layers operate on abstractions produced by lower layers.

Approximation power is not trainability

At this point it is crucial to separate two ideas:

- **approximation power:** whether a function class can represent or approximate targets of interest;
- **trainability:** whether an optimization algorithm can efficiently find good parameters from finite data.

A model may be expressive enough in principle and still be hard to train. Conversely, a model may be easy to optimize but too limited to capture the structure of the task. The theory of expressivity answers one question; optimization and generalization answer different ones.

Common confusion. Universal approximation does not prove that "depth does not matter." It shows only that sufficient width can provide broad approximation power. It says nothing about efficiency of representation, sample requirements, optimization difficulty, or the advantages of hierarchical reuse. Likewise, a class being expressive enough in principle does not mean gradient-based training will easily find a good solution.

What to remember. Expressive power concerns what functions a model class can represent or approximate. Universal approximation shows that even a one-hidden-layer network can approximate any continuous function on a compact set arbitrarily well, but this is an existence result, not a statement about efficiency or trainability. Depth matters because many functions have hierarchical structure, and deep networks can reuse intermediate computations more naturally than shallow ones. ReLU networks provide a useful geometric example: they represent continuous piecewise linear functions by partitioning input space into affine regions.

Exercises

Exercise 2.28. In your own words, explain the difference between saying that a model class is expressive and saying that it is easy to train.

Exercise 2.29. State the universal approximation theorem informally. Then list two things that the theorem does *not* imply.

Exercise 2.30. Consider the toy compositional function

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)(x_3 + x_4).$$

Identify at least one natural intermediate representation that a hierarchical model could reuse.

Exercise 2.31. Why does the proposition on ReLU networks imply that their decision boundaries can be geometrically complex even though each local piece is affine?

Exercise 2.32. Compare the two viewpoints:

1. a shallow network approximates a target function in one large hidden layer;
2. a deep network approximates the same target by building intermediate representations.

What is the possible advantage of the second viewpoint?

Exercise 2.33. Give one reason why approximation power alone is insufficient as a theory of practical deep learning.

Notes and references

This section provides the chapter's first mathematical defense of depth. Universal approximation shows that shallow networks already possess broad approximation power, so the importance of deep learning cannot be explained simply by saying that shallow networks are incapable of representing complex functions. The more refined issue is efficiency of representation and the exploitation of hierarchical structure. ReLU networks make this especially tangible because their functions are piecewise linear, built from many affine regions arranged across layers. Later sections will connect these expressive considerations to optimization, trainability, and learned internal representations.

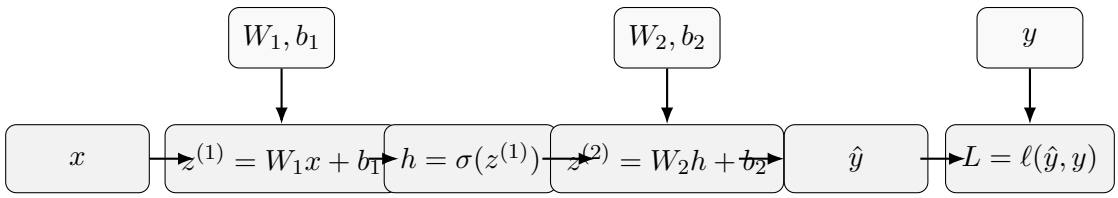


Figure 2.10: A computation graph for a two-layer network. The forward pass computes intermediate variables from left to right; backpropagation traverses the same graph in reverse to compute derivatives of the loss with respect to all parameters.

2.4 Computation Graphs, Chain Rule, and Backpropagation

Neural networks are built by composing many simple operations. Once such a composition is written down, training requires one central computational task:

> Given a loss function, how do we compute its derivatives with respect to all parameters efficiently?

The answer is *backpropagation*, which is best understood not as a mysterious separate algorithm, but as a systematic application of the chain rule to a computation graph. This section gives the formal language for that viewpoint and works through a full two-layer example in detail.

Computation graphs

A neural network is naturally described as a directed acyclic graph of intermediate quantities. Each node stores a value produced from earlier nodes, and the final node stores the loss.

Definition 2.34 (Computation graph)**.** A *computation graph* is a directed acyclic graph whose nodes represent intermediate variables and whose edges indicate functional dependence. Each non-input node is obtained by applying an elementary operation to its parent nodes. A scalar loss L is designated as an output node, and differentiation is performed with respect to this final scalar quantity.

In neural-network training, the forward pass computes all node values from input to loss, while the backward pass computes derivatives of the loss with respect to intermediate quantities and parameters.

The chain rule as the engine of gradient computation

The basic principle is the chain rule. In one variable, if

$$u = g(v), \qquad v = h(t),$$

then

$$\frac{du}{dt} = \frac{du}{dv}\frac{dv}{dt}.$$

In multivariable settings, the same idea propagates derivatives through compositions of vector-valued maps. If

$$L = L(z), \qquad z = z(\theta),$$

then

$$\nabla_\theta L = \left(\frac{\partial z}{\partial \theta}\right)^\top \nabla_z L.$$

Backpropagation is the organized repeated use of this principle across an entire computation graph.

Forward values and backward adjoints

For each node v in the computation graph, one may associate an *adjoint*

$$\bar{v} := \frac{\partial L}{\partial v},$$

that is, the derivative of the final loss with respect to that node.

The forward pass computes the values

$$v_1, v_2, \ldots, v_N.$$

The backward pass computes the corresponding adjoints

$$\bar{v}_1, \bar{v}_2, \ldots, \bar{v}_N$$

starting from

$$\bar{L} = \frac{\partial L}{\partial L} = 1$$

and moving backward through the graph.

This gives the conceptual structure of reverse-mode differentiation:

- compute all intermediate values once;
- start from the loss;
- propagate sensitivities backward using local derivatives.

Backpropagation as reverse-mode differentiation

Suppose a node u feeds into several later nodes. Then changing u affects the loss through all downstream paths. The total derivative therefore sums the contributions from all children:

$$\frac{\partial L}{\partial u} = \sum_{v:\, u \to v} \frac{\partial L}{\partial v} \frac{\partial v}{\partial u}.$$

This is the local recursion underlying backpropagation.

Proposition 2.35 (Reverse-mode differentiation is efficient)**.** *For a scalar loss defined on a computation graph, reverse-mode differentiation computes the gradient of the loss with respect to all parameters at a computational cost comparable to one forward pass, up to constant factors.*

Proof idea. A naive approach would differentiate the loss separately with respect to each parameter, repeatedly recomputing many shared intermediate derivatives. Reverse-mode differentiation avoids this redundancy.

During the forward pass, each node value is computed once and stored. During the backward pass, each edge of the graph is traversed in reverse order, and each local derivative contributes once to the adjoint of its parent node. Because shared downstream computations are reused rather than

recomputed separately for each parameter, the total cost scales with the size of the graph rather than with the number of parameters times the graph size.

Under the usual assumption that each elementary operation and its local derivative can be evaluated at comparable cost, the total backward cost is bounded by a constant multiple of the forward cost. Thus all parameter gradients are obtained simultaneously at cost comparable to one forward evaluation, up to constant factors. □

This proposition explains why deep learning is computationally feasible at all. A network may contain millions or billions of parameters, yet backpropagation does not compute each derivative from scratch.

A formal worked example: a two-layer network

We now work through a full example. Let

$$x \in \mathbb{R}^d, \qquad z^{(1)} = W_1 x + b_1 \in \mathbb{R}^m, \qquad h = \sigma(z^{(1)}) \in \mathbb{R}^m,$$
$$z^{(2)} = W_2 h + b_2 \in \mathbb{R}^k, \qquad \hat{y} = z^{(2)}.$$

For simplicity, take squared loss

$$L = \frac{1}{2}\|\hat{y} - y\|^2.$$

Example 2.36 (Backpropagation through a two-layer network)**.** For the network above, the parameter gradients are:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y,$$
$$\frac{\partial L}{\partial W_2} = (\hat{y} - y)\, h^\top, \qquad \frac{\partial L}{\partial b_2} = \hat{y} - y,$$
$$\delta^{(1)} := \frac{\partial L}{\partial z^{(1)}} = \left(W_2^\top (\hat{y} - y)\right) \odot \sigma'(z^{(1)}),$$
$$\frac{\partial L}{\partial W_1} = \delta^{(1)} x^\top, \qquad \frac{\partial L}{\partial b_1} = \delta^{(1)}.$$

Proof. Since

$$L = \frac{1}{2}\|\hat{y} - y\|^2 = \frac{1}{2}\sum_{j=1}^{k}(\hat{y}_j - y_j)^2,$$

we have

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

Because

$$\hat{y} = z^{(2)} = W_2 h + b_2,$$

the derivative with respect to the affine parameters of the second layer is

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial W_2} = (\hat{y} - y)h^\top,$$

and similarly

$$\frac{\partial L}{\partial b_2} = \hat{y} - y.$$

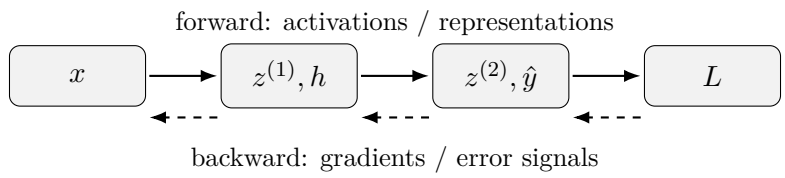


Figure 2.11: Forward and backward signal flow. The network computes activations from left to right, then propagates gradient information from the loss back toward earlier layers.

Next, we propagate the derivative backward to the hidden layer:

$$\frac{\partial L}{\partial h} = W_2^\top(\hat{y} - y).$$

Since

$$h = \sigma(z^{(1)}),$$

applied componentwise, the chain rule gives

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}} = \frac{\partial L}{\partial h} \odot \sigma'(z^{(1)}) = \Big(W_2^\top(\hat{y} - y)\Big) \odot \sigma'(z^{(1)}).$$

Finally, because

$$z^{(1)} = W_1 x + b_1,$$

the gradients of the first-layer parameters are

$$\frac{\partial L}{\partial W_1} = \delta^{(1)} x^\top, \qquad \frac{\partial L}{\partial b_1} = \delta^{(1)}.$$

This is exactly the backpropagation formula for the two-layer network. □

The pattern here is the one that will recur throughout the book:

1. compute forward activations;
2. compute loss;
3. propagate error signals backward;
4. use those signals to form parameter gradients.

Forward and backward signal flow

The forward pass carries *representations* from input to output. The backward pass carries *sensitivity information* from the loss back to earlier layers.

This dual flow explains a deep conceptual point: training requires both *evaluation* and *credit assignment*. The forward pass evaluates the model on an input; the backward pass assigns responsibility for the loss to earlier computations and parameters.

Layerwise form of backpropagation

For a multilayer feedforward network with layer equations

$$z^{(\ell)} = W_\ell h^{(\ell-1)} + b_\ell, \qquad h^{(\ell)} = \sigma_\ell(z^{(\ell)}),$$

the backpropagation recursion has the generic form

$$\delta^{(\ell)} = \left(W_{\ell+1}^{\top}\delta^{(\ell+1)}\right) \odot \sigma'_{\ell}(z^{(\ell)}),$$

together with parameter gradients

$$\frac{\partial L}{\partial W_\ell} = \delta^{(\ell)}\big(h^{(\ell-1)}\big)^{\top}, \qquad \frac{\partial L}{\partial b_\ell} = \delta^{(\ell)}.$$

This formula compresses much of the practical content of backpropagation. Each layer receives an incoming error signal from above, transforms it through the transpose of the weight matrix and the derivative of the activation, and then uses the result to compute its own parameter gradients.

Backpropagation versus learning

Backpropagation computes gradients. It does not itself specify how parameters are updated after those gradients are known.

Once gradients have been computed, one still needs an optimization rule, such as gradient descent or stochastic gradient descent:

$$W_\ell \leftarrow W_\ell - \eta\frac{\partial L}{\partial W_\ell}, \qquad b_\ell \leftarrow b_\ell - \eta\frac{\partial L}{\partial b_\ell}.$$

So backpropagation belongs to the differentiation side of learning, not the optimization side.

Common confusion. Backpropagation is not the learning rule. It is the gradient-computation procedure. The learning rule is the parameter-update method applied after gradients have been computed, such as gradient descent, SGD, or Adam. Confusing these two levels hides an important conceptual distinction: one procedure computes sensitivities, the other decides how to move in parameter space.

What to remember. A computation graph is a directed acyclic graph of intermediate variables ending in a scalar loss. Backpropagation is reverse-mode differentiation on this graph: it applies the chain rule efficiently by propagating adjoints backward from the loss. Its key computational advantage is that it computes all parameter gradients at cost comparable to one forward pass up to constant factors. In practice, training a neural network means combining forward evaluation, backward gradient computation, and then a separate optimization update.

Exercises

Exercise 2.37. Consider the scalar computation

$$u = ax + b, \qquad v = \sigma(u), \qquad L = \frac{1}{2}(v-y)^2.$$

Compute

$$\frac{\partial L}{\partial a}, \qquad \frac{\partial L}{\partial b}$$

using the chain rule.

Exercise 2.38. For the two-layer network

$$z^{(1)} = W_1 x + b_1, \qquad h = \sigma(z^{(1)}), \qquad \hat{y} = W_2 h + b_2,$$

derive

$$\frac{\partial L}{\partial W_2}$$

under squared loss

$$L = \frac{1}{2}\|\hat{y} - y\|^2.$$

Exercise 2.39. Using the same two-layer network, derive the hidden-layer error signal

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}}.$$

Explain where the Hadamard product with $\sigma'(z^{(1)})$ comes from.

Exercise 2.40. Why is reverse-mode differentiation especially well suited to neural-network training, where the output of interest is a scalar loss but the number of parameters is very large?

Exercise 2.41. Explain in words the difference between the forward pass and the backward pass in a neural network.

Exercise 2.42. A student says, "Backpropagation trains the network." What is incomplete or inaccurate about this statement?

Notes and references

This section formalizes one of the central computational ideas of deep learning: gradients in layered models are obtained efficiently by reverse-mode differentiation on a computation graph. The conceptual ingredients are simple—composition, the chain rule, stored intermediate values, and recursive sensitivity propagation—but together they make the training of large neural networks computationally practical. Later sections will build on this by studying when gradient-based training works well, when it becomes unstable, and how architecture, initialization, normalization, and residual structure influence signal propagation through depth.

2.5 Why Deep Networks Are Hard to Train

The previous sections established two major facts about neural networks.

- They are highly expressive function classes.
- Their gradients can be computed efficiently by backpropagation.

These facts are essential, but they are not enough to guarantee successful learning. A model may be expressive enough to represent a useful solution, and one may even know how to compute gradients exactly, yet optimization may still fail in practice.

Historically, this was one of the central obstacles in deep learning. Early neural networks were often difficult to optimize reliably, especially as depth increased. The problem was not merely lack of computational power. It was also the mathematical instability of training deep compositional models.

This section explains why. Its purpose is to create a genuine sense of difficulty. Depth is not free. The same layered composition that gives deep networks their expressive power also creates fragile optimization dynamics: signals may shrink, signals may explode, curvature may become badly imbalanced, and naïve gradient-based training may fail even when the desired function lies well inside the model class.

Thus one must distinguish carefully between three statements:

- the model class is expressive enough,
- gradients can be computed,
- training will succeed.

These are not equivalent. This section is about the gap between the second and third.

Why depth creates difficulty

A deep network is a repeated composition of transformations:

$$h^{(0)} = x, \qquad h^{(1)} = \sigma(W^{(1)}h^{(0)} + b^{(1)}), \qquad \dots, \qquad h^{(L)} = f_\theta(x).$$

During learning, information must flow in two directions:

- **forward:** activations and representations must propagate from input to output;
- **backward:** gradient signals must propagate from loss to early layers.

In a shallow model, these paths are short. In a deep model, they are long and repeatedly transformed. That repeated transformation can either preserve signal, attenuate it, or amplify it. As depth grows, even mild distortion at each layer can accumulate into severe instability.

This is the central intuition of the section:

> Deeper composition increases expressive structure, but it also increases the difficulty of maintaining stable information flow.

Nonconvexity

One source of difficulty is the geometry of the optimization problem itself. For a deep network, the empirical risk

$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

is typically a highly nonconvex function of the parameters θ.

This is very different from many classical models such as linear regression or logistic regression, where the objective is often convex. In convex optimization, every local minimum is global, and the geometry is comparatively well behaved. In deep learning, no such simple picture is available.

The landscape of a deep network may contain:

- local minima,
- saddle points,
- flat regions or plateaus,
- narrow valleys,
- directions with very different curvature.

This does not mean that optimization is impossible. Indeed, deep networks are trained successfully in practice. But it does mean that training cannot rely on the clean geometric guarantees associated with convex objectives. The parameter space is large, coupled, and geometrically intricate.

Common confusion. Nonconvex does *not* automatically mean impossible. A nonconvex objective can still be trainable if gradients remain informative, scales are well controlled, and the optimization geometry is manageable. Conversely, even a smooth objective can be painful to optimize if gradients vanish, explode, or point through badly conditioned directions. Nonconvexity is one source of difficulty, not the whole story.

Gradient propagation through depth

The main computational difficulty of deep learning appears most clearly in the backward pass. The chain rule says that a derivative through a composition is a product of local derivatives. In a deep network, that means the gradient reaching an early layer is obtained by multiplying together many Jacobians.

Proposition 2.43 (Gradients through depth are products of Jacobians)**.** *Let*

$$h^{(0)} = x, \qquad h^{(\ell)} = F_\ell(h^{(\ell-1)}) \quad \text{for } \ell = 1, \dots, L,$$

and let the loss be

$$L(x, y; \theta) = \mathcal{L}(h^{(L)}, y).$$

Then for every $\ell \in \{0, \dots, L-1\}$,

$$\nabla_{h^{(\ell)}} L = J_{F_{\ell+1}}(h^{(\ell)})^\top J_{F_{\ell+2}}(h^{(\ell+1)})^\top \cdots J_{F_L}(h^{(L-1)})^\top \nabla_{h^{(L)}} L,$$

where J_{F_k} denotes the Jacobian of F_k. In particular, for a standard layer

$$F_k(h) = \sigma(W^{(k)} h + b^{(k)}),$$

we have

$$J_{F_k}(h) = D^{(k)}W^{(k)}, \qquad D^{(k)} = \operatorname{diag}\big(\sigma'(z^{(k)})\big),$$

so backpropagation multiplies by both weight matrices and activation-derivative factors.

Proof. By the multivariable chain rule,

$$\nabla_{h^{(L-1)}} L = J_{F_L}(h^{(L-1)})^\top \nabla_{h^{(L)}} L.$$

Applying the same identity one step earlier gives

$$\nabla_{h^{(L-2)}} L = J_{F_{L-1}}(h^{(L-2)})^\top \nabla_{h^{(L-1)}} L = J_{F_{L-1}}(h^{(L-2)})^\top J_{F_L}(h^{(L-1)})^\top \nabla_{h^{(L)}} L.$$

Repeating this recursion yields the stated product formula. For the layer $F_k(h) = \sigma(W^{(k)}h + b^{(k)})$, the affine map contributes $W^{(k)}$ and the elementwise nonlinearity contributes the diagonal derivative matrix $D^{(k)}$, so the local Jacobian is $D^{(k)}W^{(k)}$. □

This proposition is the formal anchor for the rest of the section. It explains why depth creates both opportunity and instability: the gradient is not controlled by one local derivative, but by a long product of them.

A scalar toy derivation

The same mechanism already appears in one dimension. Consider the scalar chain

$$h^{(0)} = x, \qquad h^{(\ell)} = \phi\big(a\, h^{(\ell-1)}\big), \qquad \ell = 1, \dots, L,$$

with scalar parameter a, activation ϕ, and scalar loss

$$L = \mathcal{L}(h^{(L)}).$$

By repeated application of the chain rule,

$$\frac{\partial L}{\partial h^{(0)}} = \frac{\partial L}{\partial h^{(L)}} \prod_{\ell=1}^{L} \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} = \frac{\partial L}{\partial h^{(L)}} \prod_{\ell=1}^{L} \Big(a\, \phi'\big(a h^{(\ell-1)}\big)\Big).$$

If the local factor is typically of size ρ, meaning roughly

$$\big|a\, \phi'(a h^{(\ell-1)})\big| \approx \rho,$$

then

$$\left|\frac{\partial L}{\partial h^{(0)}}\right| \approx \left|\frac{\partial L}{\partial h^{(L)}}\right| \rho^L.$$

So:

- if $\rho < 1$, the gradient decays exponentially with depth;
- if $\rho > 1$, the gradient grows exponentially with depth;
- if $\rho \approx 1$, gradient scale is at least *capable* of remaining stable.

This toy derivation is deliberately simple, but the lesson is exact in spirit. Deep learning is hard because repeated multiplication magnifies even mild layerwise imbalance.

Vanishing gradients

The *vanishing gradient problem* occurs when backward-propagated gradient signals become extremely small in early layers.

At an intuitive level, this means that the loss provides only a weak learning signal to the beginning of the network. The later layers may still update, but the earlier layers — which are supposed to learn foundational representations — change very slowly.

In terms of the Jacobian product, vanishing gradients arise when the typical singular values or effective scalar factors along the backward path are smaller than one, so that repeated multiplication contracts:

$$\|\nabla_{h^{(\ell)}} L\| \ll \|\nabla_{h^{(L)}} L\| \qquad \text{for early } \ell.$$

This is especially problematic because early layers are often the ones that should learn the most reusable and task-important features. If they receive almost no gradient, the whole representational hierarchy becomes difficult to organize effectively.

A useful way to think about vanishing gradients is this: backpropagation communicates error information backward through the network. If that communication decays exponentially with depth, then distant layers are almost cut off from the learning signal.

Exploding gradients

The opposite problem is the *exploding gradient problem.* Here the repeated products in the backward pass amplify rather than suppress the signal, so that

$$\|\nabla_{h^{(\ell)}} L\| \gg \|\nabla_{h^{(L)}} L\|$$

for some early layers.

When this happens, updates become numerically unstable. Parameter changes may become excessively large, optimization may oscillate wildly, and the loss may jump unpredictably rather than decrease smoothly.

Exploding gradients are particularly severe in deep or recurrent models, where long chains of multiplication are unavoidable. Even when not catastrophic, they can make training extremely sensitive to learning rates and initialization scales.

The vanishing and exploding cases are opposite in direction, but similar in spirit: both are failures of stable signal propagation through depth.

Saturation of sigmoid and tanh activations

A particularly important source of vanishing gradients comes from *saturation* of activation functions. Consider activations such as the sigmoid

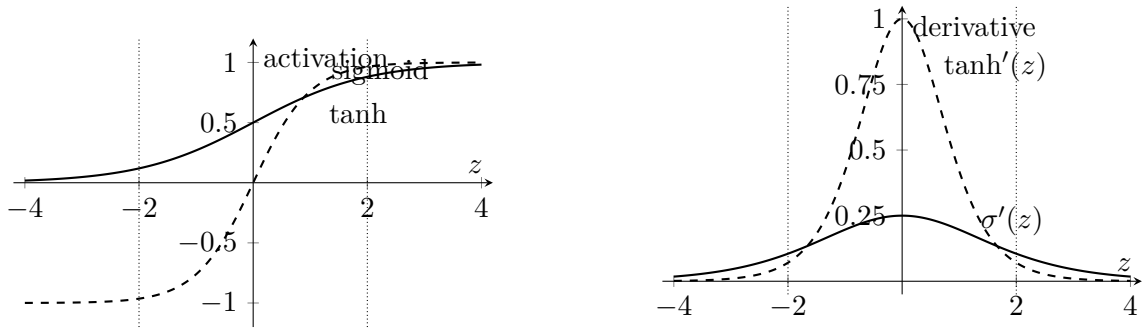
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

or the hyperbolic tangent

$$\tanh(z).$$

These functions flatten out in parts of their domain. In those regions, their derivatives are small:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \qquad \tanh'(z) = 1 - \tanh^2(z).$$



(a) Sigmoid and tanh flatten in their tails.

(b) Derivatives are small in the saturated regions.

Figure 2.12: Saturation harms gradient flow. When pre-activations spend much time in the tails of sigmoid or tanh, the local derivative is small, so the backward signal is repeatedly attenuated.

Since backpropagation multiplies by these derivatives, activations that spend much time in saturated regimes strongly attenuate gradient flow. Even if the weight matrices themselves are well behaved, repeated multiplication by near-zero activation derivatives can make gradients vanish rapidly.

This is why the geometry of the activation function matters. The issue is not merely that the network is nonlinear. The issue is how the nonlinearity behaves in the regimes actually visited during training.

Saturation also has a forward-pass interpretation. If many units are in flat regions of the activation, then their outputs become relatively insensitive to changes in input. The network then loses expressive responsiveness locally, while the backward signal simultaneously weakens.

Thus saturation harms both representation dynamics and optimization dynamics.

Ill-conditioned optimization

Even when gradients do not vanish or explode outright, optimization can still be very slow if the objective is badly conditioned.

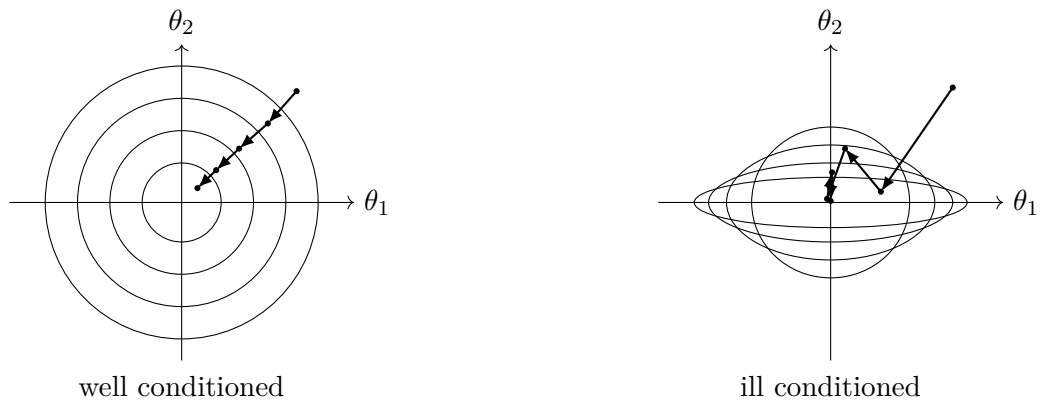
Conditioning concerns how differently the objective behaves in different parameter directions. If one direction has very high curvature while another has very low curvature, then a single learning rate is hard to choose well:

- too large a step may overshoot in steep directions,
- too small a step may barely move in flat directions.

This leads to zig-zagging, oscillation, or painfully slow progress. A familiar geometric picture is that of a narrow curved valley: gradient descent keeps bouncing across the steep walls while making only limited progress along the shallow direction.

Deep networks often suffer from such ill-conditioning because layer parameters are strongly coupled. Changing one layer changes the distribution of inputs seen by later layers, which in turn changes the local geometry of the loss. Thus curvature is not only anisotropic; it is also dynamically entangled across the network.

For students, the key lesson is this: optimization difficulty is not just about whether gradients exist. It is also about whether those gradients point in directions that can be used effectively by finite-step algorithms.



(a) Nearly isotropic contours permit direct progress.

(b) Elongated contours create zig-zagging and slow progress.

Figure 2.13: Optimization conditioning matters even when gradients exist. In poorly conditioned regions, one step size must simultaneously handle steep and flat directions, so descent can oscillate across the valley while making little progress along it.

Sensitivity to initialization and scale

Because gradients propagate through long chains of transformations, deep networks are highly sensitive to how parameters are initialized.

If initial weights are too small, then activations may collapse toward low-variation regimes and backward signals may become weak. If initial weights are too large, activations may blow up or saturate, and gradients may become unstable.

So initialization is not a minor implementation detail. It is part of the mathematical condition under which learning becomes feasible.

This sensitivity can be understood from both forward and backward viewpoints:

Forward scale. The variance or norm of activations should not grow or shrink uncontrollably across layers.

Backward scale. The magnitude of gradients should also remain within a reasonable range as they are propagated backward.

A good initialization attempts to balance both. This is why initialization schemes in deep learning are designed with layer width, activation type, and depth in mind.

The broader lesson is that the trainability of a deep model depends not only on its architecture as a function class, but also on the statistical scale at which that architecture is initialized.

Coupled co-adaptation of layers

A further source of difficulty is that all layers are being learned simultaneously. A layer is not optimizing against a fixed representation supplied by previous layers. Instead, the representations themselves are changing during training.

This creates a moving-target problem. When an early layer changes, it alters the effective input distribution seen by the next layer. That layer must then readapt. Its adaptation in turn affects the next layer, and so on.

Thus training a deep network is not like solving a sequence of isolated subproblems. It is a large coupled dynamical system in which many components co-adapt at once.

This is conceptually important because it explains why depth causes difficulty even when each individual layer, considered alone, would be easy to optimize. The challenge is not just local complexity. It is global interaction.

Why naïve training can fail

It is now possible to state the main lesson of the section more sharply.

A deep network may fail to train well even if:

- the target function lies in the hypothesis class,
- the loss is differentiable,
- backpropagation computes exact gradients.

Failure may still occur because:

- the objective is nonconvex,
- gradients decay through depth,
- gradients blow up through depth,
- activations saturate,
- curvature is badly conditioned,
- initialization is poorly scaled,
- all layers co-adapt in unstable ways.

This is an important act of intellectual discipline. One should not confuse representational adequacy with trainability. A model can be expressive enough in principle and still be difficult to optimize in practice.

Why this difficulty mattered historically

This section is not merely a list of technical obstacles. It explains why the rise of deep learning took time.

For many years, the main barrier was not the idea of multilayer representation itself. That idea was already present. The barrier was that deeper models were hard to train robustly and efficiently.

Only when the field developed practical ways to stabilize optimization — through better activations, better initialization, better architectures, better normalization, and better optimization procedures — did deep representation learning become consistently successful at scale.

Thus the modern toolbox should be understood as a collection of answers to genuine mathematical and computational problems, not as arbitrary tricks.

What to remember. Trainability problems in deep networks are not explained by one slogan. They arise from several interacting mechanisms: signal propagation through long Jacobian products, activation saturation, sensitivity to scale and initialization, ill-conditioning of the optimization geometry, and the coupled co-adaptation of layers. Nonconvexity matters, but practical trainability depends just as much on whether forward activations and backward gradients remain informative across depth.

Exercises

Exercise 2.44. Assume a scalar deep chain satisfies

$$\frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} = a \qquad \text{for every } \ell = 1, \dots, L,$$

with constant scalar a. Show that

$$\frac{\partial L}{\partial h^{(0)}} = a^L \frac{\partial L}{\partial h^{(L)}}.$$

Analyze what happens as depth grows in the three regimes $|a| < 1$, $|a| = 1$, and $|a| > 1$.

Exercise 2.45. Explain in words why saturation of sigmoid or tanh harms learning. Your answer should discuss both the forward-pass effect on responsiveness of activations and the backward-pass effect on gradient flow.

Exercise 2.46. Compare the statements "the optimization problem is nonconvex" and "the optimization problem is ill conditioned." Why are these not the same thing? Give an example of how a nonconvex problem could still be trainable in practice, and why even a smooth problem can be slow when curvature is badly imbalanced.

Notes and references

The purpose of this section is to separate expressivity from trainability. A deep network may be expressive enough to represent a useful solution and differentiable enough for exact gradients to be computed, yet still be hard to optimize because gradient signals decay or explode, activations saturate, curvature is badly conditioned, or layers co-adapt in unstable ways. The next section turns to the practical response: initialization, stochastic optimization, and signal-preserving design choices that make deep learning numerically viable.

2.6 Optimization in Practice: SGD, Initialization, and Signal Propagation

The previous section explained why deep networks are hard to train. Depth creates long chains of forward and backward transformations, and those chains can amplify instability. Even when the model class is expressive enough and gradients can be computed exactly, training may still fail if optimization is noisy in the wrong way, if scales drift across layers, or if the interaction between activations and initialization destroys signal flow.

We now turn to the first family of practical responses. These responses do not change the basic learning objective. Rather, they make gradient-based learning *numerically and statistically alive* in deep models.

The main ideas are:

- use stochastic optimization for scalability,
- use minibatches to balance noise and efficiency,
- control learning-rate dynamics over time,
- use momentum or adaptive scaling to improve optimization geometry,
- initialize parameters so that forward activations and backward gradients have reasonable scale,
- choose activations in ways that support trainability rather than destroy it.

These ideas are often introduced as practical techniques. But they are better understood as answers to a precise question:

> How can gradient-based learning remain stable and informative as signals propagate through a deep compositional model?

Minibatch stochastic gradient descent in deep networks

Let the training objective be

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i),$$

where f_θ is a deep network and θ collects all its parameters.

In principle, one could compute the full gradient

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta \ell(f_\theta(x_i), y_i)$$

at every iteration. For modern datasets and models, however, this is often too expensive. Each gradient evaluation requires a full forward and backward pass through the network for all training examples.

The standard practical alternative is *minibatch stochastic gradient descent*. At step t, one samples a minibatch $B_t \subseteq \{1, \dots, n\}$ and uses

$$g_t = \frac{1}{|B_t|}\sum_{i \in B_t} \nabla_\theta \ell(f_{\theta_t}(x_i), y_i)$$

as a gradient estimate. The update is

$$\theta_{t+1} = \theta_t - \eta_t g_t,$$

where $\eta_t > 0$ is the learning rate.

This is the basic training rule of deep learning. Its importance comes from three simultaneous advantages:

Scalability. The computation per update is far smaller than for full-batch training.

Hardware efficiency. Minibatches allow matrix-based parallel computation on GPUs and related hardware.

Controlled stochasticity. The gradient estimate is noisy, but that noise is often acceptable and sometimes even beneficial.

If examples are sampled uniformly, then the minibatch gradient satisfies

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t),$$

so it remains an unbiased estimator of the full empirical-risk gradient.

In deep networks, this stochasticity has a dual role. On the one hand, it introduces variance into optimization and can destabilize poorly tuned training. On the other hand, it reduces computational cost dramatically and may help optimization avoid getting trapped in poor local geometric behavior.

Minibatching as a compromise between noise and efficiency

Minibatch size is not merely a hardware choice. It mediates a conceptual tradeoff.

If the batch is very small, then updates are cheap but noisy. The direction g_t may fluctuate significantly from step to step.

If the batch is very large, then updates are more accurate approximations to the full gradient, but they are more expensive and may reduce the exploratory effect of stochasticity.

Thus minibatching is a compromise between two extremes:

- **single-example SGD:** maximally noisy, maximally cheap;
- **full-batch gradient descent:** minimally noisy, maximally expensive.

In practice, minibatches are attractive because they balance:

- statistical efficiency,
- computational throughput,
- gradient variance,
- memory constraints.

At a conceptual level, one may say: minibatching keeps optimization *alive* by providing enough gradient information to move meaningfully, while remaining cheap enough to make repeated deep-network training feasible.

Learning-rate schedules

The learning rate η_t is one of the most important quantities in optimization. It determines how strongly the parameter vector responds to the estimated gradient.

If η_t is too large, updates may overshoot, oscillate, or diverge. If η_t is too small, training may make little progress and appear stuck even when a descent direction is available.

In deep learning, the issue is especially delicate because the optimization landscape is nonconvex and differently scaled across layers and directions. A single poorly chosen step size can destabilize training even when gradients are otherwise informative.

For this reason, one often uses a *learning-rate schedule*, meaning that the step size changes over time. A typical idea is:

- begin with relatively large steps, to move quickly and explore the landscape;
- later reduce the step size, to refine the solution and avoid oscillation.

Mathematically, one may write

$$\theta_{t+1} = \theta_t - \eta_t g_t,$$

with η_t decreasing according to some schedule. Common conceptual strategies include:

- piecewise decay,
- exponential decay,
- cosine-style decay,
- warmup followed by decay.

The precise schedule is less important here than the principle: optimization typically benefits from different step scales at different stages of training.

Early in training, the model is far from a good solution and large steps may be helpful. Later, the model often needs smaller steps to stabilize learning and preserve already useful structures.

Momentum

Plain stochastic gradient descent reacts only to the current gradient estimate. But minibatch gradients are noisy, and the loss landscape may contain elongated valleys in which progress is slow and oscillatory.

A standard improvement is *momentum*. One introduces a running velocity variable v_t and updates according to

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta_t v_{t+1},$$

where $0 \leq \beta < 1$ controls how much past gradient information is retained.

Momentum has a useful physical intuition. Instead of responding only to the current force, the parameter vector develops inertia. Directions that persist across many steps are reinforced, while high-frequency oscillations are damped.

This can help in two important ways:

- it accelerates progress along directions where gradients are consistently aligned;
- it reduces wasteful oscillation across steep directions of curvature.

In deep networks, where curvature is often anisotropic and minibatch noise is unavoidable, momentum is one of the simplest and most effective ways to improve practical trainability.

Adaptive methods

A further idea is that different parameters or coordinates may require different effective step sizes. This leads to *adaptive methods*, which rescale updates using running statistics of past gradients.

The details vary by algorithm, but the general form is:

- estimate how large or variable gradients are in each coordinate;
- reduce steps in coordinates with consistently large gradients;
- allow relatively larger steps in coordinates with smaller effective scale.

Conceptually, adaptive methods try to compensate for ill-conditioned optimization by making the step rule more sensitive to coordinatewise behavior.

This is attractive in deep models because different parameters may participate in very different parts of the network and may therefore receive gradients with very different magnitudes. A single global learning rate may be too crude.

Adaptive methods can substantially improve optimization speed and convenience, especially in early training or in settings with heterogeneous gradient scales. At the same time, their interaction with generalization is more subtle, and different applications sometimes favor different choices.

Common confusion. Adaptive optimizers do *not* remove all trainability issues. They can rescale coordinates and often reduce tuning effort, but they do not magically fix poor signal propagation, bad initialization, saturating activations, or severe architectural mismatch. If gradients reaching early layers are already tiny or wildly unstable, an adaptive step rule has limited information to work with.

At the present level, the main lesson is that modern deep-network optimization is not just raw gradient descent. It is gradient descent equipped with mechanisms for controlling direction, scale, and variability.

Initialization as control of signal scale

Optimization is not determined only by the update rule. It also depends critically on where optimization begins.

In deep networks, initialization is not a cosmetic detail. It controls the scale of both:

- forward activations,
- backward gradients.

Consider a linear part of a layer:

$$z = Wh.$$

If the entries of W are too large, then the norm or variance of z may grow rapidly across depth. If they are too small, then the signal may collapse toward zero or near-constant behavior.

The same issue appears in backpropagation. Gradients are multiplied by transposed weight matrices and by activation derivatives. If the initialization scale is poor, then gradient magnitudes may either decay or explode before learning has had any chance to organize useful representations.

Thus initialization should be understood as the first act of signal propagation control.

A variance-preservation heuristic

A simple but extremely important heuristic is to track variance across layers. Suppose a layer has m_{in} input units and m_{out} output units, and let

$$z_i = \sum_{j=1}^{m_{\text{in}}} W_{ij} h_j.$$

Assume, heuristically, that:

- the inputs h_j have mean zero and common variance v_h,
- the weights W_{ij} are independent with mean zero and variance σ_W^2,
- the variables are sufficiently weakly correlated for a first-pass variance calculation.

Proposition 2.47 (Variance propagation through a linear layer)**.** *Under the assumptions above,*

$$\operatorname{Var}(z_i) = \sum_{j=1}^{m_{\text{in}}} \operatorname{Var}(W_{ij} h_j) \approx m_{\text{in}}\, \sigma_W^2\, v_h.$$

Hence preserving the scale of the pre-activations suggests the condition

$$m_{\text{in}} \sigma_W^2 \approx 1.$$

Proof. Because the weights and activations are centered and treated as independent,

$$\mathbb{E}[W_{ij} h_j] = \mathbb{E}[W_{ij}]\, \mathbb{E}[h_j] = 0.$$

So

$$\operatorname{Var}(W_{ij} h_j) = \mathbb{E}[W_{ij}^2 h_j^2] = \mathbb{E}[W_{ij}^2]\, \mathbb{E}[h_j^2] = \sigma_W^2 v_h.$$

Summing over j gives

$$\operatorname{Var}(z_i) \approx \sum_{j=1}^{m_{\text{in}}} \sigma_W^2 v_h = m_{\text{in}} \sigma_W^2 v_h.$$

This is heuristic rather than exact because real networks develop dependence and non-Gaussian effects, but it captures the core scaling law used in initialization. □

This proposition is the formal anchor. If $m_{\text{in}} \sigma_W^2 \gg 1$, activations tend to amplify. If $m_{\text{in}} \sigma_W^2 \ll 1$, they tend to shrink. Because deep networks apply many layers in sequence, even mild mismatch can accumulate into severe scale drift.

Deriving Xavier initialization

Suppose first that the activation function is approximately variance-preserving near the origin, as is heuristically true for linear activations and often roughly true for tanh at small scale. Then one wants

$$\operatorname{Var}(h^{(\ell)}) \approx \operatorname{Var}(h^{(\ell-1)})$$

at the start of training. By the proposition, a forward-pass requirement is therefore

$$m_{\text{in}} \sigma_W^2 \approx 1, \qquad \text{so} \qquad \sigma_W^2 \approx \frac{1}{m_{\text{in}}}.$$

Now consider the backward pass. If the layer maps $\mathbb{R}^{m_{\text{in}}}$ to $\mathbb{R}^{m_{\text{out}}}$, then the gradient entering the layer is multiplied by $W^\top$. Running the same variance heuristic backward suggests

$$m_{\text{out}}\sigma_W^2 \approx 1, \qquad \text{so} \qquad \sigma_W^2 \approx \frac{1}{m_{\text{out}}}.$$

These two goals need not coincide when $m_{\text{in}} \neq m_{\text{out}}$. A clean compromise is to balance them symmetrically and choose

$$\sigma_W^2 \approx \frac{2}{m_{\text{in}} + m_{\text{out}}}.$$

This is the basic Xavier or Glorot principle. It is not magic. It is a compromise between forward variance preservation and backward variance preservation.

In words:

- forward stability alone points toward $1/m_{\text{in}}$;
- backward stability alone points toward $1/m_{\text{out}}$;
- Xavier balances the two.

Deriving He initialization

ReLU-type activations change the variance story because they do not pass all of the signal. For a symmetric zero-mean random variable z, the ReLU output satisfies the heuristic identity

$$\mathbb{E}[\text{ReLU}(z)^2] \approx \frac{1}{2}\mathbb{E}[z^2].$$

So, at the level of second moments, ReLU keeps roughly half of the incoming energy and sets the rest to zero.

Combining this with the linear-layer calculation gives

$$\mathbb{E}[h_i^2] \approx \frac{1}{2}\, m_{\text{in}}\sigma_W^2\, v_h.$$

To keep this scale roughly unchanged across layers, one therefore wants

$$\frac{1}{2}m_{\text{in}}\sigma_W^2 \approx 1, \qquad \text{so} \qquad \sigma_W^2 \approx \frac{2}{m_{\text{in}}}.$$

This is the basic He initialization rule.

The message is simple. Because ReLU discards roughly half of a symmetric signal, the weights must start slightly larger than in the linear or tanh-style case if one wants to preserve useful scale through depth.

One worked example

A concrete example makes the scaling logic easier to see. Suppose a layer has $m_{\text{in}} = 400$ inputs and the input activations satisfy $\text{Var}(h_j) = 1$.

Example 2.48 (Variance scaling at one layer)**.** For the linear pre-activation

$$z_i = \sum_{j=1}^{400} W_{ij}h_j,$$

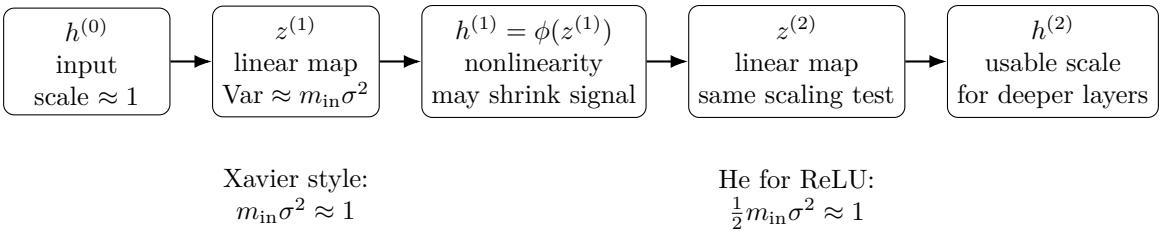


Figure 2.14: Signal propagation through layers. Initialization should keep the forward signal in a usable range as it repeatedly passes through linear maps and nonlinearities. Xavier-style scaling targets approximate preservation for near-linear or tanh-like activations, while He scaling compensates for the fact that ReLU removes roughly half of a symmetric signal.

we have the heuristic estimate

$$\mathrm{Var}(z_i) \approx 400\,\sigma_W^2.$$

So three different initialization scales behave very differently:

- if $\sigma_W^2 = 1$, then $\mathrm{Var}(z_i) \approx 400$, which is far too large;
- if $\sigma_W^2 = 1/400$, then $\mathrm{Var}(z_i) \approx 1$, matching Xavier-style forward preservation;
- if the layer is followed by ReLU and $\sigma_W^2 = 2/400$, then $\mathrm{Var}(z_i) \approx 2$, so after the ReLU gate the output second moment is brought back to about 1.

The point is not the exact decimal value. The point is that initialization scale changes the network from *immediately unstable* to *numerically plausible* before learning even begins.

A signal-propagation picture

Initialization is easiest to remember as a repeated propagation process. Each layer transforms the current signal, and the goal is not to make every number identical but to prevent systematic blow-up or collapse.

Forward and backward signal preservation

It is useful to separate two related but distinct goals of initialization.

Forward preservation. The activations

$$h^{(1)}, h^{(2)}, \ldots, h^{(L)}$$

should not systematically blow up or collapse as depth increases.

Backward preservation. The gradients

$$\delta^{(L)}, \delta^{(L-1)}, \ldots, \delta^{(1)}$$

should also remain within a useful numerical range.

A good initialization attempts to satisfy both simultaneously. This is not always possible perfectly, especially in very deep models, but the principle is central.

From this viewpoint, initialization is part of optimization, not separate from it. It shapes the geometry of the first forward pass and the first backward pass, and therefore influences the entire subsequent training trajectory.

Activation-dependent trainability

Initialization cannot be studied independently of the activation function. Trainability depends on their interaction.

Different activations change both the forward statistics and the backward derivatives:

- sigmoids and tanh may saturate, producing small derivatives and weakening backward flow;
- ReLU-type activations avoid some forms of saturation but may deactivate units or create asymmetric signal propagation;
- other activations modify smoothness, scale, and derivative behavior in different ways.

Thus a trainable network is not simply a network with many parameters. It is a network in which:

- the activations produce informative intermediate representations,
- the derivatives remain useful for learning,
- the initialization scale is matched to the activation geometry.

This is a fundamental deep-learning lesson: representation geometry and optimization geometry are inseparable. The choice of activation is not only about expressivity. It is also about whether the network can carry signal and gradient through depth effectively enough to learn.

SGD, scale, and signal propagation as one picture

The ideas of this section fit together best when viewed as one coherent system.

Minibatch SGD makes deep-network optimization computationally feasible. Learning-rate schedules control how aggressively the model moves through parameter space. Momentum and adaptive methods improve optimization dynamics in noisy and anisotropic landscapes. Initialization controls the initial scale of forward and backward signals. Activation choice determines how those signals are transformed at each layer.

These are not isolated implementation tricks. They are coordinated responses to the central challenge of deep learning: maintaining stable and informative signal propagation through a deep compositional model while performing scalable gradient-based optimization.

One may summarize the logic as follows:

- **SGD and minibatches** keep optimization scalable;
- **learning-rate control** keeps updates dynamically reasonable;
- **momentum and adaptive scaling** improve movement through difficult geometry;
- **initialization** keeps activations and gradients at viable scale;
- **activation choice** determines whether signal flow is preserved or attenuated.

The central lesson

The correct question is not simply:

> How do we optimize a deep network?

The deeper question is:

How do we keep gradient-based learning numerically and statistically alive across many layers of representation transformation?

This section has given the first part of the answer. Deep learning works in practice not only because we know how to compute gradients, but because we have learned how to preserve their usefulness.

What to remember. Optimization in deep learning is not only about picking a descent rule. Trainability depends on minibatch noise, learning-rate scale, momentum and adaptive rescaling, and especially on signal propagation through depth. Xavier initialization balances forward and backward variance preservation for near-linear or tanh-like activations, while He initialization compensates for ReLU's half-gating effect.

Exercises

Exercise 2.49. Assume a layer has m_{in} inputs, independent zero-mean weights with variance σ_W^2, and input activations of variance v. Under the same independence heuristic used in the text, compute $\text{Var}(z_i)$ for

$$z_i = \sum_{j=1}^{m_{\text{in}}} W_{ij} h_j.$$

Then determine the value of σ_W^2 that keeps $\text{Var}(z_i) \approx v$.

Exercise 2.50. Compare Xavier and He initialization for a network that uses tanh versus a network that uses ReLU. Why does the ReLU case call for a larger variance scale? Your answer should explicitly refer to what happens to the signal after the nonlinearity.

Exercise 2.51. Reason qualitatively about minibatch size and gradient noise. Why do very small batches produce noisier updates? Why can very large batches be more stable but less exploratory? How might this tradeoff interact with learning-rate choice in practice?

Notes and references

The purpose of this section is to show that practical deep learning is a problem of controlled signal propagation as much as it is a problem of optimization. Minibatches, momentum, adaptive methods, activation choice, and initialization are all responses to the same question: how to keep learning signals informative across depth. The variance calculations for Xavier and He initialization are heuristic, but they are among the most useful back-of-the-envelope derivations in modern neural-network practice because they connect linear algebra, probability, and trainability in one clean picture.

2.7 Stabilizing Deep Learning: Activations, Normalization, and Regularization

The previous sections explained why deep networks are difficult to optimize. Gradients can vanish or explode, signal scale can drift across depth, and highly expressive models can become statistically brittle even when optimization succeeds. Modern deep learning became practical only when the field learned how to control all three of these issues at once.

This section organizes that story into three subthemes:

- **activation choice and gradient flow,**
- **normalization and scale control,**
- **regularization and generalization support.**

These are not isolated tricks. They are complementary ways of making deep learning both trainable and useful. Activation functions shape the local derivatives through which signals move. Normalization keeps internal representations in controlled numerical coordinates. Regularization biases training toward solutions that are less brittle and more likely to generalize.

Activation choice and gradient flow

A hidden layer has the form

$$h^{(\ell)} = \phi(z^{(\ell)}), \qquad z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}.$$

During backpropagation the local gradient is multiplied by

$$\phi'(z^{(\ell)}).$$

So activation choice matters not only for expressivity, but also for whether gradients remain informative.

This leads to a useful design principle. One prefers activations whose derivative behavior supports learning in the regimes actually visited by the network. If the derivative is almost always tiny, then gradient flow weakens. If the activation truncates too aggressively, useful signal can be lost. If the activation preserves a healthy range of derivatives, optimization becomes easier.

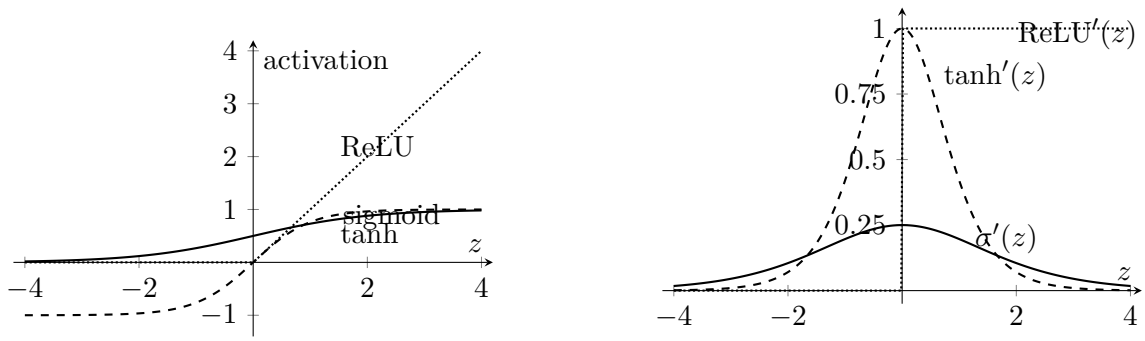
Sigmoid and tanh**.** These are smooth and bounded activations. Their historical importance is enormous, but they saturate for large $|z|$. In those regions the derivative is small, so repeated multiplication through depth weakens gradients. The centering of tanh around zero often makes it easier to optimize than sigmoid, but the saturation problem remains.

ReLU and its descendants. The rectified linear unit

$$\mathrm{ReLU}(z) = \max(0, z)$$

avoids saturation on its positive side. When a unit is active, the derivative is exactly 1. This helped make much deeper models trainable in practice. Its cost is asymmetry: inactive units have output and derivative both equal to zero. Leaky variants and smoother functions such as GELU can be viewed as attempts to preserve the trainability advantages of ReLU while softening its hard cutoff.

The broad lesson is that activation functions should be judged not only by the functions they can represent, but also by the gradient environments they create.



(a) Representative activation curves.

(b) Derivative behavior drives gradient flow.

Figure 2.15: Activation choice affects trainability because the derivative profile determines how strongly backpropagated information is attenuated or preserved. Sigmoid and tanh can saturate in their tails, while ReLU preserves derivative 1 on its active side but becomes inactive on its negative side.

Normalization and scale control

Activation choice addresses one part of the signal-propagation problem. Normalization addresses another. Its core purpose is to keep internal representations in a numerically manageable range while learning is changing the network underneath them.

Definition 2.52 (Batch normalization)**.** Let $z_1, \dots, z_m$ denote the scalar values of one feature across a minibatch of size m. Define the batch mean and variance by

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu_B)^2.$$

Batch normalization forms the normalized coordinate

$$\widehat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}},$$

and then applies a learned affine reparameterization

$$y_i = \gamma \widehat{z}_i + \beta.$$

Thus the feature is standardized across the examples of the minibatch before being rescaled and recentered by learned parameters.

Definition 2.53 (Layer normalization)**.** Let $z = (z_1, \dots, z_d)$ denote the feature vector for one example. Define the within-example mean and variance by

$$\mu_L = \frac{1}{d}\sum_{j=1}^{d} z_j, \qquad \sigma_L^2 = \frac{1}{d}\sum_{j=1}^{d}(z_j - \mu_L)^2.$$

Layer normalization forms

$$\widehat{z}_j = \frac{z_j - \mu_L}{\sqrt{\sigma_L^2 + \varepsilon}}, \qquad y_j = \gamma_j \widehat{z}_j + \beta_j.$$

Thus normalization is performed across features of a single example rather than across a minibatch.

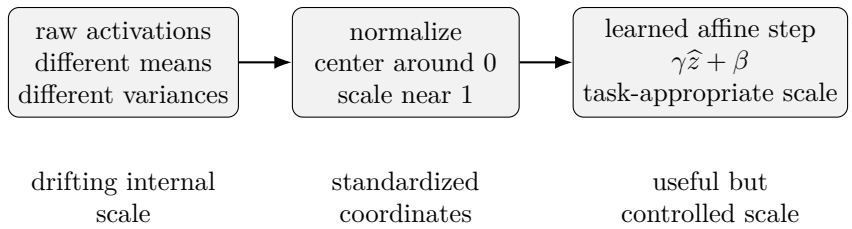


Figure 2.16: Normalization as scale control. The point is not to freeze all internal values forever, but to keep the optimization operating in standardized coordinates and then let the network relearn a useful affine scale.

These definitions look similar because the core mechanism is similar: convert a raw internal coordinate into a standardized one and then allow the network to relearn an appropriate scale and offset. The practical difference is where the statistics are computed. Batch normalization couples examples within a minibatch. Layer normalization acts within one example and is therefore natural in sequence models and transformers.

Proposition 2.54 (Normalization changes scale sensitivity and optimization geometry)**.** *Consider an idealized normalization layer with $\varepsilon = 0$. Suppose a scalar pre-activation is multiplied by a positive constant $c > 0$, so that $z_i' = cz_i$. Then the normalized coordinate is unchanged:*

$$\frac{z_i' - \mu'}{\sigma'} = \frac{z_i - \mu}{\sigma},$$

where μ', σ' are the mean and standard deviation computed from $z_1', \ldots, z_m'$ in batch normalization or from the feature vector z' in layer normalization. Consequently, normalization removes sensitivity to raw positive rescaling along certain directions, so it changes the effective optimization geometry rather than merely changing output values.

Proof. If $z_i' = cz_i$, then the corresponding mean rescales as $\mu' = c\mu$. Likewise the standard deviation rescales as $\sigma' = c\sigma$ because variance rescales by c^2. Therefore

$$\frac{z_i' - \mu'}{\sigma'} = \frac{cz_i - c\mu}{c\sigma} = \frac{z_i - \mu}{\sigma}.$$

The same algebra holds whether the statistics are taken across a minibatch or across coordinates of one example. Thus the normalized coordinate is invariant to positive rescaling of the underlying raw coordinate. □

This proposition explains why normalization is deeper than simple output rescaling. It changes which parameter directions matter. Scaling the incoming representation no longer affects the normalized coordinate in the same way it would in an unnormalized network. So optimization takes place in a different geometry, with internal coordinates continually brought back toward a controlled range.

Common confusion. Batch normalization is *not* the same thing as regularization, even though it can have regularizing side effects. Its primary role is optimization stabilization and scale control: it changes the coordinates in which learning happens and often makes training less sensitive to initialization or learning-rate choice. The minibatch noise it introduces may also regularize, but that is not the whole story and not the best first interpretation.

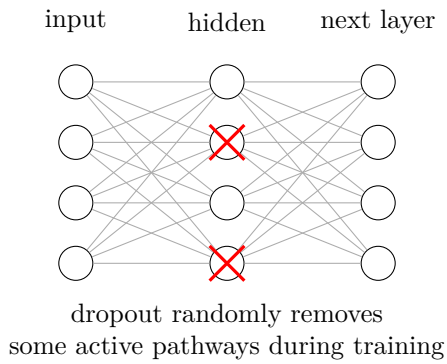


Figure 2.17: A dropout mask schematic. Randomly suppressing units during training encourages the network to spread useful information across multiple pathways rather than relying on a single fragile configuration.

Batch norm versus layer norm. The most important conceptual distinction is simple. Batch normalization standardizes each feature across examples in a minibatch. Layer normalization standardizes the full feature vector within one example. So batch norm ties examples together during training, while layer norm does not. That is why batch norm is often natural in convolutional settings with large minibatches, whereas layer norm fits sequence models and transformers where batch statistics are less central.

Regularization and generalization support

Once optimization is stable enough to train a deep model, a second question emerges. Why should the learned solution generalize rather than merely memorize? Regularization methods address that question. They do not primarily keep gradients numerically alive. Instead, they bias the learning dynamics or the objective toward solutions that are less brittle.

Remark 2.55 (Optimization stabilization versus statistical regularization). It is useful to distinguish two roles that are often blurred together. Methods such as normalization mainly stabilize optimization by controlling signal scale and changing the effective parameter geometry. Methods such as dropout, weight decay, early stopping, and data augmentation mainly support generalization by discouraging fragile or overly specialized solutions. In practice the boundary is not absolute: one method can have both optimization and regularizing effects. But pedagogically the distinction is essential.

Dropout. During training, dropout multiplies an activation vector h by a random mask m with entries in $\{0, 1\}$. In the common inverted-dropout convention,

$$\widetilde{h} = \frac{m \odot h}{p},$$

where p is the keep probability. Then each coordinate satisfies

$$\mathbb{E}[\widetilde{h}_j] = h_j,$$

so the expected activation scale is preserved even though random pathways are temporarily removed. This encourages the network not to depend too strongly on any one co-adapted configuration of units.

Weight decay, early stopping, and data augmentation. Weight decay penalizes large parameter norms and often biases learning toward less extreme solutions. Early stopping halts training before the model fully fits sample-specific noise. Data augmentation enlarges the effective training distribution by applying label-preserving transformations, thereby encouraging invariances that reflect the task structure. These methods look different, but they share one aim: improve out-of-sample behavior by discouraging brittle fitting.

Seen together, the conceptual division of labor is clear. Activation choice helps gradients remain useful. Normalization keeps internal scale under control. Regularization guides optimization toward solutions that generalize better. Modern deep learning relies on all three.

The central lesson

The right summary is not simply that deep networks became better because they became larger. They became practical because signal flow, scale control, and generalization support became manageable enough for large-scale optimization to work reliably.

In that sense, modern deep learning is not only a story of expressive model classes. It is equally a story of learning how to stabilize those classes during training.

Exercises

Exercise 2.56. Compare sigmoid, tanh, and ReLU from the perspective of derivative behavior. Which of them saturate, in what regimes, and how does that affect gradient flow in a deep network? Explain why an activation that is expressive enough in principle may still be a poor choice for optimization.

Exercise 2.57. Explain the difference between batch normalization and layer normalization. Your answer should identify where the mean and variance are computed, why this changes the training behavior, and why layer normalization is often more natural in transformer-style architectures.

Exercise 2.58. Let dropout keep each coordinate independently with probability p, and use the inverted-dropout rule

$$\widetilde{h} = \frac{m \odot h}{p}.$$

Show informally that $\mathbb{E}[\widetilde{h}_j] = h_j$ for each coordinate. Then explain why preserving the expectation does *not* mean dropout leaves training unchanged. What source of stochasticity remains?

Notes and references

The purpose of this section is to separate three roles that are often mixed together in practice: activation design for gradient flow, normalization for internal scale control, and regularization for generalization support. The exact theory behind normalization remains incomplete, but the scale-invariance proposition already captures an essential point: normalized networks are optimized in a different geometry from unnormalized ones. That is why these methods should be seen not as isolated engineering tricks, but as structural answers to trainability and robustness in deep learning.

2.8 Residual Learning and the Possibility of Very Deep Models

The previous sections explained how optimization can be stabilized through better activations, normalization, and initialization. Residual learning goes one step further. It does not merely adjust the training procedure around a deep network. It changes the architecture itself so that depth becomes easier to optimize.

This is why residual learning is a natural climax to the chapter. It answers one of the deepest practical questions in deep learning: why can very deep models work at all, rather than collapsing under their own optimization difficulty?

Why plain depth is not enough

A deeper model is more expressive in principle. It can represent all the functions of a shallower model and often many more. But expressivity is not the same thing as trainability. A plain deep network may have enough capacity to represent a useful solution and yet still be hard to optimize.

This mismatch appears in the *degradation problem*. As depth increases, a plain network may perform worse not only on test data but even on the training set. That is a sign that the issue is not simply overfitting. The issue is that optimization itself becomes harder.

Residual blocks

The key idea is to rewrite a desired transformation as identity plus correction.

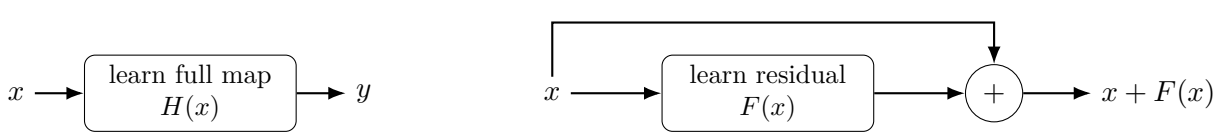
Definition 2.59 (Residual block)**.** A residual block is a map of the form

$$R(x) = x + F(x),$$

where F is a learned residual function. The direct path carrying x forward is called the *identity shortcut* or *skip connection*. When dimensions do not match exactly, the shortcut may be replaced by a simple projection, but the central idea remains the same: the block preserves a direct pathway and learns a correction rather than a full replacement.

This changes the role of depth. A plain block must reconstruct a new representation from the current one. A residual block may preserve what is already useful and modify only what needs correction.

That reformulation helps optimization because many useful deep transformations are plausibly closer to "identity plus adjustment" than to "entirely new representation at every stage." The architecture therefore builds a favorable bias directly into the parameterization.



Proposition 2.60 (Residual blocks are perturbations of the identity when the residual is small)**.** *Let $R(x) = x + F(x)$ on a normed vector space. Suppose that on some region of interest,*

$$\|F(x)\| \le \varepsilon \qquad \textit{and} \qquad \|J_F(x)\| \le \varepsilon$$

for some $\varepsilon > 0$. Then on that region,

$$\|R(x) - x\| \le \varepsilon \qquad \textit{and} \qquad \|J_R(x) - I\| \le \varepsilon.$$

So the residual block is simultaneously a small perturbation of the identity in its function values and in its local Jacobian.

Proof. By definition,

$$R(x) - x = F(x),$$

so the first bound is immediate. For the Jacobian,

$$J_R(x) = J_{x+F}(x) = I + J_F(x).$$

Hence

$$J_R(x) - I = J_F(x),$$

and therefore

$$\|J_R(x) - I\| = \|J_F(x)\| \le \varepsilon.$$

Thus both the map and its linearization remain close to the identity whenever the residual branch is small. □

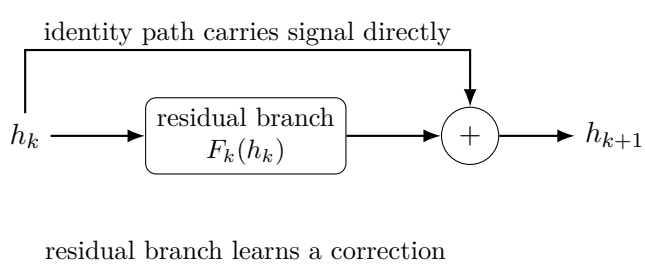
This proposition captures the main intuition of residual learning in one line. If the network has no strong reason to change the representation, it can stay near identity. That is far easier to preserve architecturally than in a plain deep stack, where each block must still learn how to carry information forward.

Identity pathways and gradient flow

The skip connection helps both forward and backward information flow. Forward information can pass through the identity path without being repeatedly distorted. Backward information can pass through a Jacobian of the form

$$J_R(x) = I + J_F(x)$$

rather than through only $J_F(x)$. This does not solve every optimization problem, but it makes gradient propagation substantially more robust.



This explains why residual connections are more than a convenience. They are an architectural mechanism for making depth compatible with optimization.

Depth as iterative refinement

Residual learning also changes the interpretation of depth. If

$$h_{k+1} = h_k + F_k(h_k),$$

then each block can be read as an update rule: keep what is already useful, then add a correction. Depth becomes a sequence of refinements rather than a sequence of complete reconstructions.

$$\boxed{h_0} \longrightarrow \boxed{h_1 = h_0 + F_0(h_0)} \longrightarrow \boxed{h_2 = h_1 + F_1(h_1)} \longrightarrow \boxed{h_3 = h_2 + F_2(h_2)}$$

first correction further refinement accumulated representation

Remark 2.61 (Residual networks, iterative refinement, and continuous-depth limits). The update rule

$$h_{k+1} = h_k + F_k(h_k)$$

resembles a discrete iterative method. If one writes it suggestively as

$$h_{k+1} - h_k \approx \Delta t\, G_k(h_k),$$

then depth begins to look like a time discretization of a flow. This is the bridge to continuous-depth viewpoints such as neural ordinary differential equations. Even before taking any limit, however, the residual-network picture is already valuable: it interprets deep representation learning as controlled iterative refinement.

Why residuals changed deep learning

Residual connections mattered historically because they made far deeper networks trainable. But their conceptual importance is even broader. They showed that optimization success in deep learning depends not only on having gradients and compute, but also on parameterizing the model in a way that makes useful solutions easier to reach.

This is the deepest lesson of the section: architecture can encode optimization bias. Residual structure tells the model that preserving information is the default and changing it is a learned correction. That is a much better starting point for very deep optimization than forcing every block to rebuild everything from scratch.

Common confusion. Residual connections do *not* magically solve every optimization issue. They improve gradient flow and make identity-like behavior accessible, but they do not remove the need for good initialization, sensible normalization, appropriate learning rates, or enough model capacity. Residual structure helps because it changes the architecture in a favorable direction; it is not a universal cure.

What to remember. A residual block has the form $R(x) = x + F(x)$. When F and its Jacobian are small, the block is a perturbation of the identity, which makes information and gradients easier to preserve. Residual networks therefore reinterpret depth as iterative refinement: each block adds a learned correction rather than replacing the representation outright. This architectural idea was one of the key reasons very deep models became practical.

Exercises

Exercise 2.62. Let $R(x) = x + F(x)$ for a differentiable map $F : \mathbb{R}^d \to \mathbb{R}^d$. Compute the Jacobian $J_R(x)$ in terms of $J_F(x)$. Then explain why the identity term matters for gradient propagation in deep residual networks.

Exercise 2.63. Explain in words why identity shortcuts help optimization. Your answer should discuss both forward information flow and backward gradient flow, and should make clear why the degradation problem is not the same thing as lack of expressive capacity.

Exercise 2.64. Consider a toy task where the desired map is very close to the identity, for example $H(x) = x + 0.1\sin(x)$ in one dimension. Why might a residual parameterization $H(x) = x + F(x)$ be easier to optimize than asking a plain network block to learn H directly from scratch? Give an informal argument based on parameterization and trainability.

Notes and references

This section presents residual learning as the architectural climax of the trainability story. The formal proposition makes precise the phrase "perturbation of identity," while the iterative-refinement remark explains why residual networks naturally connect to dynamical-systems viewpoints. The most important conceptual point is that very deep learning became practical not only because optimization algorithms improved, but because the model architecture was redesigned so that preserving information became easy and refinement became the natural role of depth.

2.9 Case Studies and Worked Examples

The chapter has now developed the main conceptual pieces of deep learning: linear models and their limits, hidden representations, backpropagation, optimization difficulty, stabilization mechanisms, and residual architecture. A final section should not merely repeat those ideas. It should force them to act on small problems where every step can be inspected.

For that reason, this section is best understood as the chapter's *consolidation lab*. The examples are deliberately tiny. They are not meant to impress by scale. They are meant to make the chapter operational. By the end of the section, students should be able to:

- identify when a linear model is sufficient and when it is not,
- explain why hidden layers become necessary on problems such as XOR,
- carry out a full forward and backward pass by hand on a tiny network,
- compare plain and residual parameterizations on a near-identity task,
- explain the difference between fixed nonlinear features and learned representations.

Perceptron on linearly separable data

We begin with the simplest successful case. Consider points in $\mathbb{R}^2$ labeled by the sign of

$$x_1 + x_2 - 1.$$

For example,

$$(0,0) \mapsto 0, \qquad (1,0) \mapsto 0, \qquad (0,1) \mapsto 0, \qquad (1,1) \mapsto 1,$$

and any point with $x_1 + x_2 > 1$ belongs to class 1. This dataset is linearly separable because the line

$$x_1 + x_2 = 1$$

separates the two classes.

A perceptron with score

$$f(x) = w^\top x + b$$

and prediction given by thresholding at zero can solve the task exactly. For instance,

$$w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad b = -1$$

works immediately.

This is the right first reminder for the chapter. Deep learning does not replace the linear viewpoint when the geometry is already linear. On linearly separable data, the simplest model class is already appropriate. The point of depth is not to abandon linear ideas, but to go beyond them when the data demand richer coordinates.

XOR and the necessity of hidden layers

The XOR problem is the smallest example showing that a linear separator can fail for structural rather than optimization reasons. Let

$$y = x_1 \oplus x_2,$$

so the four input-output pairs are

$$(0,0) \mapsto 0, \qquad (1,0) \mapsto 1, \qquad (0,1) \mapsto 1, \qquad (1,1) \mapsto 0.$$

The positive points are the off-diagonal corners of the unit square, while the negative points are the diagonal corners. No single affine decision boundary separates them.

So the failure of a linear model on XOR is not due to poor training. It is due to the geometry of the model class itself. A hidden layer becomes necessary because the representation must change.

A small ReLU construction makes this concrete. Define

$$h_1(x) = \mathrm{ReLU}(x_1 + x_2), \qquad h_2(x) = \mathrm{ReLU}(x_1 + x_2 - 1),$$

and then define the output

$$f(x) = h_1(x) - 2h_2(x).$$

Direct evaluation gives

$$f(0,0) = 0, \qquad f(1,0) = 1, \qquad f(0,1) = 1, \qquad f(1,1) = 0.$$

Thus a one-hidden-layer network can represent XOR exactly.

The important lesson is not the specific formula. It is the mechanism. The hidden layer creates new intermediate coordinates, and the output layer becomes simple in those learned coordinates. That is representation learning in miniature.

A tiny backpropagation example

The next example is the computational core of the chapter. We will perform one full forward pass and one backward pass by hand on a tiny network. Consider a network with two inputs, two hidden units, and one scalar output:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \qquad h = \sigma(z^{(1)}), \qquad \hat{y} = W^{(2)}h + b^{(2)}.$$

Take

$$x = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad y = 1,$$

choose sigmoid hidden units,

$$\sigma(t) = \frac{1}{1 + e^{-t}},$$

and use the parameter values

$$W^{(1)} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}, \qquad b^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad W^{(2)} = \begin{pmatrix} 1 & -1 \end{pmatrix}, \qquad b^{(2)} = 0.$$

Let the loss be the squared error

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

Forward pass. First compute the hidden preactivation:

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Therefore

$$h = \begin{pmatrix} \sigma(1) \\ \sigma(0) \end{pmatrix} = \begin{pmatrix} 0.7311 \\ 0.5 \end{pmatrix} \quad \text{approximately.}$$

Now compute the output:

$$\hat{y} = W^{(2)}h + b^{(2)} = 1 \cdot 0.7311 - 1 \cdot 0.5 = 0.2311.$$

So the loss is

$$L = \frac{1}{2}(0.2311 - 1)^2 \approx 0.2956.$$

Backward pass. The derivative of the loss with respect to the scalar output is

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.2311 - 1 = -0.7689.$$

Hence the gradients at the second layer are

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \hat{y}}\, h^\top = -0.7689 \begin{pmatrix} 0.7311 & 0.5 \end{pmatrix} = \begin{pmatrix} -0.5621 & -0.3845 \end{pmatrix},$$

and

$$\frac{\partial L}{\partial b^{(2)}} = -0.7689.$$

Next backpropagate into the hidden units:

$$\frac{\partial L}{\partial h} = \left(W^{(2)}\right)^\top \frac{\partial L}{\partial \hat{y}} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} (-0.7689) = \begin{pmatrix} -0.7689 \\ 0.7689 \end{pmatrix}.$$

For sigmoid units,

$$\sigma'(t) = \sigma(t)(1 - \sigma(t)).$$

Thus

$$\sigma'(1) = 0.7311(1 - 0.7311) \approx 0.1966, \qquad \sigma'(0) = 0.25.$$

So

$$\frac{\partial L}{\partial z^{(1)}} = \frac{\partial L}{\partial h} \odot \sigma'(z^{(1)}) = \begin{pmatrix} -0.7689 \\ 0.7689 \end{pmatrix} \odot \begin{pmatrix} 0.1966 \\ 0.25 \end{pmatrix} = \begin{pmatrix} -0.1512 \\ 0.1922 \end{pmatrix} \quad \text{approximately.}$$

Finally,

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial z^{(1)}} x^\top = \begin{pmatrix} -0.1512 \\ 0.1922 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} -0.1512 & 0 \\ 0.1922 & 0 \end{pmatrix},$$

and

$$\frac{\partial L}{\partial b^{(1)}} = \begin{pmatrix} -0.1512 \\ 0.1922 \end{pmatrix}.$$

This small calculation is worth doing slowly. It shows exactly what backpropagation is: not magic, but systematic reuse of intermediate derivatives through the chain rule.

Plain versus residual parameterization on a near-identity task

Now consider a task where the desired map is very close to the identity, such as

$$H(x) = x + 0.1\sin(x)$$

in one dimension. A plain block is asked to learn the entire map H directly. A residual block instead writes

$$H(x) = x + F(x)$$

with

$$F(x) = 0.1\sin(x).$$

The representational content is the same, but the parameterization is different. In the plain parameterization, the model must simultaneously preserve the input and learn the correction. In the residual parameterization, preservation is built in and the model needs only to learn the small change.

This is exactly why residual connections help optimization. When the useful transformation is close to identity, the residual branch can stay small and the shortcut carries the main signal directly. The network is therefore biased toward stable information flow rather than repeated reconstruction.

Kernel versus neural representation on a toy nonlinear dataset

Consider a nonlinear dataset such as concentric circles in $\mathbb{R}^2$. A linear classifier in the raw input coordinates fails because its decision boundaries are lines rather than circles. A classical remedy is to introduce a handcrafted nonlinear feature such as the squared radius:

$$\phi(x) = x_1^2 + x_2^2.$$

Then classification can be done by a linear rule in the lifted coordinate $\phi(x)$. This is the kernel or fixed-feature viewpoint.

A neural network approaches the same problem differently. Instead of specifying the right nonlinear feature in advance, it learns hidden coordinates from data:

$$x \mapsto h(x) \mapsto \hat{y}.$$

The final predictor may again be simple, but the hidden representation is learned rather than prescribed.

So the difference is not merely that both methods are nonlinear. The difference is *where the representation comes from*:

- a kernel or feature-engineering approach uses a fixed lifting chosen before seeing the task in full,
- a neural-network approach learns the lifting from the training process itself.

This comparison helps consolidate one of the chapter's main messages. Deep learning did not abolish the classical idea of feature maps. It made feature construction adaptive.

Common confusion. A worked example is not only for checking arithmetic. Its main value is conceptual. When students compute one full forward and backward pass by hand, they see where each derivative comes from, why the chain rule is organized layer by layer, and why hidden representations matter. The goal is understanding, not clerical repetition.

What to remember. This section is the chapter's consolidation lab. A perceptron succeeds on linearly separable data because the geometry already matches the model class. XOR shows that hidden layers can be necessary, not optional. A tiny backpropagation calculation reveals the actual mechanics of gradient computation. Residual parameterization explains why learning a correction can be easier than learning a full map. The kernel-versus-neural comparison shows the conceptual move from fixed nonlinear features to learned representations.

Exercises

Exercise 2.65. Consider the linearly separable dataset

$$(0,0) \mapsto 0, \quad (1,0) \mapsto 0, \quad (0,1) \mapsto 0, \quad (1,1) \mapsto 1.$$

Find one choice of perceptron parameters $w \in \mathbb{R}^2$ and $b \in \mathbb{R}$ that classifies all four points correctly. Then explain geometrically why the same task does not require a hidden layer.

Exercise 2.66. Show that XOR is not linearly separable. Your argument may be geometric or algebraic, but it should make clear why no affine decision boundary can place the two off-diagonal points on one side and the two diagonal points on the other.

Exercise 2.67. Repeat the tiny backpropagation example in this section, but replace the target value $y = 1$ by $y = 0$. Compute the new forward pass, the loss, and all gradients by hand. Which signs change, and why?

Exercise 2.68. Perform one gradient-descent update step on the tiny network using learning rate $\eta = 0.1$ and the gradients computed in the worked example. Then recompute the output $\hat{y}$ on the same input. Did the prediction move closer to the target?

Exercise 2.69. Explain informally why a residual parameterization may be easier to optimize than a plain parameterization on a task close to the identity. Your answer should refer to both signal preservation and the size of the learned correction.

Exercise 2.70. On a toy nonlinear classification problem such as concentric circles, compare the following three viewpoints:

1. linear prediction in the raw coordinates,
2. linear prediction after a fixed nonlinear feature map,
3. linear prediction after a learned hidden representation.

What does each viewpoint assume about where useful coordinates come from?

Notes and references

This section is designed to consolidate the chapter rather than add new theory. The perceptron example reminds students that deep methods are not always necessary, while XOR remains the canonical miniature example showing why hidden layers can qualitatively enlarge representational power. The tiny backpropagation calculation is pedagogically central: students should leave the chapter able to execute a complete forward and backward pass by hand on a very small network. The plain-versus-residual comparison connects the worked examples back to the architecture story, and the kernel-versus-neural comparison closes the loop between classical feature design and learned representation.

2.10 What We Understand and What We Still Do Not

The chapter has now developed the core mechanics of deep learning: layered representation, backpropagation, optimization difficulty, stabilization techniques, and residual architecture. A good ending should not treat these achievements as if they automatically resolve every scientific question. It should separate clearly what is established, what is empirically stable, and what remains open.

That separation is part of scientific maturity. In successful fields, one must resist two opposite mistakes. One mistake is triumphalism: success in practice is treated as if it implied full understanding. The other is excessive skepticism: because open problems remain, everything is treated as mysterious. A more disciplined view is better. Some mechanisms are mathematically clear. Some empirical regularities are robust even though their deepest explanation remains incomplete. And some central questions are genuinely open.

Statement 2.71 (Epistemic summary for this chapter)**.** *The following are well established: backpropagation computes gradients by repeated application of the chain rule on layered computation graphs; depth can enlarge expressive and representational possibilities; initialization, activation choice, normalization, and residual pathways strongly affect trainability; and modern deep learning works in practice at scales far beyond what early intuition suggested. What is* not *equally settled is a complete theory explaining why all of these ingredients interact as successfully as they do in large modern models, especially with respect to optimization dynamics and generalization.*

This statement is deliberately modest. It says neither "we understand everything" nor "we understand nothing." It states instead that the field contains a stable core together with unresolved explanatory edges.

Theory: what is established

Several central ingredients of the chapter are on firm mathematical ground.

Backpropagation. Backpropagation is not a mysterious deep-learning trick. It is reverse-mode differentiation applied to a computation graph. A scalar loss composed with layered maps can be differentiated efficiently because local derivatives combine through the chain rule. This part is conceptually and mathematically clear.

Expressive value of depth. The exact frontier of expressivity is subtle, but the basic lesson is stable: compositional depth can represent certain functions more efficiently than shallow parameterizations, and hidden layers can create intermediate coordinates in which prediction becomes easier. The XOR example is the smallest pedagogical version of this fact.

Trainability depends on signal propagation and parameterization. It is also well established that trainability is not determined only by nominal expressive power. Initialization, activation derivatives, normalization, and residual pathways all affect how signals and gradients move through depth. This is why optimization in deep learning must be studied structurally rather than only abstractly.

Empirical regularities: what is stable in practice

There are also practical facts that are not in serious doubt as observations, even if their most general explanation is incomplete.

Stochastic gradient methods work remarkably well. Minibatch stochastic optimization scales to very large nonconvex models and often finds low training loss efficiently. This is one of the most stable practical facts in the field. Its exact explanation is still debated, but the observation itself is not fragile.

Normalization and residual design reliably help. Normalization methods and residual pathways improve optimization across many model families. The details vary by architecture, but the practical pattern is robust. These are not isolated tricks that happened to work once. They are recurring design principles.

Large models often remain trainable. A striking regularity of modern practice is that overparameterized neural networks are often easier to fit than earlier intuition would suggest. This fact has motivated much of the recent theory literature. Again, the phenomenon is stable even though the complete explanatory picture remains unfinished.

Open problems: what we still do not fully understand

The deepest unresolved questions lie mainly at the level of explanation rather than raw observation.

Why optimization succeeds so broadly in overparameterized regimes. We know that large networks can often be trained effectively with gradient-based methods. We do not yet possess a single general theory explaining why the optimization landscape and the training dynamics cooperate so favorably across modern architectures and data regimes.

Why generalization can remain strong despite massive capacity. Modern networks can fit training data extremely well and yet often generalize surprisingly well. There are partial stories involving inductive bias, optimization dynamics, margins, implicit regularization, and data structure. But no single account yet serves as a universally satisfactory theory of modern deep-learning generalization.

How architecture, data, and optimization interact at scale. The chapter has treated activation choice, normalization, residual structure, and optimization as distinguishable ingredients. In reality they interact. One of the major open tasks of theory is to explain these interactions at the scale and complexity of contemporary models rather than only in simplified asymptotic regimes.

Remark 2.72 (Optimization stabilization is not the same as statistical regularization). Some methods improve optimization by stabilizing scale, gradient flow, or conditioning. Other methods improve generalization by constraining effective complexity or injecting noise. In practice, the same technique may influence both. But conceptually these roles should not be collapsed. A method can make training easier without being, in its primary mechanism, a regularizer in the statistical sense.

Common confusion. Open problems do not imply that the subject lacks foundations. Deep learning contains both well-understood mechanisms and unresolved explanatory questions. The right response is neither overconfidence nor dismissal, but careful separation of levels of certainty. In particular, a stable empirical regularity is not the same thing as a theorem, and an effective training technique is not automatically a complete explanation.

What to remember. Backpropagation, hidden representations, and the structural role of initialization, normalization, and residual pathways are established parts of the subject. Robust practical regularities such as the success of minibatch optimization are real even when theory is incomplete. The major unresolved questions concern the deepest explanations of optimization and generalization in large modern models. Scientific discipline means keeping these categories separate: theorem, empirical regularity, and open problem are not the same thing.

Reflective questions and end-of-chapter exercises

Exercise 2.73. Give one example from this chapter of each of the following:

1. a mathematically established mechanism,
2. a practically stable empirical regularity,
3. an open explanatory problem.

For each example, explain in two or three sentences why it belongs in that category rather than one of the others.

Exercise 2.74. Why is it misleading to say simply that "deep learning works because it is nonconvex"? Your answer should compare nonconvexity, expressive capacity, trainability, and optimization stabilization.

Exercise 2.75. Explain why the following two statements are different:

1. "batch normalization often improves training,"
2. "batch normalization regularizes the model."

Why does the first statement concern optimization more directly, while the second concerns a different question?

Exercise 2.76. Suppose a student says, "Because we can train very large neural networks successfully, we must already understand why they generalize." Write a short reply correcting this statement using the chapter's distinction among theory, empirical regularity, and open problem.

Notes and References

Backpropagation is classically presented as reverse-mode differentiation on computation graphs and is treated in modern deep-learning texts such as Goodfellow, Bengio, and Courville [**?**] and in standard treatments of matrix and automatic differentiation. The early importance of multilayer networks for escaping linear limitations such as XOR is historically tied to the revival of connectionism and the backpropagation era [**?**]. Residual learning as a trainability breakthrough is associated with He, Zhang, Ren, and Sun [**?**]. Batch normalization was introduced by Ioffe and Szegedy [**?**], while layer normalization was introduced by Ba, Kiros, and Hinton [**?**]. For a broad view of optimization and generalization questions in deep learning, see also contemporary textbook and survey discussions such as [**?**, **?**].

Chapter 3

Architecture and Inductive Bias in Deep Learning

3.1 Why Architecture Matters

Chapter 2 developed one of the central ideas of modern machine learning: useful internal representations should often be *learned* rather than fixed in advance. Neural networks made this possible by turning representation itself into part of the optimization problem. But once that step is taken, a deeper question immediately appears: *how should a model be organized so that it can learn the right kind of representation for a given domain?*

This is the question of *architecture*. Architecture is not a cosmetic detail. It is not merely a matter of implementation style, engineering taste, or historical fashion. It is the mechanism by which assumptions about the structure of data are built into a learning system. The thesis of this section, and of the chapter as a whole, is therefore the following:

> **Architecture is the deep-learning analogue of hand-crafted structural assumptions, but expressed in a learnable rather than fixed form.**

In classical machine learning, structural assumptions often entered through feature design, basis choice, or kernel selection. In deep learning, these assumptions are increasingly expressed through the computational organization of the model itself: which variables are allowed to interact directly, which parameters are reused across positions or times, which symmetries are preserved, and which forms of dependence are easy or hard to represent. The named architectures that follow — convolutional networks, recurrent networks, attention-based models, and transformers — should therefore not be understood as isolated inventions. They are different answers to a common question:

> What structural bias should the model embody in order to learn effectively from a particular kind of data?

From Chapter 2 to Chapter 3

The logic connecting this chapter to Chapter 2 is simple. Chapter 2 argued that the success of deep learning comes largely from learning useful representations rather than relying only on fixed ones. Chapter 3 asks what kind of *computational structure* makes those representations learnable in practice. The answer is architectural design.

A generic multilayer perceptron is an expressive function approximator, but it treats its input mainly as coordinates in a vector space. That can be appropriate in some settings, but it is often a poor match to the true organization of images, sequences, sets, graphs, or interacting objects. Real data usually has structure: locality in space, order in time, symmetry under transformation, and relational interaction among entities. Architecture matters because these regularities can be encoded directly into the model.

Generic multilayer networks versus structured models

A fully connected network is attractive because it is general. In principle, after vectorization, one may feed images, text, sensor data, or relational summaries into the same multilayer template. But this generality comes with a cost. The model is then forced to discover from data not only the task but also much of the domain structure that was already known to the practitioner.

A *structured model* changes this situation. It modifies the generic template so that the architecture itself reflects prior assumptions about the data. Such assumptions may concern locality, translation

reuse, temporal dependence, sparsity of interaction, graph connectivity, or permutation symmetry. The central contrast is not merely that generic networks are flexible while structured networks are specialized. The deeper contrast is this:

- a generic network leaves most structural assumptions implicit;
- a structured network encodes useful assumptions directly into the architecture.

This often leads to models that are easier to train, more parameter-efficient, more data-efficient, and better aligned with the geometry of the task. Architecture is therefore best understood as *inductive bias made structural.*

Core definitions

We now make this language more precise.

Definition 3.1 (Hypothesis class)**.** Let $\mathcal{X}$ be an input space and $\mathcal{Y}$ an output space. A *hypothesis class* is a set

$$\mathcal{H} \subseteq \{f : \mathcal{X} \to \mathcal{Y}\}$$

of candidate input–output maps among which the learning algorithm searches. In parametric models one often writes

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$$

for some parameter space Θ.

Definition 3.2 (Inductive bias)**.** An *inductive bias* is any restriction or preference that makes some hypotheses in $\mathcal{H}$ easier to select than others when only finite data are available. This bias may enter through the choice of $\mathcal{H}$, through regularization, through the optimization procedure, or through the architecture itself.

Definition 3.3 (Equivariance)**.** Let a group G act on the input space $\mathcal{X}$ by maps $T_g^{\mathcal{X}}$ and on the output space $\mathcal{Y}$ by maps $T_g^{\mathcal{Y}}$. A function $f : \mathcal{X} \to \mathcal{Y}$ is *equivariant* with respect to these actions if

$$f(T_g^{\mathcal{X}} x) = T_g^{\mathcal{Y}} f(x) \qquad \text{for all } g \in G,\ x \in \mathcal{X}.$$

Equivariance means that transforming the input transforms the output in a compatible way.

Definition 3.4 (Invariance)**.** Let G act on the input space $\mathcal{X}$. A function $f : \mathcal{X} \to \mathcal{Y}$ is *invariant* under that action if

$$f(T_g^{\mathcal{X}} x) = f(x) \qquad \text{for all } g \in G,\ x \in \mathcal{X}.$$

Invariance means that certain transformations of the input do not change the prediction.

Definition 3.5 (Architectural prior)**.** An *architectural prior* is an inductive bias induced by the computational graph of a model: its allowed connections, local receptive fields, parameter sharing pattern, state-update rule, attention pattern, or symmetry constraints. It specifies which interactions are easy to represent, which transformations are naturally reusable, and which forms of structure are built into the model before training begins.

These definitions clarify an important point. The architecture does not replace learning. Rather, it shapes the *space within which learning takes place.* Training still determines the numerical values of parameters, but the architecture determines the form of dependence that those parameters can express efficiently.

Why structural constraints change the function class

We now formalize the claim that architecture restricts the admissible set of functions. The cleanest way to see this is to compare a dense layer with a constrained one.

Proposition 3.6 (Architectural constraints restrict admissible function classes)**.** *Let $\mathcal{H}_{\text{dense}}$ be a class of functions realized by a dense affine map followed by a fixed nonlinearity:*

$$\mathcal{H}_{\text{dense}} = \{x \mapsto \sigma(Ax + b) : A \in \mathbb{R}^{m\times n},\ b \in \mathbb{R}^m\}.$$

Now impose two kinds of architectural constraints:

1. *a* connectivity mask $M \in \{0,1\}^{m\times n}$*, requiring* $A_{ij} = 0$ *whenever* $M_{ij} = 0$*;*
2. *a* weight-sharing relation $\sim$ *on the admissible indices, requiring* $A_{ij} = A_{i'j'}$ *whenever* $(i,j) \sim (i',j')$*.*

Let $\mathcal{H}_{\text{struct}}$ be the subclass obtained under these constraints. Then

$$\mathcal{H}_{\text{struct}} \subseteq \mathcal{H}_{\text{dense}},$$

and the number of free linear parameters in $\mathcal{H}_{\text{struct}}$ is the number of equivalence classes of allowed connections, which is at most mn and is strictly smaller whenever any connection is forbidden or any two distinct weights are tied.

Proof sketch. Every structured layer is a special case of a dense layer: one may begin with an arbitrary dense matrix A, then set forbidden entries to zero and identify tied entries so that they share a common value. Thus every map in $\mathcal{H}_{\text{struct}}$ also lies in $\mathcal{H}_{\text{dense}}$, proving the inclusion.

To count degrees of freedom, observe that in a dense $m \times n$ matrix each entry may vary independently, giving mn free linear parameters. Under masking, only entries with $M_{ij} = 1$ remain eligible to vary. Under weight sharing, several admissible entries are forced to have identical values. Hence the independent parameters are indexed not by all matrix entries, but by the equivalence classes of admissible connections. If at least one connection is removed or at least one nontrivial tying relation is imposed, the number of free parameters drops strictly below mn. □

This proposition is elementary, but its interpretation is important. Architectural design narrows the effective hypothesis class by declaring some patterns of interaction impossible and others reusable. Convolutional layers do this through locality and weight sharing. Recurrent models do it through repeated state updates across time. Attention layers do it through structured interaction rules among tokens or entities. In each case, the model is not merely made smaller; it is made *structurally biased.*

Locality, symmetry, order, and interaction

Several recurring structural principles guide modern architecture design. They should be thought of as recurring answers to the question, "what regularities should the model be built to exploit?"

Locality. Nearby elements often interact more strongly than distant ones. In images, adjacent pixels are more strongly related than far-separated ones. In speech and time series, nearby moments often carry especially strong local dependence. Architectures that respect locality make it easy to detect local patterns and reuse them across positions.

Symmetry. Sometimes the meaning of a pattern is preserved under a class of transformations. An edge remains an edge even when it appears at a different spatial location. A set-valued prediction should not depend on arbitrary ordering of the elements. Architectures that respect symmetry exploit such repeated structure rather than relearning the same detector independently at every coordinate.

Order. In sequences, arrangement matters. A sentence is not a bag of words, and a waveform is not a bag of amplitudes. Architectures for sequential data must represent not only which elements are present but how earlier ones influence later ones.

Interaction structure. In many domains the key information lies in relations among entities: which nodes are connected, which objects interact, which tokens should communicate, or which constraints couple variables. Architectures for such data should model patterns of interaction directly rather than flattening them into unrelated coordinates.

These themes — locality, symmetry, order, and interaction — recur throughout the chapter. Different architectural families emphasize them differently, but all can be viewed as design responses to structure in data.

A worked comparative example

The point of architecture becomes clearer when the same task is viewed through several model classes. Consider the following prediction problem. We observe a short sentence $w_1, \ldots, w_T$ and wish to predict whether the final token refers back to an earlier entity in the sentence. For instance, in a sentence like

The scientist thanked the assistant because she had finished the analysis,

we would like the model to decide which earlier noun phrase the pronoun "she" most naturally links to. This is a single supervised prediction problem, but different architectures embody very different assumptions about how such a problem should be solved.

Example 3.7 (One task, four architectural viewpoints)**.** Let $x = (w_1, \ldots, w_T)$ be the input sequence and let $y \in \{0, 1\}$ indicate whether the final token refers to a designated earlier entity. The prediction target is the same in every case, but the architecture changes the natural form of representation learning.

1. **MLP viewpoint.** Embed each token, concatenate the embeddings into one long vector, and apply a multilayer perceptron. This is the most generic treatment. The model may succeed, but it must infer order, locality, and long-range interaction from the flattened representation itself. Little structural guidance is built in.
2. **CNN viewpoint.** Apply one-dimensional convolutions along the token axis. The model now assumes that local patterns such as short phrases or nearby syntactic configurations are reusable across positions. It becomes easy to detect motifs like "because she" or "the assistant" wherever they appear, but long-range dependencies may still require deep stacks or larger receptive fields.
3. **RNN viewpoint.** Process the sentence left to right with a recurrent hidden state. The model now assumes that meaning can be accumulated sequentially. Earlier information is compressed into a state that is updated as tokens arrive. This naturally respects order, but it can make long-range interaction difficult if relevant information must survive through many update steps.

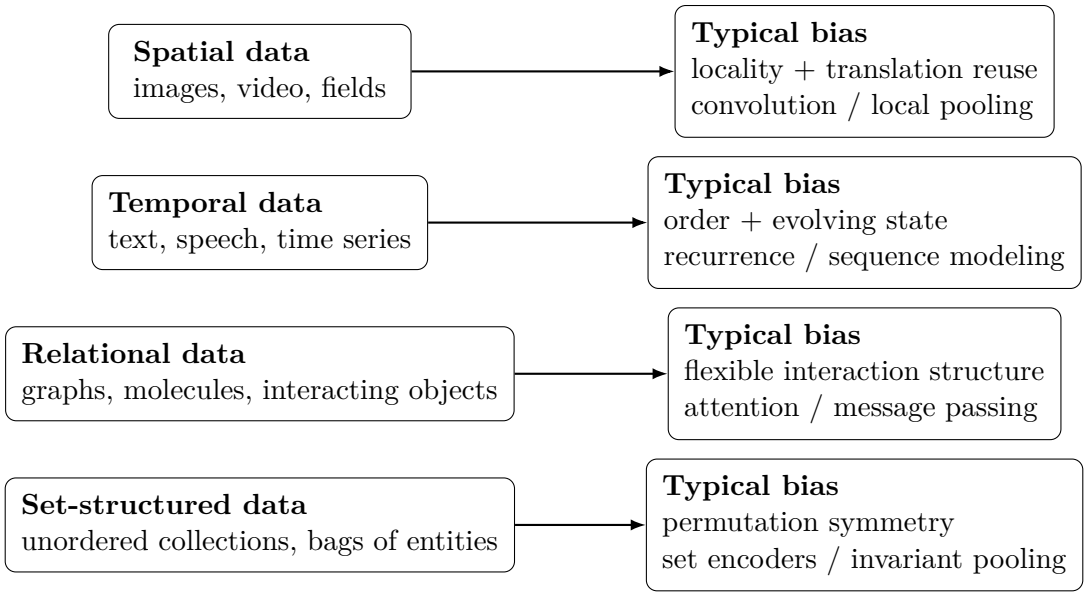


Figure 3.1: A conceptual map from common data structures to architectural biases. The purpose of the figure is not to prescribe a unique model for each domain, but to show how structure in data suggests structure in the hypothesis class.

4. **Attention viewpoint.** Compute context-dependent interactions among tokens. The final token may directly compare itself with earlier noun phrases, verbs, or clauses without passing only through a single hidden-state bottleneck. This architecture assumes that relevant dependencies are best modeled by flexible content-based interaction rather than by fixed locality or a single evolving state.

The task has not changed. What changed is the architectural prior: what kinds of regularities are easy to express, which dependencies are short or long, and whether order, locality, or flexible interaction are emphasized. This is exactly why architecture matters. The same input–output map may be representable in many ways, but some representations are far more natural, efficient, and learnable than others.

A map from data structure to architectural bias

It is helpful to summarize the relationship between common data structures and the biases that architectures typically encode. Figure 3.1 is not a theorem, but a design map. It shows how domain structure suggests architectural choices.

The figure should not be read dogmatically. Transformers are now used in vision. Convolutions can be used on sequences. Attention appears inside graph models. Set encoders may be combined with recurrent or convolutional components. The lesson is not that each domain has exactly one correct architecture. The lesson is that successful architectures usually reflect identifiable structural assumptions about the data.

Why the same neural-network template is not ideal for every domain

A generic multilayer perceptron can often approximate the target mapping after sufficient vectorization, depth, and data. That is not the main issue. The real issue is whether the chosen architecture

aligns well enough with the problem for learning to be efficient and stable. Several considerations matter.

Parameter efficiency. If the model ignores known structure, it may need many more parameters to express patterns that could otherwise be represented compactly. Convolutional weight sharing is the clearest example. A detector that should work at every spatial position need not be relearned independently at every coordinate.

Data efficiency. A poorly matched architecture may require much more data because it must infer regularities that could have been built into the model from the start. In finite-data regimes this matters greatly. The architectural prior can guide the learner toward plausible solutions before the sample size becomes enormous.

Optimization difficulty. When the architecture reflects domain structure, gradient-based learning often becomes easier. The model no longer has to discover every useful interaction from a flat representation. Instead, it can search within a hypothesis class already shaped toward the right kinds of dependencies.

Generalization quality. A well-matched bias can steer learning toward solutions that reflect stable regularities in the world rather than accidental patterns in the sample. This is one reason architectural design belongs conceptually to learning theory rather than merely to software engineering.

Thus the question is not simply whether a generic network is expressive enough. It often is. The real question is whether its structure matches the geometry of the task well enough that learning becomes practical.

Architecture as the successor to hand-crafted structural assumptions

The modern role of architecture becomes especially clear when compared with classical feature engineering. In earlier machine-learning practice, one often injected domain knowledge through hand-crafted features or kernel choice. For images one might design edge histograms or texture descriptors. For text one might build n-gram counts or syntactic indicators. For time series one might create lag features or Fourier summaries. These strategies encoded structure, but they did so in a relatively shallow and externally specified way.

Deep learning changes this arrangement. The structural assumption is no longer expressed only through a fixed feature map placed before learning. Instead, it is embedded into the architecture of the model, which then learns internal representations consistent with that structural bias. The assumption is still present, but it is now expressed as *learnable computational structure* rather than as a fully specified handcrafted representation. That is why architecture should be regarded as the deep-learning successor to classical structural design.

This perspective also clarifies the unity of the chapter. Convolutional networks are not simply "vision models." Recurrent networks are not simply "sequence models." Attention is not merely a newer mechanism that replaced older ones. Each is a design answer to the same underlying question: what regularities should the model be built to exploit?

The broader lesson

The deepest lesson of this section is that architecture is part of learning theory, not merely part of implementation. To choose an architecture is to decide which regularities should be easy for the model to express and which ones should be hard. That is a choice about inductive bias. Because learning from finite data is impossible without inductive bias, architecture is one of the central mechanisms by which deep learning becomes effective.

A generic multilayer network says:

> learn representations, but assume relatively little about how the world is organized.

A structured architecture says:

> learn representations, but do so in a way that reflects known forms of spatial, temporal, or relational organization.

That difference is the conceptual doorway into the rest of the chapter. The sections that follow should therefore be read as a study of architectural bias in action: convolutional models exploit locality and translation reuse; recurrent models exploit temporal order and evolving state; attention-based models exploit flexible interaction patterns; and transformers scale these ideas into powerful general-purpose sequence architectures.

Notes and references

The language of inductive bias comes from classical learning theory and appears in many forms across the statistical-learning literature. Useful background includes Mitchell's discussions of bias in machine learning, Vapnik's treatment of hypothesis classes and statistical learning, and standard modern texts such as Bishop or Hastie, Tibshirani, and Friedman.

The modern deep-learning perspective in which representation learning replaces fixed feature engineering is surveyed clearly by Goodfellow, Bengio, and Courville and by LeCun, Bengio, and Hinton. For the especially important idea that architectural design encodes domain structure, a particularly relevant reference is Battaglia et al. under the heading of *relational inductive biases*. Group-equivariant and symmetry-based viewpoints are developed further by Cohen and Welling and by Bronstein et al. The later sections of this chapter will connect these ideas to convolutional networks, recurrent networks, attention mechanisms, and transformers in the settings made famous by LeCun et al., Bahdanau et al., and Vaswani et al.

Historically, the crucial shift is this: classical methods often expressed structural assumptions through hand-designed representations, whereas deep learning increasingly expresses them through architectures whose internal representations are learned. That shift is one of the defining conceptual moves of modern machine learning.

3.2 Locality, Weight Sharing, and the Logic of Convolution

Section 3.1 argued that architecture matters because it encodes structural assumptions directly into the hypothesis class. Convolution is one of the clearest and most influential examples of that principle. It does not merely provide a convenient implementation for image processing. It expresses, in mathematically explicit form, three linked assumptions: local interactions matter, similar local patterns may recur across position, and larger-scale structure may be built hierarchically from smaller-scale features.

This section develops those ideas in a more formal way. We will define discrete convolution in one and two dimensions, show that it is a linear operator with highly structured matrix form, prove its translation-equivariance property, compare its parameter count with that of dense layers, and derive how receptive fields grow across stacked layers. The broader lesson is that convolution achieves statistical efficiency by restricting the model to a structured subclass of linear maps before nonlinearity is even applied.

Discrete convolution in one and two dimensions

We begin with formal definitions.

Definition 3.8 (One-dimensional discrete convolution)**.** Let $x \in \mathbb{R}^n$ be a one-dimensional signal and let $w \in \mathbb{R}^k$ be a kernel of width k. For indices at which the expression is defined, the discrete convolution of x by w is the signal $y = w * x$ given by

$$y_i = (w * x)_i = \sum_{t=0}^{k-1} w_t \, x_{i-t}.$$

Depending on the boundary convention, one may use zero padding, valid convolution, circular indexing, or other extensions.

Definition 3.9 (Two-dimensional discrete convolution)**.** Let $X \in \mathbb{R}^{H \times W}$ be an image and let $K \in \mathbb{R}^{r \times s}$ be a two-dimensional kernel. For indices at which the expression is defined, the discrete convolution of X by K is the array $Y = K * X$ with entries

$$Y_{i,j} = (K * X)_{i,j} = \sum_{u=0}^{r-1} \sum_{v=0}^{s-1} K_{u,v} \, X_{i-u,j-v}.$$

Again, the precise output shape depends on the boundary convention and any padding rule.

In practice, deep-learning libraries sometimes implement cross-correlation rather than strict mathematical convolution, since the kernel may be applied without reversal. For the structural questions that matter here — locality, parameter sharing, and equivariance — that distinction is not essential. The key point is that each output coordinate depends only on a local neighborhood of the input, and that the same coefficients are reused at every position.

Convolution as a structured linear operator

Convolution is often introduced as a sliding-window procedure. But for mathematical analysis it is better understood as a special kind of linear map.

Proposition 3.10 (Convolution is a linear operator with Toeplitz structure)**.** *For fixed kernel $w \in \mathbb{R}^k$, the map*

$$x \longmapsto w * x$$

from signals to signals is linear. Under standard zero-padding or valid-convolution conventions, its matrix representation in one dimension is Toeplitz up to boundary effects. Under circular boundary conditions it is circulant. In two dimensions, the corresponding matrix form is block Toeplitz with Toeplitz blocks.

Proof. Linearity is immediate from the definition. For scalars a, b and signals x, z,

$$w * (ax + bz) = \sum_t w_t(ax_{i-t} + bz_{i-t}) = a \sum_t w_t x_{i-t} + b \sum_t w_t z_{i-t} = a(w * x) + b(w * z).$$

To obtain the matrix form in one dimension, write the convolution as multiplication by a matrix T_w whose ith row contains the kernel coefficients aligned with the input coordinates contributing to output position i. Because shifting the output index shifts the alignment pattern by one step, the entries along each diagonal are equal. That is exactly the Toeplitz property. Under circular indexing the diagonals wrap around, producing a circulant matrix.

In two dimensions, vectorizing the image turns convolution into multiplication by a matrix whose blocks correspond to horizontal shifts and whose entries inside each block correspond to vertical shifts. The repeated shift pattern yields a block-Toeplitz matrix with Toeplitz blocks. □

This proposition is conceptually important. A dense layer is also linear before the activation function, but it allows every output coordinate to depend on every input coordinate with an independent coefficient. Convolution is more restrictive. It selects a very special subclass of linear maps: those that are sparse because of locality and repetitive because of weight sharing. That restriction is not a defect. It is the source of the model's inductive bias and one of the main reasons it can learn effectively from realistic sample sizes.

Locality and weight sharing as structural bias

Two ideas work together in convolution.

Locality. Each output unit sees only a small neighborhood of the input. In a one-dimensional kernel of width k, output coordinate y_i depends on at most k nearby coordinates of x. In a two-dimensional $r \times s$ kernel, output location (i, j) depends only on an $r \times s$ patch. This forbids arbitrary long-range interaction at a single layer.

Weight sharing. The same kernel coefficients are reused across positions. A local edge detector, texture detector, or short phrase detector does not need separate parameters for every possible location. Once the architecture assumes that the same pattern may occur anywhere, it becomes natural to tie those parameters together.

These two constraints jointly explain why convolution is statistically efficient. The model does not waste parameters on learning a separate detector for every coordinate, and it does not try to model every possible pairwise interaction at once. Instead, it begins with a simpler structural prior and then compounds that prior across depth.

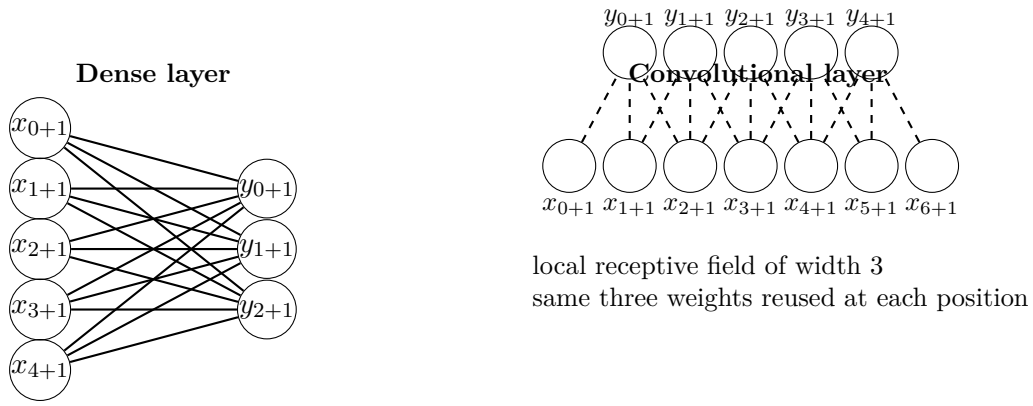


Figure 3.2: Locality and weight sharing contrasted with dense connectivity. A dense layer permits every output to depend on every input with independent coefficients. A convolutional layer restricts each output to a local neighborhood and reuses the same kernel across positions.

Translation equivariance

One of the most mathematically attractive properties of convolution is translation equivariance. Intuitively, if the input pattern shifts, the feature map shifts in the same way. The detector therefore responds consistently across location.

To state this carefully, define the translation operator T_a on one-dimensional signals by

$$(T_a x)_i = x_{i-a}.$$

A similar definition applies in two dimensions by shifting both coordinates.

Theorem 3.11 (Translation equivariance of discrete convolution)**.** *Let w be a fixed one-dimensional convolution kernel, and let T_a denote translation by offset a on signals indexed so that the operations are well-defined under a common boundary convention. Then*

$$w * (T_a x) = T_a(w * x)$$

for every signal x. Thus discrete convolution is translation equivariant. The same statement holds in two dimensions with respect to spatial translations.

Proof. Compute the ith output coordinate:

$$(w * (T_a x))_i = \sum_t w_t (T_a x)_{i-t} = \sum_t w_t x_{i-t-a}.$$

On the other hand,

$$(T_a(w * x))_i = (w * x)_{i-a} = \sum_t w_t x_{(i-a)-t} = \sum_t w_t x_{i-t-a}.$$

The two expressions are identical for every i, so

$$w * (T_a x) = T_a(w * x).$$

The two-dimensional proof is the same, with the translation acting on both spatial indices. □

This theorem helps explain why convolution is so natural for images and other spatial data. If a feature such as an edge, corner, or local texture moves in the image, the representation produced by the convolutional layer moves with it rather than changing unpredictably. The model therefore does not need to relearn essentially the same detector at each possible location.

It is worth stressing the distinction between equivariance and invariance. A convolutional feature map is typically equivariant: shifting the input shifts the feature map. A classification system may become approximately invariant only after subsequent aggregation steps such as pooling, striding, or global averaging reduce sensitivity to exact position.

Parameter efficiency: dense versus convolutional layers

The statistical advantage of convolution becomes especially visible when one counts parameters. Suppose first that an image $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ is flattened into a vector and passed into a dense layer with M output units. Then the weight matrix has

$$M \cdot HWC_{\text{in}}$$

free parameters, plus M bias terms. Every output unit has its own full set of coefficients.

Now consider a convolutional layer with kernel size $r \times s$, C_{in} input channels, and C_{out} output channels. Its parameter count is

$$rs\, C_{\text{in}} C_{\text{out}}$$

for the kernel coefficients, plus C_{out} bias terms. Crucially, this count is independent of the spatial dimensions H and W. The same filter bank is reused across all locations.

For large images the contrast can be dramatic. If $H = W = 128$, $C_{\text{in}} = 3$, and one wants $C_{\text{out}} = 64$ output channels with a 3×3 kernel, then a convolutional layer uses

$$3 \cdot 3 \cdot 3 \cdot 64 = 1728$$

weights, while even a modest dense layer producing one scalar at every spatial location would require a number of parameters proportional to 128^2 times the number of outputs. The exact comparison depends on the output shape, but the principle is universal: convolution decouples parameter count from spatial extent because it assumes translation reuse.

Proposition 3.12 (Locality and weight sharing reduce degrees of freedom)**.** *Consider a single linear layer acting on an input image with spatial size $H \times W$ and C_{in} channels, producing C_{out} output channels. A dense layer that treats each output location independently requires a number of free linear parameters that scales with HW. A convolutional layer with fixed kernel size $r \times s$ requires only $rs\, C_{\text{in}} C_{\text{out}}$ parameters, independent of H and W. Therefore locality and weight sharing reduce the number of free degrees of freedom whenever the spatial domain contains more than one receptive field position.*

Proof sketch. The dense layer assigns an independent coefficient to each admissible input–output pair, so the number of parameters scales with the number of output positions times the number of input coordinates influencing each output. A convolutional layer instead uses the same $r \times s$ local coefficient tensor at every output position. Thus once the kernel size and channel counts are fixed, spatial replication creates additional applications of the same parameters rather than new parameters. For any nontrivial spatial domain, that yields fewer independent degrees of freedom than the corresponding untied dense map. □

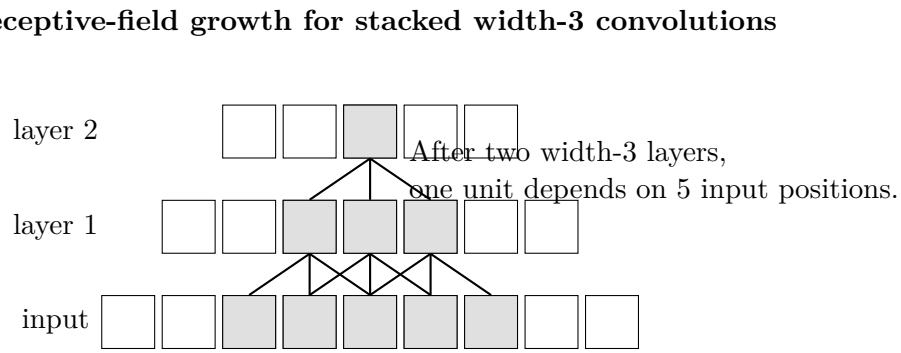


Figure 3.3: Stacking local convolutions enlarges the receptive field. Although each individual layer is local, depth allows later units to integrate information over larger regions of the original input.

This is a particularly clear instance of the general principle from Section 3.1: architecture restricts the admissible function class. Convolution does not search among all linear maps from images to feature maps. It searches among those compatible with locality and translation reuse. That restriction lowers parameter count, reduces variance, and often improves sample efficiency when the assumptions are well matched to the data.

Receptive fields and hierarchical growth

A single convolutional layer is local. But deep convolutional networks are not merely local detectors. Their effective scope grows with depth. This is captured by the idea of the receptive field.

Definition 3.13 (Receptive field)**.** The *receptive field* of a unit in a layered architecture is the subset of input coordinates that can influence that unit. For a convolutional layer, the immediate receptive field is determined by the kernel support. For a stack of layers, the receptive field is obtained by composing these local dependencies across depth.

For stride-1 convolutions with kernel widths $k_1, \ldots, k_L$ and no dilation, the receptive field size after L layers in one dimension is

$$R_L = 1 + \sum_{\ell=1}^{L} (k_\ell - 1).$$

In particular, if all layers use the same kernel width k, then

$$R_L = 1 + L(k-1).$$

Thus repeated local operations gradually create larger effective context.

More generally, with strides $s_1, \ldots, s_L$, one obtains the recursion

$$R_1 = k_1, \qquad R_\ell = R_{\ell-1} + (k_\ell - 1) \prod_{j=1}^{\ell-1} s_j, \qquad \ell \geq 2.$$

This expression shows how downsampling causes later layers to aggregate information over increasingly large regions of the original input.

This is the heart of the hierarchical story. Early layers may detect edges, corners, short motifs, or small textures. Later layers combine those local responses into larger parts, configurations, and semantic structures. Convolutional networks therefore achieve nonlocal expressivity not by abandoning locality, but by composing it repeatedly.

A worked one-dimensional example

A small example makes the structure concrete.

Example 3.14 (Convolution on a tiny one-dimensional signal)**.** Consider the signal

$$x = (2, 1, 0, 3, 1)$$

and the kernel

$$w = (1, -1).$$

Using valid convolution, the output has length 4 and is given by

$$(w * x)_i = x_i - x_{i+1}.$$

Hence

$$w * x = (2 - 1,\ 1 - 0,\ 0 - 3,\ 3 - 1) = (1, 1, -3, 2).$$

This kernel acts as a simple local difference detector. It responds strongly where adjacent entries change sharply.

Now shift the input by one position to obtain

$$T_1 x = (0, 2, 1, 0, 3, 1)$$

under a suitable padding convention. Convolving again with the same kernel produces the same pattern of local differences, shifted accordingly. This is exactly the translation-equivariance property proved above.

Even this trivial example illustrates the key ideas. The output at each position depends only on a local pair of inputs. The same detector is applied everywhere. And the detector moves consistently with shifts in the signal. A dense layer could represent the same mapping, but it would not encode any of these structural facts directly.

Pooling and the move toward coarse invariance

Convolutional architectures often interleave convolution with pooling or downsampling. Pooling does not preserve full equivariance in the exact same form, but it helps aggregate nearby responses into coarser summaries. This can make the representation less sensitive to small input translations and enlarge the effective receptive field of later layers.

Pooling should be interpreted carefully. It is not magic invariance. Rather, it is one mechanism by which exact local location is deemphasized in favor of more stable coarse structure. Modern architectures sometimes replace explicit pooling by strided convolutions or attention-like aggregation, but the underlying goal remains familiar: build increasingly global representations from local evidence.

Why structure yields statistical efficiency

We may now state the larger lesson explicitly. Convolution works well when the world approximately obeys the assumptions that convolution encodes. If local patterns recur across position, then weight sharing is sensible. If nearby coordinates interact more strongly than distant ones, then local connectivity is sensible. If larger structures are built from smaller ones, then stacked local layers are sensible.

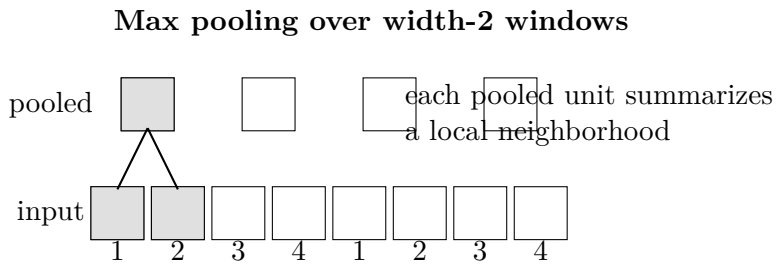


Figure 3.4: Pooling aggregates nearby activations into a coarser representation. This reduces spatial resolution, increases the effective scale of later features, and can create partial robustness to small translations.

When those assumptions are approximately correct, the reduction in degrees of freedom is not merely computationally convenient. It improves learning. The architecture no longer wastes sample complexity on rediscovering the same local detector at every location. It no longer spends parameters expressing arbitrary global couplings at the first layer. Instead, it searches within a function class aligned with the geometry of the task. That is exactly what one hopes from inductive bias.

This point also clarifies the limits of convolution. If the task depends primarily on nonlocal interactions, arbitrary permutations, or long-range relations that are not naturally built from local patterns, then convolution alone may be too restrictive. The power of convolution comes from matching the structure of the domain. Its success is therefore not universal; it is conditional on the usefulness of locality and translation reuse.

Notes and references

The mathematical view of convolution as a structured linear operator connects deep learning to classical signal processing and harmonic analysis. Standard signal-processing texts develop discrete convolution, Toeplitz structure, and the convolution theorem in more classical language. In the deep-learning literature, the central role of convolutional architectures in vision was established by LeCun and collaborators and later synthesized in modern textbooks such as Goodfellow, Bengio, and Courville.

The importance of translation equivariance and local receptive fields is now understood as part of a broader symmetry-based view of architecture. That perspective connects naturally to later developments in group-equivariant neural networks and geometric deep learning. For the purposes of this chapter, however, the main message is already visible in the elementary formulas of this section: convolution is a deliberately restricted hypothesis class, and that restriction is precisely what makes it powerful on the right domains.

3.3 Multiscale Representation and the Fourier Viewpoint

Section 3.2 emphasized the spatial side of convolution: it is local, it shares weights across position, and it builds larger receptive fields by composition. That viewpoint is indispensable, but it is not the whole story. Convolution also belongs to a classical mathematical tradition in signal processing and harmonic analysis. The same operator that acts locally in the spatial domain acts diagonally in the Fourier domain. This second viewpoint helps explain what convolutional filters do, why some filters smooth while others sharpen or detect edges, and why stacked convolutional layers may be interpreted as multiscale operators rather than merely as engineering devices.

The central thesis of this section is that convolutional networks should be understood simultaneously in two linked ways. In the spatial domain, a filter detects local configurations by sliding over the input. In the frequency domain, the same filter amplifies some Fourier modes and suppresses others. A mathematically mature understanding of convolution comes from holding both pictures at once.

Discrete Fourier transform, filtering, and frequency response

We begin with the finite-dimensional setting, where everything can be written explicitly. Because the cleanest algebra is obtained under circular boundary conditions, we state the main theorem for circular convolution. This is not a practical limitation. It isolates the core principle that Fourier analysis diagonalizes translation-invariant linear operators.

Definition 3.15 (Discrete Fourier transform)**.** Let $x = (x_0, \dots, x_{N-1}) \in \mathbb{C}^N$ and let

$$\omega_N = e^{-2\pi i/N}.$$

The *discrete Fourier transform* (DFT) of x is the vector $\widehat{x} \in \mathbb{C}^N$ with entries

$$\widehat{x}_k = \sum_{n=0}^{N-1} x_n \, \omega_N^{kn}, \qquad k = 0, \dots, N-1.$$

The inverse transform is given by

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} \widehat{x}_k \, \omega_N^{-kn}, \qquad n = 0, \dots, N-1.$$

Definition 3.16 (Circular convolution)**.** Let $x, w \in \mathbb{C}^N$. Their *circular convolution* is the vector $y = w \circledast x \in \mathbb{C}^N$ defined by

$$y_n = (w \circledast x)_n = \sum_{m=0}^{N-1} w_m \, x_{(n-m) \bmod N}, \qquad n = 0, \dots, N-1.$$

Definition 3.17 (Filtering and frequency response)**.** A *filter* is a linear operator that transforms a signal into another signal, typically by emphasizing some patterns and suppressing others. When the filter is given by convolution with a kernel w, its *frequency response* is the Fourier transform $\widehat{w}$. The magnitude $|\widehat{w}_k|$ tells how strongly the filter passes or attenuates the kth Fourier mode, while the phase of $\widehat{w}_k$ determines how that mode is shifted.

These definitions already suggest why Fourier analysis is useful. A signal may be complicated when viewed as a list of spatial values, but much simpler when decomposed into oscillatory components. A convolutional kernel may look like a short local stencil in space, yet in frequency it becomes a mode selector. Low-pass filters preserve slowly varying components. High-pass filters emphasize rapid variation. Band-pass filters isolate intermediate ranges.

The discrete convolution theorem

The key result is the finite-dimensional convolution theorem. It is one of the most important identities linking deep learning to classical signal processing.

Theorem 3.18 (Discrete convolution theorem)**.** *Let $x, w \in \mathbb{C}^N$, and let $\widehat{x}$ and $\widehat{w}$ denote their discrete Fourier transforms. Then the DFT of their circular convolution satisfies*

$$\widehat{w \circledast x}_k = \widehat{w}_k \, \widehat{x}_k, \qquad k = 0, \ldots, N-1.$$

Equivalently,

$$\mathcal{F}(w \circledast x) = \mathcal{F}(w) \odot \mathcal{F}(x),$$

where $\odot$ denotes pointwise multiplication.

Proof. Fix k. By definition,

$$\widehat{w \circledast x}_k = \sum_{n=0}^{N-1} (w \circledast x)_n \, \omega_N^{kn} = \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} w_m \, x_{(n-m) \bmod N} \, \omega_N^{kn}.$$

Interchange the order of summation:

$$\widehat{w \circledast x}_k = \sum_{m=0}^{N-1} w_m \sum_{n=0}^{N-1} x_{(n-m) \bmod N} \, \omega_N^{kn}.$$

Now make the change of variables $r = (n-m) \bmod N$. Then $n \equiv r + m \pmod{N}$, and because the summation runs over a complete residue system, we obtain

$$\sum_{n=0}^{N-1} x_{(n-m) \bmod N} \, \omega_N^{kn} = \sum_{r=0}^{N-1} x_r \, \omega_N^{k(r+m)} = \omega_N^{km} \sum_{r=0}^{N-1} x_r \, \omega_N^{kr}.$$

Therefore

$$\widehat{w \circledast x}_k = \sum_{m=0}^{N-1} w_m \, \omega_N^{km} \sum_{r=0}^{N-1} x_r \, \omega_N^{kr} = \widehat{w}_k \, \widehat{x}_k.$$

This proves the claim. □

Corollary 3.19 (Fourier modes diagonalize circular convolution)**.** *For each frequency index k, the Fourier mode*

$$\varphi_n^{(k)} = \omega_N^{-kn}$$

is an eigenvector of the convolution operator $x \mapsto w \circledast x$, with eigenvalue $\widehat{w}_k$.

Proof. Apply the inverse DFT to the theorem. Because the Fourier basis vectors are mapped to scalar multiples of themselves, each Fourier mode is an eigenvector and the scalar is exactly $\widehat{w}_k$. □

This result is worth interpreting carefully. In the spatial domain, convolution mixes nearby coordinates. In the Fourier domain, it does not mix frequencies at all: each mode is simply scaled by a complex number. That is why the frequency response is so informative. It tells us exactly how the filter treats each oscillatory component.

Remark 3.20 (Why circular convolution is the clean setting)*.* In practical neural networks one often uses valid convolution, same-padding, or reflected boundaries rather than circular boundaries. Then the matrix representation is Toeplitz or block Toeplitz rather than perfectly circulant, and the DFT no longer diagonalizes the operator exactly. Nevertheless, the circular case captures the essential principle: translation-invariant local operators are naturally analyzed in the frequency domain. The Fourier picture therefore remains pedagogically and mathematically valuable even when the implementation is not literally circular.

Low-pass, high-pass, and edge-detecting filters

The convolution theorem immediately explains a basic taxonomy of filters.

Smoothing filters. A smoothing kernel averages nearby values. In frequency language, this usually means that low frequencies are preserved while high frequencies are attenuated. Such filters are therefore called *low-pass*.

Difference and edge filters. A difference kernel responds to rapid local change. In frequency language, slowly varying components are suppressed and rapid oscillations are emphasized. Such filters are therefore *high-pass* or edge-emphasizing.

Band-pass behavior. Some filters emphasize an intermediate range of frequencies. In image analysis, these may correspond to texture-sensitive or scale-selective responses. In deeper networks, compositions of nonlinear layers make the story richer, but the linear frequency intuition remains useful.

A simple example makes this concrete. Consider the length-3 kernels

$$h_{\text{smooth}} = \frac{1}{3}(1,1,1), \qquad h_{\text{edge}} = (1,0,-1), \qquad h_{\text{high}} = (1,-2,1).$$

The first averages neighboring values and therefore suppresses rapid oscillation. The second computes a local first difference and highlights transitions. The third is a discrete second-difference operator and reacts even more strongly to curvature or sharp change.

This is one reason first-layer convolutional kernels in trained networks often look interpretable. Some resemble edge detectors, some resemble oriented band-pass filters, and some resemble coarse smoothing patterns. Even when later layers become much harder to interpret directly, the earliest filters often expose the basic local-frequency tradeoff of the architecture.

A worked example: smoothing and edge detection on a tiny signal

Let us examine a very small one-dimensional signal:

$$x = (0,1,2,1,0,-1,-2,-1).$$

This signal varies slowly at large scale, but it also contains transitions in sign and slope. We compare two filters.

Example 3.21 (Smoothing versus edge detection)**.** Using valid convolution, let

$$h_{\text{smooth}} = \frac{1}{3}(1,1,1), \qquad h_{\text{edge}} = (1,0,-1).$$

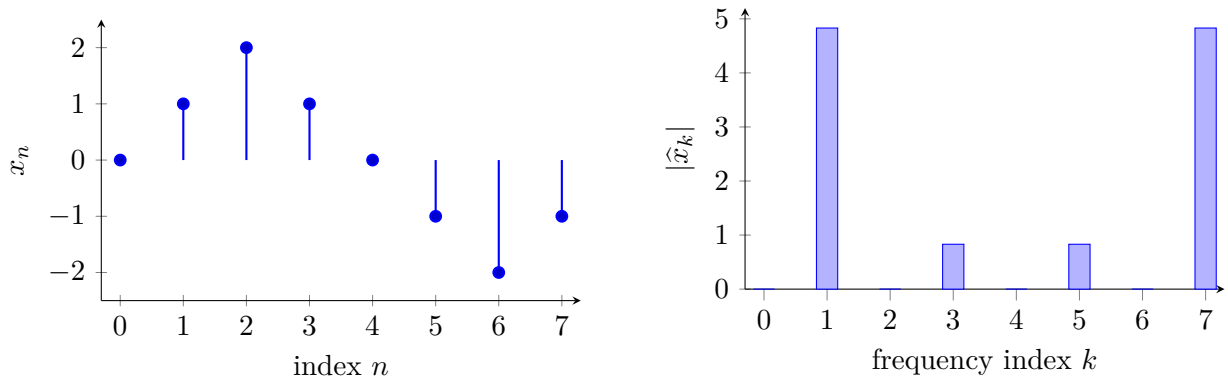
Then

$$h_{\text{smooth}} * x = \left(1, \frac{4}{3}, 1, 0, -1, -\frac{4}{3}\right),$$

and

$$h_{\text{edge}} * x = (-2,0,2,2,2,0).$$

The smoothing filter averages nearby entries and removes some sharp local variation. The edge detector responds where the signal changes strongly across a short window. In particular, it is small where the signal is nearly linear over three consecutive points and large where the local slope changes sign or magnitude.



(a) Spatial-domain signal.

(b) Magnitude of the DFT.

Figure 3.5: The same finite signal viewed in two ways. In the spatial domain one sees local amplitudes and sign changes. In the frequency domain one sees which Fourier modes carry most of the energy. Both descriptions refer to the same object, and convolution may be interpreted in either language.

The same example can be read in the frequency domain. The original signal contains more than one Fourier component. Applying h_{smooth} damps the higher-frequency part, while applying h_{edge} suppresses very low frequencies and emphasizes short-scale variation. Thus the two filters behave differently not because one is "more intelligent" than the other, but because their frequency responses differ.

Figure 3.5 is pedagogically important. The left panel displays the signal as a list of local values. The right panel displays the magnitude of its Fourier coefficients. Neither picture is more fundamental than the other. The spatial picture is better for understanding locality and pattern detection. The spectral picture is better for understanding smoothing, oscillation, and scale selection. Convolution lives exactly at the interface between them.

Learned filters in spatial and frequency language

When one says that a convolutional network *learns filters*, there are two equally legitimate interpretations.

Spatial interpretation. A learned kernel is a local template. It may respond to an edge, a corner, a short motif, or a texture. This is the language in which feature detectors are usually described.

Frequency interpretation. The same learned kernel has a frequency response. It may suppress high-frequency noise, emphasize certain orientations or scales, or selectively pass oscillatory components. This is the language in which filters are classically described in signal processing.

These two descriptions are linked by the convolution theorem. A kernel that is sharply localized in space often has a broad frequency response, while a filter highly selective in frequency tends to have more extended spatial support. This tradeoff is a discrete manifestation of a more general uncertainty principle familiar from harmonic analysis. Deep learning does not escape that mathematics; it inherits it.

This viewpoint clarifies why the phrase "learned feature extractor" is not metaphorical. A convolutional layer literally learns a family of structured linear operators before the nonlinearity

is applied. Those operators are shaped both by local geometry and by frequency selectivity. The nonlinearity and depth then compose these operators into richer representations, but the first step already has precise mathematical content.

Multiscale representation and hierarchical processing

The Fourier viewpoint also helps explain multiscale structure. Natural signals and images often contain meaningful organization at more than one scale. Edges and textures are local. Shapes and object parts are larger. Global arrangement is larger still. A useful architecture should therefore not only detect features, but also organize them across scale.

Definition 3.22 (Multiscale representation)**.** A *multiscale representation* of a signal is a family of features that capture structure at different effective spatial or temporal scales. In layered convolutional models, scale changes arise through repeated local filtering, nonlinear transformation, and downsampling or pooling.

If a single width-k filter sees only a neighborhood of size k, then stacked filters and downsampling make progressively larger contexts accessible. Section 3.2 already formalized this through receptive-field growth. The Fourier picture adds another interpretation: coarser layers may be viewed as increasingly selective to larger-scale, slower-varying structure, while fine-scale layers remain sensitive to local variation. The resulting hierarchy is one reason convolutional networks work so well on vision tasks. They do not merely detect local patterns. They convert local patterns into increasingly global and semantically useful coordinates.

This perspective also guards against a common misunderstanding. It is tempting to say that early layers learn edges and later layers learn objects as if this were an absolute law. The truth is subtler. What the network learns depends on the task, the data, and the training objective. But the architecture does make a general multiscale progression natural: small-support filters and shallow layers are biased toward local and often higher-frequency structure, while deeper and coarser layers integrate broader context.

Why this viewpoint matters for deep learning

The main pedagogical value of the Fourier and multiscale viewpoint is that it prevents convolution from appearing ad hoc. Convolutional networks are not successful merely because sliding windows happened to work in practice. They succeed because they instantiate a structured class of linear operators that matches the geometry and scale structure of many natural signals.

For mathematics students, this is especially important. Without the Fourier viewpoint, convolution may look like a useful coding trick. With it, convolution becomes recognizable as a concrete example of a translation-structured operator, linked to Toeplitz matrices, spectral analysis, filter theory, and multiscale decomposition. That is a much deeper and more satisfying picture.

What to remember. Convolution has two mathematically linked interpretations. In the spatial domain it is a local, translation-structured operator. In the Fourier domain it becomes pointwise multiplication by a frequency response. This explains smoothing, edge detection, and multiscale processing in one unified language.

Common confusion. The Fourier viewpoint does not replace the spatial viewpoint. It complements it. A convolutional kernel is not either a local detector or a frequency filter. It is both. The two descriptions are equivalent ways of understanding the same linear operation.

Exercises

Exercise 3.23. Let $x \in \mathbb{C}^N$ and let $\delta^{(a)}$ denote the signal that is 1 at index a and 0 elsewhere. Compute $\delta^{(a)} \circledast x$ and explain why this corresponds to circular translation. How does this connect with translation equivariance?

Exercise 3.24. Compute the DFT of the kernel $h = (1, -1)$ for $N = 4$ by zero-padding h to length 4. At which frequency indices does this filter have the largest magnitude response? Explain why that matches the intuition that $(1, -1)$ is a difference detector.

Exercise 3.25. Let $h_{\text{avg}} = \frac{1}{3}(1, 1, 1)$. Show that its frequency response has largest magnitude near zero frequency. Why does this justify calling it a low-pass filter?

Exercise 3.26. A signal is passed through two consecutive convolutions with kernels $w^{(1)}$ and $w^{(2)}$. Use the convolution theorem to describe the combined frequency response in terms of $\widehat{w^{(1)}}$ and $\widehat{w^{(2)}}$. How does this help explain the effect of stacking convolutional layers?

Exercise 3.27. In practice many neural networks use zero padding rather than circular convolution. Explain why the discrete convolution theorem is no longer exact in the same form. Why is the Fourier viewpoint still useful anyway?

Notes and references

The discrete Fourier transform and convolution theorem are standard topics in harmonic analysis and signal processing. Classical treatments may be found in texts on Fourier analysis, linear systems, and digital signal processing. Within machine learning, these ideas matter because convolutional networks inherit the mathematics of translation-structured filtering even when presented in modern optimization language. LeCun and collaborators established the importance of convolutional architectures for vision, while later deep-learning texts such as Goodfellow, Bengio, and Courville gave broad syntheses of their role in representation learning.

The multiscale viewpoint connects naturally to wavelets, scale-space methods, and other hierarchical signal representations, although modern convolutional networks are learned rather than hand-designed. That contrast is one of the main conceptual themes of this chapter: deep learning keeps the structural assumptions of classical signal processing — locality, translation structure, multiscale organization — but learns the filters from data rather than specifying them in advance.

3.4 Sequential Data and Recurrent Computation

Not all data are naturally organized on a spatial grid. Many important problems involve sequences: text, speech, time series, event streams, trajectories, and controlled interactions with an environment. Such data carry a form of structure different from the spatial locality emphasized in convolutional models. The crucial regularity is *order in time*. An observation at one step may change the meaning of what comes later, and an action taken now may influence all future states. A suitable architecture must therefore process information *sequentially* while carrying forward a summary of the past.

This is the central idea of recurrence. A recurrent model does not treat a sequence merely as a long vector. Instead, it updates an internal state as new elements arrive. That state plays the role of memory, context, and computational workspace. The purpose of this section is not only to define recurrent networks, but also to explain why they are mathematically delicate to train. The same architectural device that gives them temporal structure — repeated application of one transition map — also produces long products of Jacobians in the gradient. That is the origin of vanishing and exploding gradients.

Sequential structure and the state-space viewpoint

A sequence of length T is an ordered tuple

$$x_{1:T} = (x_1, x_2, \ldots, x_T),$$

where each x_t belongs to some input space $\mathcal{X}$. The ordered nature of the tuple matters. In language, reordering words changes meaning. In time series, the future depends on the past rather than vice versa. In control, the choice made at time t alters the state from which future decisions are taken.

A natural mathematical language for such problems is the language of *state-space evolution*. One introduces a hidden state $h_t \in \mathcal{H}$ that summarizes, in a compressed way, the information from the past that is relevant for future computation. The model is then built from two ingredients:

- a *recurrence map* that updates the hidden state when a new input arrives;
- an *output map* that converts the hidden state into a prediction or emitted symbol.

This viewpoint is close in spirit to dynamical systems. The state evolves step by step according to a fixed rule, except that the rule is driven by the input sequence. A recurrent neural network is therefore best understood as a learnable nonlinear dynamical system.

Core definitions

Definition 3.28 (Sequence model)**.** Let $\mathcal{X}$ be an input space, $\mathcal{Y}$ an output space, and $\mathcal{H}$ a hidden-state space. A *sequence model* processes an ordered input $x_{1:T}$ and produces outputs $y_{1:T}$ or a final output y_T by repeatedly updating an internal state.

Definition 3.29 (Recurrent neural network)**.** A *recurrent neural network* (RNN) is a parametric model specified by a recurrence map

$$F_\theta : \mathcal{H} \times \mathcal{X} \to \mathcal{H}$$

and an output map

$$G_\theta : \mathcal{H} \to \mathcal{Y},$$

with dynamics

$$h_t = F_\theta(h_{t-1}, x_t), \qquad y_t = G_\theta(h_t),$$

for $t = 1, \ldots, T$, starting from an initial state h_0.

Definition 3.30 (Discrete-time dynamical-system viewpoint)**.** An RNN is a *discrete-time input-driven dynamical system* because the hidden state evolves by repeated application of the same transition rule. The input x_t acts as an external forcing term, while the parameter vector θ determines the dynamics.

Definition 3.31 (Hidden state, recurrence map, and output map)**.** The variable h_t is called the *hidden state* at time t. The map F_θ is the *recurrence map* or *state-update rule*. The map G_θ is the *output map* or *readout*. Because the same F_θ is reused at every time step, an RNN exhibits weight sharing across time.

The most classical RNN chooses

$$F_\theta(h_{t-1}, x_t) = \sigma(W_h h_{t-1} + W_x x_t + b), \qquad G_\theta(h_t) = W_y h_t + c,$$

where σ is a nonlinearity such as tanh or a logistic sigmoid. This simple formula already contains the central architectural idea: the present state is computed from the previous state together with the new input.

Recurrence as depth through time

A recurrent network contains a loop in its abstract definition, but when the sequence length T is fixed one may *unroll* the recurrence into a depth-T computation graph:

$$h_0 \xrightarrow{F_\theta(\cdot, x_1)} h_1 \xrightarrow{F_\theta(\cdot, x_2)} h_2 \to \cdots \xrightarrow{F_\theta(\cdot, x_T)} h_T.$$

This graph is deep not because one stacked T different layers, but because one applied the *same* layer T times. That is the distinctive feature of recurrence. It is depth with parameter sharing across time.

Figure 3.6 is important conceptually. The folded diagram emphasizes the architectural idea of a state updated through time. The unrolled diagram emphasizes the optimization problem. Once unfolded, the RNN becomes an ordinary but very deep computational graph, and gradients must travel backward through every temporal step.

Backpropagation through time

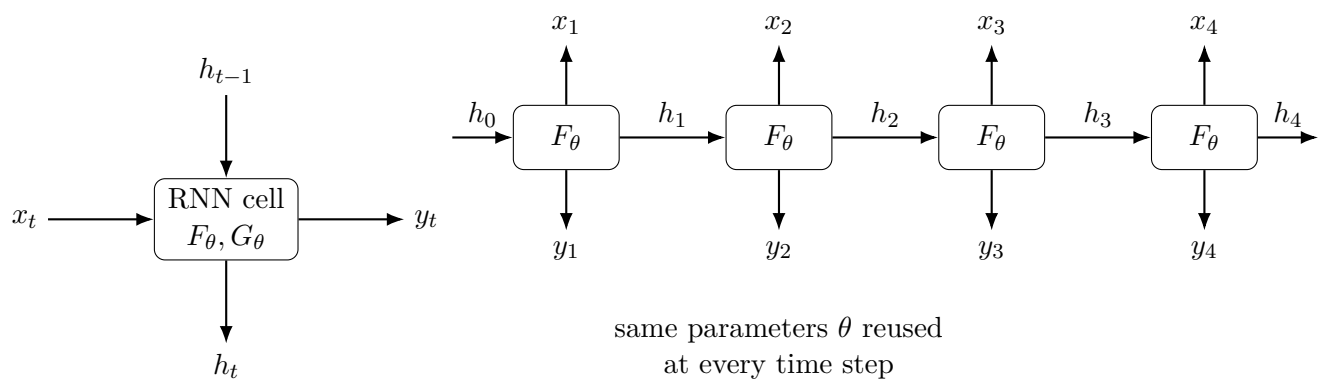
Suppose that a loss is accumulated across time:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \ell_t(y_t) = \sum_{t=1}^{T} \ell_t(G_\theta(h_t)), \qquad h_t = F_\theta(h_{t-1}, x_t).$$

Because the same parameters θ appear in every transition, differentiating $\mathcal{L}$ requires accounting for all paths by which a perturbation of θ at one time step changes later hidden states. This procedure is called *backpropagation through time* (BPTT).

A useful way to derive BPTT is to introduce the *adjoint state*

$$\delta_t := \frac{\partial \mathcal{L}}{\partial h_t},$$



(a) Folded recurrent cell.

(b) Unrolled computation graph.

Figure 3.6: A recurrent network may be viewed either as a single folded cell or as a deep computation graph unrolled through time. The unrolled view makes clear that recurrence creates depth by repeated use of the same transition map.

viewed as a row vector. Then the chain rule gives a backward recursion. Write

$$A_t := \frac{\partial F_\theta}{\partial h}(h_{t-1}, x_t) \qquad \text{and} \qquad B_t := \frac{\partial G_\theta}{\partial h}(h_t),$$

for the Jacobians of the recurrence map and output map with respect to the hidden state. If the loss at time t depends only on y_t, then

$$\delta_t = \frac{\partial \ell_t}{\partial y_t} B_t + \delta_{t+1} A_{t+1}, \qquad t = T-1, T-2, \ldots, 1,$$

with terminal condition

$$\delta_T = \frac{\partial \ell_T}{\partial y_T} B_T.$$

In words: the gradient at time t consists of a *local contribution* from the current loss plus a *future contribution* transported backward through the recurrent dynamics.

The parameter gradient is then obtained by summing the effect of the parameters at every time step:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T} \left(\frac{\partial \ell_t}{\partial \theta} + \delta_t \frac{\partial F_\theta}{\partial \theta}(h_{t-1}, x_t) \right),$$

where the first term accounts for any direct dependence of ℓ_t on θ through the output map. This is the exact analogue of reverse-mode differentiation in feedforward networks, except that the reverse pass now follows the temporal chain.

Proposition 3.32 (Gradient transport through recurrent Jacobians)**.** *Consider an RNN with hidden-state recursion $h_t = F_\theta(h_{t-1}, x_t)$ and loss*

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t(G_\theta(h_t)).$$

For $1 \le t < s \le T$, the contribution of the loss at time s to the hidden-state gradient at time t is transported by the Jacobian product

$$\frac{\partial \ell_s}{\partial h_t} = \frac{\partial \ell_s}{\partial h_s} \prod_{k=t+1}^{s} \frac{\partial h_k}{\partial h_{k-1}}.$$

Equivalently, if $A_k = \partial F_\theta / \partial h(h_{k-1}, x_k)$*, then*

$$\frac{\partial \ell_s}{\partial h_t} = \frac{\partial \ell_s}{\partial h_s} A_s A_{s-1} \cdots A_{t+1}.$$

Hence long-range gradient transport is controlled by repeated multiplication of recurrent Jacobians.

Proof. Fix $s > t$. By the chain rule,

$$\frac{\partial \ell_s}{\partial h_t} = \frac{\partial \ell_s}{\partial h_s} \frac{\partial h_s}{\partial h_t}.$$

Because h_s depends on h_t only through the intermediate states $h_{t+1}, \ldots, h_{s-1}$, another repeated application of the chain rule yields

$$\frac{\partial h_s}{\partial h_t} = \frac{\partial h_s}{\partial h_{s-1}} \frac{\partial h_{s-1}}{\partial h_{s-2}} \cdots \frac{\partial h_{t+1}}{\partial h_t}.$$

Substituting $A_k = \partial h_k / \partial h_{k-1}$ gives the stated formula. The final sentence follows because the norm of a long matrix product may decay rapidly, grow rapidly, or become highly anisotropic depending on the singular values of the factors. □

This proposition explains why recurrence is mathematically delicate. If the Jacobians A_k tend to contract vectors, then gradient signals from distant times are attenuated exponentially and one obtains *vanishing gradients*. If the Jacobians tend to expand vectors, the reverse signals may blow up and one obtains *exploding gradients*. The issue is not accidental. It is built into the repeated use of the same transition map.

Vanishing and exploding gradients

To see the mechanism more explicitly, consider a local linearization. If the recurrence is approximated near a trajectory by

$$h_t \approx A_t h_{t-1} + b_t,$$

then a gradient propagated backward over m steps is multiplied by a product of m matrices. A rough norm estimate gives

$$\left\| \frac{\partial \ell_s}{\partial h_t} \right\| \leq \left\| \frac{\partial \ell_s}{\partial h_s} \right\| \prod_{k=t+1}^{s} \|A_k\|.$$

If the typical norm $\|A_k\|$ is smaller than 1, the product tends to shrink exponentially with sequence distance. If it is larger than 1, the product may grow exponentially. Even when neither case holds uniformly, repeated multiplication can make the gradient concentrate in a few unstable directions and disappear in others.

For the classical nonlinear RNN

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b),$$

we have

$$A_t = \mathrm{Diag}(\sigma'(W_h h_{t-1} + W_x x_t + b)) W_h.$$

Thus both the recurrent weight matrix W_h and the derivative of the nonlinearity influence gradient transport. Saturating nonlinearities such as sigmoid or tanh can make the diagonal factor small, which further encourages vanishing gradients. This is one reason naive RNNs often struggle with long-term dependencies.

A worked scalar example

A simple scalar model makes the previous proposition concrete. Although the example is linear, it isolates the essential mechanism and may be viewed as a local approximation to a nonlinear recurrence.

Example 3.33 (A scalar recurrent model)**.** Consider the scalar recurrence

$$h_t = \alpha h_{t-1} + x_t, \qquad y_t = h_t,$$

with initial state $h_0 = 0$ and final-time loss

$$\mathcal{L} = \frac{1}{2}(h_T - \widetilde{h})^2.$$

Then

$$h_T = \sum_{s=1}^{T} \alpha^{T-s} x_s.$$

Moreover,

$$\frac{\partial \mathcal{L}}{\partial h_t} = \alpha^{T-t}(h_T - \widetilde{h}),$$

so the backward signal from time T to time t is scaled by the factor α^{T-t}. Therefore:

- if $|\alpha| < 1$, the gradient decays exponentially with temporal distance;
- if $|\alpha| > 1$, the gradient grows exponentially;
- if $|\alpha| \approx 1$, information can be preserved much longer.

Proof. Repeated substitution gives

$$h_1 = x_1, \quad h_2 = \alpha x_1 + x_2, \quad \dots \quad , h_T = \sum_{s=1}^{T} \alpha^{T-s} x_s.$$

Since

$$\mathcal{L} = \frac{1}{2}(h_T - \widetilde{h})^2,$$

we have

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T} \frac{\partial h_T}{\partial h_t} = (h_T - \widetilde{h})\alpha^{T-t}.$$

This is exactly the scalar version of the Jacobian-product formula. □

This worked example makes the long-dependency problem transparent. Suppose an input x_3 is important for the prediction at time $T = 100$. If $|\alpha| < 1$, then the contribution of that early event to the final gradient is suppressed by a factor of roughly $|\alpha|^{97}$. Learning to preserve that information becomes extremely difficult. The recurrence is elegant, but the optimization geometry is hostile.

Why recurrence remains important

Despite these difficulties, recurrent computation remains conceptually fundamental. It is a natural architecture whenever the task itself unfolds through time and whenever one wishes to summarize a growing past by a state. It uses parameters efficiently because the same transition map is reused at every time step. It also matches the causal structure of many real problems, especially streaming or online ones.

At the same time, the gradient analysis above shows that recurrence alone is not enough. One must also control how memory is stored and transported. This observation leads directly to gated architectures such as LSTMs and GRUs, which modify the recurrent update so that some information can move additively rather than being repeatedly multiplied through unstable Jacobians. Thus the next section is not a change of topic but a structural refinement of the dynamical systems picture developed here.

What to remember. A recurrent neural network is a learnable discrete-time dynamical system with hidden state h_t, recurrence map F_θ, and output map G_θ. When unrolled through time it becomes a deep computation graph with shared parameters. Backpropagation through time sends gradients backward through products of recurrent Jacobians, which is why long-range dependencies create vanishing or exploding gradients.

Common confusion. The hidden state is not a literal record of the entire past. It is a compressed state chosen by the model and training process. Also, the difficulty with RNNs is not merely that they are "old" or have been displaced by newer architectures. The difficulty is mathematical: long-range credit assignment requires transporting gradients through repeated Jacobian products, and that transport is unstable unless the recurrence is designed to preserve information carefully.

Exercises

Exercise 3.34. Let $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$ and $y_t = W_y h_t$. Write down the Jacobian $\partial h_t / \partial h_{t-1}$. Which two ingredients determine whether the backward signal is likely to contract or expand?

Exercise 3.35. Suppose an RNN is used only for sequence classification, so that the loss depends only on y_T. Explain why the gradient with respect to an early hidden state h_t still depends on all later Jacobians. Relate your answer to the unrolled computation graph.

Exercise 3.36. For the scalar recurrence $h_t = \alpha h_{t-1} + x_t$ with $h_0 = 0$, compute h_4 explicitly in terms of x_1, x_2, x_3, x_4 and α. Which inputs are weighted most strongly when $|\alpha| < 1$? Which are weighted most strongly when $|\alpha| > 1$?

Exercise 3.37. Assume that $\|A_k\| \le \rho < 1$ for all k. Show that the contribution of the loss at time s to the gradient at time t is bounded by a constant times ρ^{s-t}. Interpret this bound in words.

Exercise 3.38. Why is it reasonable to describe an RNN as depth with weight sharing across time? In what sense is this analogous to, and different from, weight sharing in a convolutional network?

Notes and references

Classical recurrent networks were developed for temporal context and sequence processing, with early influential work by Elman. The reverse-mode differentiation viewpoint for recurrent systems appears in the literature on ordered derivatives and backpropagation through time, including work

by Werbos. The vanishing-gradient difficulty was analyzed in influential papers by Bengio, Simard, and Frasconi, and by Hochreiter. For mathematically inclined readers, recurrent networks may also be viewed as nonlinear state-space models or input-driven dynamical systems. That viewpoint will matter again when we discuss gated recurrence in the next section and sequence alternatives such as attention later in the chapter.

3.5 Gating and the Control of Memory

The analysis of the previous section revealed the central weakness of naive recurrence. A simple recurrent update stores the past only by repeatedly rewriting one hidden state through a nonlinear map. That mechanism is expressive, but it makes long-range memory fragile. If useful information must survive for many time steps, then gradients must also survive for many time steps, and in a plain RNN both are transported through long products of Jacobians. This is why recurrence is not merely a modeling idea but also an optimization problem.

Gated recurrent architectures were designed as a structural answer to this difficulty. The main insight is that memory should not be updated in one homogeneous way. A model should be able to decide when to write new information, when to preserve existing information, when to erase obsolete information, and when to expose internal state to the output. In other words, information flow should itself be learnable. This is the meaning of gating.

The purpose of this section is to treat gating as a formal mechanism for memory control. We will write the full LSTM and GRU equations, define the gates precisely, explain why additive memory pathways help preserve gradients, and compare naive RNNs, LSTMs, and GRUs from the viewpoint of discrete-time dynamical systems. The main thesis is that gating is not cosmetic. It changes the geometry of optimization and the way information is transported through time.

From homogeneous recurrence to controlled recurrence

In a plain RNN one typically writes

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b).$$

Every coordinate of the hidden state is updated by the same broad mechanism at every time step. The model may learn to use that hidden state well, but it has no explicit structural distinction between at least four logically different operations:

- storing new information,
- preserving old information,
- forgetting information that is no longer relevant,
- exposing only part of the internal state to the outside.

A single recurrent map can in principle approximate such behavior, but the burden is placed entirely on the learned weights and nonlinearity. Gated architectures instead build these roles into the architecture itself.

This is an instance of the general chapter theme. Architecture injects prior structural assumptions into learning. For sequence problems, one useful assumption is that memory management should be decomposed into distinct, learnable channels. The resulting inductive bias does not specify *what* should be remembered; it specifies *how* remembering and forgetting may occur.

Core definitions

Definition 3.39 (Gate)**.** A *gate* is a learnable multiplicative control variable, usually taking values in $[0, 1]$ coordinatewise, that modulates the flow of information through a recurrent architecture. If $g_t \in [0, 1]^d$ is a gate and $u_t \in \mathbb{R}^d$ is a vector of candidate information, then the gated signal is often represented coordinatewise by $g_t \odot u_t$, where $\odot$ denotes the Hadamard product. Coordinates of g_t near 1 tend to pass information through, whereas coordinates near 0 tend to suppress it.

Definition 3.40 (Input, forget, and output gates)**.** In an LSTM, the *input gate* controls how much newly proposed information is written into memory, the *forget gate* controls how much of the old cell state is retained, and the *output gate* controls how much of the internal cell state is revealed to the exposed hidden state. These gates are vectors in $[0,1]^d$ computed from the current input and the previous hidden state.

Definition 3.41 (Additive memory pathway)**.** An *additive memory pathway* is a recurrence in which part of the new memory state is obtained by adding a gated version of the old memory state to a gated candidate update, rather than by applying one fully nonlinear transformation to the entire previous state. Such pathways create routes along which information may persist with less repeated distortion.

Definition 3.42 (Controlled recurrent dynamical system)**.** A gated recurrent network is a *controlled recurrent dynamical system* whose state update depends not only on the previous state and the input but also on internal control variables (the gates) that determine how information is written, preserved, or exposed.

Long short-term memory

The long short-term memory (LSTM) architecture introduces two state variables at each time step:

- the *cell state* $c_t \in \mathbb{R}^d$, which acts as internal memory,
- the *hidden state* $h_t \in \mathbb{R}^d$, which is the exposed state used for communication with the rest of the network.

Given an input $x_t \in \mathbb{R}^m$, the standard LSTM equations are

$$\begin{aligned}
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i),\\
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f),\\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o),\\
\widetilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c),\\
c_t &= f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t,\\
h_t &= o_t \odot \tanh(c_t).
\end{aligned}$$

Here:

- i_t is the *input gate*,
- f_t is the *forget gate*,
- o_t is the *output gate*,
- $\widetilde{c}_t$ is the *candidate cell update*.

The key equation is the cell-state update

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t.$$

Unlike the hidden-state update of a plain RNN, this rule has an explicit additive structure. Part of the previous memory is carried forward directly, coordinate by coordinate, and only then combined with new candidate content. That is the mechanism by which an LSTM can preserve information over longer horizons.

Gated recurrent unit

The gated recurrent unit (GRU) simplifies the LSTM while preserving the central idea of controlled memory transport. A common form of the GRU equations is

$$\begin{aligned} z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \\ r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r), \\ \widetilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t. \end{aligned}$$

Here:

- z_t is often called the *update gate*; it determines how much of the candidate state replaces the previous state,
- r_t is the *reset gate*; it determines how strongly the previous hidden state contributes when constructing the candidate update.

The GRU has no separate cell state c_t. Instead, the exposed hidden state itself plays the role of memory. Nevertheless, the same essential structural feature remains:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t,$$

which is again an additive mixture of old state and new proposal.

Gated memory transport and gradient preservation

The reason gating matters can be seen by differentiating the additive memory pathway. In an LSTM,

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t.$$

If we focus on the dependence of c_t on c_{t-1} while treating the gates and candidate update as temporarily fixed, then

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{Diag}(f_t).$$

Thus the backward transport of gradients through the cell state is controlled directly by the forget gate, rather than by an arbitrary dense Jacobian multiplied by a saturating nonlinearity at every step. This is the mathematical core of the LSTM idea.

Proposition 3.43 (Additive memory pathways improve gradient transport)**.** *Consider a recurrence of the form*

$$m_t = g_t \odot m_{t-1} + u_t,$$

where $m_t \in \mathbb{R}^d$, the gate $g_t \in [0,1]^d$, and u_t does not depend directly on m_{t-1}. Then

$$\frac{\partial m_t}{\partial m_{t-1}} = \mathrm{Diag}(g_t),$$

and for $s > t$,

$$\frac{\partial m_s}{\partial m_t} = \mathrm{Diag}(g_s)\,\mathrm{Diag}(g_{s-1}) \cdots \mathrm{Diag}(g_{t+1}).$$

In particular, if the gate coordinates remain near 1 on a relevant time interval, then gradients along the memory pathway are preserved far better than in a generic recurrent update involving repeated multiplication by unrelated dense Jacobians.

Proof. Differentiate the recurrence coordinatewise. Since u_t does not depend directly on m_{t-1},

$$\frac{\partial m_t}{\partial m_{t-1}} = \frac{\partial(g_t \odot m_{t-1})}{\partial m_{t-1}} = \mathrm{Diag}(g_t).$$

Applying the chain rule repeatedly gives

$$\frac{\partial m_s}{\partial m_t} = \frac{\partial m_s}{\partial m_{s-1}} \frac{\partial m_{s-1}}{\partial m_{s-2}} \cdots \frac{\partial m_{t+1}}{\partial m_t} = \mathrm{Diag}(g_s)\,\mathrm{Diag}(g_{s-1}) \cdots \mathrm{Diag}(g_{t+1}).$$

If the relevant gate coordinates are close to 1, then the corresponding diagonal entries in this product remain close to 1 rather than being repeatedly multiplied by arbitrary dense transformations. Thus the gradient transport can remain stable over much longer horizons. □

This proposition should be compared with the Jacobian-product formula of the previous section. A naive RNN transports gradients through matrices of the form

$$A_t = \mathrm{Diag}(\sigma'(\cdot))W_h,$$

whose repeated products may distort, contract, or expand signals unpredictably. An LSTM or GRU instead creates a distinguished pathway along which memory can move additively under coordinatewise gates. The architecture does not guarantee perfect long-term memory, but it makes long-term memory *possible* in a way that plain recurrence often does not.

A dynamical-systems comparison: plain RNN, LSTM, and GRU

From the viewpoint of dynamical systems, these architectures differ in the type of state transition they implement.

- **Plain RNN.** The entire state is rewritten at every step by one nonlinear map:

  $$h_t = \sigma(W_h h_{t-1} + W_x x_t + b).$$

  Memory is implicit and must survive repeated nonlinear transformation.
- **LSTM.** The system separates internal memory c_t from exposed state h_t. The memory itself evolves additively, with the forget and input gates controlling retention and writing:

  $$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t.$$

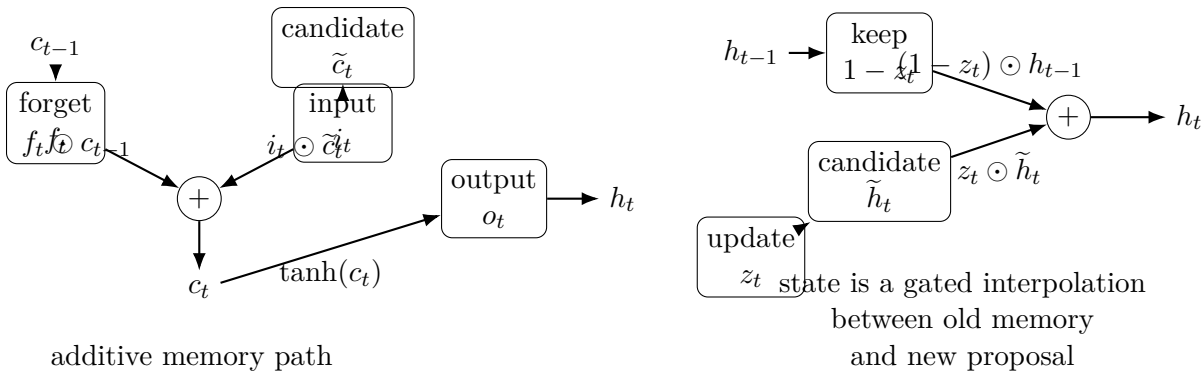
  Information may therefore travel internally without being fully rewritten at every step.
- **GRU.** The hidden state plays both roles, but the update still explicitly interpolates between the old state and a new candidate:

  $$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t.$$

  This is structurally simpler than the LSTM but preserves the key additive transport idea.

The difference is not only descriptive. In a plain RNN, one asks a single transformation to simultaneously preserve memory, react to new inputs, and create useful nonlinear features. In an LSTM or GRU, these roles are partially disentangled. That architectural decomposition changes the state-space geometry and the backpropagation geometry. Some directions of state are allowed to persist under near-identity transport, while others may be actively overwritten.

Figure 3.7 highlights the essential difference. In both the LSTM and the GRU, the old state does not disappear into one opaque nonlinear transformation. It travels along an explicit pathway, and the gates determine how much of it should persist. This is the architectural reason gated models often train better on longer dependencies.



(a) LSTM information flow.

(b) GRU information flow.

Figure 3.7: Gated architectures replace homogeneous recurrence by controlled information flow. The central structural change is the appearance of additive state transport modulated by learnable gates.

A worked memory-trace example

The following scalar example isolates the mechanism of controlled memory transport. It is deliberately simple, but it shows clearly why gates matter.

Example 3.44 (Tracing one memory coordinate through time)**.** Consider a one-dimensional LSTM-like memory update

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t,$$

with initial value $c_0 = 0$. Suppose that at time $t = 1$ the model observes a relevant event and writes it into memory by taking

$$i_1 = 1, \qquad \tilde{c}_1 = 2, \qquad f_1 = 0,$$

so that $c_1 = 2$. For the next four time steps suppose the model wants to preserve that memory, so it chooses

$$f_2 = f_3 = f_4 = f_5 = 0.98, \qquad i_2 = i_3 = i_4 = i_5 = 0.$$

Then

$$c_2 = 0.98 \cdot 2 = 1.96, \qquad c_3 = 0.98^2 \cdot 2, \qquad c_4 = 0.98^3 \cdot 2, \qquad c_5 = 0.98^4 \cdot 2 \approx 1.8447.$$

Thus the stored information persists with only slow decay. Moreover,

$$\frac{\partial c_5}{\partial c_1} = f_5 f_4 f_3 f_2 = 0.98^4 \approx 0.9224,$$

so the gradient traveling backward from time 5 to time 1 remains large. If, by contrast, a plain scalar RNN had local multiplier 0.5 at each step, then the corresponding backward factor over four steps would be $0.5^4 = 0.0625$, which is far smaller.

The point is not that LSTMs always keep gates near 1. Rather, they *can* choose to do so when the task demands memory preservation. That option does not exist in the same explicit way in a naive RNN. Gating therefore changes not merely the representation class but the available memory dynamics.

Why gating is a structural rather than cosmetic change

It is tempting to think of gates as extra engineering detail layered on top of recurrence. That interpretation misses the main mathematical point. Gates alter the class of admissible state transitions. In a plain RNN, the old state is repeatedly transformed by one map. In a gated model, some directions of memory may instead follow near-identity transport, others may be erased, and still others may be rewritten. This is a qualitatively different family of dynamical systems.

Equally important, gating changes the optimization landscape seen during training. Because important gradients can travel through additive memory paths, long-range credit assignment becomes more stable. The architecture therefore incorporates an inductive bias not only about sequence structure but also about trainable sequence structure. It says that temporal information should sometimes be preserved rather than constantly remixed. This is why gated recurrence was historically so important. It provided a workable route to sequence learning before attention-based methods became dominant in many domains.

At the same time, gating does not solve every problem. Very long contexts may still be difficult, and recurrent computation remains sequential in time, which limits parallelism. These limitations help explain why the field later moved toward attention. But conceptually, gating remains one of the clearest examples of how an architectural modification can reshape both representation and optimization.

What to remember. LSTMs and GRUs are recurrent dynamical systems with learnable gates that control writing, forgetting, and exposing information. Their key structural innovation is the additive transport of memory, for example $c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t$ in an LSTM. Because gradients along this pathway are governed by gate values rather than generic dense Jacobians, gated models can preserve information over longer horizons far better than naive RNNs.

Common confusion. The benefit of gating is not merely that it adds more parameters. Its real value is structural. A plain RNN and an LSTM with similar hidden width do not belong to the same effective family of dynamical systems, because the LSTM has explicit additive memory pathways and separate control of retention, writing, and exposure. Also, gates do not guarantee perfect memory; they create the possibility of stable memory transport when learning discovers that such transport is useful.

Exercises

Exercise 3.45. Write the LSTM cell-state update in coordinate form. For a fixed coordinate j, explain in words what happens when $(f_t)_j$ is close to 1 and $(i_t)_j$ is close to 0. What happens when $(f_t)_j$ is close to 0 and $(i_t)_j$ is large?

Exercise 3.46. Consider the recurrence $m_t = g_t m_{t-1} + u_t$ in one dimension. Show that

$$m_t = \Big(\prod_{k=1}^{t} g_k\Big) m_0 + \sum_{s=1}^{t} \Big(\prod_{k=s+1}^{t} g_k\Big) u_s.$$

Interpret this formula as a weighted memory of past updates.

Exercise 3.47. In a GRU, the update equation is $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t$. Describe the behavior of the model when z_t is close to 0 coordinatewise. Describe the behavior when z_t is close to 1. How does this compare to the role of the forget and input gates in an LSTM?

Exercise 3.48. Suppose a one-dimensional LSTM-like memory satisfies

$$c_t = 0.99c_{t-1}$$

for $t = 2, \ldots, 101$ after writing a signal at time 1. Compute $\partial c_{101}/\partial c_1$. Repeat the same calculation for a plain scalar recurrence whose local derivative is constantly 0.8. Compare the two values.

Exercise 3.49. From the viewpoint of discrete-time dynamical systems, explain one important difference between a plain RNN and an LSTM, and one important difference between an LSTM and a GRU. Your answer should refer explicitly to the structure of the state update rather than to historical popularity or benchmark performance.

Notes and references

The long short-term memory architecture was introduced by Hochreiter and Schmidhuber as a structural answer to the difficulty of learning long-range dependencies with naive recurrence. The gated recurrent unit was later proposed by Cho and collaborators as a simpler gated alternative. The mathematical motivation for gating is closely tied to the vanishing-gradient analyses of recurrent learning developed in the 1990s. From a broader perspective, gated recurrence is an important example of architectural design as inductive bias: one encodes the prior belief that sequence models should be able to preserve, overwrite, and reveal information through distinct learnable mechanisms. This idea of controlled information flow will reappear in a different form when we turn from recurrence to attention in the next sections.

3.6 From Recurrence to Attention

The previous two sections developed a precise picture of what recurrent models do well and where they struggle. Recurrence is a natural architecture for temporal data because it processes a sequence through an evolving state. That state functions as a running summary of the past. But this same mechanism also creates a structural bottleneck. If information from an early part of a long sequence is needed much later, then it must survive repeated state updates and repeated Jacobian products. Even gated architectures make that dependence travel through time one step after another.

Attention arose as a structural response to this problem. Instead of requiring all relevant information to be compressed into one fixed-size state, the model is allowed to access past representations directly and selectively. The central idea is content-based retrieval inside the network: a model forms a query describing what it currently needs, compares that query with a collection of candidate keys, and then aggregates the corresponding values according to learned relevance weights. The result is not merely a different formula. It is a different architecture for memory access.

The purpose of this section is to make that transition mathematically explicit. We will define attention as content-based weighted aggregation, derive its weighted-sum form from similarity scores, explain the encoder–decoder bottleneck in classical sequence-to-sequence models, prove a basic structural proposition about the output of attention, and work through a small alignment example. The pedagogical point is that attention did not appear as an arbitrary design fashion. It solved a precise architectural problem.

The encoder–decoder bottleneck

Consider a classical sequence-to-sequence architecture for translation. An encoder reads an input sequence $x_1, \dots, x_T$ and produces a final state h_T. A decoder then produces an output sequence $y_1, \dots, y_S$ using that final state as its initial summary of the source. At an abstract level one writes

$$h_t = F(h_{t-1}, x_t), \qquad s_u = G(s_{u-1}, y_{u-1}, h_T),$$

where h_t is the encoder state and s_u is the decoder state. The entire source sentence is therefore compressed into the single vector h_T.

This architecture can work well for short sequences, but it imposes a severe information bottleneck. A fixed-dimensional vector must simultaneously retain all the details that might later matter for every output token. When sequences become longer or dependencies become more specific, the decoder may need information that was present earlier in the source but is not adequately represented in the final summary. In that sense, classical sequence-to-sequence recurrence asks one vector to do too much.

Attention loosens this bottleneck. Instead of forcing the decoder to rely only on h_T, it allows the decoder at time u to construct a context vector from the entire collection $h_1, \dots, h_T$. The context depends on what the decoder currently needs, so different output positions may retrieve different parts of the source.

Core definitions

Definition 3.50 (Attention mechanism)**.** Let $q \in \mathbb{R}^{d_q}$ be a query, let $k_1, \dots, k_n \in \mathbb{R}^{d_q}$ be keys, and let $v_1, \dots, v_n \in \mathbb{R}^{d_v}$ be values. An *attention mechanism* is a map that assigns a relevance weight

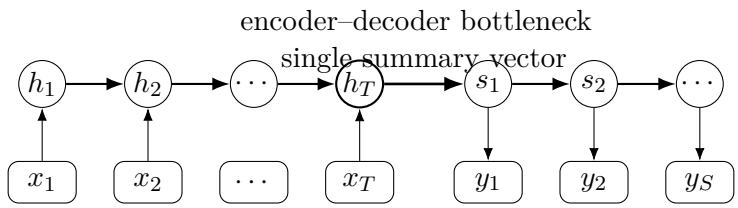


Figure 3.8: Classical sequence-to-sequence recurrence compresses the entire source into one summary state before decoding. Attention replaces this single bottleneck with selective access to many encoder states.

$\alpha_i(q)$ to each pair (q, k_i) and then returns an aggregate of the values:

$$\mathrm{Attn}(q, \{(k_i, v_i)\}_{i=1}^n) = \sum_{i=1}^n \alpha_i(q) v_i.$$

When the weights depend on content similarity between the query and the keys, the mechanism is called *content-based attention*.

Definition 3.51 (Compatibility score and alignment weights)**.** A *compatibility score* or *similarity score* is a function

$$s : \mathbb{R}^{d_q} \times \mathbb{R}^{d_q} \to \mathbb{R}$$

that measures how relevant a key is to a given query. Given scores $s(q, k_i)$, the corresponding *alignment weights* are commonly defined by the softmax normalization

$$\alpha_i(q) = \frac{\exp(s(q, k_i))}{\sum_{j=1}^n \exp(s(q, k_j))}, \qquad i = 1, \ldots, n.$$

These weights are nonnegative and sum to 1.

Definition 3.52 (Context vector)**.** In encoder–decoder attention, the *context vector* at decoder step u is the weighted combination

$$c_u = \sum_{t=1}^T \alpha_{u,t} h_t,$$

where h_t are encoder states and $\alpha_{u,t}$ are alignment weights determined by the current decoder state and the source representations. The context vector summarizes the source *relative to the current decoding need*, not in one fixed way for the whole sentence.

From similarity scores to weighted aggregation

The weighted-sum form of attention is not arbitrary. It emerges naturally from three requirements. First, each candidate memory item should receive a relevance score based on its match with the current query. Second, these scores should be normalized so that they become comparable across different numbers of candidates. Third, the retrieved information should remain differentiable with respect to both the scores and the stored representations.

Suppose the current decoder state or query is q, and suppose the encoder has produced candidate memories $h_1, \ldots, h_T$. A score function $s(q, h_t)$ assigns a real number to each source position. To

transform these scores into usable weights, one applies a normalization map. The most common choice is the softmax:

$$e_t = s(q, h_t), \qquad \alpha_t = \frac{e^{e_t}}{\sum_{j=1}^{T} e^{e_j}}.$$

Now $\alpha_t \geq 0$ and $\sum_t \alpha_t = 1$. The natural differentiable retrieval rule is then

$$c(q) = \sum_{t=1}^{T} \alpha_t h_t.$$

Thus the weighted-sum form of attention is obtained by the composition

$$\text{query and memories} \xrightarrow{\text{score}} (e_1, \ldots, e_T) \xrightarrow{\text{softmax}} (\alpha_1, \ldots, \alpha_T) \xrightarrow{\text{weighted sum}} c.$$

This formula has an important interpretation. The model does not retrieve one memory slot in a hard, discrete way. Instead it forms a soft data-dependent average over the available values. That soft selection is what makes attention trainable by gradient-based methods. At the same time, when one weight becomes much larger than the others, the mechanism approximates nearly discrete retrieval.

A structural proposition about attention outputs

The weighted-sum form immediately imposes geometric structure on what attention can produce.

Proposition 3.53 (Attention outputs lie in the span and, with softmax weights, in the convex hull of the values)**.** *Let $v_1, \ldots, v_n \in \mathbb{R}^{d_v}$ and let*

$$y = \sum_{i=1}^{n} \alpha_i v_i.$$

Then $y \in \operatorname{span}\{v_1, \ldots, v_n\}$. If in addition $\alpha_i \geq 0$ for all i and $\sum_{i=1}^{n} \alpha_i = 1$, then y lies in the convex hull of $\{v_1, \ldots, v_n\}$. In particular, standard softmax attention produces outputs that are convex combinations of the available values.

Proof. By definition, y is a linear combination of the vectors $v_1, \ldots, v_n$, so it belongs to their span. If moreover the coefficients are nonnegative and sum to 1, then y is a convex combination of those vectors, which is exactly the definition of membership in the convex hull. For softmax attention the weights satisfy these properties automatically, so the conclusion follows. □

This proposition is elementary but conceptually important. Attention does not create information from nowhere. It reweights and recombines existing value vectors. Its power comes from making the coefficients data-dependent. The architecture therefore changes *which* information can be accessed and *when* it can be accessed, even though each context vector remains a structured combination of the stored values.

Remark 3.54. The proposition also explains why learned linear projections are often placed before and after attention. If one wants the retrieved vector to live in a more expressive feature space, one can transform the stored values before aggregation or transform the resulting context afterward. Attention itself supplies adaptive selection; linear maps around it expand the representational class.

Why attention solves a precise recurrent problem

The bottleneck in classical sequence-to-sequence recurrence has two aspects. First, the source must be summarized in a single vector before the decoder begins to use it. Second, the contribution of an early source token to a late target token must travel through many recurrent transitions. Attention addresses both problems. At decoder step u, one computes a fresh query from the current decoding state and retrieves a context vector from all source positions. Thus source token x_t can influence output token y_u directly through the coefficient $\alpha_{u,t}$ rather than only indirectly through a long chain of recurrent state updates.

This does not remove all sequential structure. In an encoder–decoder model the decoder still evolves through time. But it replaces one global compressed memory with dynamic access to a whole collection of source-side representations. That is the decisive conceptual change. Memory is no longer only something carried forward; it becomes something that can be queried.

Worked example: a short alignment problem

Consider a toy translation example with source sentence

the cat sleeps

and target sentence

le chat dort.

Suppose the encoder has produced source representations h_1, h_2, h_3 for *the*, *cat*, and *sleeps*. At the decoder step for producing *chat*, the current decoder state should look most strongly at the source word *cat*. A simplified set of scores might be

$$e = (0.2,\ 2.4,\ 0.1).$$

The softmax weights are then approximately

$$\alpha \approx (0.091,\ 0.827,\ 0.082).$$

Therefore the context vector is

$$c \approx 0.091h_1 + 0.827h_2 + 0.082h_3.$$

This means that the decoder forms a source summary specialized to the current target word. For the next decoder step, when producing *dort*, the largest alignment weight would typically shift toward the source word *sleeps*.

Example 3.55 (A tiny numerical attention computation)**.** Suppose the three source values are one-dimensional for simplicity:

$$v_1 = 0, \qquad v_2 = 1, \qquad v_3 = 2,$$

and suppose the attention weights at one decoder step are

$$\alpha = (0.1, 0.8, 0.1).$$

Then the retrieved context is

$$c = 0.1 \cdot 0 + 0.8 \cdot 1 + 0.1 \cdot 2 = 1.0.$$

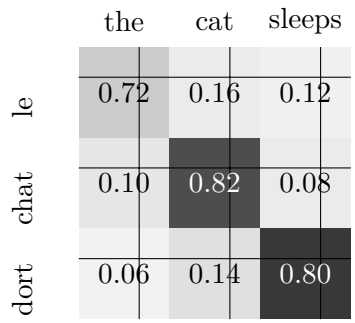


Figure 3.9: A toy alignment matrix for translating *the cat sleeps* to *le chat dort*. Darker cells indicate larger attention weights. Different target words retrieve different source positions, which is exactly what a single fixed source summary cannot do.

If the weights instead become $(0.05, 0.15, 0.80)$, then

$$c = 0.05 \cdot 0 + 0.15 \cdot 1 + 0.80 \cdot 2 = 1.75.$$

The values have not changed; what changed is the content-dependent routing. This is the basic mechanism by which attention can focus on different parts of a sequence at different times.

The alignment matrix in Figure 3.9 visualizes the same idea at the sentence level. Each row corresponds to one decoder step, and each column corresponds to one source position. The weights in each row sum to 1. For *chat*, the mass is concentrated on *cat*; for *dort*, it is concentrated on *sleeps*. Thus the decoder forms different context vectors for different output tokens. This is what it means for attention to provide dynamic, content-based access.

Relation to the later transformer viewpoint

Pedagogically it is important to pause here before moving to transformers. Attention should first be understood as a mechanism:

- it responds to the encoder–decoder bottleneck,
- it replaces one fixed summary by query-dependent retrieval,
- it creates direct interactions between distant positions,
- it remains differentiable because the retrieval is soft rather than hard.

Only after this mechanism is clear does it become natural to study architectures built primarily from attention, such as transformers. In that sense, attention is not yet the whole architecture. It is the key structural idea that makes a different architecture possible.

What to remember. Attention is content-based weighted aggregation. A query is compared with a collection of keys, the resulting scores are normalized into alignment weights, and a context vector is formed as a weighted sum of values. This directly addresses the classical sequence-to-sequence bottleneck by allowing different output positions to retrieve different source information instead of relying on one fixed summary vector.

Common confusion. Attention is sometimes described as if it were a mysterious replacement for recurrence. That is too vague. Its original role in sequence-to-sequence models was much more

specific: it allowed the decoder to access the full collection of encoder states instead of depending only on a single compressed vector. Also, attention is not hard lookup. With softmax normalization it produces a convex combination of values, so its power comes from adaptive weighting rather than from magically creating new information.

Exercises

Exercise 3.56. Let values $v_1, v_2, v_3 \in \mathbb{R}^d$ and weights $\alpha_1, \alpha_2, \alpha_3$ satisfy $\alpha_i \geq 0$ and $\alpha_1 + \alpha_2 + \alpha_3 = 1$. Show directly that $\sum_i \alpha_i v_i$ lies in the convex hull of $\{v_1, v_2, v_3\}$. Why does this apply automatically to softmax attention?

Exercise 3.57. Suppose a decoder step produces similarity scores $(1, 2, 0)$ against three source positions. Compute the corresponding softmax weights. Which source position receives the largest share of attention?

Exercise 3.58. Explain in your own words why the vector h_T in a classical encoder–decoder model is a bottleneck. Your answer should refer both to representation capacity and to long-range dependence through recurrent updates.

Exercise 3.59. In a toy translation example with source words (x_1, x_2, x_3) and target words (y_1, y_2), suppose the alignment weights are

$$(0.7, 0.2, 0.1) \quad \text{for } y_1, \qquad (0.1, 0.1, 0.8) \quad \text{for } y_2.$$

Describe qualitatively what this says about the different context vectors used at the two decoder steps.

Exercise 3.60. Why is the weighted-sum form of attention more compatible with gradient-based training than a hard rule that selects exactly one memory slot by an argmax operation? Give both an optimization-based and a representational answer.

Notes and references

The attention mechanism entered modern deep learning through sequence-to-sequence learning, where it was introduced as a remedy for the fixed-vector bottleneck of encoder–decoder recurrence. Work by Bahdanau, Cho, and Bengio was especially influential in making this point explicit for neural machine translation. Luong and collaborators developed closely related attention formulations, and the language of alignments helped connect neural translation to earlier ideas from statistical machine translation. From a broader perspective, attention can be understood as differentiable content-based retrieval over an external set of representations. That viewpoint became central once later architectures were built almost entirely from attention, but its pedagogical meaning is already clear here: attention solved a precise memory-access problem before it became the organizing principle of transformers.

3.7 Transformers as General-Purpose Sequence Architectures

The previous section introduced attention as a mechanism for content-based weighted aggregation. A transformer turns that mechanism into a full architectural template. Instead of treating attention as an auxiliary device attached to a recurrent model, the transformer makes attention the central structured operator of the network.

This shift is conceptually important. A transformer layer is not merely a collection of engineering tricks. It is a mathematically analyzable operator on a sequence of representations: it mixes information across positions by learned pairwise interactions, applies nonlinear pointwise transformations, and stabilizes depth through residual connections and normalization. Seen this way, transformers are general-purpose sequence architectures because they provide a reusable algebraic block that can model local, global, and content-dependent interaction patterns within the same framework.

Scaled dot-product self-attention

Let

$$X \in \mathbb{R}^{T\times d}$$

be a matrix of token representations, where T is the sequence length and d is the representation dimension. A self-attention layer constructs queries, keys, and values from the same input:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

where

$$W_Q \in \mathbb{R}^{d\times d_k}, \qquad W_K \in \mathbb{R}^{d\times d_k}, \qquad W_V \in \mathbb{R}^{d\times d_v}.$$

Definition 3.61 (Scaled dot-product attention)**.** For matrices $Q \in \mathbb{R}^{T\times d_k}$, $K \in \mathbb{R}^{T\times d_k}$, and $V \in \mathbb{R}^{T\times d_v}$, the scaled dot-product attention operator is

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where $\mathrm{softmax}_{\mathrm{row}}$ applies the softmax map to each row of the score matrix.

The matrix

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T\times T}$$

contains pairwise similarity scores between positions. After row-wise softmax one obtains an attention matrix

$$A = \mathrm{softmax}_{\mathrm{row}}(S),$$

whose entries satisfy $A_{ij} \geq 0$ and $\sum_{j=1}^{T} A_{ij} = 1$ for every row i. Thus the output at position i is

$$(AV)_i = \sum_{j=1}^{T} A_{ij} v_j,$$

which is a convex combination of the value vectors. This makes self-attention a structured interaction operator: each output token is built by aggregating information from all positions, but with weights determined adaptively by content.

The scaling by $\sqrt{d_k}$ is not cosmetic. Without it, dot products can grow in magnitude as the key dimension increases, pushing the softmax into very sharp regimes and making optimization less stable. The scaling keeps score magnitudes in a more controlled range.

Self-attention as a sequence operator

A recurrent model updates positions through a temporal chain. A self-attention layer instead forms an interaction graph in one step. Every position can communicate directly with every other position through the matrix A. This is one reason transformers are so expressive for long-range dependence: if token i needs information from token j, the model does not have to pass that information through $j - i$ intermediate recurrent transitions. It can create a direct content-based edge immediately.

In pure self-attention, however, the interaction pattern depends only on the token representations supplied to the layer. If no positional information is added, the layer does not intrinsically know whether one token came earlier or later in the sequence. This observation can be stated precisely.

Proposition 3.62 (Permutation equivariance of self-attention without positional information)**.** *Let $P \in \mathbb{R}^{T\times T}$ be a permutation matrix, and define*

$$\mathrm{SA}(X) = \mathrm{Attn}(XW_Q, XW_K, XW_V).$$

If no positional information is added to X, then

$$\mathrm{SA}(PX) = P\,\mathrm{SA}(X).$$

That is, self-attention is permutation equivariant.

Proof sketch. If $X' = PX$, then

$$Q' = X'W_Q = PQ, \qquad K' = X'W_K = PK, \qquad V' = X'W_V = PV.$$

Therefore the score matrix becomes

$$S' = \frac{Q'K'^\top}{\sqrt{d_k}} = \frac{(PQ)(PK)^\top}{\sqrt{d_k}} = \frac{PQK^\top P^\top}{\sqrt{d_k}} = PSP^\top.$$

Applying row-wise softmax simply permutes rows and columns in the same way, so

$$\mathrm{softmax}_{\mathrm{row}}(S') = P\,\mathrm{softmax}_{\mathrm{row}}(S)\,P^\top.$$

Hence

$$\begin{aligned}\mathrm{SA}(PX) &= \mathrm{softmax}_{\mathrm{row}}(S')V' \\ &= P\,\mathrm{softmax}_{\mathrm{row}}(S)P^\top PV \\ &= P\,\mathrm{softmax}_{\mathrm{row}}(S)V \\ &= P\,\mathrm{SA}(X).\end{aligned}$$

So permuting the inputs permutes the outputs in exactly the same way. □

This proposition is pedagogically valuable because it identifies a precise limitation. Self-attention alone is naturally suited not only to sequences but also to sets, where permutation equivariance may actually be desirable. For sequence modeling, however, order matters. Transformers therefore inject positional information explicitly, either by adding positional encodings or by using learned positional embeddings. Once order information is included, the architecture is no longer equivariant to arbitrary permutations of the token positions.

Multi-head attention

A single attention map produces one interaction pattern. In practice, transformers use *multi-head attention*, which computes several attention operations in parallel with different learned projections. For head $h = 1, \dots, H$, define

$$Q_h = XW_Q^{(h)}, \qquad K_h = XW_K^{(h)}, \qquad V_h = XW_V^{(h)},$$

and then

$$\mathrm{head}_h(X) = \mathrm{Attn}(Q_h, K_h, V_h).$$

The head outputs are concatenated and linearly mixed:

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1(X), \dots, \mathrm{head}_H(X))W_O.$$

The point is not merely to enlarge the parameter count. Different heads can represent different types of interactions: local versus long-range dependence, syntactic versus semantic relation, or one-to-one copying versus broad contextual smoothing. The transformer layer therefore learns not one interaction graph, but a small family of interaction graphs that are recombined into a new representation.

The transformer block written algebraically

A transformer layer combines self-attention with a positionwise nonlinear map. A convenient modern form is the *pre-normalization* block. If $H^{(\ell)} \in \mathbb{R}^{T\times d}$ is the input to layer ℓ, then one writes

$$U^{(\ell)} = \mathrm{LN}(H^{(\ell)}),$$

$$\widetilde{H}^{(\ell)} = H^{(\ell)} + \mathrm{MHA}(U^{(\ell)}),$$

$$Z^{(\ell)} = \mathrm{LN}(\widetilde{H}^{(\ell)}),$$

$$H^{(\ell+1)} = \widetilde{H}^{(\ell)} + \mathrm{MLP}(Z^{(\ell)}).$$

Here the positionwise feedforward network typically has the form

$$\mathrm{MLP}(z) = W_2\,\sigma(W_1 z + b_1) + b_2,$$

applied independently to each token representation.

This algebra displays the essential architecture clearly. There are two kinds of transformation in each layer:

- **interaction across positions**, produced by multi-head self-attention,
- **nonlinear feature transformation within each position**, produced by the MLP.

Residual connections preserve an identity pathway through the block, which helps optimization in deep networks, while normalization controls the scale and geometry of intermediate activations.

The original transformer formulation used a closely related *post-normalization* arrangement. The difference is architectural rather than conceptual: both forms combine attention, nonlinear pointwise transformation, skip connections, and normalization into a repeated block.

Transformer block (pre-normalization form)

Input $H^{(\ell)}$

$\downarrow$

LayerNorm

$\downarrow$

Multi-Head Self-Attention

$\downarrow$

Add residual from $H^{(\ell)}$

$\downarrow$

LayerNorm

$\downarrow$

Positionwise MLP

$\downarrow$

Add residual

$\downarrow$

Output $H^{(\ell+1)}$

Figure 3.10: A monochrome block diagram of a transformer layer. Each block mixes global interaction through self-attention with per-position nonlinear processing through an MLP, and both sublayers are wrapped in residual pathways.

A tiny worked example

A small numerical example helps make the operator concrete. Consider a sequence of length $T = 3$ with one-dimensional queries and keys:

$$Q = K = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \qquad V = I_3.$$

For simplicity, take $d_k = 1$, so no nontrivial scaling appears. Then

$$S = QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Applying row-wise softmax gives

$$A = \mathrm{softmax}_{\mathrm{row}}(S) \approx \begin{bmatrix} 0.422 & 0.155 & 0.422 \\ 0.333 & 0.333 & 0.333 \\ 0.422 & 0.155 & 0.422 \end{bmatrix}.$$

Since $V = I_3$, the attention output is simply

$$AV = A.$$

Thus the first and third tokens place most of their weight on positions 1 and 3, while the second token averages all positions equally.

Example 3.63 (Reading the tiny attention matrix)**.** This matrix already illustrates several essential facts. First, positions 1 and 3 can interact directly in one layer even though they are not adjacent. Second, the rows of A sum to one, so each output is a weighted average of values. Third, without positional information, the first and third positions behave identically because their representations are identical. A positional encoding would break this symmetry and allow the model to distinguish "first" from "third."

One should not mistake this toy example for a realistic language model computation. Its purpose is structural. It shows that self-attention is an explicit matrix operator whose pattern of interaction can be inspected directly. In larger models the same principle survives, only with higher-dimensional features and many heads.

Complexity and comparison with recurrence

Transformers are often contrasted with recurrent models because both are used for sequential data, but the contrast is subtler than "new versus old." They organize computation differently.

For a recurrent layer with hidden width d, the dependence on sequence length is typically linear: processing T positions requires T recurrent updates, each using matrix multiplications of size roughly $d \times d$. This leads to arithmetic cost on the order of $O(Td^2)$, with an inherently sequential computation graph.

For dense self-attention, one still pays $O(Td^2)$ for the linear projections, but the interaction matrix $QK^\top$ and the multiplication by V contribute an additional $O(T^2d)$ term. Thus, with width fixed, dense self-attention has quadratic dependence on sequence length. The price of direct all-to-all interaction is the T^2 attention matrix.

Yet there is an architectural advantage. The pairwise interactions for all positions can be computed in parallel once the token representations for a layer are available. Moreover, the path length between two arbitrary positions is very short: one self-attention layer can connect any token directly to any other token, whereas a recurrent model may require many sequential steps for the same information to propagate.

Architecture	Cost in T (with width fixed)	Sequential depth	Long-range path length
Recurrent layer	linear in T	T	on the order of T
Dense self-attention layer	quadratic in T	essentially constant per layer	1

This comparison explains both the success and the limitations of transformers. They are powerful because they make global interaction easy and parallelizable. They can be expensive because unrestricted interaction across all token pairs scales poorly for very long sequences. Many later architectural refinements can be understood as attempts to preserve the expressive advantages of attention while reducing this quadratic cost.

Why transformers became general-purpose

The transformer block is now used far beyond classical text sequences. It can process tokenized image patches, audio frames, multimodal streams, sets, and structured collections of objects. The reason is not that all such data are "really language." The reason is that the block itself is abstract

and modular. It only assumes that we have a collection of representations together with some way of encoding whatever structural information matters — positional, spatial, temporal, or relational.

From this point of view, a transformer is a general-purpose architecture because it separates three ingredients:

- a representation of the elements to be processed,
- an interaction operator that mixes information across elements by learned relevance,
- a per-element nonlinear map that enriches features after interaction.

That decomposition is flexible enough to be reused across many domains.

What to remember. A transformer layer is a repeated algebraic block built from multi-head self-attention, a positionwise MLP, residual connections, and normalization. Scaled dot-product attention computes a row-stochastic interaction matrix from learned similarities. Without positional information, self-attention is permutation equivariant, so explicit positional encoding is necessary for genuine sequence modeling. Transformers replace long sequential interaction paths with direct pairwise interaction, but dense self-attention incurs quadratic cost in sequence length.

Common confusion. It is easy to say that transformers "remove order." That is not quite right. The attention operator by itself is insensitive to order, but the full model can represent order once positional information is supplied. It is also easy to think that the transformer is "just attention." In fact, the architecture depends equally on residual structure, normalization, and the repeated alternation between cross-position interaction and per-position nonlinear transformation.

Exercises.

1. Let $X \in \mathbb{R}^{T\times d}$ and let P be a permutation matrix. Write out carefully why $QK^\top$ transforms as $P(QK^\top)P^\top$ under the substitution $X \mapsto PX$.
2. Suppose all token representations in a self-attention layer are identical and no positional information is added. Show that every row of the attention matrix is the same. What does this imply about the outputs?
3. For the toy example in this section, replace
$$Q = K = [1, 0, 1]^\top$$
by
$$Q = K = [2, 0, -2]^\top.$$
Compute the new attention matrix and explain how the sharper score differences change the interaction pattern.
4. Compare a recurrent layer and a self-attention layer as mechanisms for modeling a dependency between the first and last token of a long sequence. Which architecture has the shorter interaction path, and which has the cheaper asymptotic dependence on sequence length?
5. Derive the output dimensions of a multi-head attention block with H heads, model width d, per-head key dimension d_k, and per-head value dimension d_v.

Notes and references. The transformer architecture was introduced by Vaswani and collaborators in *Attention Is All You Need*. Its attention mechanism builds on earlier encoder–decoder attention ideas, especially the work of Bahdanau, Cho, and Bengio on neural machine translation. For

pedagogical development, it is useful to view the transformer not as a historical slogan but as a structured operator composed of attention, residual pathways, normalization, and per-position nonlinear maps. That viewpoint also clarifies why positional encoding is essential and why complexity in sequence length became a central issue in later transformer variants.

3.8 Scaling Transformer Architectures and Multimodal Extensions

The previous section treated the transformer as a structured operator on a sequence of representations. The natural next question is what happens when that operator is enlarged, specialized, or connected to non-text modalities. This section studies that question at three levels. First, there is a fairly clean *theoretical* layer: one can write down the algebra of encoder-only, decoder-only, and encoder–decoder blocks, and one can analyze how computational cost depends on depth, width, and context length. Second, there is a large *empirical* layer: many scaling trends are observed reliably in practice, but they are not consequences of a single theorem. Third, there is a broad region of *design practice*: tokenization choices, multimodal interfaces, and long-context approximations are often guided by engineering tradeoffs more than by settled theory.

That distinction matters pedagogically. Students often hear that modern transformer systems became powerful simply by "scaling up." But scaling is not a magic word. It means enlarging specific architectural axes, paying specific computational costs, and making specific modeling decisions about how representations are formed and how information is allowed to interact.

Scaling axes: depth, width, and context length

Let a transformer have:

- depth L, meaning the number of repeated blocks,
- model width d, meaning the hidden dimension of token representations,
- context length T, meaning the number of tokens processed in one forward pass.

To first approximation, these are the three principal architectural scaling axes.

Definition 3.64 (Architectural scaling axes)**.** For a transformer family with fixed block design, scaling means increasing one or more of the quantities L, d, and T:

1. **depth scaling:** increase the number of layers L,
2. **width scaling:** increase the hidden size d, the feedforward size, or the number of heads,
3. **context scaling:** increase the sequence length T on which attention is computed.

These axes do not behave symmetrically. Depth mainly enlarges the number of repeated transformations through which information can be refined. Width enlarges the dimensional capacity of each representation and usually the parameter count of each block. Context length enlarges the set of positions that may interact, and in dense attention it is the most computationally expensive axis.

To make this precise, recall from the previous section that one transformer block applies linear projections to obtain Q, K, and V, forms an attention matrix of size $T \times T$, applies that matrix to values, and then applies a positionwise multilayer perceptron. If the width is d and the feedforward dimension is proportional to d, then for one layer the dominant arithmetic cost is approximately

$$O(Td^2) + O(T^2 d).$$

The first term comes from linear maps and pointwise MLP transformations. The second term comes from all-pairs attention interactions. Across L layers this becomes approximately

$$O(LTd^2 + LT^2 d).$$

Likewise, with fixed feedforward ratio, the parameter count grows roughly like

$$O(Ld^2).$$

Remark 3.65 (How the scaling axes differ). At fixed T, increasing depth by a factor of two roughly doubles the blockwise computation. At fixed L and T, increasing width by a factor of two typically multiplies parameters and much of the arithmetic cost by roughly four. At fixed L and d, increasing context length by a factor of two roughly doubles the projection cost but multiplies the dense attention interaction term by roughly four. This is why long-context modeling is often the first place where naive scaling becomes prohibitive.

These asymptotics are theoretical bookkeeping statements. They tell us how costs scale when architectural dimensions increase. They do *not* by themselves guarantee better learning behavior, better generalization, or better sample efficiency. Those are empirical questions. In practice, larger transformers often do perform better when data, optimization, and regularization are also scaled appropriately, but that observation is a broad empirical pattern rather than a theorem deduced from the complexity formulas above.

Architectural variants: encoder-only, decoder-only, and encoder–decoder

Although the transformer block has a common algebraic core, different tasks organize that core differently. The distinctions are not superficial. They determine which positions are allowed to interact and what output object the network is trying to produce.

Definition 3.66 (Encoder-only transformer)**.** An encoder-only transformer takes an input sequence

$$X = (x_1, \dots, x_T)$$

and maps it to contextualized representations

$$H^{(L)} \in \mathbb{R}^{T \times d}$$

using repeated *unmasked self-attention*. Every position may attend to every other observed position. Such architectures are natural when the goal is representation learning, classification, tagging, retrieval, or any task where the full input is available at once.

Formally, if $H^{(0)}$ denotes token embeddings plus positional information, an encoder layer has the same general form as in Section 3.7:

$$\widetilde{H}^{(\ell)} = H^{(\ell)} + \mathrm{MHA}(\mathrm{LN}(H^{(\ell)})),$$
$$H^{(\ell+1)} = \widetilde{H}^{(\ell)} + \mathrm{MLP}(\mathrm{LN}(\widetilde{H}^{(\ell)})).$$

No causal mask is imposed, so attention is bidirectional across the observed sequence.

Definition 3.67 (Decoder-only transformer)**.** A decoder-only transformer also processes a sequence of token representations, but its self-attention is *causally masked* so that position i can attend only to positions $j \le i$. This makes the model suitable for autoregressive generation, where the next token is predicted from previous tokens.

A convenient masked-attention notation is

$$\mathrm{Attn}_{\mathrm{mask}}(Q, K, V; M) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

where the mask $M \in \mathbb{R}^{T \times T}$ satisfies

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Then a decoder-only block uses masked multi-head self-attention in place of bidirectional self-attention. The difference in formula is small, but conceptually it is large: the architecture now represents a factorization of a sequence distribution into one-step conditional predictions.

Definition 3.68 (Encoder–decoder transformer)**.** An encoder–decoder transformer contains two streams:

1. an encoder producing source-side representations from an observed input sequence,
2. a decoder producing target-side representations by combining masked self-attention on the partial target with *cross-attention* to the encoder outputs.

This structure is natural for transduction problems such as translation, summarization, or speech-to-text mapping.

The encoder builds a memory of the source sequence. The decoder then generates a target sequence while consulting that memory. This division of labor is mathematically clean and architecturally important. The encoder is typically bidirectional over the source. The decoder is causal over the target. Cross-attention links the two.

Architectural roles of common transformer variants

Variant	Self-attention pattern	Typical output role	Canonical use case
Encoder-only	bidirectional over observed input	contextual representations of all input tokens	classification, tagging, retrieval, representation learning
Decoder-only	causal over prefix tokens	next-token distribution or generated continuation	language modeling and autoregressive generation
Encoder–decoder	bidirectional encoder, causal decoder, plus cross-attention	output sequence conditioned on source sequence	translation, summarization, conditional generation

Figure 3.11: A comparison figure emphasizing that the three major transformer organizations differ primarily in which interactions are permitted and in what object the network is designed to produce.

Cross-attention written formally

Cross-attention is the algebraic device that lets one representation stream query another. Suppose the decoder has hidden states

$$Y \in \mathbb{R}^{T_y \times d}$$

and the encoder has produced memory states

$$Z \in \mathbb{R}^{T_x \times d}.$$

Then cross-attention forms queries from Y, but keys and values from Z:

$$Q = YW_Q, \qquad K = ZW_K, \qquad V = ZW_V.$$

The resulting attention operator is

$$\mathrm{CrossAttn}(Y, Z) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Since Q has T_y rows and K has T_x rows, the attention matrix has shape

$$T_y \times T_x.$$

Thus each target position forms a content-based weighted average over source-side value vectors.

In a pre-normalization encoder–decoder block one may write:

$$\begin{aligned}
U^{(\ell)} &= \mathrm{LN}(H^{(\ell)}),\\
\widehat{H}^{(\ell)} &= H^{(\ell)} + \mathrm{MaskedMHA}(U^{(\ell)}),\\
V^{(\ell)} &= \mathrm{LN}(\widehat{H}^{(\ell)}),\\
\widetilde{H}^{(\ell)} &= \widehat{H}^{(\ell)} + \mathrm{CrossMHA}(V^{(\ell)}, E),\\
H^{(\ell+1)} &= \widetilde{H}^{(\ell)} + \mathrm{MLP}(\mathrm{LN}(\widetilde{H}^{(\ell)})),
\end{aligned}$$

where E denotes the final encoder memory. This equation makes the architectural role of cross-attention explicit: self-attention organizes target-side interactions, whereas cross-attention injects source-conditioned information.

Example 3.69 (Why cross-attention is different from concatenation)**.** Suppose the source sentence has length $T_x = 5$ and the partial target has length $T_y = 3$. Cross-attention yields a 3×5 matrix of alignment weights. The first target token may focus mostly on source token 2, the second on tokens 3 and 4, and the third on token 5. This is not the same as simply concatenating source and target tokens into one long sequence. The architectural asymmetry remains visible in the equations: target positions are the queries, while source positions provide the keys and values.

Multimodal tokenization and representation interfaces

Transformers became especially influential because their basic algebra is indifferent to the semantic meaning of a token. What matters is that each element of the input is represented as a vector in a shared hidden space. This shifts an important part of architecture design to the *representation interface* between raw modality-specific data and token sequences.

Let $x^{(m)}$ denote an input from modality m, for example text, image, audio, or video. A multimodal system introduces a modality-specific map

$$\phi_m : \mathcal{X}_m \to \mathbb{R}^{T_m \times d},$$

which converts the raw input into a sequence of d-dimensional token vectors. Typical examples include:

- text token embeddings for discrete vocabulary items,
- image patch embeddings for a grid of pixels,
- audio frame or spectrogram patch embeddings,
- video tubelet or frame embeddings.

After this stage one may add positional or spatial encodings and obtain

$$Z^{(m)} = \phi_m(x^{(m)}) + P^{(m)}.$$

The transformer then operates on these token sequences through either shared self-attention, cross-attention, or a hybrid of the two.

Two common multimodal design patterns are worth separating. First, one may form a *single token stream*

$$Z = \mathrm{Concat}(Z^{(1)}, Z^{(2)}, \ldots, Z^{(M)}),$$

and run self-attention over the concatenated sequence. This encourages direct all-to-all interaction, but it can become expensive when the total token count is large. Second, one may keep modality-specific streams and use cross-attention modules to couple them. This preserves a clearer architectural separation and often gives more control over what information is exchanged.

Remark 3.70 (What is formal and what is design practice). The formal part is simple: each modality is mapped into vectors of common width d, and attention operators are then applied to those vectors. What is much less canonical is how the tokenization map ϕ_m should be chosen. Patch size, frame rate, quantization scheme, positional encoding, and fusion schedule are usually design decisions guided by empirical performance and computational budget rather than by a complete theory.

This observation helps explain why multimodal transformers feel simultaneously unified and heterogeneous. The unification comes from the shared algebra of token interaction. The heterogeneity comes from the fact that raw modalities have very different local structure, sampling rates, invariances, and compression requirements. A text tokenizer, an image patch embedder, and an audio frontend all feed the transformer, but they solve very different preprocessing problems.

Long-context modeling and sparse approximations

Dense self-attention is attractive because every token may interact with every other token in one layer. Its main weakness is the quadratic T^2 dependence on context length. This has motivated a large family of approximations and architectural modifications for long-context modeling. The details vary widely, but most methods can be grouped into a few conceptual classes:

- **local or windowed attention,** where each token interacts mainly with nearby positions,
- **block-sparse or pattern-based attention,** where only selected token pairs are allowed to interact,
- **low-rank or kernelized approximations,** which try to approximate dense attention more cheaply,
- **memory or compression mechanisms,** which summarize distant context into a smaller set of states.

The central mathematical tradeoff is clear. Dense attention gives maximum interaction flexibility at cost $O(T^2 d)$. Sparse or approximate attention reduces computation and memory, but it also constrains or approximates the family of interaction matrices the model can realize exactly.

What is known rigorously here is limited but useful. One can analyze the asymptotic reduction in arithmetic or memory cost and sometimes characterize what sparsity pattern is being imposed. What remains largely empirical is how much modeling power is lost for a given approximation and which approximation is best for a given domain. In practice, long-context architecture is still heavily shaped by design experimentation. For some tasks local patterns are sufficient most of the time. For others, a few carefully chosen global tokens or retrieval-style memory mechanisms are important. There is no single universally optimal replacement for dense attention.

A useful synthesis: theory, empirics, and practice

At this point it is worth separating three claims that are often blended together.

Theoretical statements. One can state precisely how encoder-only, decoder-only, and encoder–decoder forms differ. One can write exact equations for self-attention and cross-attention. One can also derive the dominant asymptotic dependence of arithmetic cost and memory use on L, d, and T. These are mathematical properties of the architecture itself.

Empirical statements. Larger transformer systems often improve when model size, data, and optimization scale together. Shared token-space architectures often transfer surprisingly well across modalities. Long-context approximations sometimes preserve much of the useful behavior of dense attention while reducing cost. These are broad empirical regularities, but they are not universal theorems.

Design-practice statements. How many layers should be used. How wide each layer should be. How context should be truncated or extended. Whether modalities should be concatenated early, fused late, or coupled by cross-attention. Which sparse attention pattern is best. These are often decisions made by architectural judgment, ablation studies, hardware constraints, and application-specific goals.

Students should keep these categories distinct. Doing so prevents two opposite mistakes. The first is to treat all of modern transformer engineering as if it were already fully understood mathematically. The second is to conclude that because many choices are empirical, no clean theory exists at all. In reality, transformer architecture contains a solid algebraic core together with a large and active outer layer of empirical design.

What to remember. Transformer scaling has three principal axes: depth L, width d, and context length T. With dense attention, a useful leading-order cost model is $O(LTd^2 + LT^2d)$, so long context is often the most expensive axis to scale. Encoder-only, decoder-only, and encoder–decoder transformers differ primarily in the attention constraints they impose and the output object they are designed to produce. Cross-attention uses queries from one stream and keys and values from another. Multimodal transformers rely on representation interfaces that map different raw modalities into token sequences in a common hidden space. Sparse and long-context variants reduce cost by restricting or approximating dense all-to-all interaction.

Common confusion. It is tempting to think that "a transformer" is one fixed architecture. In fact, the common name covers several distinct organizations of the same basic operators. It is also easy to confuse asymptotic complexity with learning performance. A model may be more expensive in T yet more effective because its interaction structure is richer. Finally, multimodal modeling is not obtained automatically just by concatenating heterogeneous inputs. A great deal of the actual modeling work lies in the interface that converts raw modality-specific data into useful token representations.

Exercises.

1. Starting from the cost of one dense self-attention layer, derive the leading-order complexity $O(LTd^2 + LT^2d)$ for a stack of L layers. State clearly which terms come from projections, attention scores, and value aggregation.

2. Compare an encoder-only architecture and a decoder-only architecture on the same input sequence of length T. Which one permits token i to depend on token $j > i$ in a single layer? Explain the modeling consequence.
3. Let $Y \in \mathbb{R}^{T_y \times d}$ and $Z \in \mathbb{R}^{T_x \times d}$. Derive the dimensions of the matrices Q, K, V, the cross-attention score matrix, and the final cross-attention output.
4. Consider a multimodal system with text tokens $Z^{(t)} \in \mathbb{R}^{T_t \times d}$ and image tokens $Z^{(i)} \in \mathbb{R}^{T_i \times d}$. Compare early fusion by concatenation with fusion by cross-attention. What interaction patterns are possible in each design?
5. Suppose the context length is doubled while depth and width are fixed. How do the leading projection term and the leading dense-attention interaction term change asymptotically?
6. Give one advantage and one disadvantage of local-window attention compared with dense attention. Your answer should distinguish computational benefit from possible representational loss.

Notes and references. The basic transformer framework comes from Vaswani et al. Encoder-only and decoder-only specializations became central in large-scale representation learning and autoregressive language modeling, while encoder–decoder designs remained especially important for conditional sequence transduction. Cross-attention should be viewed as a formal operator that couples two representation streams, not merely as an implementation detail. For multimodal extensions, the shared architectural theme is the conversion of raw inputs from different modalities into token sequences inside a common representation space, followed by self-attention or cross-attention based interaction. The literature on long-context and sparse attention is broad and still evolving; the cleanest unifying perspective is to treat these models as attempts to preserve the benefits of content-based interaction while weakening the quadratic cost of dense all-to-all attention.

3.9 Worked Examples and Comparative Case Studies

This section is meant to consolidate the whole chapter. Up to this point, convolution, recurrence, gating, attention, and transformers have been introduced as different architectural responses to different kinds of structure. The most useful final step is to see these ideas operating on small, concrete examples. A worked example forces us to ask the right question: what structural assumption is the model making, what operator is it applying, and why is that operator well matched — or badly matched — to the task?

The examples below are deliberately small. Their purpose is not to impress by scale, but to reveal the logic of the architecture. In each case, one should pay attention not only to the numerical computation, but also to the inductive bias that computation expresses.

A mini CNN example: detecting a vertical edge

Consider the following toy grayscale image:

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}.$$

The image is bright on the left and dark on the right, so it contains a vertical transition near the middle columns. A natural convolutional filter for this kind of pattern is

$$K = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}.$$

This filter compares the left and right parts of a small patch. If both sides of the patch are similar, the output is small. If the left side is bright and the right side is dark, the output is large and positive.

Using stride 1 and no padding, the convolution output has size 3×3. Each entry is obtained by placing the filter on a 2×2 patch and taking the inner product. For the first row of patches we obtain

$$\langle K, X_{1:2,1:2} \rangle = \left\langle \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \right\rangle = 1 - 1 + 1 - 1 = 0,$$

$$\langle K, X_{1:2,2:3} \rangle = \left\langle \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \right\rangle = 1 + 0 + 1 + 0 = 2,$$

$$\langle K, X_{1:2,3:4} \rangle = \left\langle \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \right\rangle = 0.$$

By the same reasoning for the remaining rows, the full feature map is

$$X * K = \begin{bmatrix} 0 & 2 & 0 \\ 0 & 2 & 0 \\ 0 & 2 & 0 \end{bmatrix}.$$

After a ReLU nonlinearity, nothing changes because all entries are already nonnegative. A max-pooling or global-max step would then return the value 2, signaling that the image contains the desired vertical edge pattern somewhere in the field of view.

Example 3.71 (What this example shows)**.** The important point is not the specific number 2. The important point is that the *same* filter was applied at every location. The model did not need one parameter set for the top edge, another for the middle edge, and another for the bottom edge. That is the inductive bias of convolution: local pattern detectors are shared across space. If the same kind of edge appears in another location, the same filter can detect it there too.

A dense fully connected layer could also learn to detect this image pattern, but it would not build translation sensitivity and parameter sharing into the operator itself. The CNN does so directly. This is why convolution is especially natural for images: the operator already encodes locality and repeated spatial motifs.

Recurrence versus attention on a toy sequence memory task

Now consider a toy sequence problem. A sequence consists of symbols from the set $\{A, B\}$, and the target at the final position is: *repeat the first symbol of the sequence.* For example,

$$(A, B, B, A, ?) \longmapsto A.$$

This is a deliberately simple task, but it separates two architectural ideas very clearly. The model must preserve or recover information from the beginning of the sequence when it reaches the end.

Recurrent solution. A recurrent model computes

$$h_t = \phi(W_h h_{t-1} + W_x x_t),$$

where x_t is the embedding of the current token and h_t is the hidden state. To solve the task, the network must somehow store the information contained in x_1 inside the evolving state and prevent later updates from destroying it. For a short sequence this is possible. For a long sequence it becomes harder, because the hidden state must continuously compress the past into a single vector while still processing new inputs. The required information travels through the chain

$$x_1 \to h_1 \to h_2 \to h_3 \to \cdots \to h_T.$$

Thus the relevant interaction path from the first token to the last output has length on the order of T.

Attention-based solution. A decoder-style or self-attention model can treat the same task differently. At the final position, it may compute

$$y_T = \sum_{j=1}^{T} a_{Tj} v_j, \qquad \sum_{j=1}^{T} a_{Tj} = 1,$$

where v_j is the value vector for token j. If the model learns that the final query should focus on the first position, then one possible attention pattern is

$$(a_{T1}, a_{T2}, a_{T3}, a_{T4}, a_{T5}) = (0.92, 0.03, 0.02, 0.02, 0.01).$$

Then

$$y_T \approx 0.92 v_1 + 0.03 v_2 + 0.02 v_3 + 0.02 v_4 + 0.01 v_5,$$

so the output is dominated by the representation of the first symbol. The relevant information need not survive repeated state updates; it can be retrieved directly.

Example 3.72 (Same task, different operator)**.** For this toy task, the recurrent architecture behaves like a running memory state, while self-attention behaves like a content-based retrieval rule. The RNN says: "compress the past into a state and carry it forward." Attention says: "when the final query is formed, look back and pull the needed representation directly." Neither statement is universally better in every setting, but they represent sharply different inductive biases.

This example also clarifies why gated recurrence was historically important. An LSTM or GRU modifies the recurrent update so that important information can be preserved more selectively, partially narrowing the gap between plain recurrence and attention-based access. Still, the underlying logic remains different: recurrence transports information through state evolution, while attention creates direct interaction edges.

A toy machine-translation alignment example

Machine translation is a particularly instructive case because it requires both representation and alignment. Suppose the source sentence is

$$x = (\text{le}, \text{chat}, \text{dort})$$

and the target sentence is

$$y = (\text{the}, \text{cat}, \text{sleeps}).$$

Let the encoder produce source representations

$$H = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \in \mathbb{R}^{3\times d},$$

corresponding to the three source words. At target step i, the decoder forms a query q_i and computes cross-attention weights

$$\alpha_i = \text{softmax}\left(\frac{q_i K^\top}{\sqrt{d_k}}\right), \qquad K = HW_K, \qquad V = HW_V.$$

Equivalently, for all target positions at once,

$$C = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where the rows of Q come from target-side decoder states.

To make this concrete, suppose the decoder produces the following score matrix before softmax:

$$S = \begin{bmatrix} 3 & 1 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{bmatrix}.$$

Applying row-wise softmax gives approximately

$$A = \text{softmax}_{\text{row}}(S) \approx \begin{bmatrix} 0.84 & 0.11 & 0.05 \\ 0.02 & 0.96 & 0.02 \\ 0.02 & 0.02 & 0.96 \end{bmatrix}.$$

The interpretation is immediate:

- when producing *the*, the decoder attends mostly to *le*,
- when producing *cat*, it attends overwhelmingly to *chat*,
- when producing *sleeps*, it attends overwhelmingly to *dort*.

The corresponding context vectors are

$$c_1 = 0.84v_1 + 0.11v_2 + 0.05v_3,$$

$$c_2 = 0.02v_1 + 0.96v_2 + 0.02v_3,$$

$$c_3 = 0.02v_1 + 0.02v_2 + 0.96v_3.$$

These are not hard dictionary lookups. They are soft weighted combinations of encoded source representations. That softness matters because translation is rarely a one-word-at-a-time substitution rule. Sometimes multiple source words contribute jointly, and sometimes word order differs between languages.

Example 3.73 (Why attention solved a real bottleneck)**.** In a classical encoder–decoder model without attention, the entire source sentence had to be compressed into a single fixed-size vector before decoding began. For longer or more complex sentences, this creates an information bottleneck. Cross-attention removes much of that bottleneck by allowing each target token to consult the full source representation sequence directly. Thus attention is not just a convenience for visualization; it solves a precise architectural problem.

This machine-translation example also shows why alignment matrices became such powerful pedagogical objects. They let us inspect, at least approximately, which source positions influenced which target outputs. That makes the operator visible in a way that a hidden recurrent state often is not.

A time-series forecasting example

Consider a one-step-ahead forecasting problem for a short noisy time series:

$$(20, 22, 21, 23).$$

Suppose we believe the underlying signal evolves through a slowly changing latent level and that short-term noise should be smoothed. A very simple recurrent state model for this idea is

$$h_t = 0.7h_{t-1} + 0.3x_t, \qquad \hat{x}_{t+1} = h_t,$$

with initial state $h_1 = x_1 = 20$. Then

$$h_2 = 0.7 \cdot 20 + 0.3 \cdot 22 = 20.6,$$

$$h_3 = 0.7 \cdot 20.6 + 0.3 \cdot 21 = 20.72,$$

$$h_4 = 0.7 \cdot 20.72 + 0.3 \cdot 23 \approx 21.40.$$

So the one-step-ahead forecast is

$$\hat{x}_5 = h_4 \approx 21.40.$$

This forecast does not simply copy the last observation 23. Instead it forms a smoothed state that reflects the recent history. That is precisely the inductive bias of recurrent or state-space style forecasting: observations are treated as partial evidence about an evolving hidden state.

Now compare this with a causal convolutional rule such as

$$\hat{x}_{t+1} = 0.5x_t + 0.5x_{t-1}.$$

Applied at $t = 4$, this gives

$$\hat{x}_5 = 0.5 \cdot 23 + 0.5 \cdot 21 = 22.$$

This rule is also reasonable, but it encodes a different assumption. It says that the next value depends mainly on a local window of recent observations, not on a hidden state that is updated over time. An attention-based forecaster would express yet another assumption: the next value may depend on selected past times, perhaps even distant ones, if their patterns are relevant.

Example 3.74 (What the forecasting example teaches)**.** There is no unique "time-series architecture." If the series is governed by a slowly evolving latent state, recurrence is very natural. If short local motifs dominate, causal convolution may be simpler and more effective. If irregular long-range interactions matter — for example, if today should be compared with the same day in earlier cycles — attention can be very attractive. The architecture should follow the dependence structure one expects in the data.

Final comparison across the chapter

The worked examples point toward one central lesson. Architectural names are secondary. What matters is the structural claim each architecture makes about the data and the operator by which it uses that claim. Table 3.1 summarizes the chapter in that language.

Architecture	**Assumption**	**Operator**	**Strength**	**Weakness**	**Natural domain**
CNN	Local spatial structure and repeated patterns across location	Shared local convolution filters, often followed by pooling	Parameter sharing; strong locality prior; translation-friendly features	Less natural for arbitrary long-range dependence without added mechanisms	Images, spatial signals
RNN	Sequence is processed through evolving state	Recurrent update $h_t = \phi(W_h h_{t-1} + W_x x_t)$	Natural state evolution; compact online processing	Long-range information must survive many updates; sequential computation	Time series, streaming sequences
LSTM / GRU	Important state information should be selectively preserved or overwritten	Gated recurrent state transitions	Better memory control than plain RNNs; robust on many sequential tasks	Still fundamentally sequential; state bottleneck remains	Language, speech, temporal data
Attention / encoder–decoder attention	Relevance should be computed content-dependently between positions or streams	Weighted aggregation from learned similarity scores	Explicit alignment; direct access to relevant source information	Dense interaction can be expensive; needs careful representation design	Translation, retrieval, cross-modal fusion
Transformer	Rich sequence behavior can be built from repeated attention blocks plus MLPs, residuals, and normalization	Multi-head self-attention and feedforward blocks	Parallelizable global interaction; flexible across many modalities	Dense attention scales quadratically in context length; positional structure must be supplied	Language, vision patches, multimodal token streams

Table 3.1: A comparative summary of the main architectures in this chapter. The key comparison is not historical order, but the relation between structural assumption and operator.

The table should not be read as a ranking. It is a map. Each row describes a way of embedding prior assumptions into a learnable system. The central unifying message of the chapter is therefore the following: architecture is the mechanism by which inductive bias becomes operational.

What to remember. Convolution, recurrence, gating, attention, and transformers should be compared through the structures they assume and the operators they apply. A CNN uses shared local filters and is naturally suited to repeated spatial patterns. A recurrent model uses an evolving state and is naturally suited to temporal evolution. Attention uses content-dependent weighted aggregation and is naturally suited to alignment and selective retrieval. A transformer turns attention into a full repeated architectural block that supports flexible global interaction. The best architecture is not the most fashionable one, but the one whose operator matches the dependence structure of the task.

Common confusion. Students often ask which architecture is "best." That question is usually too vague. A better question is: what does the task require the model to preserve, compare, or aggregate? It is also easy to confuse representational power in principle with suitability in practice. A dense network can represent many of the same functions as a CNN or transformer, but it may do so with a much weaker built-in prior and much less efficient learning. Finally, one should not confuse a worked toy example with a full empirical benchmark. The role of a toy example is to reveal the structural logic of the operator.

Exercises.

1. In the CNN example, replace the filter
$$\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$
by
$$\begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}.$$
Compute the new feature map and explain what pattern this second filter is designed to detect.
2. For the toy memory task, explain from first principles why the information path from the first token to the last output has length on the order of T in a recurrent model but length 1 in a self-attention layer.
3. In the translation example, verify numerically that each row of the alignment matrix A sums to one. Then explain why this implies that each context vector is a convex combination of value vectors.
4. Compare the recurrent forecasting rule
$$h_t = 0.7h_{t-1} + 0.3x_t$$
with the causal convolutional rule
$$\hat{x}_{t+1} = 0.5x_t + 0.5x_{t-1}.$$
What hidden assumption about temporal structure is built into each method?
5. Give one task for which a CNN is likely to be more natural than a transformer, and one task for which a transformer is likely to be more natural than a plain RNN. Justify both answers in terms of inductive bias rather than popularity.
6. Choose any two rows from Table 3.1 and write a short proof-style argument explaining why their operators impose genuinely different constraints on how information can move through the network.

Notes and references. The convolutional viewpoint developed in this chapter traces back to the early CNN literature associated with LeCun and collaborators. The recurrent and gated examples are closely connected to the RNN, LSTM, and GRU traditions developed by Elman, Hochreiter and Schmidhuber, and Cho and collaborators. The translation alignment example reflects the conceptual importance of attention in neural machine translation, especially in the work of Bahdanau, Cho, and Bengio. The transformer synthesis inherits the operator viewpoint of Vaswani and collaborators and later general-purpose sequence modeling work. Pedagogically, these examples are best read not as historical trophies, but as comparative demonstrations of how architectural assumptions become computational operators.

3.10 What Is Understood and What Remains Open

The theory and practice of architecture in deep learning contain both mature ideas and major open questions.

What is well understood

This chapter developed a central theme of deep learning architecture: different model classes should be understood as different structured operators acting on data with different kinds of regularity. Convolution exploits locality and translation structure. Recurrence exploits ordered state evolution. Gated recurrence modifies state evolution so that memory can be preserved or overwritten selectively. Attention introduces content-dependent interaction. Transformers organize these interaction rules into reusable sequence-processing blocks.

A good closing section should do more than summarize these ideas rhetorically. It should also separate what the chapter established mathematically, what it distilled as durable design principles, and what remains open. That distinction matters for students. Without it, one can either become too skeptical and treat all architecture design as ad hoc engineering, or too optimistic and imagine that the current empirical success of modern architectures is already fully explained by theory. The right view lies between these extremes. Some architectural facts are precise and stable. Some design lessons are strong but empirical. Some central questions remain genuinely unresolved.

A theorem map of the chapter

The word *theorem* should be used carefully. Not every useful architectural statement is a theorem in the narrow mathematical sense. Still, this chapter did establish a coherent map of mathematically clean claims and exact operator identities. Table 3.2 collects the most important ones.

Several remarks help interpret this map correctly. First, the cleanest results in architecture are often *structural* rather than *statistical*. For example, one can prove permutation equivariance of self-attention without positional encoding exactly, but one cannot yet give a comparably complete theory of why transformer pretraining scales so effectively across domains. Second, many important architectural insights arise from algebraic form. The fact that attention produces convex combinations is already informative because it constrains what a single layer can and cannot represent directly. Third, asymptotic cost formulas are valuable but limited. They tell us how arithmetic and memory scale, but not by themselves whether the corresponding model class is best for a given problem.

Design principles distilled from the chapter

A chapter on architecture should end not only with claims but also with principles. These are not all theorems. They are higher-level lessons that help one decide which architectural family is appropriate for which data geometry.

Principle 1: Build the operator around the dominant structure of the data. Images exhibit local spatial regularity and approximate translation symmetry, so convolutions are natural. Sequences exhibit order and evolving context, so recurrent or attention-based operators are natural. Graphs and manifolds exhibit relational rather than grid-like structure, so architectures must respect adjacency, symmetry, and geometry directly rather than pretending the data lie on a Euclidean lattice. The most durable lesson of the chapter is that architecture is inductive bias made operational.

Principle 2: Separate local processing from long-range interaction. One recurring design pattern is to use one mechanism for nearby structure and another for distant dependence. CNNs use local filters and then enlarge receptive fields by stacking or pooling. RNNs use local temporal updates but must transport information through state. Transformers make long-range interaction explicit through attention. Modern hybrid designs often combine these ideas rather than choosing only one.

Principle 3: Memory is an architectural object, not only an optimization side effect. Plain recurrence stores the past in a continuously updated hidden state. Gated recurrence makes preservation and forgetting controllable. Attention turns memory retrieval into a weighted lookup over representations. These are not cosmetic differences. They reflect different hypotheses about what should be remembered, how it should be accessed, and what computational path information should take through the model.

Principle 4: Depth composes simple operators into richer ones. A single local filter sees only a small patch. A single recurrent step sees only one update. A single attention layer computes one round of adaptive interaction. Depth matters because repeated composition enlarges receptive field, increases representational hierarchy, and allows iterative refinement of intermediate states. This principle is broad enough to cover CNNs, RNNs, and transformers alike.

Principle 5: Symmetry and equivariance should be used when they are real, not merely convenient. Architectural sharing is powerful when it matches actual invariance or equivariance in the domain. Weight sharing in convolution is effective because the same local pattern may matter across positions. Permutation equivariance is appropriate for set-like inputs. Graph neural networks exploit relational symmetries. But the chapter also showed the danger of overextending symmetry assumptions. A model that ignores order entirely is unsuitable for language unless positional or relational information is reintroduced.

Principle 6: Keep theory, empirical regularity, and engineering practice distinct. A theorem about equivariance is not the same kind of statement as an empirical scaling law. An empirical scaling law is not the same kind of statement as a hardware-driven design convention. Students often blur these levels together. Doing so makes the literature seem either more chaotic or more settled than it really is. Part of architectural maturity is learning to ask which category a claim belongs to.

Remark 3.75 (What is probably most stable over time). Specific fashionable architectures may change. The most stable ideas are deeper than the current brand names: locality, symmetry, hierarchy, state, memory control, content-based interaction, and representation interfaces. These are the concepts most likely to survive shifts in implementation style.

What appears well understood

It is now reasonable to say that several parts of architectural deep learning are conceptually mature. Convolution is no longer mysterious. Its relation to locality, translation structure, and hierarchical feature extraction is clear. Recurrent models are also conceptually clear as sequential state machines, even if their optimization can be difficult. Attention is mathematically transparent at the operator level: it forms data-dependent weighted aggregation according to learned similarities. The transformer block itself is algebraically explicit.

What is mature here is not that every practical question has been solved. Rather, it is that the conceptual and mathematical backbone is in place. A student can understand what these operators do, what structures they encode, and why they differ. This is already a major intellectual achievement. A field becomes more mature when its main architectural objects can be written, analyzed, and compared in a common language. That is exactly what this chapter tried to provide.

Open questions

The open problems begin where architectural algebra meets modern large-scale learning. These questions should be stated explicitly rather than hidden inside vague closing prose.

Why does scale help as much as it does? We understand how cost grows with depth, width, and context length. We do not yet possess a fully satisfactory theory explaining why large transformer systems, trained on massive corpora, acquire such broad and transferable behavior. This is partly an optimization question, partly a representation-learning question, and partly a question about the interaction of architecture with data distribution.

What is the right theory of in-context computation? Attention layers clearly permit flexible content-based interaction. But modern transformer systems seem to use context not only as memory but sometimes as a substrate for task adaptation, pattern matching, or implicit algorithm execution. How much of this can be characterized cleanly in terms of the architecture itself, and how much depends on large-scale training dynamics, remains open.

When should architectural bias dominate, and when should generic scale dominate? General-purpose transformers have been remarkably successful. At the same time, vision, audio, scientific data, and geometric data often retain domain-specific structure that seems too important to ignore. A central open design question is where the boundary lies between highly specialized architectural bias and sufficiently large generic token-based modeling.

How should multimodal interfaces be designed? The transformer core is relatively uniform, but the map from raw data to tokens is not. For language, tokenization is discrete and symbolic. For images, audio, video, or scientific measurements, the interface involves patching, framing, quantization, or learned latent coding. These choices strongly affect sample efficiency, compute, and downstream behavior, yet much of their current design remains empirical.

What are the right long-context approximations? Dense attention is expensive in context length. Sparse patterns, memory compression, retrieval mechanisms, and low-rank approximations all aim to reduce this burden. The asymptotic savings are easy to state. What remains difficult is to predict, from first principles, which approximation preserves the interactions that matter for a given task.

How should geometry enter modern deep learning? The chapter emphasized classical structures such as grids and sequences, but many domains are inherently geometric or relational. One open frontier is to unify the lessons of convolution, attention, and equivariance with deeper geometric principles on graphs, manifolds, and symmetry groups. This is one reason geometric deep learning has become such an important extension of the architectural viewpoint.

Remark 3.76 (A useful intellectual discipline). When reading current papers, ask whether a claim is about expressivity, optimization, statistical generalization, computational cost, or empirical benchmark behavior. Many apparent disagreements in the literature are really disagreements between different levels of analysis.

Annotated bibliography

The references below are organized by architectural theme. They are not meant to be exhaustive. They are meant to help a mathematically inclined reader orient themselves historically and conceptually.

CNNs. The conceptual ancestor is Fukushima's *Neocognitron*, which introduced a hierarchical pattern-recognition architecture with local receptive fields and shared feature extraction ideas. LeCun and collaborators established the modern practical CNN tradition through early convolutional networks for document recognition, culminating in the well-known LeNet line. For a broader retrospective synthesis, the review by LeCun, Bengio, and Hinton is valuable because it situates convolution inside the larger rise of deep learning and explains why locality and hierarchical composition matter.

Recurrence. Early simple recurrent networks associated with Elman and Jordan provide the cleanest entry point to recurrence as state evolution. They are especially useful pedagogically because the update equations are minimal and the dynamical-systems viewpoint is easy to see. For gated recurrence, the foundational source is the LSTM paper of Hochreiter and Schmidhuber, which addresses long-term dependency through controlled memory pathways. The GRU work of Cho and collaborators gives a streamlined variant that is often easier to teach comparatively.

Attention. The turning point for neural sequence modeling is Bahdanau, Cho, and Bengio, where attention was introduced to overcome the fixed-bottleneck problem of classical encoder–decoder models. That paper is still the best place to see attention in its original role as differentiable soft alignment. Luong-style attention variants are also worth reading because they clarify several alternative scoring forms and make the alignment picture more concrete for students.

Transformers. The central reference is *Attention Is All You Need* by Vaswani and collaborators. It is foundational both because it introduced the transformer block and because it made attention a first-class sequence operator rather than an auxiliary mechanism. For broader textbook treatment, the deep learning texts by Goodfellow, Bengio, and Courville, and later modern treatments of sequence modeling, help place transformers in relation to earlier recurrent architectures. When reading transformer literature, it is useful to separate the original algebraic block from the later ecosystem of scaling methods, positional schemes, multimodal extensions, and long-context modifications.

Geometric deep learning. For readers interested in how the architectural viewpoint extends beyond grids and sequences, the geometric deep learning program associated with Bronstein, Bruna, Cohen, Veličković, and others is the natural next step. Its central message is that many modern neural architectures can be organized around symmetry, invariance, equivariance, and relational structure. This literature helps unify convolution on Euclidean domains, message passing on graphs, and more general group- or geometry-aware operators.

How to use these references. A productive reading order is often historical first, unifying second. Read one foundational paper in each category to see the motivating problem clearly. Then read a modern survey or textbook chapter to understand how the idea was absorbed into the larger field. This prevents the common mistake of learning only the most recent implementation pattern without understanding the conceptual reason it was invented.

Closing perspective

The main intellectual lesson of this chapter is that architecture is the language in which learning systems express prior assumptions about structure. A model is never just a black box with more or fewer parameters. Its operator form decides which interactions are easy, which symmetries are respected, which information paths are short or long, and which regularities are encoded before any data are seen.

The chapter also leaves an important balanced conclusion. Architecture is neither fully solved nor completely mysterious. It contains a stable mathematical core, a set of durable design principles, and a frontier of open questions that are still being negotiated by theory, experiment, and engineering practice. That is precisely why architectural study matters. It sits at the point where mathematics, modeling judgment, and empirical performance meet.

The next chapter turns from *architectural structure* to *representation learning at scale*. Once one has operators capable of expressing useful inductive bias, the next question becomes how those operators are trained on massive data to produce transferable internal representations.

What to remember. Chapter 3 established a set of mathematically clean architectural claims: convolution as a structured local operator, recurrence as state evolution, gating as controlled memory flow, attention as weighted aggregation, self-attention without positional information as permutation equivariant, and transformer scaling as governed in part by the cost of all-to-all token interaction. Beyond these claims, the chapter distilled broader design principles about matching operators to data structure, separating local and global interaction, and distinguishing theory from empirical design practice. The major open questions concern scale, in-context computation, multimodal interfaces, long-context approximation, and the integration of geometry into modern deep learning.

Common confusion. Students sometimes think that because modern architectures achieve impressive benchmark performance, the theoretical picture must already be complete. The opposite mistake is to conclude that because much of the frontier is empirical, the existing theory is shallow. Both views are wrong. The correct picture is layered: some results are exact and structural, some principles are strong but empirical, and some central design questions remain open. A second common confusion is to treat architectural categories such as CNN, RNN, and transformer as isolated historical episodes. A better perspective is to view them as different answers to one general question: what operator should be used when the data exhibit a given form of structure?

Exercises.

1. Pick three entries from Table 3.2 and classify each one as a theorem, a proposition-level structural statement, an exact architectural identity, or an asymptotic derivation. Explain why these are different kinds of mathematical knowledge.
2. Compare convolution and self-attention as inductive biases. In what sense does convolution build in stronger prior assumptions than dense self-attention? In what sense does self-attention allow a richer interaction family?

3. Starting from the recurrence $h_t = \phi(W_h h_{t-1} + W_x x_t)$, explain why the path length from token 1 to token T grows with T. Then compare this with the direct interaction path in self-attention.
4. Give a concrete example of a domain in which explicit symmetry or equivariance should guide architecture design. Explain what would likely be lost if one used a completely generic token model without encoding that structure.
5. "Transformers succeeded, so architecture no longer matters." Critically evaluate this statement using ideas from multimodal interfaces, long-context modeling, and geometric deep learning.
6. Choose one category from the annotated bibliography and trace a short historical arc from a foundational paper to a modern architectural family. Identify what stayed conceptually stable and what changed in implementation practice.

Notes and references. This closing section is intentionally synthetic. Its role is to separate the mathematically established content of the chapter from larger empirical and conceptual questions. For convolutional architectures, early foundational works by Fukushima and LeCun remain important historically, while broader retrospectives help situate CNNs within deep learning as a whole. For recurrence, Elman, Jordan, Hochreiter–Schmidhuber, and Cho et al. mark the key conceptual steps from simple state evolution to controlled gating. For attention and transformers, Bahdanau et al. and Vaswani et al. are the canonical turning points. For geometric deep learning, modern surveys provide a useful bridge from classical equivariance ideas to graph- and geometry-aware architectures. The pedagogical point is not merely to know these names, but to see that each major architecture arose in response to a precise structural problem.

Several architectural principles are by now conceptually clear.

Convolution and equivariance. The logic of locality, weight sharing, and translation structure is mathematically clean and well motivated.

Recurrence as state evolution. RNNs provide a natural framework for sequential dependence and temporal memory.

Gating as memory control. LSTM and GRU architectures embody the useful principle that information flow should be selectively regulated.

Attention as adaptive interaction. Attention mechanisms provide direct, content-dependent communication among representations.

What remains only partially understood

Other issues are much less settled.

Why transformers are so dominant across modalities. Their empirical success is undeniable, but the full theoretical explanation for their breadth of applicability remains incomplete.

What scaling really changes. Increasing depth, width, and context length clearly alters performance, but the mechanisms behind large-scale emergent behavior are still not fully understood.

The theory of in-context behavior. Attention-based models often perform sophisticated context-dependent adaptation, yet a full theory of this phenomenon remains open.

When architectural bias still beats raw scale. There are many settings where strong domain-specific structure remains valuable. Understanding the boundary between built-in architectural bias and broad general-purpose scale is an ongoing question.

Closing perspective

Architecture is not a superficial design choice. It is one of the main ways machine learning expresses assumptions about the world.

This chapter showed that different architectures encode different structural hypotheses:

- spatial locality,
- temporal state,
- controlled memory,
- adaptive interaction,
- scalable compositional processing.

The next chapter shifts from *architectural structure* to *learning structure at scale*. Once architectures can express useful inductive bias, the next question is:

> How do modern systems learn reusable representations from massive data, and how are those representations adapted across many tasks?

Topic	**Established claim**	**Nature of claim**
Convolution	A discrete convolution layer is a structured linear operator with local connectivity and weight sharing; translation of the input leads to a corresponding translation of the feature responses, up to boundary effects.	Algebraic operator identity / equivariance statement
Pooling and hierarchy	Stacking local operators increases effective receptive field with depth, allowing progressively larger-scale features to be represented.	Structural derivation from composition
Recurrence	An RNN defines a nonlinear state-space evolution $h_t = \phi(W_h h_{t-1} + W_x x_t + b)$, so sequential processing can be studied as iterative state update.	Exact dynamical-systems formulation
Gated recurrence	LSTM- and GRU-style updates create additive memory pathways and multiplicative control signals that regulate preservation, deletion, and exposure of information.	Exact operator decomposition
Attention	Attention outputs are weighted sums of value vectors, and hence lie in the span of the values and, under normalized nonnegative weights, in their convex hull.	Proposition-level structural statement
Self-attention	Self-attention without positional information is permutation equivariant.	Proposition with proof sketch
Transformer block	A transformer layer can be written algebraically as repeated composition of normalization, attention, residual addition, and pointwise nonlinear maps.	Exact architectural identity
Scaling model	For dense transformers, a useful leading-order cost model is $O(LTd^2 + LT^2d)$, separating projection/MLP cost from all-to-all token interaction cost.	Asymptotic complexity derivation
Cross-attention	Encoder–decoder interaction is described exactly by letting one stream supply queries and another stream supply keys and values.	Exact operator identity

Table 3.2: A theorem map for Chapter 3. Some entries are formal propositions, while others are exact algebraic or asymptotic statements derived from the model definitions. The point of the map is to distinguish mathematically established structure from broader empirical belief.

Chapter 4

Representation Learning, Transfer, and Foundation Models

4.1 From Task-Specific Prediction to Reusable Representation

Chapters 1 and 2 introduced learning as function approximation and deep learning as the practice of learning representations rather than hand-crafting features. Chapter 3 then emphasized that architecture matters because it imposes inductive bias. This chapter studies a further modern shift: we increasingly train models not only to solve a single task, but to acquire internal structure that can be reused across many tasks.

The central thesis of the chapter is simple but important. Modern machine learning increasingly learns *reusable internal coordinates*. A good representation organizes raw inputs into a space in which many downstream tasks become easier to express, learn, and adapt. This is the conceptual core of pretraining, transfer learning, self-supervision, and foundation models.

From a single predictor to a representation-plus-head decomposition

In the most classical supervised-learning picture, one trains a predictor

$$f : X \to Y$$

directly for one specified task. Given training examples (x_i, y_i), the objective is to choose a function from a hypothesis class that minimizes empirical risk and hopefully generalizes. This perspective remains correct, but it hides an important internal factorization. Many modern systems are better viewed as

$$f = g \circ \phi,$$

where

$$\phi : X \to Z$$

is a representation map and

$$g : Z \to Y$$

is a downstream predictor.

Definition 4.1 (Representation)**.** Let X be an input space and Z a feature or latent space. A *representation* is a map

$$\phi : X \to Z$$

that assigns to each input $x \in X$ a coded description $z = \phi(x)$. The representation is useful for a task if a predictor acting on z can solve that task more simply, more robustly, or with less labeled data than a predictor acting directly on raw inputs.

This definition is deliberately broad. The representation might be a hidden layer of a neural network, an embedding vector, a sequence of token features, a set of patch embeddings, or a learned state summarizing a time series. The key point is that the representation is not itself the final answer. It is an intermediate coordinate system in which answers become easier to extract.

Once we write $f = g \circ \phi$, the learning problem splits into two conceptual parts:

- ϕ learns how to organize the input space,
- g learns how to read out the information needed for a particular task.

This decomposition is one of the deepest structural ideas in modern machine learning. It says that part of what is learned should be *reusable*.

Task-specific features versus transferable features

A feature is *task-specific* if it is useful mainly for one narrowly defined prediction problem. For example, if one trains a classifier only to distinguish cats from dogs, the internal representation may organize the data in a way highly tuned to that distinction. That can work well for the original task while being of limited use for retrieval, captioning, segmentation, or medical anomaly detection.

By contrast, a feature is *transferable* if it participates in many tasks. Edges, shapes, object parts, syntax, semantic relations, speaker characteristics, and latent dynamical state are all examples of structure that can be useful beyond one label space. A representation is therefore more transferable when it captures regularities of the data-generating process rather than merely memorizing the decision boundary of one benchmark.

A useful formal way to express this is through a family of tasks. Let $\mathcal{T}$ index tasks, with each task $t \in \mathcal{T}$ having target space Y_t, distribution P_t over $X \times Y_t$, and downstream predictor class $\mathcal{G}_t$ acting on Z. The same representation ϕ is used for all tasks, while the readout g_t may vary with the task.

Definition 4.2 (Transferable representation relative to a task family)**.** Let $\{(P_t, \mathcal{G}_t) : t \in \mathcal{T}\}$ be a family of downstream learning problems. A representation $\phi : X \to Z$ is *transferable* relative to this family if, for many tasks t, there exists a comparatively simple predictor $g_t \in \mathcal{G}_t$ such that

$$R_t(g_t \circ \phi)$$

is small, where R_t denotes the population risk on task t.

This definition is intentionally qualitative in one place: what counts as "simple" depends on context. Sometimes simplicity means linear probes. Sometimes it means small sample complexity, shallow heads, or minimal fine-tuning. The main idea is stable: a good representation turns many tasks into easy readout problems.

A fixed representation induces a new hypothesis class

Once ϕ is fixed, the downstream learner no longer searches over arbitrary functions on X. Instead it searches over functions on Z, then composes them with ϕ. This induces a precise new hypothesis class.

Proposition 4.3 (A fixed representation induces a hypothesis class over X)**.** *Let $\phi : X \to Z$ be fixed, and let $\mathcal{G}$ be a class of predictors $g : Z \to Y$. Define*

$$\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}.$$

Then $\mathcal{H}_\phi$ is a hypothesis class on X. Moreover, empirical risk minimization over $\mathcal{G}$ on transformed examples $(\phi(x_i), y_i)$ is exactly empirical risk minimization over $\mathcal{H}_\phi$ on the original examples (x_i, y_i).

Proof sketch. For every $g \in \mathcal{G}$, the composition $g \circ \phi$ is a function from X to Y, so $\mathcal{H}_\phi$ is a valid hypothesis class on X. Now consider the empirical objective

$$\frac{1}{n} \sum_{i=1}^{n} \ell(g(\phi(x_i)), y_i).$$

This is numerically identical to

$$\frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i) \quad \text{with} \quad h = g \circ \phi.$$

Thus minimizing over $g \in \mathcal{G}$ on transformed data is the same optimization problem as minimizing over $h \in \mathcal{H}_\phi$ on the original data. $\square$

This proposition may look obvious, but it is conceptually powerful. A representation is not just a convenient hidden layer. It changes the effective hypothesis class available to the learner. When ϕ is well chosen, difficult nonlinear structure in X may become easy linear or low-complexity structure in Z. When ϕ is poorly chosen, even simple downstream tasks may remain hard.

Example 4.4 (Linear probes as a restricted readout class)**.** If $\mathcal{G}$ is the class of affine maps on $Z = \mathbb{R}^m$, then

$$\mathcal{H}_\phi = \{x \mapsto w^\top \phi(x) + b : w \in \mathbb{R}^m,\ b \in \mathbb{R}\}.$$

This is the hypothesis class induced by *linear probing*. The probe is linear in representation space even if the resulting predictor is highly nonlinear as a function of the original input x.

So linear probing is not a contradiction. It is simple in Z, but it can correspond to a sophisticated decision rule in X because the nonlinearity has been delegated to the representation map.

Why reusable representations are valuable

The task-specific paradigm asks us to collect labeled data and solve each task separately. That strategy becomes expensive when tasks are numerous or labels are costly. In many realistic domains, however, unlabeled or weakly labeled data are abundant. Language is everywhere. Images and video are easy to collect. Sensor streams, scientific measurements, and interaction traces can be massive. What is scarce is often not data itself but task-specific supervision.

This asymmetry motivates pretraining. One first learns a representation on broad data, often using a generic objective such as prediction, masking, contrast, reconstruction, or next-token modeling. One then adapts the model to a downstream task by training a small head, fine-tuning part of the model, or fine-tuning all of it. The reusable component is the representation.

Pretrain–then–adapt pipeline

Large raw dataset $\{x\}$
$\downarrow$ pretraining objective
Learn shared representation map $\phi_\theta : X \to Z$
$\downarrow$ reuse across tasks
Task A head g_A Task B head g_B Task C head g_C
$\downarrow$
Predictors $g_A \circ \phi_\theta$, $g_B \circ \phi_\theta$, $g_C \circ \phi_\theta$

Figure 4.1: A schematic picture of the modern pipeline. The shared representation is learned first and then adapted or probed for multiple downstream tasks. The central claim of representation learning is that the same internal coordinates can support many different predictors.

The pipeline can be realized in different ways. Sometimes the representation is frozen and only a probe is trained. Sometimes the whole model is fine-tuned. Sometimes adaptation is done by prompts, low-rank updates, or small inserted modules. The engineering details differ, but the conceptual structure is the same: learn reusable internal organization first, specialize later.

A worked example: linear probing after representation learning

A toy example makes the geometry explicit. Suppose our input space is $X = \mathbb{R}^2$, and the classification problem is to distinguish points on an inner circle of radius 1 from points on an outer circle of radius 2. In the original space X, these classes are not linearly separable by a straight line. No affine function

$$x \mapsto a^\top x + b$$

can separate one circle from the other.

Now define the representation

$$\phi(x) = \|x\|^2.$$

This maps each point in $\mathbb{R}^2$ to a scalar in $Z = \mathbb{R}$. For points on the inner circle,

$$\phi(x) = 1,$$

and for points on the outer circle,

$$\phi(x) = 4.$$

A linear probe on Z takes the form

$$g(z) = wz + b.$$

Choose $w = 1$ and $b = -\frac{5}{2}$. Then

$$g(1) = -\tfrac{3}{2} < 0, \qquad g(4) = \tfrac{3}{2} > 0.$$

Thus the classifier

$$\hat{y} = \operatorname{sign}(g(\phi(x)))$$

perfectly separates the two classes.

Example 4.5 (What the probe is really telling us)**.** The downstream classifier is linear only in the representation space Z, not in the original input space X. The nonlinear geometric work was done by ϕ, which collapsed radial information into a one-dimensional coordinate. This is why linear probes are informative. If a simple probe succeeds, it suggests that the representation has already organized the relevant task information into an accessible form.

This example is intentionally small, but it captures a general pattern. A representation can convert a hard decision boundary in raw input space into a simple separator in latent space. Modern deep learning uses very high-dimensional and learned versions of the same idea. Instead of hand-defining $\phi(x) = \|x\|^2$, the model learns a representation from data. A linear or shallow downstream head then tests what information has become explicit.

What is modern about this picture

It is tempting to think that representation learning is merely an implementation detail inside neural networks. That would miss the historical shift. Earlier machine learning often treated features as external to the learner: one engineered them first and then trained a predictor. Deep learning already changed this by learning internal features jointly with the task. The more recent development goes further still: large-scale pretraining seeks features that are not tied to one task at all. The representation becomes a reusable computational resource.

This is why the language of "foundation models" emerged. A foundation model is not defined only by parameter count or benchmark accuracy. Its defining ambition is to provide a broadly

reusable representational substrate from which many downstream capabilities can be adapted. That ambition raises scientific questions as well as engineering ones. What kinds of pretraining objectives induce transferable structure? How should one measure transferability? When does a representation support linear probing, and when does it require deeper adaptation? What parts of a learned representation are robust, spurious, or task-dependent? These are central themes of the rest of the chapter.

What to remember. A representation is a map $\phi : X \to Z$ that organizes raw data into a space where downstream prediction becomes easier. Modern predictors are often best understood as compositions $g \circ \phi$. Once ϕ is fixed, the downstream learner operates in an induced hypothesis class $\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}$. Transferable representations are those that allow many tasks to be solved by relatively simple readouts. The pretrain–then–adapt paradigm is built on the claim that reusable internal coordinates can be learned once and reused many times.

Common confusion. A reusable representation is not the same thing as a universally optimal representation. Transfer is always relative to a family of tasks and a notion of downstream simplicity. It is also easy to think that linear probing means the original problem was linear. That is false: a problem can be highly nonlinear in X while becoming linearly separable after a learned map into Z. Finally, pretraining does not guarantee transfer. A representation can be excellent for one distribution or objective and poor for another.

Exercises.

1. Let $\phi : X \to Z$ be fixed and let $\mathcal{G}$ be a downstream hypothesis class on Z. Prove carefully that empirical risk minimization over $\mathcal{G}$ on transformed data is equivalent to empirical risk minimization over $\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}$ on the original data.
2. Give an example of a feature that is useful for one classification task but unlikely to transfer well to a broader family of tasks. Explain why it is task-specific rather than reusable.
3. In the concentric-circle example, verify that no affine classifier on $\mathbb{R}^2$ can separate the inner circle from the outer circle, but that a threshold on $\|x\|^2$ can.
4. Suppose $\phi : \mathbb{R}^d \to \mathbb{R}^m$ is learned and a downstream task uses a linear probe. Write the resulting predictor explicitly and explain why it may still define a nonlinear decision boundary in the original input space.
5. Compare two adaptation strategies after pretraining: freezing ϕ and training only g, versus fine-tuning both ϕ and g. What are the likely advantages and risks of each?

Notes and references. For broad perspectives on representation learning, see Bengio, Courville, and Vincent on representation learning and Goodfellow, Bengio, and Courville on deep learning. For transfer learning as a general paradigm, Pan and Yang remain a standard reference. The diagnostic use of linear classifiers on top of learned features appears throughout the modern literature on probing and representation evaluation. Historically, the conceptual shift in this section leads directly to self-supervised learning, large-scale pretraining, and foundation-model thinking: the common thread is the attempt to learn internal coordinates that outlive any one benchmark task.

4.2 Self-Supervised Learning as Predictive Representation Learning

Section 4.1 introduced the chapter's main thesis: modern machine learning increasingly seeks a representation

$$\phi : X \to Z$$

that can be reused across many downstream tasks. The next question is where such a representation comes from when task-specific labels are scarce. One major answer is *self-supervised learning*. In self-supervision, the learning signal is extracted from structure already present in the data. The model is not given an external class label for every example. Instead it is trained to solve a predictive problem constructed from the example itself.

This section is one of the conceptual cores of the chapter because it explains why modern pretraining is not merely "training without labels." It is better understood as *predictive representation learning*. The pretraining objective is chosen so that success requires the model to organize inputs into internal coordinates that preserve reusable regularities. In language, those regularities may include syntax, discourse, and semantic compatibility. In vision, they may include shape, object identity, spatial composition, and invariances to nuisance variation. In temporal data, they may include latent state and dynamical structure.

An abstract view of self-supervised learning

Let X be an input space and let P_X be a distribution over raw observations. A self-supervised system first constructs from $x \sim P_X$ one or more derived objects: masked views, augmented views, contexts, future targets, or paired modalities. These derived objects define a prediction problem. The representation map ϕ_θ and possibly an auxiliary head h_θ are then trained to minimize a loss that depends only on these internally constructed targets.

Definition 4.6 (Abstract self-supervised objective)**.** A self-supervised learning problem is an optimization problem of the form

$$\min_\theta \ \mathbb{E}_{x\sim P_X} \ \mathbb{E}_{\omega\sim Q(\cdot|x)}\big[\ell_{\mathrm{ssl}}(\phi_\theta, h_\theta; x, \omega)\big],$$

where ω is an auxiliary random object derived from x and sampled from a rule $Q(\cdot \mid x)$. The variable ω may encode a mask, an augmentation, a context window, a time index, or a pairing relation. The loss is self-supervised when its target is determined from x and ω rather than from an externally supplied task label.

This abstraction is useful because it separates the enduring mathematical structure from implementation details. Every self-supervised method chooses:

- what predictive relation should be extracted from the data,
- what family of transformations or corruptions defines the task,
- what representation must be learned to solve that task.

In this sense self-supervision is not one method but a design pattern.

A taxonomy of self-supervised objectives

Three large families organize most modern self-supervised learning: masked prediction, autoregressive prediction, and contrastive learning. They differ in what is hidden, what is predicted, and what kind of invariance or dependency structure is encouraged.

1. Masked prediction. Part of the input is hidden and the model is asked to infer the missing content from the visible part. If $x = (x_1, \ldots, x_T)$ is a structured input and $M \subseteq \{1, \ldots, T\}$ is a set of masked positions, we write the visible context as $x_{\bar{M}}$ and the masked content as x_M. A canonical masked objective is

$$\mathcal{L}_{\text{mask}}(\theta) = \mathbb{E}_{x,M}\big[-\log p_\theta(x_M \mid x_{\bar{M}}, M)\big].$$

This trains the model to reconstruct hidden information from context. The objective does not directly tell the model what semantic abstractions to learn, but it does force the representation to capture dependencies among observed parts.

2. Autoregressive prediction. In sequential data one may ask the model to predict each token from the past. For a sequence $x_1, \ldots, x_T$, the objective is

$$\mathcal{L}_{\text{AR}}(\theta) = \mathbb{E}_x\Big[\sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t})\Big].$$

This is a standard conditional log-likelihood. It encourages the representation at time t to summarize the preceding context in a way useful for continuation. Next-token prediction in language models is the most visible modern example.

3. Contrastive learning. Rather than reconstructing hidden content, contrastive learning distinguishes compatible pairs from incompatible ones. Let a and a' be random augmentations, and let

$$u = a(x), \qquad v = a'(x)$$

be two views of the same underlying example. Let $\tilde{v}_1, \ldots, \tilde{v}_K$ be views of other examples. With an encoder ϕ_θ and similarity function $s(\cdot, \cdot)$, a typical contrastive loss takes the form

$$\mathcal{L}_{\text{con}}(\theta) = \mathbb{E}\left[-\log \frac{\exp(s(z, z^+)/\tau)}{\exp(s(z, z^+)/\tau) + \sum_{k=1}^{K} \exp(s(z, z_k^-)/\tau)}\right],$$

where $z = \phi_\theta(u)$, $z^+ = \phi_\theta(v)$, and $z_k^- = \phi_\theta(\tilde{v}_k)$. The temperature $\tau > 0$ controls how sharply similarities are weighted. The objective pushes together representations of related views and pushes apart representations of unrelated examples.

Positives, negatives, augmentations, and contexts

The notation above is standard enough that it deserves explicit definitions.

Definition 4.7 (Augmentation)**.** An *augmentation* is a transformation $a : X \to X'$ that preserves enough of the underlying content for the transformed object to remain a valid view of the same example for the pretraining task. In images, augmentations may include cropping, color perturbation, or flipping. In language, they may be more constrained because arbitrary edits can destroy meaning.

Definition 4.8 (Positive pair and negative pair)**.** In contrastive learning, a *positive pair* is a pair of views (u, v) deemed compatible because they are generated from the same underlying source example or matched context. A *negative pair* is a pair $(u, \tilde{v})$ deemed incompatible because the two views come from different source examples or mismatched contexts.

Definition 4.9 (Context)**.** A *context* is the observed part of a structured input that is used to predict another part. For masked learning the context is typically $x_{\bar{M}}$. For autoregression the context at position t is the prefix $x_{<t}$. More generally, a context is any information retained after a task-specific corruption or restriction has been applied.

These definitions matter because a large part of self-supervised design is hidden inside them. Choosing an augmentation implicitly decides what information the representation should ignore. Choosing positives and negatives decides what counts as semantic compatibility. Choosing a context rule decides what dependencies the model is forced to model.

What contrastive learning is trying to do

There are two complementary ways to understand contrastive learning. One is geometric: views of the same object should map nearby, while views of different objects should not collapse into one indistinguishable cluster. The other is information-theoretic: the representation of one view should retain information predictive of another compatible view. A standard formal statement linking these viewpoints is the InfoNCE lower-bound result.

Proposition 4.10 (InfoNCE lower-bound intuition)**.** *Let (U, V) be jointly distributed random variables, and let $V_1, \ldots, V_N$ be candidates such that one candidate is paired with U according to the joint law and the others are sampled independently from the marginal law of V. If a scoring function $f_\theta(U, V)$ is trained with the multiclass contrastive objective*

$$\mathcal{L}_{\mathrm{NCE}}(\theta) = \mathbb{E}\left[-\log \frac{\exp(f_\theta(U, V_1))}{\sum_{j=1}^{N} \exp(f_\theta(U, V_j))}\right],$$

then, under the standard sampling construction above,

$$I(U; V) \geq \log N - \mathcal{L}_{\mathrm{NCE}}(\theta)$$

for the optimal scoring rule and, more generally, low contrastive loss is evidence that the score retains information useful for identifying the correct paired view among negatives.

Proof sketch and interpretation. The classification problem hidden inside InfoNCE asks the model to identify which candidate V_j is paired with U. If U and V share substantial mutual information, then this identification problem is easier than chance. The optimal logit is related to a log-density ratio between the joint distribution and the product of marginals. Substituting that optimal score into the cross-entropy objective yields the inequality above. The proposition should be read carefully. It does not say that every practical contrastive pipeline estimates mutual information well. It says that under a mathematically clean negative-sampling construction, successful contrastive discrimination is linked to preserving shared information between two views.

This proposition is useful because it elevates contrastive learning above pure heuristic. There is a real theorem-level bridge from a discrimination objective to an information quantity. At the same time, the theorem does not automatically settle practice. Finite batch size, imperfect negatives, architecture choice, projector heads, and augmentation design all affect what the learned representation actually becomes.

What is known, what is empirical, and what remains heuristic

Because self-supervised learning is now a central methodology, it is important to distinguish three layers of understanding.

What is mathematically well grounded. Masked prediction and autoregressive prediction are conditional likelihood objectives on induced prediction tasks. The optimization problem is explicit. If the model class is rich enough and optimization succeeds, the optimum is tied to modeling a conditional distribution. Contrastive learning also has formal analyses, including the lower-bound interpretation above and geometric analyses of alignment and non-collapse in simplified settings. Thus self-supervision is not merely folklore.

What is strongly supported empirically. Large-scale pretraining can produce representations that transfer remarkably well across downstream tasks. Masked language modeling, next-token prediction, and contrastive image learning have all been shown in practice to learn features useful far beyond the pretraining objective itself. Linear probing and fine-tuning experiments repeatedly demonstrate that pretrained representations encode substantial reusable structure. These are empirical facts about current systems.

What remains design practice or heuristic. Many decisive choices still come from engineering judgment rather than closed theory. How much masking should be used? What augmentations preserve semantic identity rather than destroy it? How many negatives are helpful? What temperature should be used in contrastive loss? Why do predictor heads and stop-gradient tricks help in some image methods? Why do some objectives transfer better than others even when training loss looks similar? These questions are only partially understood. The current field combines theorem, experiment, and craft.

Worked example: masked language modeling

A compact toy example makes the predictive structure concrete. Consider the sentence

$$\text{“the cat sat on the mat”}.$$

Suppose positions 3 and 6 are masked. The visible context is therefore

$$(\text{the}, \text{cat}, \texttt{[MASK]}, \text{on}, \text{the}, \texttt{[MASK]}).$$

A masked language model produces contextual representations

$$h_t = \phi_\theta(x_{\bar{M}}, M)_t$$

for each position t, and then predicts the token at each masked position via a softmax head:

$$p_\theta(x_t = w \mid x_{\bar{M}}, M) = \frac{\exp(W_w^\top h_t + b_w)}{\sum_{w'} \exp(W_{w'}^\top h_t + b_{w'})}.$$

If the correct targets are “sat” and “mat,” the contribution to the loss is

$$-\log p_\theta(\text{sat} \mid x_{\bar{M}}, M, t = 3) - \log p_\theta(\text{mat} \mid x_{\bar{M}}, M, t = 6).$$

Why can this teach useful representation rather than mere word filling? Because correct prediction requires the model to exploit several kinds of dependency at once. At position 3, the local context “the cat _ on” strongly suggests a verb compatible with the subject. At position 6, the phrase “on the _” suggests a noun, but selecting “mat” over alternatives is easier if the model also preserves coherence with the earlier words. Thus the internal state must encode grammatical and semantic compatibility across the sentence.

Example 4.11 (What is learned in the toy masked example)**.** The model is not explicitly told to learn syntax, parts of speech, or semantic role. Those structures emerge as useful internal variables because they help predict the masked tokens. This is a recurring pattern in self-supervision: the objective names one task, but the representation internalizes latent regularities needed to solve it.

One should also note a limitation. Masked prediction is not the same as direct generation. It teaches the model to use bidirectional context for infilling, not necessarily to produce a left-to-right continuation policy. This is one reason why different self-supervised objectives can lead to different downstream strengths.

Worked example: contrastive image learning

Now consider a toy image example. Let x be an image of a bicycle. Apply two random augmentations:

$$u = a(x), \qquad v = a'(x),$$

where one augmentation crops the left side of the bicycle and the other changes color and brightness. Although the two resulting views are visually different, they still correspond to the same underlying object. Take another image x' of a dog and form a negative view $\tilde{v} = a''(x')$. Suppose the encoder produces normalized embeddings

$$z = \phi_\theta(u), \qquad z^+ = \phi_\theta(v), \qquad z^- = \phi_\theta(\tilde{v}),$$

and use dot-product similarity $s(z, z') = z^\top z'$.

Assume, for illustration, that

$$s(z, z^+) = 2.0, \qquad s(z, z^-) = 0.5, \qquad \tau = 1.$$

With one negative, the contrastive loss for this anchor is

$$-\log \frac{e^{2.0}}{e^{2.0} + e^{0.5}} \approx 0.20.$$

If the positive similarity were smaller, say 0.8, then the loss would be

$$-\log \frac{e^{0.8}}{e^{0.8} + e^{0.5}} \approx 0.55,$$

which is worse. Thus the training rule explicitly rewards making the two bicycle views more compatible than the bicycle–dog pair.

The important conceptual point is that the representation is being shaped by invariance assumptions hidden in the augmentations. If cropping and color perturbation preserve object identity, then a good representation should not change radically under those transformations. But this is also where design enters. An augmentation that destroys semantic content teaches the wrong invariance. For example, an extreme crop that removes the bicycle entirely should not remain a positive view in any meaningful sense.

Why self-supervision can support transfer

We can now state the chapter-level intuition more sharply. Self-supervised learning tends to help downstream tasks when solving the pretext objective requires the model to preserve regularities that many downstream tasks also need. The mechanism is not magical. A representation becomes

Two common self-supervised pipelines

Masked prediction	**Contrastive learning**
Raw example x	Raw example x
$\downarrow$ apply mask M	$\downarrow$ sample augmentations a, a'
Visible context $x_{\bar{M}}$	Two views $u = a(x),\ v = a'(x)$
$\downarrow$ encoder ϕ_θ	$\downarrow$ shared encoder ϕ_θ
Contextual states h_t	Embeddings z, z^+
$\downarrow$ predict hidden content x_M	$\downarrow$ compare with negatives $z_1^-, \ldots, z_K^-$
Loss: reconstruction / conditional likelihood	Loss: align positives, separate negatives

Figure 4.2: A monochrome comparison of two major self-supervised pipelines. Masked methods learn by recovering omitted information from context. Contrastive methods learn by making compatible views more similar than incompatible ones.

reusable when the pretraining objective rewards internal coordinates that expose broadly useful structure.

Masked prediction often encourages context-sensitive features. Autoregression encourages representations that summarize history and continuation constraints. Contrastive learning often encourages invariances to nuisance transformations while preserving semantic identity. These are not identical biases. Each objective selects a somewhat different notion of what information should be accessible in Z. That is why different self-supervised methods can lead to different probing behavior, adaptation behavior, and domain suitability.

A good representation is therefore not simply one that minimizes a pretraining loss. It is one whose induced hypothesis class

$$\{g \circ \phi : g \in \mathcal{G}\}$$

supports many downstream tasks with simple readout or modest adaptation. This connects self-supervised learning back to the representation-learning picture of Section 4.1. The pretext task is not the final goal. It is an instrument for shaping a useful latent coordinate system.

What to remember

- Self-supervised learning constructs predictive targets from the data itself rather than from externally supplied labels.
- The three main objective families are masked prediction, autoregressive prediction, and contrastive learning.
- Augmentations, positives, negatives, and contexts are not minor implementation details; they determine what invariances and dependencies the representation is encouraged to learn.
- Contrastive learning has real theorem-level support through results such as the InfoNCE lower-bound interpretation, but many practical design choices remain heuristic.
- The reason self-supervision matters is not only that it avoids labels, but that it can induce reusable internal coordinates for downstream tasks.

Common confusion

- **"Self-supervised" does not mean unsupervised in the sense of having no target at all.** There is a target, but it is generated from the example itself.

- **Low pretraining loss is not the same as good transfer.** A method can optimize its pretext objective well while learning the wrong invariances for downstream use.
- **Contrastive learning is not only geometry and not only information theory.** In practice it has both interpretations, and neither fully replaces the other.
- **Masked prediction and autoregression are related but not identical.** They expose different conditional structure and often produce different downstream behavior.

Exercises

1. Write the masked-prediction loss for a sequence in which exactly one token position is chosen uniformly at random for masking.
2. Show that autoregressive next-token prediction is a maximum-likelihood objective for a factorized model of a sequence.
3. In the one-negative contrastive example above, recompute the loss when $s(z, z^+) = 1.5$, $s(z, z^-) = 1.5$, and explain the result geometrically.
4. Give one example of an augmentation that is likely valid for natural images and one that is likely invalid, and explain what invariance each would encourage.
5. Compare masked language modeling and contrastive image learning as representation-learning mechanisms. What information is each objective trying to preserve, and what information is each willing to discard?
6. Suppose a representation collapses so that $\phi(x)$ is the same vector for every x. Explain why this is bad for contrastive learning and for downstream linear probing.
7. Derive from first principles why the softmax cross-entropy in contrastive learning rewards larger positive similarity than negative similarity.
8. Consider a time series with latent periodic structure. Which of the three objective families in this section would you try first, and why?

Notes and references

Self-supervised learning has roots in several traditions rather than a single founding paper. Predictive coding, language modeling, denoising autoencoding, masked prediction, and contrastive estimation all contributed pieces of the modern picture. For denoising and representation learning, a classical reference is Vincent et al. For language-model pretraining, early neural language-model and contextual-representation work is foundational. For masked language modeling and bidirectional contextual pretraining, Devlin et al. is central. For contrastive objectives and their information-theoretic interpretation, the Noise-Contrastive Estimation and InfoNCE lines are especially important. For modern contrastive image learning, representative references include methods in the SimCLR and MoCo families. A useful reading strategy is to separate three questions while reading the literature: what predictive task is defined, what invariances are encoded through the data pipeline, and what evidence is provided that the learned representation transfers.

4.3 Transfer Learning and Adaptation

Section 4.1 introduced the representation map

$$\phi : X \to Z$$

and the downstream decomposition

$$f = g \circ \phi.$$

Section 4.2 then explained how self-supervised learning can train such a map without task-specific labels. The next question is therefore structural rather than merely procedural: once a representation has been learned, how should it be reused on a new task or a new domain? This is the subject of transfer learning and adaptation.

The central idea of this section is that transfer is not a single method but a design space. One may freeze a pretrained representation and learn only a new head. One may fine-tune all parameters. One may insert small trainable modules. One may constrain the update to lie in a low-rank subspace. These choices are different computationally, statistically, and geometrically. They also respond differently to domain shift.

Basic definitions: source, target, and adaptation under shift

We begin by formalizing the setting. A pretrained model is typically obtained from one data source and then reused on another. Those two environments need not coincide.

Definition 4.12 (Source domain and target domain)**.** A *source domain* is a probability distribution

$$\mathcal{D}_S$$

on an input–label space $X \times Y_S$ or, more generally, on an input space X together with whatever supervision or self-supervision is used during pretraining. A *target domain* is a probability distribution

$$\mathcal{D}_T$$

on $X \times Y_T$ corresponding to the downstream task of interest. The source and target may differ in input marginal, label space, or task objective.

This deliberately broad definition covers several common cases. Sometimes the source and target share the same label semantics but differ in acquisition conditions, such as photographs in daylight versus photographs at night. Sometimes the source objective is self-supervised while the target objective is supervised. Sometimes the source and target have different label spaces altogether, as when image pretraining is reused for medical classification.

Definition 4.13 (Domain shift)**.** There is *domain shift* when the source and target environments are not identical. At the most basic level this means

$$\mathcal{D}_S \neq \mathcal{D}_T.$$

In practice one often distinguishes at least three possibilities:

- **covariate shift:** the input marginals differ while the predictive relationship is relatively stable,
- **label or task shift:** the target labels or task definition differ from those in pretraining,

- **representation mismatch:** the pretrained features fail to expose the distinctions needed downstream.

The third item is especially important in modern transfer learning. Even when source and target inputs look similar, the internal coordinates learned during pretraining may or may not align with what the downstream task needs. This is why adaptation is fundamentally about representation, not only about data overlap.

Definition 4.14 (Feature extraction)**.** Let $\phi_{\theta_0} : X \to Z$ be a pretrained representation map. *Feature extraction* means keeping θ_0 fixed and learning a downstream predictor

$$g_\psi : Z \to Y_T$$

on top of the frozen features $z = \phi_{\theta_0}(x)$. The optimization problem is

$$\min_\psi \ \widehat{R}_T(g_\psi(\phi_{\theta_0}(x)), y).$$

Definition 4.15 (Fine-tuning)**.** *Fine-tuning* means initializing a model from pretrained parameters and then updating some or all of those parameters on the target task. If both the representation and the head are updated, one solves

$$\min_{\theta,\psi} \ \widehat{R}_T(g_\psi(\phi_\theta(x)), y)$$

starting from (θ, ψ) near pretrained values rather than random initialization.

These two definitions already express an important distinction. Feature extraction restricts the learner to the induced hypothesis class

$$\mathcal{H}_{\phi_{\theta_0}} = \{g \circ \phi_{\theta_0} : g \in \mathcal{G}\}.$$

Fine-tuning instead changes the representation itself and therefore changes the effective hypothesis class. The gain in flexibility comes with extra optimization cost and greater risk of overfitting or catastrophic drift.

A formal taxonomy of adaptation strategies

Transfer learning is often described in broad language, but the main strategies can be written precisely. Suppose a pretrained network is parameterized by θ_0 and produces hidden states $h = \phi_{\theta_0}(x)$. Let g_ψ be the target head. Then the main adaptation choices are as follows.

Definition 4.16 (Linear probing)**.** *Linear probing* is the special case of feature extraction in which the downstream predictor is restricted to an affine map on representation space:

$$g_\psi(z) = Wz + b.$$

Thus the only trainable parameters are those of a linear classifier or regressor on top of frozen features.

Definition 4.17 (Full fine-tuning)**.** *Full fine-tuning* updates all pretrained parameters. Equivalently, if the model weights are written collectively as θ, then one optimizes over all perturbations

$$\Delta\theta$$

through the parameterization $\theta = \theta_0 + \Delta\theta$. No structural restriction is imposed on $\Delta\theta$ beyond those already implicit in the model class and optimization method.

Definition 4.18 (Adapter-based tuning)**.** *Adapters* are small trainable modules inserted into a mostly frozen backbone. A standard form adds a bottleneck residual module

$$h \mapsto h + U\sigma(Vh),$$

where $V \in \mathbb{R}^{r\times d}$, $U \in \mathbb{R}^{d\times r}$, and $r \ll d$. The backbone parameters remain fixed, while the adapter parameters (U, V) and the task head are trained.

Definition 4.19 (Low-rank adaptation)**.** *Low-rank adaptation* constrains weight updates to be low rank. If a pretrained linear map has weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$, then a low-rank update has the form

$$W = W_0 + BA,$$

where

$$B \in \mathbb{R}^{d_{\text{out}}\times r}, \qquad A \in \mathbb{R}^{r\times d_{\text{in}}}, \qquad r \ll \min(d_{\text{out}}, d_{\text{in}}).$$

Only A and B are trained.

These definitions describe four distinct points in the adaptation design space. Linear probing changes only the readout. Full fine-tuning changes everything. Adapters preserve the main backbone while introducing small task-specific nonlinear modules. Low-rank adaptation allows weight changes but restricts them to a low-dimensional subspace.

Remark 4.20 (What changes geometrically)*.* Each strategy modifies a different part of the predictor geometry. Linear probing changes only the separating surface in the learned representation space. Full fine-tuning can change both the representation geometry and the readout. Adapters and low-rank updates occupy an intermediate position: they allow controlled reshaping of the representation without rewriting the whole model.

Why adaptation is not only optimization

It is tempting to treat transfer learning as simply a warm start for gradient descent. That would be too narrow. A pretrained model does not merely provide convenient initial parameters. It provides an already structured coordinate system for the data. Adaptation decisions are therefore decisions about how much of that coordinate system should be trusted.

If the target task is close to the source and the representation already exposes the relevant distinctions, then a linear probe may suffice. If the representation contains the needed information but in a form that is not linearly accessible, a richer head or a small adapter may help. If the source and target domains differ substantially, the representation itself may need to move. This is why transfer learning has a genuine statistical question behind its engineering surface: when does success on the source domain imply anything useful on the target domain?

A theorem-level result from domain adaptation theory

A widely cited answer comes from the theory of domain adaptation for binary classification. The theorem below is given in a simplified form. It does not solve the full modern transfer problem, but it gives an important and rigorous decomposition of target error.

Let $\mathcal{H}$ be a hypothesis class of binary classifiers on X. For a distribution $\mathcal{D}$ on $X \times \{0, 1\}$, let

$$\varepsilon_{\mathcal{D}}(h)$$

denote the population classification error of h under $\mathcal{D}$. Write $\mathcal{D}_S^X$ and $\mathcal{D}_T^X$ for the source and target input marginals.

Definition 4.21 ($\mathcal{H}\Delta\mathcal{H}$ divergence, informal form)**.** Given a hypothesis class $\mathcal{H}$, the quantity

$$d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S^X, \mathcal{D}_T^X)$$

measures how well classifiers formed by disagreements of hypotheses in $\mathcal{H}$ can distinguish source inputs from target inputs. If this divergence is small, the two domains look similar to the discriminative power of the class. If it is large, the domains are readily distinguishable and transfer is harder.

Theorem 4.22 (Simplified domain-adaptation bound)**.** *Let $\mathcal{H}$ be a hypothesis class for binary classification. Then for every $h \in \mathcal{H}$,*

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S^X, \mathcal{D}_T^X) + \lambda_{\mathcal{H}},$$

where

$$\lambda_{\mathcal{H}} = \inf_{h \in \mathcal{H}} \big(\varepsilon_S(h) + \varepsilon_T(h)\big)$$

is the error of the best joint classifier in the class across both domains.

Interpretation. The theorem says that small source error alone is not enough. To guarantee small target error, one also needs the source and target domains to be similar as seen through the hypothesis class, and one needs the class to contain a classifier that performs well on both domains simultaneously. The three terms have distinct meanings:

- $\varepsilon_S(h)$: how well the chosen classifier fits the source domain,
- $\frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}$: how different source and target look to the class,
- $\lambda_{\mathcal{H}}$: whether the two tasks are even jointly compatible inside the chosen representation and hypothesis class.

Why this matters for representation learning. The bound is usually presented for a hypothesis class on raw inputs, but in transfer learning the relevant class is often induced by a representation. If one uses a fixed pretrained map ϕ, then the downstream class is effectively

$$\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}.$$

A good transferable representation can therefore help in three ways at once: it can reduce source error by making the task easier, it can make source and target appear more similar in representation space, and it can enlarge the chance that one simple readout works well on both domains.

What the theorem does and does not say. This result is rigorous, but its scope is limited. It is a statement for an abstract hypothesis class under binary classification, not a complete theory of large-scale fine-tuning in deep networks. It does not tell us which representation to learn, how gradient descent finds it, or why a particular adapter design works better than another. Still, it captures a central truth: transfer depends jointly on source performance, domain discrepancy, and representational compatibility.

Worked example: frozen backbone versus full fine-tuning

A toy example makes the distinction concrete. Consider inputs $x = (x_1, x_2) \in \mathbb{R}^2$. Suppose pretraining on the source task produced a one-dimensional representation

$$\phi_{\theta_0}(x) = x_1.$$

This is plausible if the source labels depended only on x_1. For example, the source task might have been

$$y_S = \mathbf{1}\{x_1 > 0\}.$$

Now consider a target task with labels

$$y_T = \mathbf{1}\{x_1 + x_2 > 0\}.$$

The target requires information from both coordinates.

Frozen backbone with linear probe. If we freeze the backbone, then every downstream predictor has the form

$$\hat{y} = \mathbf{1}\{w\phi_{\theta_0}(x) + b > 0\} = \mathbf{1}\{wx_1 + b > 0\}.$$

No choice of w and b can represent the diagonal decision boundary $x_1 + x_2 = 0$, because the predictor does not depend on x_2 at all. Thus frozen feature extraction fails for a structural reason: the required information is absent from the representation.

Full fine-tuning. Now allow the representation to adapt within the family

$$\phi_\alpha(x) = x_1 + \alpha x_2.$$

With a linear probe $g(z) = \mathbf{1}\{z > 0\}$ and the choice $\alpha = 1$, we obtain

$$\hat{y} = \mathbf{1}\{x_1 + x_2 > 0\},$$

which matches the target task exactly. Here fine-tuning succeeds because it changes the representation itself rather than only the readout.

Example 4.23 (What the toy example teaches)**.** The lesson is not that fine-tuning is always better. The lesson is that frozen features succeed only when the downstream task can already be expressed within the induced class $\mathcal{H}_{\phi_{\theta_0}}$. When the target distinction is missing from the representation, no probe can recover it. Fine-tuning, adapters, or low-rank updates are valuable precisely because they can alter what information becomes explicit.

A practical version of the same story occurs constantly. A vision encoder pretrained on natural photographs may already support linear probing for coarse object categories, yet require fine-tuning for radiology, satellite imagery, or fine-grained species classification. A language model may support probing for sentiment or topic, yet need adaptation for legal drafting, protein sequences, or code repair. The key question is always the same: does the current representation already expose the target structure, or must the representation itself move?

The adaptation design space in one picture

This figure should be read as a structural summary rather than a rigid ranking. More flexibility is not always better. With small target data, a low-capacity adaptation strategy may generalize better because it preserves useful source structure and reduces overfitting. Conversely, if the source and target are substantially misaligned, high rigidity can lock the model into the wrong representation.

Adaptation design space

Strategy	Trainable part	Flexibility	Cost	Best when
Linear probe	head only	low	very low	representation already separates target task
Feature extraction with richer head	head only	moderate	low	pretrained features are useful but linear readout is too weak
Adapters	small inserted modules + head	moderate to high	moderate	some representation reshaping is needed but full tuning is expensive
Low-rank adaptation	restricted weight updates + head	moderate to high	moderate	large model must adapt within a compact update budget
Full fine-tuning	most or all parameters	highest	highest	target differs substantially or maximum specialization is needed

Figure 4.3: Transfer learning is not a binary choice between freezing and retraining everything. It is a spectrum of design decisions about where adaptation is allowed to occur and how much geometric reshaping of the learned representation is permitted.

What is understood, and what remains partly empirical

Transfer learning today sits between theory and design practice. Some points are reasonably well understood. A fixed representation induces a restricted hypothesis class. Domain adaptation theory explains why low source error need not imply low target error under shift. Low-rank or bottleneck adaptation reduces parameter count by imposing algebraic structure on the update. These are mathematically clear facts.

But many central questions remain mostly empirical. When does full fine-tuning outperform low-rank adaptation by a large margin? How much target data is needed before unfreezing the backbone is worthwhile? Which layers should be adapted first? Why do some tasks benefit from preserving early layers while others require deep representational change? How do optimization dynamics interact with catastrophic forgetting and feature drift? These questions are answered today more by experiment and engineering judgment than by closed theory.

This is why transfer learning is a good place to see modern machine learning in its full character. There are formal definitions and meaningful theorems, but there is also a substantial design layer in which one chooses an adaptation mechanism to balance compute, robustness, and specialization. The science is real, but it is not yet complete.

What to remember

- Transfer learning studies how a pretrained representation or model is reused on a new target task or domain.
- Feature extraction freezes the representation and learns a new head; fine-tuning updates some or all pretrained parameters.
- Linear probing, full fine-tuning, adapters, and low-rank adaptation are formally distinct points in the adaptation design space.
- A simplified domain-adaptation bound shows that target error depends not only on source error but also on domain discrepancy and on whether one classifier can perform well on both domains.
- Frozen backbones succeed only when the target task lies inside the induced hypothesis class of the pretrained representation.

Common confusion

- **"Pretrained" does not mean "already optimal for every target."** A pretrained model may still require substantial adaptation under shift.
- **Linear probing is not the same as feature extraction in general.** Linear probing is a special, especially restrictive case of feature extraction.
- **Full fine-tuning is not automatically superior.** More flexibility can also mean more overfitting, more compute, and more instability on small target datasets.
- **Domain shift is not only a change in visual appearance or data source.** It can also be a mismatch between what the representation makes accessible and what the target task needs.

Exercises

1. Let $\phi : X \to Z$ be fixed and let $\mathcal{G}$ be a class of downstream predictors on Z. Show that feature extraction restricts learning to the induced class $\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}$. Explain why fine-tuning changes this class.
2. In the toy example above, prove that no linear probe on the frozen feature $\phi_{\theta_0}(x) = x_1$ can realize the target decision rule $\mathbf{1}\{x_1 + x_2 > 0\}$.
3. Derive from first principles the parameter count of a low-rank update BA and compare it with the parameter count of a full update to a matrix in $\mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$.
4. Compare two methods: adapters and low-rank adaptation. In what sense do both restrict the update space, and in what sense are their restrictions geometrically different?
5. Explain the three terms in the simplified domain-adaptation bound and give a concrete failure mode corresponding to each term being large.
6. Suppose a target dataset is extremely small but very close to the source domain. Which adaptation strategy would you try first, and why? Then answer the same question when the target domain is small but strongly shifted from the source.
7. Consider a pretrained language model adapted to two tasks: sentiment classification and specialized biomedical question answering. Argue which task is more likely to work with linear probing and which is more likely to require deeper adaptation.
8. Give an example of covariate shift in vision or time series and explain whether you would expect frozen features, adapters, or full fine-tuning to be most promising.

Notes and references

Transfer learning as a general framework is classically surveyed by Pan and Yang. For a broad deep-learning treatment, Goodfellow, Bengio, and Courville provide useful background on representation reuse and fine-tuning. The domain-adaptation bound presented here is associated with the theory developed by Ben-David and collaborators, which remains one of the clearest formal accounts of why transfer under shift is delicate. For parameter-efficient adaptation, the modern literature on adapters and low-rank adaptation is central, especially in large language and vision models. A good reading strategy is to keep three levels separate: representation reuse as a conceptual framework, domain adaptation as the source of rigorous error decompositions, and parameter-efficient tuning as an engineering response to scale and deployment constraints.

4.4 Foundation Models and Learning at Scale

Sections 4.1–4.3 argued that modern learning increasingly separates the acquisition of representation from the final downstream use of that representation. Foundation models are a large-scale realization of that idea. They are not best understood as "very large neural networks" in the abstract. Rather, they are systems trained on broad data with a reusable interface, so that one parameter set can support many later tasks through prompting, fine-tuning, retrieval, or other adaptation mechanisms. This section gives a sober account of that regime. Its aim is not to celebrate scale, but to make precise what changes when representation learning, pretraining, and deployment are organized around one large pretrained model.

A precise working definition

We begin with a definition tailored to the logic of this chapter.

Definition 4.24 (Foundation model)**.** A *foundation model* is a pretrained parametric model

$$F_{\theta_0}$$

obtained from broad data distribution P_{pre} by optimizing a generic pretraining objective

$$\mathcal{L}_{\text{pre}}(\theta),$$

such that the resulting parameter vector θ_0 supports a family of downstream tasks $\mathcal{T}$ through one or more adaptation mechanisms and deployment interfaces, with comparatively modest task-specific modification relative to training the model from scratch for each task.

This definition is intentionally narrower than "large model" and broader than "language model." Three ingredients matter. First, the model is trained on broad rather than narrowly task-specific data. Second, the pretraining objective is generic enough to produce reusable internal structure. Third, the resulting system is reused across many tasks through a stable interface. The word "foundation" therefore refers to a role in a learning pipeline, not merely to parameter count.

A small model can in principle play this role within a restricted domain, and a large model can fail to play it if it is trained only for one narrow benchmark. In practice, however, the models usually discussed under this heading are large because broad reuse often benefits from large capacity, diverse data, and substantial compute.

Pretraining objective, adaptation mechanism, and deployment interface

One source of confusion in current discussion is that several distinct layers are mixed together. For a mathematically cleaner account, it is useful to separate them.

Definition 4.25 (Pretraining objective)**.** A *pretraining objective* is a loss functional

$$\mathcal{L}_{\text{pre}}(\theta) = \mathbb{E}_{x\sim P_{\text{pre}}}\left[\ell_{\text{pre}}(x;\theta)\right]$$

or its empirical approximation on a large dataset. Examples include masked prediction, autoregressive next-token prediction, denoising, contrastive objectives, or multimodal matching objectives. Its role is to determine what regularities the model is rewarded for internalizing during large-scale training.

Definition 4.26 (Adaptation mechanism)**.** An *adaptation mechanism* is a procedure that converts a pretrained model into a task-specialized system. Formally, it is a map of the form

$$\mathcal{A} : (\theta_0, \mathcal{D}_T) \mapsto \Theta_T,$$

where $\mathcal{D}_T$ is task-specific information and Θ_T denotes either updated parameters, auxiliary modules, prompts, retrieval state, or some combination thereof. Full fine-tuning, linear probing, adapters, low-rank adaptation, and instruction tuning are examples.

Definition 4.27 (Deployment interface)**.** A *deployment interface* is the protocol by which a user or downstream system queries the pretrained or adapted model. It specifies what object may be presented to the model at inference time and what output is requested. For example, a prompt interface may accept text context c and return a conditional distribution over continuations,

$$y \mapsto p_\theta(y \mid c),$$

while a classification interface may accept an input x and return a label or score.

These three notions should not be conflated. The pretraining objective explains how the shared representation was learned. The adaptation mechanism explains whether and how the parameters or auxiliary modules are changed for a new task. The deployment interface explains how the resulting system is queried at use time. Much confusion around foundation models arises when prompting is discussed as if it were a training objective, or instruction tuning is discussed as if it were only an inference trick.

Example 4.28 (One model, three different layers)**.** Consider a decoder-only transformer trained with next-token prediction. Its pretraining objective is autoregressive likelihood. Its adaptation mechanism might be none at all (zero-shot prompting), or supervised instruction tuning, or low-rank fine-tuning. Its deployment interface might be a text prompt followed by generation, a chat interface, or an API that returns log-probabilities over candidate outputs. These are related, but they are not the same mathematical object.

A formal view of prompting and conditional behavior

For concreteness, suppose a pretrained decoder defines

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

If a context or prompt c is provided, then the model induces a conditional distribution on continuations:

$$p_\theta(y \mid c).$$

This is the basic mathematical content of prompting. One does not change the parameters. One changes the conditioning context.

Definition 4.29 (Prompting)**.** Let θ be fixed. *Prompting* is the use of a context variable c at inference time to select a conditional prediction rule

$$q_{\theta,c}(y) = p_\theta(y \mid c).$$

The prompt may contain an instruction, demonstrations, formatting constraints, retrieved external information, or a question to be answered.

This definition is deliberately simple. It captures both ordinary continuation and more elaborate prompting patterns. Few-shot prompting is the case in which the prompt includes example input–output pairs. Chain-of-thought prompting is the case in which the prompt encourages intermediate reasoning-like text. Retrieval-augmented prompting is the case in which the prompt is extended by externally fetched context. All of these remain, mathematically, conditional inference with fixed parameters.

Proposition 4.30 (Prompting changes context, not weights)**.** *Fix model parameters θ. Then the family of predictors available through prompting is*

$$\mathcal{Q}_\theta = \{q_{\theta,c} : c \in \mathcal{C}\}, \qquad q_{\theta,c}(y) = p_\theta(y \mid c).$$

Prompting can select different conditional behaviors within $\mathcal{Q}_\theta$, but it does not alter the underlying parameter vector and therefore does not perform parameter adaptation.

Proof sketch. For fixed θ, every admissible prompt c defines a conditional distribution $p_\theta(\cdot \mid c)$. Varying c therefore varies the member of the family $\mathcal{Q}_\theta$ being queried. But the map $\theta \mapsto p_\theta$ is unchanged, since no optimization step or parameter update occurs. Thus prompting can reveal or organize capabilities already encoded by the pretrained parameters, but it does not itself move the model to a new point in parameter space. □

The proposition is elementary, but it clarifies a major conceptual distinction. When a prompted model performs a new task, two different explanations are possible. One is that the pretrained model already contains a sufficiently rich conditional mechanism, and the prompt selects the relevant behavior. The other is that the model is being genuinely improved by additional parameter adaptation. Prompting belongs to the first category; instruction tuning belongs to the second.

Instruction tuning and post-pretraining alignment

Instruction tuning is a parameter update stage applied after broad pretraining. Suppose one has instruction–response pairs (u, r) drawn from a distribution P_{inst}. Starting from pretrained parameters θ_0, one optimizes

$$\mathcal{L}_{\text{inst}}(\theta) = \mathbb{E}_{(u,r)\sim P_{\text{inst}}}\big[-\log p_\theta(r \mid u)\big].$$

This does not merely select among existing conditionals. It changes the parameters so that the model becomes easier to steer by instruction-like inputs.

In the current literature, this stage is often grouped under broader labels such as alignment or post-training. The exact procedures vary: supervised instruction tuning, preference-based optimization, rejection sampling from stronger models, and combinations with retrieval or tool use all appear in practice. What matters conceptually is the distinction between two questions:

- how the broad internal representation was acquired during pretraining,
- how the deployed behavior was later shaped toward task following, helpfulness, safety constraints, or conversational format.

It is therefore misleading to say that prompting, instruction tuning, and alignment are interchangeable. Prompting changes the input to a fixed model. Instruction tuning changes the model itself. Alignment, in a broad engineering sense, usually refers to post-pretraining procedures intended to shape deployed behavior, of which instruction tuning is one important example.

In-context behavior: what can be stated cleanly

A striking empirical feature of large pretrained sequence models is that they may behave as if they had adapted to a task merely from examples given in the context. For instance, one may provide several input–output pairs in a prompt and then ask the model to continue with the output for a new input. This is commonly called *in-context learning* or *in-context behavior*.

A careful exposition should separate the mathematical fact from the stronger interpretation. The mathematical fact is simple: because the model computes $p_\theta(y \mid c)$ for arbitrary admissible context c, one may include demonstrations inside c. The stronger interpretation—that the model is internally implementing a fast learning algorithm over the prompt—is plausible in some cases and supported by several theoretical toy models, but it is not a general theorem about deployed systems.

Definition 4.31 (In-context behavior)**.** Let θ be fixed. We say that a model exhibits *in-context behavior* for a task family if its conditional predictor

$$y \mapsto p_\theta(y \mid c)$$

changes systematically when the context c is augmented by task descriptions or demonstrations, so that performance on the implied task improves without an explicit weight update.

This definition is intentionally behavioral. It does not claim that the model literally updates an internal parameter vector in the way an external training algorithm would. It only claims that the input–output map depends on the context in a task-relevant way. That is the phenomenon one can observe directly.

Example 4.32 (Prompting versus instruction tuning)**.** Suppose a pretrained language model is asked to classify sentiment. With no adaptation, one may prompt it by writing:

> Text: "The film was unexpectedly moving."
> Label:

and sampling or scoring candidate outputs such as "positive" and "negative." If one instead provides several labeled examples before the query, this becomes few-shot prompting. By contrast, instruction tuning would optimize the model parameters on many such instruction–response pairs before deployment. The first is conditional use of a fixed model; the second is parameter adaptation.

Learning at scale: what is algorithmic fact and what is empirical observation

Foundation models are strongly associated with scale. To discuss this responsibly, it helps to separate three kinds of statement.

First, there are *architectural and algorithmic facts*. For a dense transformer with depth L, width d, and context length T, one can analyze parameter count, memory footprint, and per-step computational cost. At a high level, the parameter count of the feedforward and attention projections scales on the order of

$$O(Ld^2),$$

while dense self-attention contributes sequence-length dependence on the order of

$$O(LT^2d)$$

in time and roughly $O(T^2)$ per layer in attention-score storage before implementation details are taken into account. These are not empirical laws but consequences of the architecture. They explain why larger width, depth, and context require substantial engineering resources.

Second, there are *empirical scaling observations*. Across substantial but finite training regimes, several studies report that validation loss or perplexity often follows approximate power-law trends as model size, data, and compute increase. Symbolically, one often writes a relation of the form

$$\mathcal{L}(N, D, C) \approx AN^{-\alpha} + BD^{-\beta} + \cdots ,$$

where N denotes parameter count, D effective training data, and C compute budget. This notation is useful, but it should not be mistaken for a theorem of learning theory. The exponents are fitted in restricted regimes, depend on modeling choices, and need not be universal.

Third, there are *design practices and frontier claims*. Claims about sudden capability transitions, the best post-training recipe, the correct balance between synthetic and human data, or the ideal mixture of retrieval, tools, and prompting are largely empirical and engineering-driven. Some are well supported by repeated practice. Very few are settled mathematically.

This separation is essential. It is true, in a precise computational sense, that larger models usually cost more to train and serve. It is true, in a repeated empirical sense, that performance often improves predictably over useful ranges as scale increases. It is not true that current theory provides a complete first-principles account of why broad competence emerges at particular scales or why one post-training pipeline dominates another.

Lifecycle of a modern foundation-model pipeline

Broad data source P_{pre}

↓

Pretraining objective $\mathcal{L}_{\text{pre}}$

↓

Shared base model F_{θ_0}

↓

Post-pretraining shaping: instruction tuning, preference optimization, safety constraints

↓

Deployment-time interfaces: prompting, retrieval, tools, API calls

↓

Optional task adaptation: probe, adapter, low-rank update, or full fine-tuning

↓

Deployed system for downstream use

Figure 4.4: A print-friendly schematic of the lifecycle. Pretraining, post-training alignment, prompting, and task adaptation occupy different levels of the pipeline. The same base model may support several deployment routes, and the mathematical role of each route is different.

Why scale matters for representation learning

Why should large data and large models help at all? At the chapter level, the most conservative answer is representational. A broad pretraining distribution exposes the model to many contexts, tasks, styles, and regularities. A sufficiently expressive architecture can then allocate parameters to internal variables that are not tied to one benchmark label space. Scale increases the opportunity to learn stable reusable coordinates rather than brittle task-specific shortcuts.

This statement should also be interpreted carefully. Scale does not guarantee semantic understanding, factual reliability, or robust reasoning in every setting. It changes the regime in which representation learning takes place. The resulting models can then often support a larger collection of downstream behaviors through the interfaces described above. That is already a substantial claim, and it is the one most closely aligned with the rest of this chapter.

What is understood, what is empirical, and what remains design practice

We can now summarize the epistemic status of the main claims in this area.

Relatively well understood. The probabilistic semantics of autoregressive and masked objectives are clear. The distinction between prompting and parameter adaptation is clear. The computational scaling of dense architectures in parameters, memory, and sequence length is clear. The existence of transfer through shared representations is supported both by theory in simplified settings and by overwhelming empirical evidence.

Strongly empirical but stable enough to teach. Large-scale pretraining often produces features that transfer broadly. Instruction tuning often improves task following and interface usability. Approximate scaling-law regularities are often visible over substantial ranges. Prompt design, retrieval augmentation, and post-training methods materially affect deployment behavior. These statements are not universal theorems, but they are robust enough to form part of modern practice.

Still largely design practice or open theory. There is no complete theory of in-context learning in realistic models. There is no settled first-principles account of why some capabilities appear gradually while others seem to improve abruptly under certain evaluation protocols. There is no single mathematically justified recipe for post-training alignment. Questions of robustness, calibration, tool use, long-horizon reasoning, and multimodal grounding remain only partially understood.

That balance matters pedagogically. A mathematically serious treatment should neither understate what is genuinely established nor overstate what remains heuristic. Foundation models are important partly because they work, but academic exposition should still distinguish clean formal structure from well-supported empirical regularity and from frontier design judgment.

What to remember

- A foundation model is defined here by its role: broad pretraining, reusable internal structure, and support for many downstream tasks through stable interfaces.
- Pretraining objective, adaptation mechanism, and deployment interface are distinct objects and should not be conflated.
- Prompting changes context for a fixed model, whereas instruction tuning changes parameters after pretraining.
- In-context behavior is an empirical phenomenon of task-sensitive conditioning without explicit weight updates.
- Computational scaling of dense architectures is analyzable, but observed scaling laws and post-training recipes are primarily empirical.

Common confusion

- **"Foundation model" means only "very large model."** Size matters in practice, but the more important idea is broad reusable pretraining plus many downstream uses.
- **Prompting and fine-tuning are the same.** They are not: prompting leaves parameters fixed, while fine-tuning updates them.
- **Instruction tuning is identical to pretraining.** It is usually a later stage with different data and different behavioral goals.
- **Scaling laws are theorems.** They are fitted empirical regularities, not universal derivations from first principles.
- **In-context learning is fully understood.** It is observed and partially modeled, but a complete theory for realistic systems is still missing.

Exercises

1. Let a pretrained decoder define $p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$. Show directly that a prompt c induces a conditional predictor $q_{\theta,c}(y) = p_\theta(y \mid c)$. Why does this formalize prompting without parameter adaptation?
2. Give one example each of a pretraining objective, an adaptation mechanism, and a deployment interface for a language model. Explain why confusing any two of them leads to conceptual error.
3. Compare zero-shot prompting, few-shot prompting, instruction tuning, and full fine-tuning. For each, say what changes mathematically: the input context, the parameter vector, or both.
4. Suppose a dense transformer has depth L, width d, and context length T. Using the chapter's rough scaling discussion, explain qualitatively how training cost changes when one doubles d versus when one doubles T.
5. A student claims that because prompting can produce different behaviors, it should count as learning in exactly the same sense as gradient-based fine-tuning. Give a careful response using the proposition in this section.
6. Write a short essay comparing two deployment routes for the same base model: prompt-only use versus adapter-based task adaptation. Discuss flexibility, cost, and control.

Notes and references

For the phrase *foundation model* and a broad systems-level framing, a standard reference is the survey by Bommasani et al. Canonical large-scale language-model papers include BERT for masked language modeling, GPT-style autoregressive models, T5 for text-to-text transfer, and later large decoder-only systems such as PaLM and related families. For empirical scaling-law work, the Kaplan et al. study is the classic early reference, while Hoffmann et al. ("Chinchilla") is central for the compute–data tradeoff discussion. For multimodal large-scale pretraining, CLIP is a standard reference on contrastive vision–language alignment, and Flamingo-style work is representative of multimodal prompting with frozen or partially frozen backbones. On instruction tuning and post-training alignment, common reference points include the FLAN line of work, InstructGPT, and subsequent preference-optimization methods. Theoretical work on in-context learning exists in several simplified regimes, including analyses of transformers as implicit learners in stylized settings, but the field remains much less settled than the empirical literature.

4.5 Representation Learning as a Unifying Principle

4.5.1 Overview

The preceding sections introduced representation as a map $\phi : \mathcal{X} \to \mathcal{Z}$, self-supervised objectives that shape ϕ, and transfer mechanisms that reuse it. This section elevates these ideas into a unifying principle:

Thesis. Modern machine learning can be understood as the construction of intermediate coordinate systems $\mathcal{Z}$ in which multiple tasks become simpler, more stable, or more transferable.

This section formalizes key properties of representations and shows how they operate across supervised learning, self-supervised learning, generative modeling, and reinforcement learning.

4.5.2 Core Properties of Representations

Definition 4.33 (Invariance)**.** A representation ϕ is invariant to a transformation $T : \mathcal{X} \to \mathcal{X}$ if

$$\phi(Tx) = \phi(x) \quad \text{for all } x \in \mathcal{X}.$$

Definition 4.34 (Sufficiency)**.** A representation ϕ is sufficient for a task y if there exists a predictor g such that

$$y = g(\phi(x)),$$

or more generally, the conditional distribution satisfies

$$P(y \mid x) = P(y \mid \phi(x)).$$

Definition 4.35 (Transferability)**.** A representation ϕ is transferable across a family of tasks $\mathcal{T}$ if for each $T \in \mathcal{T}$ there exists a simple predictor g_T (e.g., linear or low-complexity) such that

$$f_T(x) = g_T(\phi(x)).$$

Definition 4.36 (Semantic Structure)**.** A representation exhibits semantic structure if distances or directions in $\mathcal{Z}$ correspond to meaningful variations in the data-generating process.

Remark 4.37. These properties are not independent. Increasing invariance may reduce sufficiency, and transferability depends on both.

4.5.3 A Cross-Domain Representation Framework

Across learning paradigms, we observe a shared decomposition:

$$x \overset{\phi}{\longmapsto} z \overset{g}{\longmapsto} y.$$

Supervised learning.

- ϕ is learned jointly with g
- sufficiency is enforced by labels
- invariance emerges implicitly

Self-supervised learning.

- ϕ is trained via predictive objectives

- invariance is explicitly induced (augmentations, masking)
- transferability is evaluated downstream

Generative modeling.

- ϕ corresponds to latent variables
- structure is imposed through likelihood or score objectives
- sufficiency relates to reconstruction or density estimation

Reinforcement learning.

- ϕ defines state representations
- sufficiency relates to Markov structure
- transferability appears in policy reuse and generalization

Proposition 4.38 (Induced Simplicity Across Tasks)**.** *If ϕ is sufficient for a family of tasks $\mathcal{T}$ and invariant to nuisance transformations common to $\mathcal{T}$, then each task admits a predictor g_T of reduced complexity compared to operating directly on $\mathcal{X}$.*

Sketch. Sufficiency ensures no relevant information is lost. Invariance removes irrelevant variation. Thus the effective input space for g_T is both smaller and more structured, reducing hypothesis complexity. □

4.5.4 Worked Examples

Example 1: Vision (Invariance) Let x be an image and T a small translation. A convolutional representation ϕ approximately satisfies

$$\phi(Tx) \approx \phi(x),$$

making object classification invariant to position.

Example 2: Language (Contextual Representation) A token embedding $\phi(w_i)$ depends on context:

$$\phi(w_i) = f(w_1, \ldots, w_T).$$

Syntactic and semantic relations become approximately linear or geometrically structured in $\mathcal{Z}$.

Example 3: Reinforcement Learning (State Abstraction) Let x be raw observations and $\phi(x)$ a learned state. If

$$P(s_{t+1} \mid s_t, a_t) \approx P(s_{t+1} \mid x_t, a_t),$$

then ϕ defines a near-Markov representation, simplifying control.

4.5.5 Representation as Geometry

A useful abstraction is to view ϕ as defining a coordinate system on data:

$$\mathcal{X} \xrightarrow{\phi} \mathcal{Z},$$

where:

- distances reflect similarity,

- directions reflect variation factors,
- subspaces correspond to task-relevant structure.

Multiple tasks then act as functions on the same latent space:

$$g_1, g_2, g_3 : \mathcal{Z} \to \mathcal{Y}.$$

Interpretation. Representation learning constructs a shared geometric substrate on which diverse tasks are defined.

4.5.6 Remarks: Limits of the Representation View

Remark 4.39 (Non-identifiability). Many representations yield identical performance. There is no unique "correct" latent space.

Remark 4.40 (Interpretability is not guaranteed). Semantic structure is not automatic. A representation may be useful for prediction while remaining opaque.

Remark 4.41 (Task-dependence). A representation that is optimal for one task may discard information needed for another.

Remark 4.42 (Overclaiming risk). Statements such as "the model understands concepts" should be treated cautiously. Most learned representations are aligned with objectives, not necessarily with human semantics.

4.5.7 Synthesis

This chapter can now be read as a progression:

- Section 4.1: representations as maps
- Section 4.2: objectives that shape representations
- Section 4.3: reuse through transfer
- Section 4.4: scaling representation learning to foundation models
- Section 4.5: unification across paradigms

Conclusion. Representation learning is not a subfield but a structural principle: learning useful coordinate systems in which problems become simpler.

4.5.8 What to Remember

- Representations balance invariance and sufficiency
- Transferability is a central design goal
- Many learning paradigms share the same representation structure
- Representations define geometry, not just features

4.5.9 Common Confusion

- Invariance does not mean information loss is harmless
- Good performance does not imply interpretable representations
- Transferability depends on task families, not universality

4.5.10 Exercises

Exercise 4.43. Give an example where increasing invariance reduces performance due to loss of sufficiency.

Exercise 4.44. Show that if ϕ is invertible, it is always sufficient but not necessarily invariant.

Exercise 4.45. Compare representations learned by supervised and contrastive objectives on the same dataset.

Exercise 4.46. Construct a simple RL example where a poor representation violates the Markov property.

4.5.11 Notes and References

Key directions include:

- early representation learning and feature learning literature
- invariance and equivariance in convolutional architectures
- contrastive and self-supervised learning frameworks
- latent-variable models in generative modeling
- representation learning in reinforcement learning

This area remains partially formalized, with strong empirical guidance and evolving theoretical foundations.

4.6 State Representation in Sequential Decision-Making

4.6.1 Overview

In supervised and self-supervised learning, data is typically assumed to be static. In contrast, reinforcement learning (RL) models *sequential interaction* between an agent and an environment.

Key shift. Data is no longer given; it is generated through actions. Learning must therefore account for feedback loops between decisions and future observations.

4.6.2 Markov Decision Processes

Definition 4.47 (Markov Decision Process (MDP))**.** A Markov Decision Process is a tuple

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where:

- $\mathcal{S}$ is a state space
- $\mathcal{A}$ is an action space
- $P(s' \mid s, a)$ is the transition probability
- $R(s, a)$ is the reward function
- $\gamma \in [0, 1)$ is the discount factor

Definition 4.48 (Markov Property)**.** An environment satisfies the Markov property if

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t).$$

4.6.3 Policies and Returns

Definition 4.49 (Policy)**.** A policy is a mapping

$$\pi(a \mid s),$$

giving the probability of taking action a in state s.

Definition 4.50 (Return)**.** The return from time t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k}).$$

Definition 4.51 (Value Function)**.** The state-value function of a policy π is

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s].$$

Definition 4.52 (Action-Value Function)**.** The action-value function is

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

4.6.4 Bellman Equations

Proposition 4.53 (Bellman Expectation Equation)**.** *For any policy π,*

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s') \right].$$

Proposition 4.54 (Bellman Optimality Equation)**.** *The optimal value function satisfies*

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right].$$

Remark 4.55. The Bellman equations express value functions as fixed points of nonlinear operators.

4.6.5 Bellman Fixed-Point Theorem

Theorem 4.56 (Contraction of the Bellman Operator)**.** *Let $\mathcal{T}$ be the Bellman optimality operator:*

$$(\mathcal{T}V)(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \right].$$

Then for $\gamma \in [0,1)$, $\mathcal{T}$ is a contraction in the sup norm:

$$\|\mathcal{T}V - \mathcal{T}W\|_\infty \le \gamma \|V - W\|_\infty.$$

Proof. For any s,

$$|(\mathcal{T}V)(s) - (\mathcal{T}W)(s)| \le \max_a \gamma \sum_{s'} P(s' \mid s,a) |V(s') - W(s')|.$$

Using $\sum_{s'} P(s' \mid s,a) = 1$,

$$\le \gamma \|V - W\|_\infty.$$

Taking supremum over s yields the result. □

Corollary 4.57. *The Bellman operator has a unique fixed point V^*, and iterative updates*

$$V_{k+1} = \mathcal{T} V_k$$

converge to V^*.

4.6.6 Worked Example: A Finite MDP

Consider an MDP with two states $\{s_1, s_2\}$ and one action per state:

- $R(s_1) = 1$, $R(s_2) = 0$
- $P(s_2 \mid s_1) = 1$
- $P(s_2 \mid s_2) = 1$

Then:

$$V(s_2) = 0 + \gamma V(s_2) \Rightarrow V(s_2) = 0,$$
$$V(s_1) = 1 + \gamma V(s_2) = 1.$$

Interpretation. The value reflects immediate reward plus discounted future reward.

4.6.7 State-Transition View

An MDP can be visualized as a directed graph:

- nodes: states
- edges: actions and transitions
- edge weights: rewards and probabilities

Interpretation. Reinforcement learning is path optimization over this graph under uncertainty.

4.6.8 Connection to Representation Learning

In high-dimensional settings, raw observations x may not satisfy the Markov property. A representation $\phi(x)$ is often learned such that

$$P(s_{t+1} \mid s_t, a_t) \approx P(s_{t+1} \mid x_t, a_t).$$

Insight. Representation learning in RL aims to construct state variables that make the environment approximately Markov.

4.6.9 What to Remember

- RL models sequential decision making under uncertainty
- Value functions satisfy fixed-point equations
- The Bellman operator is a contraction
- Representations in RL aim to recover Markov structure

4.6.10 Common Confusion

- The Markov property is a modeling assumption, not always satisfied
- Value functions depend on policies
- Optimality equations are nonlinear due to the max operator

4.6.11 Exercises

Exercise 4.58. Compute the value function for a three-state chain with deterministic transitions.

Exercise 4.59. Show that policy evaluation defines a linear system in V^π.

Exercise 4.60. Prove that the Bellman expectation operator is also a contraction.

Exercise 4.61. Construct an example where a poor representation violates the Markov property.

4.6.12 Notes and References

Key sources include:

- classical dynamic programming and optimal control
- foundational RL texts (e.g., Sutton and Barto)
- modern deep RL and representation learning work

This area is comparatively well understood in finite settings, but becomes substantially more complex in large-scale or partially observed environments.

4.7 Deep Reinforcement Learning as Learned Representation Plus Control

4.7.1 Overview

Classical reinforcement learning assumes that value functions and policies can be represented exactly. In modern settings, state and action spaces are large or continuous, and learning must rely on *function approximation*.

Key idea. Deep reinforcement learning (deep RL) replaces tabular representations with parameterized functions:

$$V(s) \approx V_\theta(s), \quad Q(s,a) \approx Q_\theta(s,a), \quad \pi(a \mid s) \approx \pi_\theta(a \mid s).$$

This introduces both expressive power and new sources of instability.

4.7.2 Function Approximation in Reinforcement Learning

Definition 4.62 (Value Function Approximation)**.** A value function is approximated by a parameterized model $V_\theta : \mathcal{S} \to \mathbb{R}$.

Definition 4.63 (Action-Value Approximation)**.**

$$Q_\theta : \mathcal{S} \times \mathcal{A} \to \mathbb{R}.$$

Definition 4.64 (Policy Approximation)**.** A policy is parameterized as

$$\pi_\theta(a \mid s).$$

Representation role. In deep RL, these functions typically factor as:

$$s \xmapsto{\phi_\theta} z \xmapsto{g_\theta} \text{value or policy}.$$

Thus, learning control is also learning representations.

4.7.3 Value-Based Methods and Bootstrapping

Value-based methods aim to approximate the Bellman optimality equation:

$$Q^*(s,a) = R(s,a) + \gamma \mathbb{E}_{s'} \max_{a'} Q^*(s',a').$$

Temporal-difference (TD) learning. Given a sample transition (s,a,r,s'), define the target:

$$y = r + \gamma \max_{a'} Q_\theta(s',a').$$

Then update θ to minimize:

$$\mathcal{L}(\theta) = \big(Q_\theta(s,a) - y\big)^2.$$

Remark 4.65 (Bootstrapping)*.* The target depends on the current estimate Q_θ. Learning thus uses its own predictions as targets.

DQN intuition. Deep Q-Networks stabilize this process by:

- using a *target network* for y
- sampling from a replay buffer to decorrelate data

4.7.4 Policy-Based Methods

Instead of learning value functions, one may directly optimize policies.

Definition 4.66 (Objective)**.**

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right].$$

Theorem 4.67 (Policy Gradient Identity)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a)\right].$$

Sketch. Write $J(\theta)$ as an expectation over trajectories. Differentiate under the integral and apply the log-derivative trick:

$$\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta.$$

Rearranging yields the result. □

Remark 4.68. In practice, Q^{π_θ} is replaced by sampled returns or learned critics.

4.7.5 Actor–Critic Methods

Actor–critic methods combine value and policy learning:

- **actor:** policy π_θ
- **critic:** value function V_ϕ or Q_ϕ

Interpretation. The critic provides a learned representation of long-term outcomes, guiding the actor's updates.

4.7.6 Model-Free vs Model-Based Reinforcement Learning

Definition 4.69 (Model-Free RL)**.** Learns value functions or policies without explicitly modeling $P(s' \mid s, a)$.

Definition 4.70 (Model-Based RL)**.** Learns or uses a model of the environment:

$$\hat{P}(s' \mid s, a), \quad \hat{R}(s, a).$$

Comparison.

- Model-free: simpler, often more robust, data-hungry
- Model-based: more sample-efficient, but sensitive to model error

Remark 4.71. Modern approaches often combine both through learned latent dynamics.

4.7.7 Worked Example: Learned State Representation

Consider an agent observing high-dimensional input x_t (e.g., images). A learned encoder ϕ_θ produces:

$$z_t = \phi_\theta(x_t).$$

Suppose:

$$Q_\theta(x_t, a) = g_\theta(z_t, a).$$

If ϕ_θ is trained jointly with Q_θ, it learns to:

- discard irrelevant visual detail
- retain task-relevant structure (e.g., object positions)

Interpretation. The representation ϕ_θ is shaped by control objectives rather than reconstruction or prediction alone.

4.7.8 Deep RL Training Loop

A typical deep RL loop consists of:

1. collect transitions (s, a, r, s')
2. store in replay buffer
3. sample mini-batches
4. update value/policy networks
5. periodically update target networks

Interpretation. Learning interleaves data collection, representation learning, and control optimization.

4.7.9 Connection to Representation Learning

Deep RL integrates representation learning with decision making:

- representations must support prediction of future outcomes
- invariances are task-dependent (control-relevant)
- transfer requires reusable state abstractions

Key insight. Unlike supervised learning, representations are evaluated by their effect on long-term return.

4.7.10 What to Remember

- Deep RL uses function approximation for value and policy
- Bootstrapping introduces both efficiency and instability
- Policy gradients provide a principled optimization framework
- Representations are shaped by control objectives

4.7.11 Common Confusion

- Value-based and policy-based methods are not mutually exclusive
- Bootstrapping is not the same as supervised learning
- Good representations in RL are not necessarily interpretable

4.7.12 Exercises

Exercise 4.72. Derive the TD update for V_θ using the Bellman expectation equation.

Exercise 4.73. Show how the policy gradient identity changes when using a baseline.

Exercise 4.74. Construct an example where model-based RL fails due to model bias.

Exercise 4.75. Explain how representation learning differs between supervised learning and RL.

4.7.13 Notes and References

Key directions include:

- temporal-difference learning and Q-learning
- policy gradient methods (REINFORCE)
- deep Q-networks (DQN)
- actor–critic methods
- model-based RL and latent dynamics models

Deep RL remains less theoretically understood than supervised learning, particularly in the presence of function approximation and non-stationary data.

4.8 Case Studies in Modern Representation Learning

The preceding sections developed the formal language of representations, self-supervised objectives, transfer, foundation models, and reinforcement learning. This section studies four concrete cases through a common analytic template. For each case we ask five questions:

1. What is the training objective?
2. What representation is learned?
3. How is that representation transferred or adapted?
4. What structure does the representation appear to capture?
5. Where does the representation fail?

The purpose is not to celebrate successful models in general terms. It is to compare, in a controlled way, how different learning signals shape different kinds of reusable internal structure.

Case study I: BERT and masked language modeling

A masked language model is trained by hiding selected tokens in a sequence and predicting them from context. If a sentence is written as $x = (x_1, \ldots, x_T)$ and $M \subseteq \{1, \ldots, T\}$ is the set of masked positions, then a simplified masked-language-model objective is

$$\mathcal{L}_{\mathrm{MLM}}(\theta) = -\sum_{i \in M} \log p_\theta(x_i \mid x_{\setminus M}).$$

Thus the model is trained to use surrounding context to infer missing content.

Objective. The objective is predictive but not task-specific. It does not directly optimize sentiment classification, translation, or question answering. It instead encourages the model to estimate local and global linguistic regularities from partially observed text.

Learned representation. The learned representation is a family of contextual token embeddings

$$h_i = \phi_\theta(x)_i,$$

one for each token position i. Unlike static word embeddings, these vectors depend on the entire sentence. The representation of the word "bank," for example, can differ substantially between a financial and a river context. Many downstream systems also use a pooled sequence representation derived from the token states.

Transfer mechanism. Transfer usually occurs in one of two ways:

- a linear probe or shallow classifier is trained on top of frozen contextual embeddings,
- or the full network is fine-tuned on a supervised downstream task.

The first tests how linearly accessible the task information already is; the second allows the representation itself to deform toward the new task.

What the representation captures. Empirically, masked language models learn strong syntactic and semantic structure. Their hidden states often encode agreement, phrase boundaries, lexical disambiguation, and broad semantic similarity. This should not be interpreted as a theorem about language understanding, but it is well supported by probing and downstream performance studies. The important point is that contextual prediction induces a representation in which many linguistic tasks become substantially easier than in raw token space.

Where it fails. The representation also has limits. Masked prediction does not by itself guarantee strong discourse planning, grounded reference, or robust long-horizon reasoning. It may rely on superficial distributional cues, and it can degrade under domain shift, rare vocabulary, or tokenization mismatch. Thus the case supports a careful conclusion: masked prediction yields powerful reusable linguistic structure, but not a complete or theory-certified model of meaning.

Case study II: CLIP and contrastive image–text learning

Contrastive image–text learning starts from paired data $(x^{\text{img}}, x^{\text{text}})$. An image encoder and a text encoder produce vectors

$$z^{\text{img}} = \phi_\theta(x^{\text{img}}), \qquad z^{\text{text}} = \psi_\eta(x^{\text{text}}).$$

A contrastive objective then raises the similarity of matched pairs and lowers the similarity of mismatched pairs in the same batch. A simplified symmetric objective has the form

$$\mathcal{L}_{\text{con}} = -\sum_{i=1}^{B} \log \frac{\exp(\langle z_i^{\text{img}}, z_i^{\text{text}} \rangle / \tau)}{\sum_{j=1}^{B} \exp(\langle z_i^{\text{img}}, z_j^{\text{text}} \rangle / \tau)} + (\text{text-to-image term}).$$

Objective. The objective is not to assign a fixed class label. It is to align two modalities in a shared embedding geometry. Learning therefore depends on correspondence across views rather than explicit task annotations.

Learned representation. The representation is a pair of embedding spaces that are trained to be mutually comparable. Images and texts that describe the same semantic content are placed nearby. This geometry is the central learned object.

Transfer mechanism. Transfer often occurs through prompting rather than ordinary fine-tuning. A downstream class label such as "cat" is turned into a short text description, embedded by the text encoder, and compared against the image embedding. Thus zero-shot classification becomes a nearest-neighbor or similarity-ranking problem in the shared representation space. The same geometry also supports cross-modal retrieval.

What the representation captures. Empirically, CLIP-style models capture high-level semantic correspondences that ordinary supervised classifiers do not naturally expose. They can connect images to flexible textual descriptions and often generalize better across open-vocabulary settings than closed-set classifiers. This is strong evidence that contrastive cross-modal objectives can produce transferable semantic structure.

Where it fails. The learned geometry is imperfect. CLIP-style systems can struggle with fine spatial relations, counting, OCR-heavy inputs, subtle compositional distinctions, and dataset-specific language biases. They may also inherit social or stylistic biases from web-scale data. Thus the case shows both the power and the fragility of alignment-based representations: they are broad, but not uniformly precise.

Case study III: supervised pretraining and transfer in vision

A classical transfer-learning case begins with supervised pretraining on a large dataset such as ImageNet. A convolutional or transformer-based vision backbone learns a representation

$$z = \phi_\theta(x)$$

by minimizing a labeled classification objective on the source domain. The backbone is then reused on a smaller target dataset.

Objective. The source objective is ordinary supervised prediction. For source examples (x, y), one minimizes a loss such as

$$\mathcal{L}_{\text{sup}}(\theta, w) = -\log p_{\theta,w}(y \mid x).$$

This is task-specific at training time, but the resulting representation often contains features that survive the original task boundary.

Learned representation. The learned visual representation tends to organize edges, textures, parts, object fragments, and increasingly abstract category-level information across depth. Early layers often transfer broadly, whereas later layers may be more specialized to the source label taxonomy.

Transfer mechanism. Three standard transfer mechanisms are common:

- **feature extraction:** freeze the backbone and train a new head,
- **linear probing:** test whether the target task is already linearly recoverable,
- **full fine-tuning:** update all layers on the target data.

A typical evidence-driven comparison is to evaluate these three settings on a small target dataset. If linear probing performs well, the source representation already exposes the relevant structure. If full fine-tuning yields a large improvement, the features are useful but not fully aligned with the target domain.

What the representation captures. This case shows that even a source objective designed for one label set can learn visual primitives useful for many other tasks. In practice, pretrained backbones often reduce sample complexity and improve optimization on small target datasets. This is among the clearest empirical demonstrations of representation reuse in modern machine learning.

Where it fails. Transfer is not automatic. When the source and target domains differ sharply, for example from natural images to specialized medical images or industrial imagery, later-layer features may be poorly aligned. A frozen backbone may miss fine texture, measurement, or domain-specific cues. This case therefore illustrates an important general lesson: transfer depends not only on source accuracy, but on compatibility between the source-induced representation and the target task.

Case study IV: deep reinforcement learning from raw observations

In deep reinforcement learning, the system receives observations sequentially, acts, and collects rewards. A value-based example uses a representation network together with an action-value head:

$$z_t = \phi_\theta(x_t), \qquad Q_{\theta,w}(x_t, a) = g_w(z_t, a).$$

Training then minimizes a temporal-difference loss based on bootstrapped targets such as

$$\mathcal{L}_{\mathrm{TD}}(\theta, w) = \Big(Q_{\theta,w}(x_t, a_t) - [r_t + \gamma \max_{a'} Q_{\bar{\theta},\bar{w}}(x_{t+1}, a')]\Big)^2.$$

Objective. The objective is neither supervised labeling nor ordinary self-supervised prediction. It is control: the representation is shaped by its contribution to long-term return through value estimation or policy optimization.

Learned representation. The learned representation is a latent state useful for decision making. From raw pixels, for example, the encoder may emphasize controllable objects, positions, velocities, or imminent hazards while suppressing irrelevant background variation. In that sense, deep RL learns task-oriented state abstractions.

Transfer mechanism. Transfer can occur by reusing the encoder across related control tasks, initializing a new agent from a pretrained visual backbone, or combining auxiliary self-supervised objectives with policy learning. Compared with standard transfer learning, however, reuse in RL is often less stable because the target policy changes the data distribution itself.

What the representation captures. When successful, the representation captures control-relevant geometry: which states are similar for planning, which features predict reward, and which transitions are dynamically important. This is a distinct notion of usefulness from classification or language modeling. A good RL representation need not be human-interpretable; it must support effective action.

Where it fails. Deep RL representations are often brittle. They may overfit to one reward specification, fail under visual distractors or small environment changes, and learn shortcuts tied to exploration history. Because data is generated by interaction, not passively given, representation learning is coupled to nonstationary sampling. This makes transfer and reliability substantially harder than in standard supervised settings.

Comparative analysis

These four cases support a disciplined comparison. The choice of objective determines which invariances and distinctions are encouraged:

- masked prediction emphasizes contextual reconstruction,
- contrastive alignment emphasizes agreement across views or modalities,
- supervised pretraining emphasizes source-label discrimination,
- reinforcement learning emphasizes control-relevant state abstraction.

The representations are therefore not interchangeable. Each case learns a useful internal coordinate system, but the geometry of that coordinate system is shaped by the learning signal.

Case	Objective	Learned representation	Transfer mechanism	Typical failure modes
BERT / MLM	Predict masked tokens from context	Contextual token and sequence embeddings	Linear probe or fine-tuning	Weak grounding, long-context limits, domain shift
CLIP / image–text contrast	Align matched image–text pairs, separate mismatches	Shared cross-modal embedding geometry	Prompting, zero-shot classification, retrieval	Counting, fine relations, OCR, data bias
Supervised vision transfer	Source-domain label prediction	Hierarchical visual features from edges to semantics	Frozen backbone, linear probe, full fine-tuning	Domain mismatch, over-specialized late features
Deep RL control	Maximize long-run return through bootstrapped or policy losses	Task-oriented latent state for control	Encoder reuse, auxiliary pretraining, related-task initialization	Sample inefficiency, brittleness under shift, reward dependence

What these cases show

The evidence across the cases supports three chapter-level conclusions.

First, representation quality is objective-dependent. There is no single universally best representation. The usefulness of a learned feature space depends on what information the training objective rewards the model for preserving and organizing.

Second, transfer is strongest when source and target structure overlap. A representation transfers well when the invariances and distinctions learned in pretraining remain appropriate for the downstream task. This is why image classifiers can transfer to related visual tasks, why BERT-like encoders transfer broadly across NLP tasks, and why RL transfer is harder when rewards or transition structure change.

Third, failure analysis matters as much as success analysis. A representation should be judged not only by benchmark gains but by the structure of its errors. Failure modes reveal what the model has and has not organized well in its internal coordinates. This is especially important in multimodal systems and control, where apparent success can conceal brittle abstractions.

Notes and references

Canonical examples include BERT for masked language modeling, CLIP for contrastive image–text learning, the large literature on ImageNet pretraining and downstream visual transfer, and deep Q-learning or actor–critic systems for learned control from high-dimensional observations. The case-study literature is primarily empirical. Its main contribution is comparative evidence about which objectives produce which kinds of reusable structure. The theoretical language introduced earlier in the chapter—invariance, sufficiency, transferability, and state abstraction—provides a way to interpret that evidence more systematically.

4.9 What Is Established and What Remains Open

This chapter has argued that modern machine learning increasingly relies on *learned representations* that can be reused, adapted, or conditioned across tasks. That broad claim is now well supported by practice, but the surrounding theory is uneven. Some parts of the chapter rest on firm mathematical foundations. Other parts are established primarily by repeated empirical success. Still others remain conjectural, heuristic, or only partially explained.

Accordingly, the conclusion of this chapter should separate three levels of understanding:

- results that are mathematically established within clear assumptions,
- empirical facts that are robust across many experiments and applications,
- theoretical conjectures or explanatory narratives that remain incomplete.

This distinction is especially important in representation learning, where practical success often runs ahead of general theory.

Established mathematical structure

A first group of claims is established at the level of definitions, operators, and finite-model analysis.

Representations induce new hypothesis classes. If one fixes a representation map $\phi : \mathcal{X} \to \mathcal{Z}$ and then learns only a downstream predictor g on $\mathcal{Z}$, one has effectively replaced a hypothesis class on $\mathcal{X}$ by an induced class

$$\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}.$$

This is a precise mathematical statement. It formalizes the idea that representation learning changes the geometry and complexity of the learning problem. What is *not* automatically established is that the induced class is better for every downstream task. That depends on how well ϕ preserves relevant information and suppresses nuisance variation.

Self-supervised objectives can be written as well-defined optimization problems. Masked prediction, autoregressive prediction, and contrastive learning all admit clean abstract formulations. In that sense, the chapter's self-supervised learning material is not merely conceptual. One can define contexts, augmentations, positives, negatives, and predictive losses precisely. Some objectives also admit limited theorem-level interpretations, such as lower-bound viewpoints for contrastive criteria. However, a mathematically clean objective is not the same thing as a complete theory of the representations it will produce in realistic large-scale settings.

Transfer design spaces can be formalized. Linear probing, frozen feature extraction, full fine-tuning, adapters, and low-rank updates are not just engineering terms. They define distinct optimization and function-class constraints. One can state exactly which parameters are fixed and which are allowed to move. Domain adaptation theory also provides rigorous upper bounds that relate target-domain error to source-domain error, discrepancy terms, and irreducible mismatch. These results do not solve transfer learning in practice, but they do provide a mathematical language for saying why low source error alone is insufficient.

Finite-state reinforcement learning has strong foundations. For Markov decision processes with finite state and action spaces and discount factor $\gamma \in [0, 1)$, Bellman operators are contractions in the sup norm, have unique fixed points, and support convergent dynamic-programming procedures.

This is one of the chapter's clearest theorem-level domains. The gap appears when moving from tabular value functions to large-scale function approximation, partial observability, and nonstationary data collection.

Established empirical facts

A second group of claims is not purely theorem-driven, but is nevertheless strongly established by repeated experimental evidence.

Pretraining often improves downstream performance. Across language, vision, multimodal learning, and many practical pipelines, models trained first on large upstream corpora often outperform models trained only on the final target task. This is now a stable empirical pattern rather than an isolated observation. The size of the gain depends on data, architecture, and adaptation method, but the broad fact of useful reuse is no longer controversial.

Self-supervised objectives can produce transferable features. Masked language modeling yields representations useful for classification, tagging, retrieval, and many other language tasks. Contrastive image or image–text learning yields spaces that can support retrieval, linear evaluation, zero-shot transfer, and flexible adaptation. The precise mechanism remains debated, but the empirical fact that broad utility can arise without task-specific labels is firmly established.

Scaling changes capability profiles. Larger models trained on larger and more diverse corpora often exhibit better transfer, better few-shot or in-context adaptation, and broader coverage of downstream tasks. This statement should be read carefully: it is a robust empirical observation under particular training regimes, not a theorem that larger always means better. Nevertheless, scaling behavior is one of the central empirical regularities of modern representation learning.

Adaptation mechanism matters. Frozen backbones, linear probes, full fine-tuning, and lightweight parameter-efficient methods can behave quite differently even when built on the same pretrained model. In some settings a linear probe already performs strongly, indicating that the representation is well aligned with the target task. In other settings significant weight updates are needed. This too is a repeatable empirical fact: transfer success is not a binary property of a representation, but depends on how adaptation is carried out.

Deep reinforcement learning can learn useful state abstractions. In control tasks with high-dimensional observations, end-to-end systems can learn internal variables that support successful action selection. This is empirically important even when the internal states are not directly interpretable. What is established is that learned control can work and that representations matter. What remains less settled is a general account of which representation properties make such control sample-efficient, robust, and transferable.

Theoretical conjectures, partial explanations, and design narratives

A third group of claims should be treated more cautiously. These are ideas that are plausible, often useful, and sometimes strongly supported by experiments, but not yet settled as general theory.

Why broad transfer emerges. It is tempting to say that pretraining works because models discover abstract semantic structure that is shared across tasks. This may be substantially true, but as a general theorem it is not available. Different models may transfer for different reasons: some may encode reusable syntax, some robust local statistics, some broad world regularities, and some simply exploit overlap between pretraining and downstream distributions. "The model learned a reusable latent world model" is therefore an interpretive claim, not a universally established conclusion.

Why in-context learning works. Large sequence models can often condition on examples in the prompt and improve performance without parameter updates. There are partial explanatory frameworks: implicit Bayesian analogies, meta-learning analogies, induction-head mechanisms in transformers, and linearized analyses in restricted regimes. But there is not yet one agreed theory that covers realistic large models across tasks. It is more accurate to say that in-context behavior is an established empirical phenomenon with several promising but incomplete explanatory accounts.

What representation geometry means semantically. It is common to describe latent spaces as if directions correspond to concepts and distances correspond to meaning. Sometimes this is a productive approximation. Sometimes it is misleading. A representation can be highly useful for transfer while remaining poorly aligned with human-interpretable factors. Thus statements about "semantic structure" should be read as claims about task-relevant organization, not automatic guarantees of transparent meaning.

When scaling laws explain performance. Empirical scaling laws describe useful regularities relating loss, data, model size, and compute in certain regimes. They are invaluable for engineering and forecasting. But they are not a complete theory of capability, transfer, or safety. In particular, extrapolating from smooth loss curves to qualitative behavioral claims requires caution.

Unresolved questions

The chapter closes naturally with a short list of unresolved questions that remain central to current research.

Open questions on transfer.

- What properties of a representation best predict successful transfer across distant tasks rather than merely nearby tasks?
- When does full fine-tuning improve a representation, and when does it overwrite broadly useful structure?
- Can we characterize, in advance, whether a downstream task will be well served by linear probing, adapters, or full model updating?

Open questions on in-context learning.

- Under what conditions does prompt conditioning behave like true task adaptation rather than superficial pattern matching?
- Which parts of in-context learning can be explained by sequence statistics alone, and which require stronger internal algorithmic structure?

- How should one separate genuine context-dependent computation from hidden memorization effects?

Open questions on representation semantics.

- When do latent directions correspond to meaningful factors of variation, and when are they merely convenient coordinates for optimization?
- Can usefulness, interpretability, and robustness be aligned simultaneously, or do they often compete?
- What should count as evidence that a representation captures "concepts" rather than only predictive correlations?

Open questions on reinforcement learning representations.

- How can one learn state abstractions that preserve control-relevant information while remaining sample-efficient?
- Why do some deep RL representations transfer poorly across environments that look similar to humans?
- Can representation-learning principles unify model-free and model-based control more cleanly than current design practice allows?

A sober closing synthesis

The central scholarly conclusion of this chapter is not that one grand theory has already unified self-supervision, transfer, foundation models, and reinforcement learning. Rather, the conclusion is more measured and more useful:

> Modern machine learning is increasingly organized around the learning of reusable internal structure, but the degree to which that structure is understood depends strongly on the regime under study.

In some regimes, such as finite Markov decision processes or formal decompositions of induced hypothesis classes, the mathematics is comparatively clear. In others, especially large-scale pretraining and in-context adaptation, practice is ahead of comprehensive theory. That is not a weakness of the subject so much as an honest description of its present state.

The next chapter turns from reusable representations to another major theme of modern AI: models that do not only support prediction and control, but also learn to describe, generate, and sample from complex data distributions.

Annotated bibliography

The references below are grouped by topic and annotated briefly to indicate why they matter for this chapter.

Self-supervised learning.

- **Devlin et al. (BERT).** A canonical masked-language-model pretraining paper showing that bidirectional contextual prediction can produce broadly reusable language representations.
- **He et al. (MAE).** A representative masked-prediction approach in vision, useful for understanding how reconstruction from partial information can scale beyond language.

- **Chen et al. (SimCLR).** A clean contrastive-learning framework emphasizing the role of augmentations, positive pairs, and large-batch discrimination.
- **Grill et al. (BYOL).** Important because it showed that strong representation learning can emerge even without the usual explicit negative-pair construction.

Transfer learning and adaptation.

- **Pan and Yang.** A standard early survey on transfer learning, useful for terminology such as source domain, target domain, and adaptation settings.
- **Ben-David et al.** Foundational for domain-adaptation bounds that relate target error to source error and discrepancy measures.
- **Howard and Ruder.** An influential practical account of transfer in modern NLP through fine-tuning, showing how pretrained representations can be adapted effectively.
- **Hu et al. (LoRA).** A standard reference for low-rank adaptation and parameter-efficient transfer in large models.

Foundation models and large-scale pretraining.

- **Brown et al. (GPT-3).** Canonical for large autoregressive language models and few-shot prompting behavior.
- **Bommasani et al.** A widely cited systems-level framework for discussing what foundation models are, how they are trained, and how they are deployed.
- **Kaplan et al.** A central scaling-law reference describing empirical regularities relating performance, model size, data, and compute.
- **Chowdhery et al. (PaLM) or Touvron et al. (LLaMA).** Representative large-model studies showing how scaling and design choices affect broad downstream performance.
- **Ouyang et al. (InstructGPT).** Useful for post-pretraining alignment, instruction tuning, and the distinction between base-model capability and deployed assistant behavior.

Reinforcement learning and learned control.

- **Sutton and Barto.** The standard textbook foundation for Markov decision processes, value functions, Bellman equations, and policy gradients.
- **Mnih et al. (DQN).** Canonical for deep value-based reinforcement learning from high-dimensional observations.
- **Williams (REINFORCE).** A foundational policy-gradient reference.
- **Lillicrap et al. (DDPG), Schulman et al. (TRPO/PPO), or Haarnoja et al. (SAC).** Standard examples of deep policy optimization and actor–critic style methods in continuous control.
- **Lesort et al. or related surveys on representation learning in RL.** Helpful for connecting state abstraction and learned control directly to the chapter's representation-learning theme.

What to remember

- Some parts of this chapter are theorem-level; others are established mainly by repeated empirical success.

- Broad transfer, self-supervised reuse, and scaling effects are real empirical facts, even when full explanatory theories are missing.
- In-context learning and representation semantics remain active areas of explanation rather than settled theory.
- A mature view of modern AI must separate mathematical certainty, empirical regularity, and open conjecture.

Common confusion

- "Empirically established" does not mean "proved in general."
- "Semantic representation" does not automatically mean human-interpretable latent variables.
- "Scaling law" does not mean a universal theorem about intelligence or capability.

Exercises

1. Give one example from this chapter of a claim that is mathematically established and one example of a claim that is primarily empirical.
2. Explain why a domain-adaptation bound does not by itself guarantee successful transfer in a new application.
3. Compare two possible explanations of in-context learning and state clearly what each explanation leaves open.
4. Describe a situation in which a representation is useful for prediction but not meaningfully interpretable.
5. Choose one item from the annotated bibliography and explain how it supports the chapter's central thesis about reusable learned structure.

Chapter 5

Generative Modeling and Generative AI

5.1 What Does It Mean to Model a Data Distribution?

The previous chapters emphasized prediction, representation, transfer, and control. In those settings the central object was often a map from an input x to an output y or to a useful internal representation. Generative modeling changes the point of view. Instead of asking only how to predict from data, we ask how to model the data distribution itself.

Chapter role. This opening section sets the notation and taxonomy for the chapter. It introduces the main kinds of generative models, distinguishes likelihood evaluation from sample generation, and explains why latent variables and implicit samplers lead to different mathematical and algorithmic questions.

Basic notation and the chapter's main question

Let $x \in \mathcal{X}$ denote an observed data point, for example an image, a sentence, an audio segment, or a trajectory. A dataset is written as

$$D = \{x_1, \ldots, x_n\}.$$

A generative model with parameters θ attempts to capture the structure of the unknown data-generating distribution by introducing one or more of the following objects:

$$p_\theta(x), \qquad p_\theta(x, z), \qquad x \sim p_\theta, \qquad x = G_\theta(\varepsilon).$$

These symbols already suggest that generative modeling is not a single method but a family of related viewpoints. The main question of the chapter is therefore the following:

> What mathematical object is the model learning, and what operations does that object support?

Some models support exact or approximate evaluation of probabilities. Some support efficient sampling. Some introduce latent variables that organize the data through hidden structure. Some provide only a simulation mechanism without a tractable density. The rest of the chapter will develop these alternatives systematically.

Definitions: generative model, density model, sampler, latent model, implicit model

Definition 5.1 (Generative model)**.** A *generative model* is a parameterized family of distributions or sampling procedures intended to approximate the distribution of observed data. Depending on the framework, this may mean learning a density $p_\theta(x)$, a joint model $p_\theta(x, z)$ with latent variables, or a sampling map that produces synthetic observations with realistic statistics.

Definition 5.2 (Density model)**.** A *density model* is a generative model that assigns a probability mass or density value to each observation x through an explicit expression such as

$$p_\theta(x).$$

When this quantity is tractable, one can evaluate likelihoods, compare models by log-likelihood, and reason directly about probability mass.

Definition 5.3 (Sampler)**.** A *sampler* is a mechanism that generates synthetic data points from a model distribution. We write

$$x \sim p_\theta$$

to indicate that x is sampled from the model. The ability to sample does not by itself imply that $p_\theta(x)$ can be evaluated in closed form.

Definition 5.4 (Latent-variable model)**.** A *latent-variable model* introduces a hidden variable $z \in \mathcal{Z}$ and specifies a joint distribution

$$p_\theta(x, z) = p_\theta(x \mid z) p_\theta(z).$$

The observed-data distribution is obtained by marginalization:

$$p_\theta(x) = \sum_z p_\theta(x, z)$$

for discrete z, or

$$p_\theta(x) = \int p_\theta(x, z)\, dz$$

for continuous z.

Definition 5.5 (Implicit model)**.** An *implicit model* defines a procedure for generating data, often in the form

$$\varepsilon \sim p(\varepsilon), \qquad x = G_\theta(\varepsilon),$$

without necessarily providing a tractable formula for $p_\theta(x)$. Such models are judged primarily through sample quality, distributional matching, or downstream utility rather than direct likelihood evaluation.

These definitions organize much of modern generative modeling. Likelihood-based models, latent-variable models, adversarial generators, normalizing flows, and diffusion models all fit into this language, but they emphasize different parts of it.

Likelihood evaluation versus sample generation

Two goals in generative modeling are closely related but mathematically distinct.

Likelihood evaluation. A model supports likelihood evaluation if it can compute or approximate quantities such as

$$p_\theta(x) \quad \text{or} \quad \log p_\theta(x).$$

This is useful for density estimation, uncertainty quantification, compression, and comparison by probabilistic criteria.

Sample generation. A model supports sample generation if it can draw synthetic observations

$$x^{\text{new}} \sim p_\theta.$$

This is useful for simulation, creative generation, imputation, and scenario construction.

A key lesson of the chapter is that these two capabilities do not always coincide. A model may assign a tractable density but generate poor-looking samples, or generate highly realistic samples while providing no tractable pointwise likelihood. Thus one must always ask which operation the model is designed to support.

Example 5.6 (A density that is easy to evaluate)**.** Suppose

$$p_\theta(x) = \mathcal{N}(x \mid \mu, \Sigma).$$

Then both evaluation and sampling are straightforward. For any x, one can compute $p_\theta(x)$ explicitly, and one can also generate synthetic samples by drawing from the Gaussian. This is the idealized case in which density estimation and sampling align cleanly.

Example 5.7 (A sampler without tractable likelihood)**.** Suppose

$$\varepsilon \sim \mathcal{N}(0, I), \qquad x = G_\theta(\varepsilon),$$

where G_θ is a deep neural network. Then generating x may be easy, but the induced density on x can be difficult or impossible to evaluate tractably. This is the typical situation for implicit generative models.

Latent variables and hidden structure

One of the chapter's main principles is that complicated observations may arise from simpler hidden causes. If z denotes a latent variable, then the model is written as

$$p_\theta(x, z) = p_\theta(x \mid z) p_\theta(z).$$

The variable z may represent class identity, geometric structure, semantic content, style, trajectory intent, or other hidden factors. The advantage of this factorization is conceptual and computational: modeling becomes easier if the complexity of the observations can be explained through lower-dimensional or more structured latent causes.

Sampling from a latent-variable model typically proceeds in two stages:

$$z \sim p_\theta(z), \qquad x \sim p_\theta(x \mid z).$$

This separates the model into a prior over hidden structure and a decoder or observation model that maps hidden structure to visible data.

Inference, however, runs in the opposite direction. Given an observed x, one would like to understand the posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}.$$

This is often much harder than sampling, and it is one of the central reasons variational methods arise later in the chapter.

Explicit models, latent models, and implicit samplers: three worked contrasts

This section is intended to set notation, but it should also make the distinctions concrete. The following three examples give a compact contrast among the main model types.

Example 5.8 (Explicit density model: a Gaussian on $\mathbb{R}^d$)**.** Let

$$p_\theta(x) = \mathcal{N}(x \mid \mu, \Sigma), \qquad \theta = (\mu, \Sigma).$$

Then:

- the model directly defines a density on observations,
- likelihood evaluation is tractable,
- sampling is tractable,
- there is no latent variable separate from the observed x.

This is the cleanest form of explicit generative modeling, but it may be too restrictive for high-dimensional structured data.

Example 5.9 (Latent-variable model: finite mixture)**.** Let $z \in \{1, \dots, K\}$ and define

$$p_\theta(z = k) = \pi_k, \qquad p_\theta(x \mid z = k) = \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

Then the joint model is

$$p_\theta(x, z = k) = \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

and the marginal observed-data density is

$$p_\theta(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

Compared with a single Gaussian, this model introduces hidden structure. The latent variable z explains which component generated the data point. Likelihood evaluation remains tractable, but latent inference becomes part of the problem.

Example 5.10 (Implicit sampler: neural generator)**.** Let

$$\varepsilon \sim \mathcal{N}(0, I), \qquad x = G_\theta(\varepsilon).$$

Then sampling is immediate: draw ε and push it through G_θ. However, unless G_θ has special structure, there may be no tractable closed-form expression for the density $p_\theta(x)$. The model is therefore suitable when realistic generation is primary, but it is less suitable when exact likelihood evaluation is required.

These three examples already preview the chapter's later routes: explicit density modeling, latent-variable inference, and sample-driven implicit generation. Normalizing flows and diffusion models will later show that some modern methods mix these viewpoints rather than falling perfectly into one category.

A taxonomy of generative models

Figure 5.1 gives a compact taxonomy that will guide the rest of the chapter.

Conditional generation and representation

Many generative tasks are conditional rather than unconditional. One may wish to generate an image from text, speech from phonetic content, or a completion from a prompt. In such cases one writes

$$p_\theta(x \mid c)$$

for a condition c. The condition may be a class label, a text description, a partial observation, or a control variable.

Model family	**Core object**	**Main operation**	**Typical question**
Explicit density model	$p_\theta(x)$	Evaluate likelihood and sample	How much probability does the model assign to this x?
Latent-variable model	$p_\theta(x, z)$	Marginalize or infer z	Which hidden structure explains this observation?
Implicit model	$x = G_\theta(\varepsilon)$	Sample generation	Can the model generate realistic observations?
Conditional generative model	$p_\theta(x \mid c)$ or $x = G_\theta(c, \varepsilon)$	Controlled generation	How do we generate an x consistent with condition c?
Hybrid models	combinations of the above	Trade off tractability and realism	Which operations are preserved and which are approximated?

Figure 5.1: A compact taxonomy of generative models. The same model may belong to more than one row; for example, latent-variable models may also be explicit density models, and some modern architectures approximate several operations at once.

Conditional generation is especially important in modern generative AI because the system is often evaluated by whether it can produce outputs consistent with instructions or side information. Mathematically, however, the same distinctions remain: one may seek a conditional density, a conditional latent-variable model, or a conditional implicit sampler.

This also connects the chapter to representation learning. A useful representation may serve as the condition c, and conversely a generative model may learn a latent code z that functions as a representation. Thus generative modeling and representation learning are not separate topics. They are often two views of the same internal structure.

What this section establishes, and what it does not

At this stage, the mathematical claims are modest but important. The section establishes a clean vocabulary:

- $p_\theta(x)$ denotes an explicit model of observed data,
- $p_\theta(x, z)$ denotes a joint model with latent variables,
- $x \sim p_\theta$ denotes sampling from the model,
- $x = G_\theta(\varepsilon)$ denotes an implicit sampling mechanism.

It also establishes that likelihood evaluation and sample generation are distinct operations.

What the section does *not* yet establish is which generative route is best. That depends on the modeling objective, the data modality, the notion of tractability one requires, and the trade-off between statistical faithfulness and computational convenience. The remaining sections of the chapter develop these trade-offs in detail.

What to remember

- A generative model may define a density, a joint model with latent variables, or a sampling procedure.
- The notation $p_\theta(x)$, $p_\theta(x, z)$, and $x = G_\theta(\varepsilon)$ corresponds to genuinely different modeling commitments.
- Likelihood evaluation and sample generation are related but not identical capabilities.
- Latent-variable models explain visible data through hidden structure.
- Implicit models may generate realistic data without offering tractable pointwise likelihoods.

Common confusion

- **"Generative" does not always mean "can compute exact likelihood."** Some generative models are defined primarily through sampling.
- **"Can sample" does not imply "understands probability well."** High sample quality does not automatically provide calibrated density values.
- **Latent variables are not required in every generative model.** They are one important route, not the only route.

Exercises

1. Let $p_\theta(x) = \mathcal{N}(x \mid 0, I)$ on $\mathbb{R}^d$. Explain why this model is both an explicit density model and a sampler.
2. For a finite mixture model with K components, derive the formula

$$p_\theta(x) = \sum_{k=1}^{K} \pi_k p_\theta(x \mid z = k).$$

   Why is this a latent-variable model?
3. Give an example of a task in which tractable likelihood evaluation matters more than sample realism, and another in which sample realism matters more than tractable likelihood evaluation.
4. Compare an explicit density model and an implicit sampler on the following five criteria: tractable evaluation, sample quality, hidden structure, inference, and ease of training.

Notes and references

The basic distinction between explicit density modeling, latent-variable modeling, and implicit generation runs through the modern generative-modeling literature. Classical probability texts supply the background for density estimation and latent variables. Mixture models and the EM tradition develop one of the oldest latent-variable routes. Adversarial generation emphasizes implicit sampling. Normalizing flows restore tractable density through invertible transformations. Diffusion models show that high-quality generation can also emerge from iterative denoising constructions. The purpose of this chapter is to place these families in one coherent mathematical picture rather than to treat them as unrelated model brands.

5.2 Latent Variables, Incomplete Data, and the EM Principle

Section 5.1 introduced the idea that a generative model may explain observations through hidden causes. That idea becomes mathematically nontrivial as soon as the latent variables are unobserved. Even when the joint model $p_\theta(x, z)$ is simple, the observed-data likelihood may become awkward because the latent variable must be summed or integrated out. This section develops the expectation–maximization (EM) principle as the canonical response to that difficulty. It is one of the oldest and still one of the clearest examples of how lower bounds, latent inference, and parameter estimation interact in generative modeling.

Section role. The goal here is not merely to describe EM procedurally. The section formalizes the distinction between complete-data and incomplete-data likelihoods, derives the EM lower bound using Jensen's inequality, proves the monotonic-improvement property in a clean finite setting, and works through Gaussian mixtures as the chapter's first full latent-variable example.

Complete data, incomplete data, and the source of the difficulty

Suppose observations $x_1, \ldots, x_n$ are accompanied by latent variables $z_1, \ldots, z_n$. The model is specified through a joint density or probability mass function

$$p_\theta(x, z).$$

If the latent variables were observed, parameter estimation would often be straightforward. This motivates the following distinction.

Definition 5.11 (Complete-data likelihood)**.** For observations $x = (x_1, \ldots, x_n)$ and latent variables $z = (z_1, \ldots, z_n)$, the complete-data likelihood is

$$L_c(\theta; x, z) = p_\theta(x, z),$$

and the complete-data log-likelihood is

$$\ell_c(\theta; x, z) = \log p_\theta(x, z).$$

Definition 5.12 (Incomplete-data or observed-data likelihood)**.** When only x is observed, the relevant likelihood is

$$L(\theta; x) = p_\theta(x) = \sum_z p_\theta(x, z)$$

for discrete latent variables, or

$$L(\theta; x) = p_\theta(x) = \int p_\theta(x, z)\, dz$$

for continuous latent variables. Its logarithm

$$\ell(\theta; x) = \log p_\theta(x)$$

is called the observed-data or incomplete-data log-likelihood.

The complete-data problem is often easy because the logarithm distributes over products and because the latent structure appears explicitly. The incomplete-data problem is harder because one must first marginalize and only then take the logarithm. For independent observations,

$$\ell(\theta; x_1, \ldots, x_n) = \sum_{i=1}^{n} \log p_\theta(x_i) = \sum_{i=1}^{n} \log \sum_{z_i} p_\theta(x_i, z_i),$$

or the corresponding integral form. The expression

$$\log \sum_{z_i} p_\theta(x_i, z_i)$$

is the characteristic obstruction: the logarithm of a sum destroys much of the simple algebra available in the complete-data setting.

A lower bound from Jensen's inequality

The central idea behind EM is to introduce an auxiliary distribution over the latent variables. For one observation x, let $q(z)$ be any distribution on the latent space with $q(z) > 0$ whenever $p_\theta(x, z) > 0$. Then

$$\log p_\theta(x) = \log \sum_z q(z) \frac{p_\theta(x, z)}{q(z)}.$$

Since log is concave, Jensen's inequality gives

$$\log p_\theta(x) \geq \sum_z q(z) \log \frac{p_\theta(x, z)}{q(z)}.$$

For a dataset, summing over i yields the lower bound

$$\ell(\theta; x_1, \ldots, x_n) \geq \sum_{i=1}^n \sum_{z_i} q_i(z_i) \log \frac{p_\theta(x_i, z_i)}{q_i(z_i)}.$$

It is useful to name the right-hand side. Define

$$\mathcal{F}(q, \theta) = \sum_{i=1}^n \sum_{z_i} q_i(z_i) \log p_\theta(x_i, z_i) - \sum_{i=1}^n \sum_{z_i} q_i(z_i) \log q_i(z_i).$$

Then

$$\ell(\theta; x_1, \ldots, x_n) \geq \mathcal{F}(q, \theta).$$

The first term is an expected complete-data log-likelihood; the second is the entropy of the auxiliary distributions.

A more informative form comes from comparing the bound to the exact posterior.

Proposition 5.13 (Likelihood decomposition)**.** *For any choice of auxiliary distributions $q_i(z_i)$,*

$$\ell(\theta; x_1, \ldots, x_n) = \mathcal{F}(q, \theta) + \sum_{i=1}^n \mathrm{KL}\big(q_i(z_i) \,\|\, p_\theta(z_i \mid x_i)\big),$$

where $\mathrm{KL}(q\|p) = \sum_z q(z) \log \frac{q(z)}{p(z)}$ in the discrete case. In particular, $\mathcal{F}(q, \theta)$ is a lower bound on the observed-data log-likelihood, and equality holds if and only if

$$q_i(z_i) = p_\theta(z_i \mid x_i) \quad \textit{for all } i.$$

Proof. Starting from Bayes' rule,

$$p_\theta(z_i \mid x_i) = \frac{p_\theta(x_i, z_i)}{p_\theta(x_i)},$$

we obtain

$$\log p_\theta(x_i) = \log p_\theta(x_i, z_i) - \log p_\theta(z_i \mid x_i).$$

Taking expectation with respect to q_i gives

$$\log p_\theta(x_i) = \sum_{z_i} q_i(z_i) \log p_\theta(x_i, z_i) - \sum_{z_i} q_i(z_i) \log p_\theta(z_i \mid x_i).$$

Add and subtract $\sum_{z_i} q_i(z_i) \log q_i(z_i)$ to get

$$\log p_\theta(x_i) = \sum_{z_i} q_i(z_i) \log \frac{p_\theta(x_i, z_i)}{q_i(z_i)} + \sum_{z_i} q_i(z_i) \log \frac{q_i(z_i)}{p_\theta(z_i \mid x_i)}.$$

The first term is the ith contribution to $\mathcal{F}(q, \theta)$ and the second term is the KL divergence. Summing over i proves the claim. □

This proposition explains EM conceptually. The lower bound becomes tight when the auxiliary distribution is chosen to be the exact posterior under the current parameters. The algorithm then improves the parameters while keeping that posterior-based bound in view.

The EM algorithm

Let θ^{old} denote the current parameter value. EM alternates between an inference step and an optimization step. The inference step chooses the posterior under the current parameters:

$$q_i^{\text{old}}(z_i) = p_{\theta^{\text{old}}}(z_i \mid x_i).$$

Because the KL divergence then vanishes,

$$\ell(\theta^{\text{old}}; x) = \mathcal{F}(q^{\text{old}}, \theta^{\text{old}}).$$

If we now keep q^{old} fixed and maximize $\mathcal{F}(q^{\text{old}}, \theta)$ over θ, the entropy term is constant in θ, so the relevant objective reduces to

$$Q(\theta, \theta^{\text{old}}) = \sum_{i=1}^{n} \mathbb{E}_{z_i \sim p_{\theta^{\text{old}}}(z_i \mid x_i)} \big[\log p_\theta(x_i, z_i)\big].$$

This is the classical EM surrogate.

E-step. Compute the posterior distributions

$$q_i^{\text{old}}(z_i) = p_{\theta^{\text{old}}}(z_i \mid x_i).$$

M-step. Choose a new parameter value θ^{new} satisfying

$$Q(\theta^{\text{new}}, \theta^{\text{old}}) \geq Q(\theta^{\text{old}}, \theta^{\text{old}}).$$

In the exact version of EM, θ^{new} maximizes $Q(\theta, \theta^{\text{old}})$. In generalized EM, it is enough to improve it.

Monotonic improvement of EM

The most classical theoretical fact about EM is that each iteration does not decrease the observed-data likelihood. This does not imply convergence to the global optimum, but it does explain why EM is algorithmically stable and conceptually appealing.

Theorem 5.14 (Monotonic improvement of EM)**.** *Assume the observed-data log-likelihood is finite and that, at iteration t:*

- *the E-step sets*
$$q_i^{(t)}(z_i) = p_{\theta^{(t)}}(z_i \mid x_i),$$
- *the M-step chooses $\theta^{(t+1)}$ such that*
$$Q(\theta^{(t+1)}, \theta^{(t)}) \geq Q(\theta^{(t)}, \theta^{(t)}).$$

Then
$$\ell(\theta^{(t+1)}; x_1, \ldots, x_n) \geq \ell(\theta^{(t)}; x_1, \ldots, x_n).$$

Proof. By the likelihood decomposition,
$$\ell(\theta; x) = \mathcal{F}(q, \theta) + \sum_{i=1}^{n} \mathrm{KL}\big(q_i \,\|\, p_\theta(z_i \mid x_i)\big).$$

At iteration t, choose $q = q^{(t)}$ from the E-step. Because $q_i^{(t)}(z_i) = p_{\theta^{(t)}}(z_i \mid x_i)$, every KL divergence vanishes at $\theta = \theta^{(t)}$. Hence
$$\ell(\theta^{(t)}; x) = \mathcal{F}(q^{(t)}, \theta^{(t)}).$$

Now compare the same bound at the new parameter value $\theta^{(t+1)}$. Since KL divergences are always nonnegative,
$$\ell(\theta^{(t+1)}; x) \geq \mathcal{F}(q^{(t)}, \theta^{(t+1)}).$$

During the M-step, $q^{(t)}$ is fixed, so the entropy part of $\mathcal{F}(q^{(t)}, \theta)$ is constant in θ. Therefore improving $Q(\theta, \theta^{(t)})$ is equivalent to improving $\mathcal{F}(q^{(t)}, \theta)$. The M-step assumption gives
$$\mathcal{F}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \theta^{(t)}).$$

Combining the last three displays yields
$$\ell(\theta^{(t+1)}; x) \geq \mathcal{F}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \theta^{(t)}) = \ell(\theta^{(t)}; x).$$

This proves monotonic improvement. □

The theorem is intentionally modest. It does not say that EM finds the best parameter value, only that the observed-data likelihood cannot decrease under exact E-steps and improving M-steps. In nonconvex models, the algorithm may still converge to local optima or stationary points.

A figure for the EM principle

Figure 5.2 summarizes the logic of EM as alternating hidden-assignment inference and parameter re-estimation.

Observed data $x_1, \ldots, x_n$ and current parameters $\theta^{(t)}$

$\Downarrow$ E-step: compute latent assignments or soft responsibilities

Posterior weights $q_i^{(t)}(z_i) = p_{\theta^{(t)}}(z_i \mid x_i)$

$\Downarrow$ M-step: maximize expected complete-data log-likelihood

Updated parameters $\theta^{(t+1)}$ from weighted latent assignments

$\circlearrowleft$ repeat until the likelihood stops improving

Figure 5.2: The logic of EM. The E-step infers hidden assignments under the current model, and the M-step re-estimates parameters as if those assignments were softly observed.

Gaussian mixtures: the canonical EM example

The Gaussian mixture model (GMM) is the standard worked example because every part of EM can be written explicitly. Assume data $x_i \in \mathbb{R}^d$ are generated from one of K Gaussian components. Introduce a latent variable $z_i \in \{1, \ldots, K\}$ indicating the component identity, with mixture weights $\pi_k \geq 0$ and $\sum_{k=1}^K \pi_k = 1$. The joint model is

$$p_\theta(x_i, z_i = k) = \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k),$$

where

$$\theta = (\pi_1, \ldots, \pi_K, \mu_1, \ldots, \mu_K, \Sigma_1, \ldots, \Sigma_K).$$

Marginalizing over z_i gives

$$p_\theta(x_i) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k).$$

It is often convenient to represent z_i through indicator variables $z_{ik} \in \{0, 1\}$ with $\sum_{k=1}^K z_{ik} = 1$. Then the complete-data log-likelihood for the dataset is

$$\ell_c(\theta; x, z) = \sum_{i=1}^n \sum_{k=1}^K z_{ik} \Big(\log \pi_k + \log \mathcal{N}(x_i \mid \mu_k, \Sigma_k) \Big).$$

Because the indicators would be observed in the complete-data problem, maximizing this expression would simply be weighted Gaussian estimation for each component. EM replaces the unknown indicators by their posterior expectations.

E-step for GMMs. Given current parameters θ^{old}, compute the responsibilities

$$\gamma_{ik} = \mathbb{E}[z_{ik} \mid x_i, \theta^{\text{old}}] = p_{\theta^{\text{old}}}(z_i = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}.$$

These are soft assignment weights. They tell us how strongly the current model believes point x_i came from component k.

M-step for GMMs. Define the effective component counts

$$N_k = \sum_{i=1}^{n} \gamma_{ik}.$$

Maximizing the expected complete-data log-likelihood subject to $\sum_k \pi_k = 1$ gives the updates

$$\pi_k^{\text{new}} = \frac{N_k}{n},$$

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} x_i,$$

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^\top.$$

Thus each parameter update takes exactly the form one would obtain if the latent labels were observed, except that the hard assignments have been replaced by posterior weights. This is the clearest operational meaning of EM.

A worked one-dimensional mixture example

To make the alternation explicit, consider a one-dimensional two-component mixture with fixed variance $\sigma^2 = 1$. Let the data be

$$x_1 = 0, \qquad x_2 = 0.5, \qquad x_3 = 3,$$

and initialize

$$\pi_1^{\text{old}} = \pi_2^{\text{old}} = 0.5, \qquad \mu_1^{\text{old}} = 0, \qquad \mu_2^{\text{old}} = 2.5.$$

With variance fixed, the unknown parameters are only the mixture weights and means.

For a standard normal density, we use the approximate values

$$\mathcal{N}(0 \mid 0, 1) \approx 0.399, \qquad \mathcal{N}(0 \mid 2.5, 1) \approx 0.0175,$$

$$\mathcal{N}(0.5 \mid 0, 1) \approx 0.352, \qquad \mathcal{N}(0.5 \mid 2.5, 1) \approx 0.0540,$$

$$\mathcal{N}(3 \mid 0, 1) \approx 0.00443, \qquad \mathcal{N}(3 \mid 2.5, 1) \approx 0.352.$$

The E-step responsibilities for component 1 are therefore

$$\gamma_{11} = \frac{0.5 \cdot 0.399}{0.5 \cdot 0.399 + 0.5 \cdot 0.0175} \approx 0.958,$$

$$\gamma_{21} = \frac{0.5 \cdot 0.352}{0.5 \cdot 0.352 + 0.5 \cdot 0.0540} \approx 0.867,$$

$$\gamma_{31} = \frac{0.5 \cdot 0.00443}{0.5 \cdot 0.00443 + 0.5 \cdot 0.352} \approx 0.012.$$

Hence the responsibilities for component 2 are

$$\gamma_{12} \approx 0.042, \qquad \gamma_{22} \approx 0.133, \qquad \gamma_{32} \approx 0.988.$$

The effective counts become

$$N_1 \approx 0.958 + 0.867 + 0.012 = 1.837, \qquad N_2 \approx 1.163.$$

The M-step then gives updated weights

$$\pi_1^{\text{new}} \approx \frac{1.837}{3} \approx 0.612, \qquad \pi_2^{\text{new}} \approx 0.388,$$

and updated means

$$\mu_1^{\text{new}} \approx \frac{0.958 \cdot 0 + 0.867 \cdot 0.5 + 0.012 \cdot 3}{1.837} \approx 0.256,$$
$$\mu_2^{\text{new}} \approx \frac{0.042 \cdot 0 + 0.133 \cdot 0.5 + 0.988 \cdot 3}{1.163} \approx 2.605.$$

After one EM iteration, the first component shifts toward the two smaller observations, while the second shifts toward the larger observation. The model has not yet produced hard clusters; instead it has produced *softly weighted evidence* and then re-estimated the parameters accordingly. That is the characteristic behavior of EM in mixture models.

What is established here, and what remains limited

This section establishes several mathematically precise facts. First, it formalizes the gap between complete-data and observed-data likelihood. Second, it shows that Jensen's inequality yields a lower bound whose slack is exactly a KL divergence to the posterior. Third, it proves that exact E-steps together with improving M-steps produce monotonic improvement of the observed-data likelihood. Fourth, it shows concretely in Gaussian mixtures how posterior soft assignments become weighted sufficient statistics.

What remains limited is equally important. The monotonicity theorem does not guarantee the global maximum. It does not guarantee fast convergence. It does not say that the E-step or M-step is always tractable in complicated latent-variable models. Those limitations are precisely what motivate approximate inference and variational methods later in the chapter. EM should therefore be understood both as a successful algorithm and as a conceptual prototype for much of modern latent-variable learning.

What to remember

- Complete-data likelihood treats latent variables as if they were observed; incomplete-data likelihood marginalizes them out.
- The logarithm of a sum is the main obstacle in latent-variable maximum likelihood.
- Jensen's inequality yields a lower bound on the observed-data log-likelihood.
- The E-step makes this bound tight at the current parameter value by choosing the exact posterior.
- The M-step improves the bound by maximizing the expected complete-data log-likelihood.
- In Gaussian mixtures, EM reduces to soft cluster assignment followed by weighted re-estimation of mixture parameters.

Common confusion

- **EM does not integrate out the latent variables once and for all.** It alternates between posterior inference under the current model and parameter update under those posterior weights.
- **Monotonic improvement does not mean global optimality.** The likelihood can still have many local maxima.

- **The lower bound is not arbitrary.** Its tightness is controlled exactly by the KL divergence to the posterior.
- **Responsibilities are not hard labels.** They are posterior probabilities or soft assignments.

Exercises

1. Starting from Bayes' rule, derive the decomposition

$$\ell(\theta; x) = \mathcal{F}(q, \theta) + \mathrm{KL}(q \| p_\theta(z \mid x))$$

   for a single observation.
2. Show that if the latent variables were directly observed, maximizing the complete-data likelihood of a finite Gaussian mixture reduces to weighted estimation of Gaussian parameters for each component.
3. For a two-component one-dimensional Gaussian mixture with known variance, carry out one additional EM iteration after the worked example in this section.
4. Explain carefully why improving $Q(\theta, \theta^{\mathrm{old}})$ is equivalent to improving $\mathcal{F}(q^{\mathrm{old}}, \theta)$ during the M-step.
5. Compare EM with ordinary gradient ascent on the observed-data log-likelihood. What algebraic structure does EM exploit that direct gradient ascent does not?

Notes and references

EM is one of the foundational algorithms of latent-variable modeling and remains conceptually central far beyond mixture models. The basic monotonicity argument and the likelihood-lower-bound view are classical. Gaussian mixtures are the standard pedagogical example because the E-step and M-step can both be written explicitly. The same lower-bound logic later reappears in variational inference, where the exact posterior is replaced by a tractable approximation. For the purposes of this chapter, EM is important not only as an algorithm in its own right but also as the cleanest entry point into the broader principle that hard marginal-likelihood problems can be approached through auxiliary distributions and alternating optimization.

5.3 Variational Autoencoders: Deep Latent-Variable Modeling

Section 5.2 showed that latent-variable learning becomes difficult when the posterior distribution

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}$$

is not easy to compute exactly. Variational autoencoders (VAEs) respond to that difficulty by learning a tractable approximation to the posterior while keeping a probabilistic latent-variable model. They are therefore one of the clearest bridges between classical latent-variable modeling and modern deep learning.

Section role. This section develops the VAE in textbook form. It derives the evidence lower bound (ELBO) as an identity, proves that the ELBO equals the log-likelihood minus a posterior KL term, explains the reparameterization trick carefully, discusses the standard Gaussian encoder parameterization, and works through a one-dimensional VAE example in full. The section should be read as the variational counterpart to the EM section that precedes it.

The latent-variable model and the variational approximation

A VAE begins with a latent-variable generative model

$$p_\theta(x, z) = p_\theta(x \mid z)p(z),$$

where $z \in \mathcal{Z}$ is latent, $p(z)$ is usually a simple prior such as $\mathcal{N}(0, I)$, and $p_\theta(x \mid z)$ is a flexible decoder or likelihood model. The induced data model is

$$p_\theta(x) = \int p_\theta(x, z)\, dz = \int p_\theta(x \mid z)p(z)\, dz.$$

In principle one would like to maximize

$$\log p_\theta(x),$$

but this requires integrating over all latent values. The posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}$$

therefore becomes the central intractable object.

The VAE introduces a tractable approximate posterior

$$q_\phi(z \mid x),$$

parameterized by an encoder network. The decoder parameters θ and encoder parameters ϕ are optimized jointly.

The ELBO as an identity

The key mathematical fact is not merely that the ELBO is a lower bound. More strongly, it is part of an exact decomposition of the log-likelihood. This is the variational identity underlying the whole method.

Theorem 5.15 (ELBO identity)**.** *For any observation x, any model $p_\theta(x, z)$, and any variational distribution $q_\phi(z \mid x)$ with support containing the support of the posterior, define*

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_{q_\phi(z|x)} \left[\log p_\theta(x, z) - \log q_\phi(z \mid x)\right].$$

Then

$$\log p_\theta(x) = \mathcal{L}(x; \theta, \phi) + D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big).$$

Equivalently,

$$\mathcal{L}(x; \theta, \phi) = \log p_\theta(x) - D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big).$$

In particular,

$$\mathcal{L}(x; \theta, \phi) \leq \log p_\theta(x),$$

with equality if and only if $q_\phi(z \mid x) = p_\theta(z \mid x)$ almost everywhere.

Proof. Start from Bayes' rule:

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}.$$

Taking logarithms gives

$$\log p_\theta(x) = \log p_\theta(x, z) - \log p_\theta(z \mid x).$$

Now take expectation with respect to $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \mathbb{E}_{q_\phi}\big[\log p_\theta(x, z) - \log p_\theta(z \mid x)\big].$$

Add and subtract $\mathbb{E}_{q_\phi}[\log q_\phi(z \mid x)]$ to obtain

$$\log p_\theta(x) = \mathbb{E}_{q_\phi}\big[\log p_\theta(x, z) - \log q_\phi(z \mid x)\big] + \mathbb{E}_{q_\phi}\big[\log q_\phi(z \mid x) - \log p_\theta(z \mid x)\big].$$

The first term is $\mathcal{L}(x; \theta, \phi)$ and the second is

$$D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big).$$

Since KL divergence is nonnegative, the lower-bound statement follows immediately. Equality holds exactly when the KL divergence is zero, which occurs precisely when the two distributions coincide almost everywhere. □

This theorem is the precise analogue of the decomposition used in EM. The difference is that EM uses the exact posterior in the E-step whenever possible, whereas the VAE uses a parameterized family $q_\phi(z \mid x)$ that may not contain the exact posterior.

Proposition 5.16 (Standard ELBO decomposition)**.** *If the model factorizes as*

$$p_\theta(x, z) = p_\theta(x \mid z) p(z),$$

then the ELBO can be written as

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big).$$

Proof. Expand the joint term:

$$\log p_\theta(x, z) = \log p_\theta(x \mid z) + \log p(z).$$

Substituting into the definition of $\mathcal{L}$ gives

$$\mathcal{L} = \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi}[\log p(z) - \log q_\phi(z \mid x)].$$

The second expectation is exactly

$$-D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z)).$$

□

The two terms have a clean interpretation. The expected log-likelihood term asks whether samples from the approximate posterior contain enough information to explain the observation. The KL term regularizes the approximate posterior toward the prior, which keeps the latent space organized and makes sampling possible.

A Jensen derivation of the lower bound

The previous theorem gives the cleanest conceptual identity, but it is also useful to see how Jensen's inequality produces the lower bound directly. Starting from

$$\log p_\theta(x) = \log \int q_\phi(z \mid x) \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \, dz,$$

concavity of the logarithm implies

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right] = \mathcal{L}(x; \theta, \phi).$$

This derivation explains the phrase "evidence lower bound." The theorem above then explains the exact slack in the bound: the slack is the KL divergence to the true posterior.

Encoder and decoder as probabilistic maps

A VAE is often summarized by the schematic

$$x \longrightarrow q_\phi(z \mid x) \longrightarrow z \longrightarrow p_\theta(x \mid z).$$

However, the probabilistic meaning matters more than the picture. The encoder does not deterministically compress x into a code. Rather, it outputs a distribution over plausible latent causes of x. The decoder then maps latent samples back into a distribution over observations.

Gaussian encoder parameterization

The standard practical choice is a Gaussian variational family. For a latent dimension m, one writes

$$q_\phi(z \mid x) = \mathcal{N}(z \mid \mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x))),$$

where both

$$\mu_\phi(x) \in \mathbb{R}^m, \qquad \sigma_\phi(x) \in \mathbb{R}^m_{>0}$$

Observation x
$\Downarrow$ encoder network produces parameters of $q_\phi(z \mid x)$
Approximate posterior $q_\phi(z \mid x)$ with mean $\mu_\phi(x)$ and scale $\sigma_\phi(x)$
$\Downarrow$ sample latent variable by reparameterization
Latent point $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$
$\Downarrow$ decoder network defines $p_\theta(x \mid z)$
Decoder likelihood, reconstruction path, and sampling path from $z \sim p(z)$

Figure 5.3: The VAE pipeline. The encoder produces a variational posterior, reparameterization turns posterior sampling into a differentiable transformation, and the decoder maps latent samples to an observation model. At generation time, one samples from the prior and follows only the decoder path.

are produced by the encoder network. In implementation one usually outputs a log-variance or log-standard-deviation vector, for example

$$\alpha_\phi(x) = \log \sigma_\phi^2(x),$$

and then sets

$$\sigma_\phi(x) = \exp\Big(\tfrac{1}{2}\alpha_\phi(x)\Big).$$

This has two advantages. First, the positivity constraint on variances is enforced automatically. Second, the KL term against a standard normal prior has a closed form:

$$D_{\mathrm{KL}}(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \,\|\, \mathcal{N}(0, I)) = \frac{1}{2}\sum_{j=1}^{m}(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

This closed-form term is one of the reasons diagonal-Gaussian VAEs became the standard starting point.

The reparameterization trick

The ELBO contains an expectation with respect to $q_\phi(z \mid x)$. If z is sampled directly from a distribution depending on ϕ, naive differentiation through that sampling step is problematic. The reparameterization trick resolves this by moving the randomness to a parameter-free auxiliary variable.

Proposition 5.17 (Reparameterization for diagonal Gaussians)**.** *Suppose*

$$q_\phi(z \mid x) = \mathcal{N}\big(z \mid \mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x))\big).$$

Let $\varepsilon \sim \mathcal{N}(0, I)$ and define

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon.$$

Then $z \sim q_\phi(z \mid x)$, and for any integrable function f,

$$\mathbb{E}_{z \sim q_\phi(z|x)}[f(z)] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)}\big[f(\mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon)\big].$$

Proof. If $\varepsilon \sim \mathcal{N}(0, I)$, then each component $z_j = \mu_j + \sigma_j \varepsilon_j$ is Gaussian with mean μ_j and variance σ_j^2. Because the covariance of ε is the identity and the transformation is diagonal, the resulting covariance matrix is $\operatorname{diag}(\sigma_1^2, \ldots, \sigma_m^2)$. Hence z has exactly the claimed Gaussian distribution. The expectation identity is then just a change of variables induced by this affine transformation. □

The practical significance is that the Monte Carlo estimator

$$\mathcal{L}(x; \theta, \phi) \approx \frac{1}{L} \sum_{\ell=1}^{L} \log p_\theta\big(x \mid \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon^{(\ell)}\big) - D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

with $\varepsilon^{(\ell)} \sim \mathcal{N}(0, I)$ is differentiable with respect to both θ and ϕ. The noise source does not depend on the network parameters; all parameter dependence sits inside an ordinary differentiable map.

A worked one-dimensional VAE example

To make the algebra concrete, consider a scalar latent variable $z \in \mathbb{R}$ and a scalar observation $x \in \mathbb{R}$. Let the prior be

$$p(z) = \mathcal{N}(0, 1),$$

and let the decoder be

$$p_\theta(x \mid z) = \mathcal{N}(x \mid z, 1).$$

This means the decoder uses z itself as the mean of a unit-variance Gaussian. For one observed value $x = 1.2$, choose the variational encoder

$$q_\phi(z \mid x) = \mathcal{N}(z \mid \mu, \sigma^2)$$

with

$$\mu = 0.8, \qquad \sigma^2 = 0.25.$$

The ELBO is

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_q[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q(z \mid x) \| p(z)).$$

We compute the two terms separately.

For the reconstruction term,

$$\log p_\theta(x \mid z) = -\frac{1}{2} \log(2\pi) - \frac{1}{2}(x - z)^2.$$

Therefore

$$\mathbb{E}_q[\log p_\theta(x \mid z)] = -\frac{1}{2} \log(2\pi) - \frac{1}{2} \mathbb{E}_q[(x - z)^2].$$

Since

$$\mathbb{E}_q[(x - z)^2] = (x - \mu)^2 + \sigma^2,$$

we get

$$\mathbb{E}_q[(1.2 - z)^2] = (1.2 - 0.8)^2 + 0.25 = 0.16 + 0.25 = 0.41.$$

Hence

$$\mathbb{E}_q[\log p_\theta(x \mid z)] = -\frac{1}{2} \log(2\pi) - 0.205.$$

Using $\frac{1}{2} \log(2\pi) \approx 0.9189$, this becomes

$$\mathbb{E}_q[\log p_\theta(x \mid z)] \approx -1.1239.$$

Next compute the KL term from $q = \mathcal{N}(\mu, \sigma^2)$ to $p = \mathcal{N}(0, 1)$:

$$D_{\mathrm{KL}}(q\|p) = \frac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1).$$

Substituting $\mu = 0.8$ and $\sigma^2 = 0.25$ gives

$$D_{\mathrm{KL}}(q\|p) = \frac{1}{2}(0.64 + 0.25 - \log 0.25 - 1).$$

Since $\log 0.25 \approx -1.3863$,

$$D_{\mathrm{KL}}(q\|p) \approx \frac{1}{2}(0.64 + 0.25 + 1.3863 - 1) = \frac{1}{2}(1.2763) \approx 0.6382.$$

Therefore the ELBO is

$$\mathcal{L}(x; \theta, \phi) \approx -1.1239 - 0.6382 = -1.7621.$$

This worked example shows the trade-off directly. A posterior centered close to the observation improves reconstruction, but moving too far from the standard normal prior increases the KL penalty. The VAE objective balances these two pressures.

Posterior collapse and the role of the decoder

The ELBO identity also explains one of the most important pathologies of VAEs. If the decoder is so expressive that it can model x well with little help from z, optimization may drive

$$q_\phi(z \mid x) \approx p(z),$$

in which case the KL term becomes small and the latent variable carries little information about the observation. This is called *posterior collapse*. In that regime the latent channel is underused even though the model may still achieve a reasonable data likelihood.

Posterior collapse is not a contradiction of the ELBO identity. It is a consequence of the fact that the ELBO balances two terms rather than maximizing mutual information between x and z directly. This is why later variants such as β-VAEs, KL annealing, weakened decoders, and richer priors are often introduced.

What is mathematically established, and what remains practical design

The mathematically established parts of the section are the ELBO identity, the lower-bound interpretation, and the reparameterization equality for Gaussian variational families. The diagonal-Gaussian KL formula is also exact. What is not guaranteed by theory alone is that the chosen variational family is expressive enough, that the learned latent variables will be semantically meaningful, or that optimization will avoid posterior collapse. Those features depend strongly on architecture, objective weighting, decoder strength, and data regime. Thus the VAE is both a principled probabilistic model and an area where design choices matter substantially.

What to remember

- A VAE is a latent-variable model with a learned approximate posterior $q_\phi(z \mid x)$.
- The ELBO is not merely a heuristic objective; it is the log-likelihood minus a posterior KL divergence.

- The standard ELBO contains a reconstruction term and a KL regularization term.
- Reparameterization makes Monte Carlo training differentiable by moving randomness to an auxiliary variable.
- Gaussian encoders are standard because they are easy to parameterize and their KL term is closed form.
- Posterior collapse occurs when the latent variable becomes uninformative under the learned solution.

Common confusion

- **The encoder is not a deterministic compression map.** It defines a distribution over latent explanations.
- **The ELBO is not the same as the log-likelihood.** The gap is exactly the KL divergence to the true posterior.
- **Sampling from the prior and encoding a data point are different operations.** Generation uses $z \sim p(z)$; inference uses $q_\phi(z \mid x)$.
- **A small KL term is not always good.** It may indicate regularization, but it may also signal posterior collapse.

Exercises

1. Starting from Bayes' rule, re-derive the ELBO identity for a single observation and identify the term that becomes zero when the variational posterior equals the true posterior.
2. For the one-dimensional worked example, change the variational variance from $\sigma^2 = 0.25$ to $\sigma^2 = 1$ and recompute both ELBO terms. How does the balance between reconstruction and regularization change?
3. Show that for a diagonal-Gaussian encoder and standard normal prior,

$$D_{\mathrm{KL}}(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

4. Explain in words why reparameterization helps gradient-based training but direct sampling from a parameter-dependent distribution does not lead as cleanly to low-variance pathwise gradients.
5. Give a careful argument for why an expressive decoder can encourage posterior collapse. Which term in the ELBO makes this possible?
6. Compare EM and VAEs as two responses to latent-variable learning. What does EM do exactly that the VAE replaces by amortized approximation?

Notes and references

VAEs are one of the central modern latent-variable models because they combine probabilistic generative modeling with neural function approximation and scalable optimization. The ELBO identity belongs to the larger framework of variational inference; the distinctive contribution of the VAE is the use of an amortized encoder and the pathwise reparameterization idea that makes end-to-end training practical. The standard diagonal-Gaussian setup is pedagogically useful because the KL term is explicit, but many later variants enrich the prior, posterior family, or decoder architecture. For the logic of this chapter, the most important conceptual point is that VAEs

preserve the latent-variable viewpoint of EM while replacing exact posterior computation by learned approximate inference in a flexible deep model.

5.4 Adversarial Generative Modeling

Sections 5.1–5.3 developed generative modeling through explicit densities, latent-variable models, and variational inference. Adversarial methods take a different route. They define a sampler

$$x = G_\theta(z), \qquad z \sim p(z),$$

and train it not by maximizing a tractable likelihood, but by forcing its samples to become hard to distinguish from real data. This makes adversarial modeling conceptually important: it shows that one can pursue distribution matching even when likelihood evaluation is unavailable or inconvenient.

Section role. This section develops the classical GAN objective in textbook form. It defines the minimax game, proves the formula for the optimal discriminator, derives the generator's induced divergence objective, explains the Jensen–Shannon interpretation, and then separates two important extensions: conditional GANs and the Wasserstein viewpoint. It closes with an explicit account of instability mechanisms, because the elegance of the theory should not obscure the practical difficulty of the method.

The classical minimax objective

Let p_{data} denote the data distribution on $\mathcal{X}$. Let $p(z)$ be a simple latent prior, typically a standard Gaussian or a uniform distribution, and let $G_\theta : \mathcal{Z} \to \mathcal{X}$ be a generator network. The generator induces a model distribution p_G on $\mathcal{X}$ through the sampling rule

$$z \sim p(z), \qquad x = G_\theta(z).$$

A discriminator $D_\phi : \mathcal{X} \to (0,1)$ is then trained to output the probability that an input came from real data rather than the generator.

The classical GAN objective is

$$\min_\theta \max_\phi V(D_\phi, G_\theta) = \min_\theta \max_\phi \left[\mathbb{E}_{x\sim p_{\text{data}}}[\log D_\phi(x)] + \mathbb{E}_{x\sim p_G}[\log(1 - D_\phi(x))]\right].$$

Equivalently, writing the generator expectation through the latent prior,

$$V(D_\phi, G_\theta) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D_\phi(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D_\phi(G_\theta(z)))].$$

The discriminator tries to maximize classification accuracy between real and generated samples. The generator tries to minimize the same criterion, thereby pushing generated samples toward the real data manifold. This is a two-player zero-sum game rather than an ordinary loss minimization problem.

Optimal discriminator for a fixed generator

For fixed generator distribution p_G, the discriminator optimization can be solved pointwise. This is the first key theoretical fact.

Proposition 5.18 (Optimal discriminator)**.** *Fix a generator distribution p_G. For each x such that $p_{\text{data}}(x) + p_G(x) > 0$, the value function is maximized by*

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

At points where both densities vanish, the value is irrelevant to the objective.

Real data $x \sim p_{\text{data}}$ and latent noise $z \sim p(z)$ feed two branches.

$\Downarrow$

Generator branch	Data branch
$z \mapsto G_\theta(z)$ produces fake sample $\tilde{x}$	real sample $x \sim p_{\text{data}}$

$\Downarrow$ both are shown to the discriminator D_ϕ

Discriminator outputs $D_\phi(x)$ or $D_\phi(\tilde{x})$ and sends gradients back. The discriminator maximizes real/fake discrimination; the generator updates to make $\tilde{x}$ look real.

Figure 5.4: Generator–discriminator interaction in a classical GAN. The discriminator receives both real and generated samples, while the generator is updated only through the discriminator's feedback.

Proof. For fixed p_G, the objective can be written as

$$V(D, G) = \int [p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))]\, dx.$$

The integrand depends on $D(x)$ pointwise. For a fixed x, define

$$f_x(d) = a \log d + b \log(1 - d),$$

where $a = p_{\text{data}}(x)$ and $b = p_G(x)$. Then

$$f_x'(d) = \frac{a}{d} - \frac{b}{1 - d}.$$

Setting the derivative to zero gives

$$\frac{a}{d} = \frac{b}{1 - d} \quad \Longrightarrow \quad d = \frac{a}{a + b}.$$

Since

$$f_x''(d) = -\frac{a}{d^2} - \frac{b}{(1 - d)^2} < 0,$$

this critical point is the maximizer. Substituting back yields the stated formula. □

This proposition makes the discriminator statistically interpretable. At optimum it returns the posterior probability that a point came from the data rather than the generator under a balanced mixture model. So the discriminator is not arbitrary: it estimates relative density evidence.

The generator's induced divergence objective

Once the optimal discriminator is substituted back into the game, the minimax objective becomes a pure distribution-comparison criterion.

Proposition 5.19 (Value at the optimal discriminator)**.** *For fixed generator distribution p_G and optimal discriminator D^*,*

$$V(D^*, G) = \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}\, dx + \int p_G(x) \log \frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)}\, dx.$$

Equivalently, if

$$m(x) = \frac{1}{2}(p_{\text{data}}(x) + p_G(x)),$$

then

$$V(D^*, G) = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_G),$$

where JS *denotes the Jensen–Shannon divergence.*

Proof sketch. Insert the formula for D^* into the value function:

$$V(D^*, G) = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{p_{\text{data}} + p_G}\right] + \mathbb{E}_{p_G}\left[\log \frac{p_G}{p_{\text{data}} + p_G}\right].$$

Now use

$$p_{\text{data}} + p_G = 2m$$

and split the logarithms:

$$\log \frac{p_{\text{data}}}{p_{\text{data}} + p_G} = \log \frac{p_{\text{data}}}{m} - \log 2,$$

with the analogous identity for p_G. Therefore

$$V(D^*, G) = -2\log 2 + \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{m}\right] + \mathbb{E}_{p_G}\left[\log \frac{p_G}{m}\right].$$

The last two terms are precisely

$$\mathrm{KL}(p_{\text{data}} \,\|\, m) + \mathrm{KL}(p_G \,\|\, m) = 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_G).$$

Since $-2\log 2 = -\log 4$, the identity follows. □

This is the classical divergence interpretation of GANs. Under idealized assumptions—infinite discriminator capacity, exact optimization, and well-defined densities—training the generator is equivalent to minimizing a Jensen–Shannon divergence between the generated distribution and the data distribution.

Corollary 5.20. *The global minimum of the ideal GAN objective is achieved when $p_G = p_{\text{data}}$. At that point,*

$$\mathrm{JS}(p_{\text{data}} \,\|\, p_G) = 0 \quad \textit{and} \quad V(D^*, G) = -\log 4.$$

Moreover, the optimal discriminator becomes the constant function $D^*(x) = \frac{1}{2}$.

Proof. The Jensen–Shannon divergence is nonnegative and vanishes exactly when its two arguments agree almost everywhere. Therefore the smallest achievable value of $V(D^*, G)$ is $-\log 4$, attained when $p_G = p_{\text{data}}$. Substituting this equality into the formula for D^* gives $D^*(x) = \frac{1}{2}$. □

What is elegant in theory, and what is limited in practice

The previous derivation is mathematically clean, but it rests on idealizations. In practical deep learning, neither player is optimized exactly, both are represented by finite networks, and gradients are computed from mini-batches rather than population expectations. So the Jensen–Shannon theorem is best read as a guiding idealization, not as a full theory of training dynamics.

There is also a subtle geometric issue. If the supports of p_{data} and p_G lie on thin and nearly disjoint manifolds, then a strong discriminator can separate them almost perfectly. In that regime, the objective may saturate and the generator can receive weak or unstable gradients. Thus the divergence interpretation is correct at the level of the classical objective, but it does not by itself guarantee smooth optimization.

A worked one-dimensional example

Consider the toy setting in which

$$p_{\text{data}} = \tfrac{1}{2}\delta_{-1} + \tfrac{1}{2}\delta_1, \qquad p_G = \delta_\theta,$$

so the true data has two modes at -1 and 1, while the generator currently puts all mass at a single point θ. If $\theta = 1$, then the generator matches only one of the two modes. The optimal discriminator places high confidence on the unmatched real point -1 and reduced confidence on the shared point 1. The generator therefore receives a signal that it is missing part of the data distribution.

This toy example illustrates two lessons. First, the GAN objective compares whole distributions rather than individual reconstructions. Second, if the generator can improve its objective by concentrating on a few high-payoff regions, it may collapse onto only part of the target distribution. This is the beginning of mode collapse.

Instability mechanisms

The promise of GANs is sample quality; the cost is training instability. Several distinct mechanisms contribute.

1. Saturating gradients. If the discriminator becomes nearly perfect, then $D_\phi(G_\theta(z)) \approx 0$ for generated samples. In the original minimax form, the generator update uses

$$\nabla_\theta \log(1 - D_\phi(G_\theta(z))),$$

which may become very small or poorly conditioned. A common practical modification is the non-saturating generator loss

$$\min_\theta \; -\mathbb{E}_{z\sim p(z)}[\log D_\phi(G_\theta(z))],$$

which has the same fixed point but usually stronger gradients early in training.

2. Mode collapse. The generator may map many latent values to similar outputs. This can locally fool the discriminator while ignoring substantial parts of the data distribution. Mode collapse is not fully explained by the ideal Jensen–Shannon derivation; it is a property of the finite, alternating optimization process.

3. Non-stationary targets. In supervised learning, the target is fixed. In GANs, each player's target changes as the other player updates. So optimization is inherently non-stationary. The generator does not descend a fixed loss surface; it responds to a moving critic.

4. Game dynamics rather than plain minimization. Even simple minimax systems can rotate or cycle rather than converge directly. This phenomenon is already visible in bilinear games such as

$$\min_x \max_y \; xy.$$

The point is not that GANs reduce to such a toy game, but that adversarial optimization has dynamical features absent from ordinary gradient descent.

5. Finite-capacity discriminators. The clean theorem assumes the discriminator reaches its population optimum for a fixed generator. In practice, the discriminator is only partially optimized and represented by a finite network. Thus the generator is updated with respect to an approximate and evolving divergence surrogate.

Conditional adversarial modeling

Many applications require generation conditional on side information c, such as a class label, text description, segmentation map, or another image. A conditional GAN therefore uses

$$G_\theta(z, c), \qquad D_\phi(x, c).$$

The classical conditional objective is

$$\min_\theta \max_\phi \mathbb{E}_{(x,c)\sim p_{\text{data}}}[\log D_\phi(x, c)] + \mathbb{E}_{z\sim p(z),\, c\sim p(c)}[\log(1 - D_\phi(G_\theta(z, c), c))].$$

More carefully, one can think of the generator as trying to match the conditional data distribution $p_{\text{data}}(x \mid c)$ for each context c. So conditional adversarial learning is not merely adding labels; it changes the modeling target from one unconditional distribution to a family of conditional distributions.

This is why conditional GANs became important in image-to-image translation and controllable generation. They separate two questions: what should be generated, and under what condition should it be generated?

The Wasserstein viewpoint

A major critique of the classical GAN objective is that Jensen–Shannon geometry can become poorly behaved when the model and data distributions are far apart or lie on low-dimensional sets. The Wasserstein GAN (WGAN) addresses this by replacing the discriminator with a critic whose objective approximates an optimal-transport distance. At a high level, one writes

$$\max_{f\in\mathcal{F}_{\text{Lip}}} \left[\mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_G}[f(x)]\right],$$

where $\mathcal{F}_{\text{Lip}}$ denotes a class of 1-Lipschitz functions. The generator then tries to minimize this quantity.

The significance of this reformulation is not merely technical. It changes the geometry of distribution comparison. The Wasserstein distance can vary continuously even when two distributions have disjoint supports, so the generator often receives more informative gradients. This does not remove all optimization difficulty, but it explains why Wasserstein ideas became influential in adversarial learning.

Modeling viewpoint: explicit, implicit, and adversarial

GANs should be understood as a special case of implicit generative modeling. They provide a learned sampler

$$z \sim p(z), \qquad x = G_\theta(z),$$

without necessarily providing a tractable formula for $p_G(x)$. Their strength is therefore sample generation rather than likelihood evaluation. Compared with VAEs, GANs often produce sharper perceptual samples but offer less direct probabilistic structure. Compared with explicit density models, they may avoid awkward likelihood assumptions but inherit an adversarial optimization problem.

What to remember

- A GAN trains a generator and discriminator through a minimax game.
- For a fixed generator, the optimal discriminator is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

- Under ideal assumptions, substituting D^* turns the generator objective into Jensen–Shannon divergence minimization.
- Conditional GANs target conditional distributions, while Wasserstein GANs change the geometry of distribution comparison.
- Practical GAN training is unstable because of saturating gradients, mode collapse, non-stationarity, and game dynamics.

Common confusion

- GANs do not generally provide a tractable likelihood, even though they define a model distribution implicitly.
- The Jensen–Shannon theorem is an idealized population statement; it is not a guarantee of easy optimization.
- Mode collapse is not the same as overfitting, though both can reduce diversity.
- Conditional GANs are not a different family of models in principle; they are adversarial models for conditional distributions.

Exercises

Exercise 5.21. Derive the optimal discriminator formula by maximizing the integrand pointwise for fixed p_{data} and p_G.

Exercise 5.22. Show that if $p_G = p_{\text{data}}$, then $D^*(x) = \frac{1}{2}$ and the optimal value is $-\log 4$.

Exercise 5.23. Why can the original minimax generator loss produce weak gradients when the discriminator is very strong? Explain without computation and then verify by differentiating the loss formally.

Exercise 5.24. Compare a VAE and a GAN in terms of: (i) tractable likelihood, (ii) sampling path, (iii) training stability, and (iv) latent-space interpretation.

Exercise 5.25. In a conditional GAN, explain carefully what distribution is being matched when the condition variable c is fixed.

Notes and references

The original GAN framework was introduced by Goodfellow et al. The optimal-discriminator and Jensen–Shannon derivation come from that classical paper and remain the standard theoretical entry point. Conditional GAN ideas were developed rapidly in the years that followed and became important for controllable generation and image-to-image translation. The Wasserstein viewpoint was introduced to address shortcomings of the Jensen–Shannon geometry in practical optimization. For a modern textbook treatment, GANs are best read alongside latent-variable and diffusion models, because their strengths and weaknesses become clearer in comparison.

5.5 Exact Density Modeling Through Invertible Maps

A third major route to generative modeling is based on exact transformation of probability distributions. This leads to *normalizing flows*: models that transport a simple base distribution into a more complex data distribution through an invertible map.

Compared with adversarial models, flows retain exact density evaluation. Compared with latent-variable models such as VAEs, they avoid variational approximation in the likelihood itself. Their main burden is architectural: the map must be expressive, invertible, and accompanied by a tractable Jacobian determinant.

Invertibility as a modeling strategy

Let $z \in \mathbb{R}^d$ be drawn from a simple base density p_Z, for example a standard Gaussian, and let

$$x = f_\theta(z),$$

where $f_\theta : \mathbb{R}^d \to \mathbb{R}^d$ is bijective and differentiable with differentiable inverse. Then sampling is straightforward:

$$z \sim p_Z, \qquad x = f_\theta(z).$$

Because f_θ is invertible, one may also map a data point x back to latent coordinates

$$z = f_\theta^{-1}(x),$$

and evaluate its density exactly by accounting for local volume distortion. This is the central mathematical idea behind flows.

The finite-dimensional change-of-variables theorem

Theorem 5.26 (Change of variables in finite dimensions)**.** *Let $f : U \to V$ be a C^1 bijection between open sets $U, V \subset \mathbb{R}^d$, and assume that the Jacobian matrix $J_f(z)$ is nonsingular for all $z \in U$. If Z has density p_Z on U and $X = f(Z)$, then X has density*

$$p_X(x) = p_Z(f^{-1}(x)) \left|\det J_{f^{-1}}(x)\right|, \qquad x \in V.$$

Equivalently, writing $x = f(z)$,

$$p_X(f(z)) = p_Z(z) \left|\det J_f(z)\right|^{-1}.$$

Proof sketch. For small regions in latent space, a differentiable map behaves approximately like its Jacobian matrix. Thus a small volume element dz near z is mapped to a volume element near $x = f(z)$ whose size is approximately

$$\left|\det J_f(z)\right| dz.$$

Probability mass is preserved under deterministic transformation, so

$$p_Z(z)\, dz \approx p_X(x)\, dx = p_X(f(z)) \left|\det J_f(z)\right| dz.$$

Dividing by dz yields

$$p_X(f(z)) = p_Z(z) \left|\det J_f(z)\right|^{-1}.$$

Replacing z by $f^{-1}(x)$ gives the first formula. □

Interpretation. The determinant measures how much the map expands or contracts local volume. If $|\det J_f(z)| > 1$, the transformation spreads probability mass over a larger region and the resulting density decreases. If $|\det J_f(z)| < 1$, probability mass is concentrated and the density increases. Thus exact density modeling is possible whenever the transport map and its local volume change are tractable.

Likelihood under an invertible model

Given data $x \in \mathbb{R}^d$ and an invertible model $x = f_\theta(z)$ with base density p_Z, the log-likelihood is

$$\log p_\theta(x) = \log p_Z(f_\theta^{-1}(x)) + \log \left|\det J_{f_\theta^{-1}}(x)\right|.$$

Equivalently, if $z = f_\theta^{-1}(x)$, then

$$\log p_\theta(x) = \log p_Z(z) - \log |\det J_{f_\theta}(z)|.$$

This is one reason flows are mathematically elegant:

- sampling is exact,
- density evaluation is exact,
- training can proceed by direct maximum likelihood.

Composition of invertible layers

A practical flow is usually a composition of simpler invertible maps,

$$f_\theta = f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

If we define intermediate states

$$h_0 = z, \qquad h_\ell = f_\ell(h_{\ell-1}), \qquad h_L = x,$$

then the chain rule gives the total Jacobian

$$J_{f_\theta}(z) = J_{f_L}(h_{L-1}) \cdots J_{f_1}(h_0).$$

Proposition 5.27 (Composition formula for log-determinants)**.** *For the composition above,*

$$\log |\det J_{f_\theta}(z)| = \sum_{\ell=1}^{L} \log |\det J_{f_\ell}(h_{\ell-1})|.$$

Proof. The determinant of a product of square matrices equals the product of determinants:

$$\det(AB) = \det(A)\det(B).$$

Applying this repeatedly to the chain-rule factorization and then taking logarithms gives the result. □

Why this matters. The overall transformation may be highly nonlinear, yet exact density evaluation reduces to summing layerwise contributions. Thus the engineering problem becomes: design invertible layers for which the inverse and the Jacobian determinant are both cheap.

Coupling layers and tractable Jacobians

A common flow layer splits the variable into two blocks,

$$z = (z_a, z_b),$$

and transforms only one block at a time using the other as context. An affine coupling layer has the form

$$x_a = z_a, \qquad x_b = z_b \odot \exp\big(s(z_a)\big) + t(z_a),$$

where $s(\cdot)$ and $t(\cdot)$ are arbitrary functions and $\odot$ denotes elementwise multiplication.

Proposition 5.28 (Why coupling layers are tractable)**.** *The Jacobian matrix of an affine coupling layer is block triangular. In particular,*

$$J = \begin{pmatrix} I & 0 \\ * & \operatorname{diag}(\exp(s(z_a))) \end{pmatrix},$$

so that

$$\det J = \exp\left(\sum_i s_i(z_a)\right).$$

Moreover, the inverse map is explicit:

$$z_a = x_a, \qquad z_b = \big(x_b - t(x_a)\big) \odot \exp\big(-s(x_a)\big).$$

Proof. Because $x_a = z_a$, the derivative of x_a with respect to z_b is zero. Thus the Jacobian is block lower triangular. The diagonal block corresponding to $z_b \mapsto x_b$ is simply

$$\operatorname{diag}(\exp(s(z_a))),$$

whose determinant is the product of its diagonal entries. The inverse formula follows by solving the affine transformation for z_b. □

Interpretation. Coupling layers are powerful because the functions s and t may be expressive neural networks, yet the determinant remains easy to compute. This is the basic architectural trick that makes exact density modeling feasible in high dimensions.

A worked two-dimensional example

Consider a two-dimensional standard Gaussian base variable

$$Z = (Z_1, Z_2) \sim \mathcal{N}(0, I_2),$$

and define the coupling transformation

$$X_1 = Z_1, \qquad X_2 = e^{aZ_1} Z_2 + bZ_1,$$

where $a, b \in \mathbb{R}$ are constants. This is the special case

$$s(z_1) = az_1, \qquad t(z_1) = bz_1.$$

The inverse map is

$$Z_1 = X_1, \qquad Z_2 = e^{-aX_1}(X_2 - bX_1).$$

The Jacobian matrix of the forward map is

$$J_f(z_1, z_2) = \begin{pmatrix} 1 & 0 \\ aze^{az_1} z_2 + b & e^{az_1} \end{pmatrix},$$

so

$$\det J_f(z_1, z_2) = e^{az_1}.$$

Hence the model density is

$$p_X(x_1, x_2) = \frac{1}{2\pi} \exp\left(-\frac{1}{2} \left[x_1^2 + e^{-2ax_1} (x_2 - bx_1)^2 \right] \right) \cdot e^{-ax_1}.$$

What the formula shows. Even though the base density is Gaussian, the transformed density need not be Gaussian. The first coordinate controls both a translation and a scale change in the second coordinate. Thus the map can bend, shear, and stretch the density while keeping exact likelihood available.

For example, if $a = 0.5$ and $b = 1$, then at the point $(x_1, x_2) = (1, 2)$ we have

$$z_1 = 1, \qquad z_2 = e^{-0.5}(2 - 1) = e^{-0.5},$$

and therefore

$$\log p_X(1, 2) = -\log(2\pi) - \frac{1}{2}\left(1 + e^{-1}\right) - 0.5.$$

This gives a completely explicit likelihood calculation for a nontrivial nonlinear transport.

A geometric view: density warping

Flows are often easiest to understand geometrically. One begins with a simple cloud of probability mass in latent space and applies a sequence of invertible warpings. Each layer preserves total probability but bends and stretches regions differently. This can create complex curved level sets in data space while retaining exact density evaluation.

Density warping under invertible transport

Base space z	$\xrightarrow{f_1}$	Intermediate space h
concentric level sets		sheared / stretched level sets

$\Downarrow$ composition of further invertible layers

Data space x: curved, anisotropic, non-Gaussian density contours

Figure 5.5: A normalizing flow warps a simple base density into a complex target density through a sequence of invertible transformations. The important point is not only that samples are transported, but that local volume change remains exactly trackable.

What is elegant, and what is restrictive

Flows appeal strongly to mathematically inclined readers because they provide an unusually clean synthesis of geometry and likelihood. Still, this elegance comes with constraints.

Strengths.

- exact likelihood evaluation,
- exact ancestral sampling,
- transparent relation between architecture and density geometry,
- direct maximum-likelihood training.

Limitations.

- invertibility constrains layer design,
- input and latent dimensions must match unless one augments the architecture,
- tractable Jacobians may restrict expressivity or efficiency,
- excellent likelihood does not always coincide with best perceptual sample quality.

Thus flows occupy a distinctive position in generative modeling: they are among the most exact and mathematically transparent models, but exactness imposes architectural discipline.

What to remember

- A normalizing flow models data by transporting a simple base density through an invertible map.
- The change-of-variables theorem gives exact density evaluation.
- Compositions turn global complexity into a sum of layerwise log-determinants.
- Coupling layers work because their Jacobians are triangular.

Common confusion

- Invertibility does not mean linearity; highly nonlinear flows are possible.
- Exact likelihood does not automatically imply the best visual samples.
- The determinant term is not an arbitrary correction; it is the precise account of local volume change.

Exercises

1. Verify directly that the inverse formula for the affine coupling layer is correct.
2. Let $x = Az + b$ for an invertible matrix A. Derive the exact density of x when $z \sim \mathcal{N}(0, I)$.
3. For the two-dimensional example in this section, compute $p_X(x_1, x_2)$ when $a = 0$ and explain why the model reduces to a simple shear transformation.
4. Show that if each layer in a flow is volume preserving, then the log-determinant term vanishes and likelihood depends only on the base density evaluated at the inverse image.

Notes and references

The flow viewpoint goes back to the idea of exact density modeling through invertible transformations. Important modern developments include NICE, RealNVP, Glow, and related architectures based on coupling layers, invertible 1×1 convolutions, and other tractable Jacobian constructions. Conceptually, flows also connect to transport maps and, at a broader level, to geometric perspectives on probability distributions.

5.6 Generation by Denoising and Iterative Refinement

The previous sections presented three major paradigms for generative modeling: latent-variable decoding, adversarial generation, and exact transport by invertible maps. Diffusion introduces a fourth viewpoint. Instead of generating a sample in one shot, or evaluating density by a single formula, it models generation as the gradual reversal of corruption.

Guiding idea. A complicated global distribution may be easier to learn through many small local denoising steps than through one direct global map.

This section is intentionally slower and more conceptual than the next one. Its goal is to make precise what is being corrupted, what is being learned, and why repeated denoising can become a generative mechanism.

From clean data to noisy data

Suppose $x_0 \in \mathbb{R}^d$ denotes a data point drawn from the unknown data distribution. A denoising-based generative method begins by defining a family of corrupted versions

$$x_1, x_2, \ldots, x_T,$$

of x_0, where larger t corresponds to more corruption. The variable x_T should be so heavily corrupted that it is close to a simple reference distribution such as an isotropic Gaussian.

Definition 5.29 (Forward corruption process)**.** A forward corruption process is a conditional distribution

$$q(x_{1:T} \mid x_0)$$

that maps clean data x_0 into progressively noisier variables $x_1, \ldots, x_T$. In diffusion models this process is usually chosen by design rather than learned from data.

The choice to fix the forward process is important. It means that the difficult modeling problem is shifted away from "how should we corrupt data?" and toward "how should we reverse that corruption?"

A clean probabilistic formulation of gradual corruption

A convenient formulation is a Markov chain,

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$

where each step adds only a small amount of noise. A standard Gaussian choice is

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t \mid \sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\Big),$$

with $0 < \beta_t < 1$ typically small.

This formula has a direct interpretation:

- the factor $\sqrt{1-\beta_t}$ slightly shrinks the previous signal,
- the term $\beta_t I$ injects fresh Gaussian noise,
- repeated application gradually destroys fine structure.

For a single corruption level, one may also study a simpler conditional model of the form

$$q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x} \mid x, \sigma^2 I),$$

where $\tilde{x} = x + \sigma\varepsilon$ and $\varepsilon \sim \mathcal{N}(0, I)$. This "single-step corruption" viewpoint is historically useful because it connects diffusion to denoising autoencoders.

Denoising as learning reverse conditional structure

If corruption is probabilistic, then denoising is naturally a conditional inference problem. Given a noisy observation x_t, one would like to understand the conditional distribution of the less noisy variable that preceded it. This suggests learning reverse conditionals of the form

$$p_\theta(x_{t-1} \mid x_t).$$

Definition 5.30 (Reverse denoising model)**.** A reverse denoising model is a parameterized family of conditional distributions

$$p_\theta(x_{t-1} \mid x_t), \qquad t = 1, \ldots, T,$$

intended to approximate the true reverse conditionals induced by the forward process.

If these reverse conditionals were exact, then one could generate by starting from a simple sample x_T and successively drawing

$$x_{T-1} \sim p_\theta(x_{T-1} \mid x_T), \quad x_{T-2} \sim p_\theta(x_{T-2} \mid x_{T-1}), \quad \ldots, \quad x_0 \sim p_\theta(x_0 \mid x_1).$$

Thus generation becomes iterative refinement: noise is converted into structure one small step at a time.

What is being learned? The reverse model does not need to memorize entire clean samples. Rather, at each stage it must capture the local conditional geometry of the data distribution: which slightly cleaner configurations are plausible given the current noisy one.

From denoising autoencoders to iterative generation

Before diffusion models were formulated as full latent Markov chains, denoising autoencoders already used corruption as a learning device. A denoising autoencoder samples a corrupted input

$$\tilde{x} \sim q_\sigma(\tilde{x} \mid x),$$

and trains a network $r_\theta(\tilde{x})$ to reconstruct the clean target x by minimizing a loss such as

$$\mathcal{L}_{\mathrm{DAE}}(\theta) = \mathbb{E}_{x \sim p_{\mathrm{data}},\, \tilde{x} \sim q_\sigma(\tilde{x}|x)} \big[\|x - r_\theta(\tilde{x})\|^2 \big].$$

This is already a meaningful representation-learning problem: the model must learn which directions in input space correspond to signal and which correspond to nuisance perturbation. However, by itself, a denoising autoencoder is not yet a complete generative model in the same direct sense as a latent-variable sampler or a flow. It learns local restoration, but not necessarily a full multi-step sampling scheme.

Diffusion extends this idea by:

- defining a whole family of corruption levels rather than one fixed noise level,

- learning denoising rules indexed by noise scale or time step,
- chaining these rules into a generative process from nearly pure noise back to data.

This is the conceptual bridge:

> A denoising autoencoder learns to undo corruption. A diffusion model organizes many such denoising tasks into a stochastic path from noise to data.

A one-step Gaussian denoising calculation

The basic denoising idea can be seen in the simplest Gaussian setting. Assume the clean variable is one-dimensional with prior

$$X \sim \mathcal{N}(0,1),$$

and the observed noisy variable is

$$Y = X + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\sigma^2),$$

with ε independent of X. Then the conditional distribution $p(x \mid y)$ is also Gaussian. Its mean is the Bayes-optimal denoiser under squared loss.

Proposition 5.31 (Posterior mean for Gaussian denoising)**.** *In the model above,*

$$X \mid Y = y \sim \mathcal{N}\left(\frac{1}{1+\sigma^2}y,\ \frac{\sigma^2}{1+\sigma^2}\right).$$

In particular, the minimum mean-squared error estimator is

$$\mathbb{E}[X \mid Y = y] = \frac{1}{1+\sigma^2}y.$$

Proof sketch. Because (X,Y) is jointly Gaussian, the conditional distribution is Gaussian. Using

$$\mathrm{Var}(X) = 1, \qquad \mathrm{Var}(Y) = 1+\sigma^2, \qquad \mathrm{Cov}(X,Y) = 1,$$

the standard conditional Gaussian formula gives

$$\mathbb{E}[X \mid Y = y] = \frac{\mathrm{Cov}(X,Y)}{\mathrm{Var}(Y)}y = \frac{1}{1+\sigma^2}y,$$

and

$$\mathrm{Var}(X \mid Y) = 1 - \frac{\mathrm{Cov}(X,Y)^2}{\mathrm{Var}(Y)} = \frac{\sigma^2}{1+\sigma^2}.$$

□

Interpretation. The optimal denoiser does not simply output y. It shrinks y toward the mean 0, because the model knows that part of the observation is noise. As $\sigma^2 \to 0$, the estimator approaches y. As $\sigma^2 \to \infty$, the observation becomes uninformative and the estimator shrinks strongly toward 0.

This toy example is small, but it captures the essential probabilistic logic of denoising: learning a reverse conditional distribution means learning how clean structure is statistically related to noisy structure.

A worked iterative toy example

Consider now a three-stage refinement picture in one dimension. Suppose $x_0 \sim \mathcal{N}(0,1)$, and define two corrupted versions

$$x_1 = x_0 + \varepsilon_1, \qquad x_2 = x_1 + \varepsilon_2,$$

with independent noises $\varepsilon_1 \sim \mathcal{N}(0, 0.25)$ and $\varepsilon_2 \sim \mathcal{N}(0,1)$. Then

$$x_2 = x_0 + (\varepsilon_1 + \varepsilon_2),$$

so the total noise variance from x_0 to x_2 is 1.25. The optimal one-step estimator of x_0 from x_2 is therefore

$$\mathbb{E}[x_0 \mid x_2] = \frac{1}{1+1.25} x_2 = \frac{4}{9} x_2.$$

Likewise, since $x_2 = x_1 + \varepsilon_2$ and $x_1 \sim \mathcal{N}(0, 1.25)$,

$$\mathbb{E}[x_1 \mid x_2] = \frac{1.25}{1.25+1} x_2 = \frac{5}{9} x_2.$$

Then from x_1, the optimal denoiser back to x_0 is

$$\mathbb{E}[x_0 \mid x_1] = \frac{1}{1+0.25} x_1 = 0.8 x_1.$$

If, for instance, we observe $x_2 = 1.8$, then the best intermediate denoised estimate is

$$\hat{x}_1 = \frac{5}{9} \cdot 1.8 = 1.0,$$

and the next denoised estimate is

$$\hat{x}_0 = 0.8 \cdot 1.0 = 0.8.$$

Direct denoising gives

$$\mathbb{E}[x_0 \mid x_2] = \frac{4}{9} \cdot 1.8 = 0.8$$

as expected.

Why keep the multi-step view? In this simple Gaussian example, direct denoising and iterative denoising agree because everything is linear and exactly tractable. But in complex high-dimensional data, the multi-step decomposition is useful because each local denoising task may be much simpler than the full jump from noise to data. This is the practical intuition behind diffusion.

Noise prediction versus clean-sample prediction

There are several equivalent-looking ways to parameterize a denoiser. One may try to predict:

- the clean sample x_0 from a noisy input,
- the previous state x_{t-1} from x_t,
- or the actual noise that was added.

These choices are related, but they are not merely cosmetic. Different parameterizations lead to different optimization behavior and different interpretations of what the network is learning. In modern diffusion practice, predicting the added noise is particularly common because it interacts conveniently with the Gaussian forward process and leads to simple training objectives. The next section makes this algebra explicit.

Progressive corruption and iterative recovery

clean data	$\longrightarrow$	lightly noisy	$\longrightarrow$	heavily noisy
x_0		x_1		$x_T \approx \mathcal{N}(0, I)$
$\mathcal{N}(0, I)$	$\longrightarrow$	coarse structure	$\longrightarrow$	sample
x_T		$x_{T-1}, x_{T-2}, \ldots$		x_0

Forward process: fixed stochastic corruption.
Reverse process: learned conditional denoising steps.

Figure 5.6: Diffusion-style generation as reversal of a progressive corruption process.

Progressive corruption and recovery

The forward and reverse arrows should not be confused. The forward process is usually chosen analytically and known exactly. The reverse process is what must be learned from data.

What is already understood, and what remains heuristic

It is useful to separate three levels of claim.

Probabilistically well defined. Once a forward corruption process is specified, the induced reverse conditionals are mathematically meaningful objects. The question "what is the true distribution of a slightly cleaner sample given a noisier one?" is a legitimate conditional inference question.

Conceptually persuasive. The idea that complex generation can be decomposed into many local denoising problems is strongly supported by modern practice. It explains why diffusion can be more stable than adversarial training and why denoising losses can be easier to optimize.

Still design practice. Specific noise schedules, parameterizations, samplers, and acceleration tricks often come from empirical design rather than from a single complete theory that tells us the unique correct choice. Thus diffusion is mathematically structured, but not fully determined by theory alone.

What to remember

- Diffusion starts from a forward corruption process and learns to reverse it.
- Denoising is a conditional inference problem, not merely a heuristic cleanup step.
- Iterative refinement turns many local denoising tasks into a generative procedure.
- Denoising autoencoders provide the conceptual bridge to full diffusion models.

Common confusion

- The forward process is usually fixed, not learned.
- A denoiser is not automatically a full generative model unless it is embedded into a sampling procedure.

- Predicting noise, predicting x_0, and predicting x_{t-1} are related but not identical parameterizations.

Exercises

1. Let $X \sim \mathcal{N}(0,1)$ and $Y = X + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0,4)$. Compute $\mathbb{E}[X \mid Y = y]$ and $\mathrm{Var}(X \mid Y)$. Interpret the result.
2. Suppose $q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x} \mid x, \sigma^2 I)$. Explain why minimizing $\mathbb{E}\|x - r_\theta(\tilde{x})\|^2$ trains the posterior mean estimator when the model class is rich enough.
3. Compare one-step denoising and multi-step denoising conceptually. Why might the multi-step version be easier in high dimensions?
4. In your own words, explain the conceptual difference between a denoising autoencoder and a diffusion sampler.

Notes and References

Denoising as a learning principle predates modern diffusion models and appears prominently in work on denoising autoencoders and score-based viewpoints. Modern diffusion models combine this denoising intuition with a carefully designed forward noising process and a learned reverse chain. Historically important reference points include denoising autoencoders, score matching ideas, and the later diffusion and score-based generative-model literature. The next section develops the formal diffusion equations that make these ideas computationally effective.

5.7 Diffusion Models: Mathematical Formulation

The previous section introduced diffusion conceptually as generation by repeated denoising. We now make that picture precise. This section is one of the mathematical anchors of the chapter: it states the forward Markov chain exactly, derives the Gaussian marginals and posterior conditionals that make training tractable, and explains why the practical noise-prediction objective is a simplified variational objective rather than an unrelated heuristic.

Guiding question. How can one turn a fixed corruption process into a trainable generative model whose reverse dynamics map simple noise back to structured data?

The formal forward Markov chain

Let $x_0 \in \mathbb{R}^d$ denote a clean data point. A discrete-time diffusion model begins by fixing a noising schedule $\beta_1, \ldots, \beta_T$ with $0 < \beta_t < 1$, and defining

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

Definition 5.32 (Forward diffusion chain)**.** The forward process is the Markov chain

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$

with Gaussian transitions

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t \mid \sqrt{\alpha_t}\, x_{t-1},\, \beta_t I\big).$$

Each step preserves most of the current signal but injects a small amount of Gaussian noise. When T is large enough and the schedule is chosen appropriately, the terminal variable x_T is close to an isotropic Gaussian. The fixed forward chain is exact; no learning enters here.

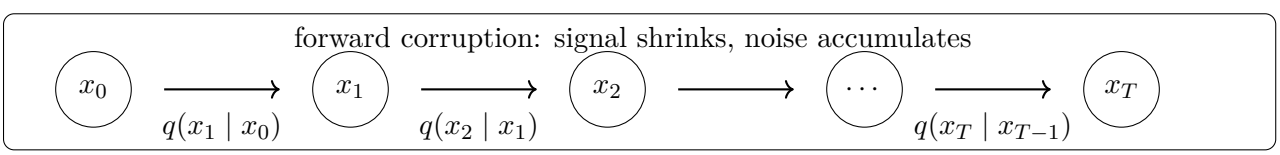


Figure 5.7: The forward diffusion chain is a fixed Gaussian Markov process that gradually destroys structure.

Closed-form forward marginals

The usefulness of the Gaussian forward chain lies in the fact that one can move from x_0 to any intermediate time t in closed form. This is what makes training efficient.

Proposition 5.33 (Closed-form forward marginal)**.** *For the forward chain above,*

$$q(x_t \mid x_0) = \mathcal{N}\Big(x_t \mid \sqrt{\bar{\alpha}_t}\, x_0,\, (1 - \bar{\alpha}_t) I\Big).$$

Equivalently,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Proof. We argue by induction on t. For $t = 1$ the claim is exactly the definition of $q(x_1 \mid x_0)$. Assume

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\, \varepsilon_{t-1}, \qquad \varepsilon_{t-1} \sim \mathcal{N}(0, I).$$

From the transition equation,

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, z_t, \qquad z_t \sim \mathcal{N}(0, I).$$

Substituting the induction hypothesis gives

$$x_t = \sqrt{\alpha_t \bar{\alpha}_{t-1}}\, x_0 + \sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})}\, \varepsilon_{t-1} + \sqrt{\beta_t}\, z_t.$$

Because the last two terms are independent centered Gaussians, their sum is Gaussian with covariance

$$\alpha_t (1 - \bar{\alpha}_{t-1}) I + \beta_t I = (1 - \alpha_t \bar{\alpha}_{t-1}) I = (1 - \bar{\alpha}_t) I.$$

Also $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$, so the mean is $\sqrt{\bar{\alpha}_t}\, x_0$. This proves the claim. □

Interpretation. The entire history $x_1, \ldots, x_{t-1}$ can be collapsed into a single effective corruption level. Training therefore does not require simulating the whole chain step by step. One may sample a time index t, draw one Gaussian noise vector ε, and construct x_t directly.

Posterior conditionals and reverse-process parameterization

To learn generation, one would ideally model the exact reverse conditionals induced by the forward chain. For the Gaussian process above, the relevant posterior is itself Gaussian when conditioned on both x_t and the original x_0.

Proposition 5.34 (Gaussian posterior for one reverse step)**.** *For $t \geq 1$,*

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(x_{t-1} \mid \tilde{\mu}_t(x_t, x_0),\, \tilde{\beta}_t I\big),$$

where

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\, \beta_t,$$

and

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\, \beta_t}{1 - \bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, x_t.$$

Proof sketch. Use Bayes' rule with the two Gaussian factors

$$q(x_t \mid x_{t-1}) \qquad \text{and} \qquad q(x_{t-1} \mid x_0).$$

Their product, viewed as a function of x_{t-1}, is proportional to a Gaussian density. Completing the square yields the stated posterior mean and variance. □

This proposition explains why the reverse model is commonly parameterized as

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \qquad p(x_T) = \mathcal{N}(0, I),$$

with Gaussian reverse transitions

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1} \mid \mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big).$$

The true reverse conditional depends on the unknown x_0, so the network must infer from x_t what clean structure likely produced the noisy observation.

A standard and extremely useful parameterization predicts the noise rather than the mean directly. If a network $\varepsilon_\theta(x_t, t)$ estimates the forward noise, then one defines

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

and substitutes $\hat{x}_0$ into the posterior-mean formula. Equivalently, when the reverse variance is chosen as a fixed known schedule,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(x_t, t)\right).$$

Thus reverse-process learning can be organized around estimating either x_0, the reverse mean, or the forward noise; these are different parameterizations of the same Gaussian structure.

A clean variational decomposition

Diffusion training admits a variational derivation analogous in spirit to the ELBO for latent-variable models. Here the latent variables are the intermediate noise states $x_1, \ldots, x_T$. For a datapoint x_0, define

$$\mathcal{L}_{\mathrm{vlb}}(x_0; \theta) = \mathbb{E}_{q(x_{1:T}|x_0)}\left[\log q(x_{1:T} \mid x_0) - \log p_\theta(x_{0:T})\right].$$

Then

$$-\log p_\theta(x_0) \le \mathcal{L}_{\mathrm{vlb}}(x_0; \theta),$$

so minimizing $\mathcal{L}_{\mathrm{vlb}}$ maximizes a lower bound on the log-likelihood.

Expanding the chain terms gives the decomposition

$$\begin{aligned}\mathcal{L}_{\mathrm{vlb}}(x_0; \theta) = {} & D_{\mathrm{KL}}\big(q(x_T \mid x_0) \,\|\, p(x_T)\big) \\ & + \sum_{t=2}^{T} \mathbb{E}_{q(x_t|x_0)}\left[D_{\mathrm{KL}}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\big)\right] \\ & - \mathbb{E}_{q(x_1|x_0)}\big[\log p_\theta(x_0 \mid x_1)\big].\end{aligned}$$

Interpretation. This formula separates the training problem into three parts: matching the terminal noise distribution to a simple Gaussian prior, matching each learned reverse step to the exact Gaussian posterior, and reconstructing the final clean sample from a slightly noisy version. The first term is usually fixed by design, the middle terms are the main diffusion-specific KL penalties, and the last term acts like a data-reconstruction term.

From the variational bound to the noise-prediction objective

The practical success of diffusion models depends on the fact that the complicated variational objective above reduces to a very simple supervised denoising objective under common parameter choices.

Suppose the reverse variance is fixed and only the mean is learned. Then each intermediate KL term becomes, up to constants independent of θ,

$$\mathbb{E}\Big[\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2\Big].$$

If we use the noise parameterization of μ_θ, then the mean-matching loss becomes a weighted noise-prediction loss.

Proposition 5.35 (Simplified objective under noise parameterization)**.** *If $p_\theta(x_{t-1} \mid x_t)$ is Gaussian with fixed variance $\tilde{\beta}_t I$ and mean parameterized by $\varepsilon_\theta(x_t, t)$ as above, then each variational term satisfies*

$$L_t = \mathbb{E}_{x_0,\varepsilon}\left[\frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1-\bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right] + C_t,$$

where $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$ and C_t does not depend on θ.

Proof sketch. The KL between two Gaussians with the same covariance is proportional to the squared distance between their means. Substitute the exact posterior mean $\tilde{\mu}_t(x_t, x_0)$ and the parameterized mean $\mu_\theta(x_t, t)$. After algebraic simplification, the difference between the two means is a known scalar multiple of $\varepsilon - \varepsilon_\theta(x_t, t)$. The stated coefficient is exactly the square of that scalar divided by the Gaussian variance term appearing in the KL. □

This is the origin of the standard training objective

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x_0,\varepsilon,t}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right], \qquad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon.$$

Strictly speaking, this objective omits time-dependent weights and certain auxiliary terms from the full variational bound. Empirically, however, it works extremely well and is the training rule most readers encounter first.

Relation to score matching

Diffusion is closely related to score-based generative modeling. Recall that the score of a density p is

$$\nabla_x \log p(x).$$

For the Gaussian corruption model,

$$q(x_t \mid x_0) = \mathcal{N}\Big(x_t \mid \sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I\Big),$$

so the conditional score is explicit.

Proposition 5.36 (Noise prediction and conditional score estimation)**.** *For fixed x_0 and t,*

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1-\bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}.$$

Hence predicting ε is equivalent, up to a known scalar factor, to predicting the conditional score of the noisy distribution.

Proof. Differentiate the log-density of the Gaussian $q(x_t \mid x_0)$ with respect to x_t. Then substitute

$$x_t - \sqrt{\bar{\alpha}_t} x_0 = \sqrt{1-\bar{\alpha}_t}\,\varepsilon.$$

□

This proposition does not say that the network directly learns the unconditional score $\nabla_{x_t} \log q_t(x_t)$ in exactly this form. Rather, it explains why denoising and score estimation are so closely connected: once Gaussian perturbations are introduced, the direction in which one should move to increase data density is encoded in the same residual term that appears in the denoising task.

DDPM and DDIM as formal sampling viewpoints

The same trained denoiser can support different reverse-time samplers. Two important viewpoints are DDPM and DDIM.

DDPM. In the original diffusion probabilistic model, sampling is stochastic. At time t, one forms $\hat{x}_0$ from the predicted noise, computes the reverse mean $\mu_\theta(x_t, t)$, and samples

$$x_{t-1} \sim \mathcal{N}\big(\mu_\theta(x_t, t),\, \sigma_t^2 I\big),$$

with σ_t^2 chosen from the variance schedule. This closely follows the probabilistic reverse-chain interpretation.

DDIM. DDIM keeps the same training objective but uses a non-Markovian sampling construction that can be written as

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\,\hat{\varepsilon} + \sigma_t z,$$

where $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$, $z \sim \mathcal{N}(0, I)$, and σ_t is chosen by the sampler. Setting $\sigma_t = 0$ gives a deterministic update.

Formal comparison. DDPM is a stochastic approximation to the reverse diffusion chain and naturally emphasizes probabilistic sampling. DDIM uses the same learned denoiser but chooses a family of trajectories consistent with the same training objective, including deterministic ones. Thus the model is the learned denoiser; the sampler is a separate algorithmic choice built on top of it.

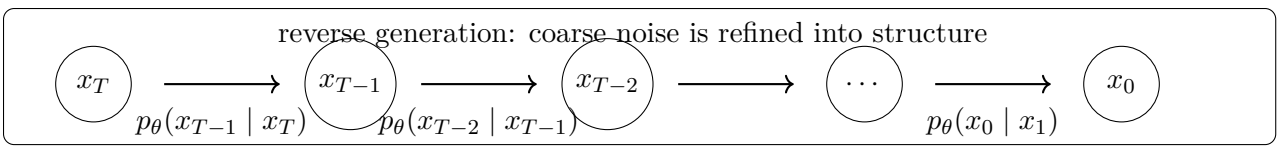


Figure 5.8: A learned reverse chain maps a simple Gaussian terminal state back to data. DDPM uses stochastic steps; DDIM can use deterministic ones.

Worked one-dimensional example

Consider the one-dimensional case with $T = 1$ and variance schedule $\beta_1 = 0.36$, so $\alpha_1 = 0.64$ and $\bar{\alpha}_1 = 0.64$. Then

$$q(x_1 \mid x_0) = \mathcal{N}(x_1 \mid 0.8x_0, 0.36).$$

If $x_0 = 2$ and the sampled forward noise is $\varepsilon = -1$, then

$$x_1 = 0.8 \cdot 2 + 0.6(-1) = 1.0.$$

Suppose a denoiser predicts $\varepsilon_\theta(x_1, 1) = -0.9$. Then the reconstructed clean sample is

$$\hat{x}_0 = \frac{x_1 - \sqrt{1 - \bar{\alpha}_1}\,\varepsilon_\theta(x_1, 1)}{\sqrt{\bar{\alpha}_1}} = \frac{1.0 - 0.6(-0.9)}{0.8} = 1.925.$$

The noise-prediction loss for this single example is

$$(\varepsilon - \varepsilon_\theta)^2 = (-1 + 0.9)^2 = 0.01.$$

Even in this tiny example one sees the basic logic: the model never receives the exact posterior; it learns to infer what noise must have been added, and that estimate is enough to reconstruct an approximation of the clean data point.

What is exact, approximate, and heuristic

The mathematical structure of diffusion is unusually clean, but different parts of the method live at different levels of certainty.

- **Exact:** the forward Gaussian chain, its closed-form marginals, and the Gaussian posterior formulas are exact consequences of the chosen corruption process.
- **Variationally justified:** the reverse-chain likelihood bound and the resulting KL decomposition are exact identities once the forward process and reverse family are specified.
- **Approximate but principled:** the learned reverse conditionals are approximations to the true reverse conditionals, and the simplified noise-prediction loss drops weights and terms from the full variational objective.
- **Design practice:** noise schedules, sampler choices, parameterizations, architectural details, and acceleration rules remain partly empirical.

This separation matters pedagogically. The elegance of diffusion does not come from pretending every successful implementation detail is theoretically inevitable. It comes from recognizing that a large and useful class of those details sits on top of a remarkably transparent probabilistic core.

What to remember

- The forward diffusion process is a fixed Gaussian Markov chain with closed-form marginals.
- The reverse model learns Gaussian conditionals intended to approximate the true reverse posterior steps.
- The diffusion ELBO decomposes into terminal, reverse-step, and reconstruction terms.
- The practical noise-prediction loss is a simplified variational objective.
- Diffusion is closely connected to score estimation on noisy data.

Common confusion

- Predicting noise is not a separate idea from reverse-process learning; it is a convenient parameterization of it.
- The forward process is not learned in the standard formulation.
- DDPM and DDIM differ mainly in sampling dynamics, not in the fundamental denoiser-training objective.
- A simplified training loss can still be principled even if it is not the full likelihood objective written term by term.

Exercises

1. Prove the closed-form forward marginal again, but this time by composing Gaussian means and covariances recursively rather than using the explicit noise representation.
2. Starting from the Gaussian formula for $q(x_t \mid x_{t-1})$ and the closed-form formula for $q(x_{t-1} \mid x_0)$, derive the posterior $q(x_{t-1} \mid x_t, x_0)$ by completing the square.
3. Show directly that the conditional score formula

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}$$

follows from the Gaussian density and the reparameterized form of x_t.

4. Explain in words why the simplified noise-prediction objective can be interpreted as supervised learning, even though the overall model is generative.
5. Compare DDPM and DDIM from the viewpoint of stochastic versus deterministic trajectories. What remains common between them?

Notes and references

The modern discrete-time diffusion formulation was developed in the diffusion probabilistic-model literature and later clarified and generalized through score-based generative-model work. Important reference points include denoising score matching, diffusion probabilistic models, improved DDPM-style training, and DDIM-style non-stochastic sampling. Conceptually, the section also inherits ideas from denoising autoencoders, variational latent-variable modeling, and stochastic-process viewpoints on generation.

5.8 Comparing the Main Generative Paradigms

We have now studied four major families of generative models in this chapter: variational latent-variable models, adversarial models, exact invertible models, and diffusion models. At this point the main pedagogical task is no longer definition but judgment. A student who has understood the previous sections should be able to answer a more difficult question than "what is a VAE?" or "what is a diffusion model?" The real question is:

> for a given modeling goal, which generative paradigm is structurally appropriate, which tradeoffs does it make, and which claims about it are mathematical, empirical, or merely conventional?

This section therefore compares the main paradigms along a fixed set of dimensions: objective, tractability, latent inference, sample quality, controllability, training stability, and sampling cost. The goal is not to declare a single winner. The goal is to learn how to choose.

The comparison axes

A comparative section is only useful if the axes of comparison are made explicit. For this chapter, the most informative axes are the following.

Objective. What quantity is optimized during training? Examples include a variational lower bound, an adversarial game objective, exact log-likelihood, or a denoising-based variational surrogate.

Tractability. Can one evaluate or optimize the training objective exactly, approximately, or only indirectly? This includes tractable likelihood evaluation, tractable posterior inference, and tractable gradients.

Latent inference. Does the model provide a natural map from data back to hidden variables? In VAEs this is built in through an approximate posterior. In flows it is exact by inversion. In GANs it is not canonical unless one adds an encoder. In diffusion models the internal noise states are useful computational variables, but they do not usually play the same role as a single global latent code.

Sample quality. How realistic are generated samples in practice? This is mostly an empirical question rather than a theorem. The answer depends strongly on architecture, dataset, and evaluation metric.

Controllability. How easily can one steer generation through labels, prompts, latent editing, or conditional structure? The answer depends both on the family and on how conditioning is implemented.

Training stability. Does optimization behave like a single-objective problem, or like a fragile game or long iterative chain? Again, this is partly mathematical and partly practical.

Sampling cost. Once trained, how expensive is sampling? A model may be elegant statistically but slow operationally, or fast operationally but hard to train.

Family-by-family comparison

We now compare the main families from earlier sections using these axes. The important point is not that each family has one permanent rank, but that each one embodies a different answer to the central generative-modeling problem.

Variational autoencoders and related latent-variable models. The objective is a variational lower bound on $\log p_\theta(x)$. Likelihood training is therefore principled, but only indirect when the posterior is intractable. A major strength is that latent inference is built into the formulation through $q_\phi(z \mid x)$. This makes VAEs natural when one wants compressed latent descriptions, interpolation, or amortized inference. Their traditional weakness is that sample sharpness may lag behind the best adversarial or diffusion systems unless the decoder family and training scheme are very strong. Sampling is fast once the latent code is drawn, but the learned latent space may mix semantically useful directions with nuisance structure.

Adversarial models. The objective is a minimax game rather than a direct likelihood objective. This can align well with perceptual realism, which is why GANs historically became important for high-fidelity image generation. However, there is no tractable likelihood in the standard formulation, and latent inference is not canonical unless one explicitly adds an inverse map. Sampling is typically fast because generation is one forward pass through the generator. The main difficulty is instability: the generator and discriminator co-evolve, so gradients can become uninformative or oscillatory. Thus GANs are attractive when one-shot high-quality synthesis matters more than density evaluation or stable latent inference.

Normalizing flows. The objective is exact log-likelihood under an invertible change of variables. This is mathematically clean: likelihood evaluation is exact, sampling is exact, and latent inference is exact by inversion. The main price is architectural constraint. Because each layer must remain invertible and its Jacobian determinant must be tractable, one often sacrifices some modeling freedom. Flows are therefore especially appealing when exact densities, exact inverses, or principled likelihood-based comparison matter. They are less compelling when the most important criterion is best-in-class sample quality under minimal architectural restriction.

Diffusion models. The objective is usually implemented as a denoising or noise-prediction loss derived from a variational formulation. Training is typically much more stable than adversarial training because it reduces generation to supervised prediction over corrupted inputs. In modern practice, diffusion models often provide very strong sample quality and good conditioning behavior. Their main cost is sampling: naive generation requires many reverse steps. Their internal variables are useful as a sequence of refinement states, but not always as a single semantically interpretable latent code. Thus diffusion is attractive when quality and controllability matter more than one-pass generation speed.

A structured summary table

The comparison in prose is easier to use once condensed into a single reference table. Table 5.1 should be read as a comparative guide rather than as a ranking. Words such as “strong” or “weak” are intentionally relative and should be interpreted as broad patterns across standard versions of each family.

Family	**Objective**	**Tractable likelihood**	**Latent inference**	**Sample quality**	**Controllability**	**Training stability**	**Sampling cost**
VAE / latent-variable	ELBO or related variational objective	Approximate	Approximate posterior available	Often moderate to strong, depends on decoder	Good when latent structure is well designed	Usually good	Low after training
GAN	Adversarial minimax game	No, not in standard form	Not canonical without added encoder	Often very high in domains where GANs work well	Can be good conditionally, but latent control may be fragile	Often difficult	Very low
Flow	Exact log-likelihood by change of variables	Yes, exact	Yes, exact by inversion	Often good but constrained by invertibility requirements	Good when conditioning is built into the transform	Usually good to moderate	Low to moderate
Diffusion	Variational denoising / noise-prediction objective	Bound or surrogate; exact likelihood usually not the main practical interface	Reverse refinement states rather than a simple global code	Often very high in modern practice	Often strong, especially with conditional guidance	Usually good	High unless accelerated

Table 5.1: A comparative summary of the main generative paradigms studied in this chapter.

Which model family for which goal?

A table is compact, but a decision figure is better for judgment. Figure 5.9 gives a simple decision-oriented map. It is deliberately schematic. Real model choice depends on domain, scale, compute budget, and engineering maturity. Still, this diagram captures the structural logic of the comparison.

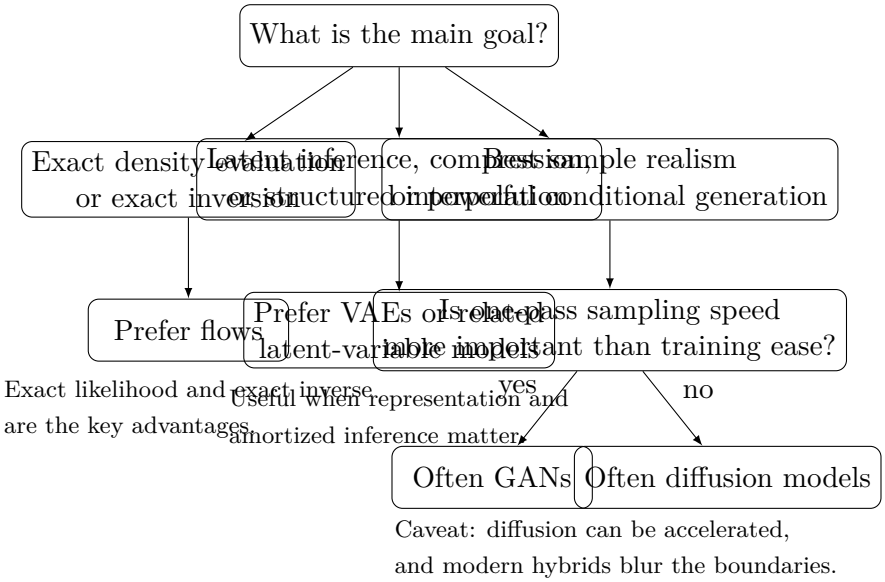


Figure 5.9: A decision-oriented guide to the main generative-model families. This is not a theorem and not a universal ranking; it is a structured heuristic for matching modeling goals to paradigms.

What is mathematical, and what is empirical?

A mature comparison should distinguish between statements that follow from the mathematical formulation and statements that come from repeated empirical experience.

Mathematical facts. Flows provide exact density evaluation and exact inversion by construction. Standard GANs do not provide tractable likelihoods by construction. VAEs optimize a lower bound on log-likelihood rather than log-likelihood itself. Diffusion models implement generation through a learned reverse process and usually sample by many refinement steps. These are structural statements.

Empirical patterns. Diffusion models often achieve very strong visual and multimodal sample quality. GANs can provide excellent one-pass synthesis when training succeeds. VAEs often provide useful latent spaces and stable training. Flows often shine when likelihood and invertibility matter. These are broad empirical regularities rather than universal laws.

Design-dependent conclusions. Controllability, semantic latent structure, and practical efficiency depend strongly on architecture and conditioning strategy. For example, a diffusion model may be slow in its naive form but far faster after distillation. A VAE may yield weak samples with a small decoder but much stronger samples with a more expressive decoder and improved objective. A GAN may support meaningful editing if paired with a well-behaved latent space, but this is not guaranteed by the minimax objective alone.

How to choose in practice

The comparison above can be compressed into a few practical recommendations.

- Choose a flow when exact density evaluation, exact inversion, or principled likelihood-based comparison is central.
- Choose a VAE or related latent-variable model when one wants representation learning, amortized inference, compression, or latent-variable reasoning together with principled probabilistic structure.
- Choose a GAN when sample realism and one-shot synthesis dominate, and one is willing to manage adversarial instability and the lack of tractable likelihood.
- Choose a diffusion model when sample quality, conditional steering, and robust training matter more than raw sampling speed.

These recommendations are useful, but they should not harden into slogans. Real systems often mix ideas. Latent diffusion combines diffusion with latent-variable structure. Flow matching and consistency-style methods alter the speed–quality tradeoff. Conditional and multimodal systems increasingly borrow components across families. Thus the right long-term lesson is not merely "pick one box from the chart." The deeper lesson is to identify which modeling commitments actually matter for the task.

What to remember

- Different generative paradigms solve different mathematical subproblems well: likelihood, inference, realism, invertibility, or iterative refinement.
- No single family dominates every axis simultaneously.
- The most important comparison axes are objective, tractability, latent inference, controllability, stability, and sampling cost.
- Good model choice requires matching the paradigm to the goal rather than repeating current fashion.

Common confusion

- "Best sample quality" does not mean "best for every task."
- "Has a latent space" does not mean the latent variables are automatically interpretable or easy to control.
- "Likelihood-based" does not mean samples will necessarily look best perceptually.
- "Fast sampling" and "easy training" are different desiderata and often trade off against each other.

Exercises

1. You need a model for anomaly detection in which exact or at least principled density evaluation matters. Which family from this chapter is the strongest first candidate, and why?
2. You need a model for image compression and smooth latent interpolation between examples. Compare VAEs and GANs for this goal. Which structural features of the objectives matter most?
3. You need conditional image synthesis for an offline pipeline where sample quality matters more than generation speed. Which family is most attractive, and what cost does it incur?
4. You need one-pass generation for an interactive system. Which family is attractive for speed, and what training risks come with that choice?
5. Explain why exact invertibility is an advantage in some scientific settings but not automatically decisive for perceptual media generation.
6. Give one modeling situation in which a diffusion model is a poor first choice, and justify your answer using the comparison axes from this section.
7. Design your own decision chart for a domain such as molecules, audio, or robotics. Which axes from this section remain central, and which new axes would you add?

Notes and References

The four-way comparison in this section reflects the conceptual structure of the chapter rather than an exhaustive survey of generative modeling. Autoregressive models are also central to modern generative AI, especially in language modeling, but the present chapter has focused on latent-variable, adversarial, invertible, and diffusion-based viewpoints because they illuminate different mathematical strategies for modeling a distribution. For historical orientation, standard reference points include the VAE literature, classical GAN papers and their stability variants, the normalizing-flow literature, and the modern diffusion and score-based modeling literature. A useful reading habit is to ask of every new generative paper the same questions used in this section: what objective is actually optimized, what inference problem is solved, what is tractable, and what practical cost is paid at sampling time?

5.9 Worked Examples and Visual Case Studies

The earlier sections of this chapter introduced the main generative paradigms one by one. A reader may understand each paradigm locally and still not yet have the comparative judgment that matters most in practice. This section therefore slows down and works through a small set of low-dimensional examples. The point is not to showcase large-scale engineering success. The point is to see, on the same kinds of toy problems, how different generative models represent uncertainty, hidden structure, and generation itself.

Whenever possible, we use either the same target distribution or closely related toy distributions: a bimodal one-dimensional dataset, a low-dimensional latent line between two examples, a two-dimensional warped density, and an iterative denoising path from noise back to structure. These examples are intentionally simple enough that one can inspect the equations and the pictures at the same time.

Case study 1: Gaussian mixture and EM walk-through

A Gaussian mixture model is still the cleanest worked example of latent-variable generative modeling. Suppose the observed one-dimensional data are

$$\mathcal{D} = \{-2.2, -1.8, 2.1, 2.4\},$$

and consider a two-component Gaussian mixture with fixed variance $\sigma^2 = 1$:

$$p_\theta(x) = \pi_1 \mathcal{N}(x \mid \mu_1, 1) + \pi_2 \mathcal{N}(x \mid \mu_2, 1), \qquad \pi_1 + \pi_2 = 1.$$

The latent variable $z_i \in \{1, 2\}$ indicates which component generated x_i.

We initialize

$$\pi_1^{(0)} = \pi_2^{(0)} = \tfrac{1}{2}, \qquad \mu_1^{(0)} = -1, \qquad \mu_2^{(0)} = 1.$$

The E-step computes responsibilities

$$r_{ik} = \mathbb{P}(z_i = k \mid x_i, \theta^{(0)}) = \frac{\pi_k^{(0)} \mathcal{N}(x_i \mid \mu_k^{(0)}, 1)}{\sum_{j=1}^{2} \pi_j^{(0)} \mathcal{N}(x_i \mid \mu_j^{(0)}, 1)}.$$

Because the data are well separated, the responsibilities are already nearly hard assignments:

$$r_{11} \approx 0.992, \quad r_{21} \approx 0.973, \qquad r_{31} \approx 0.043, \quad r_{41} \approx 0.032,$$

so points near -2 are assigned mostly to component 1 and points near 2 mostly to component 2.

The M-step updates mixture weights and means:

$$\pi_k^{(1)} = \frac{1}{n} \sum_{i=1}^{n} r_{ik}, \qquad \mu_k^{(1)} = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}.$$

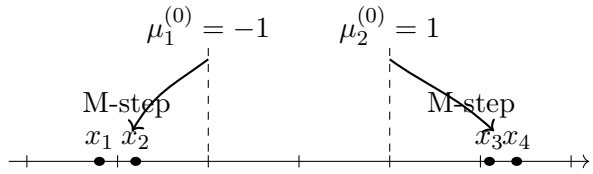
With the approximate responsibilities above,

$$\pi_1^{(1)} \approx 0.510, \qquad \pi_2^{(1)} \approx 0.490,$$

and

$$\mu_1^{(1)} \approx \frac{0.992(-2.2) + 0.973(-1.8) + 0.043(2.1) + 0.032(2.4)}{0.992 + 0.973 + 0.043 + 0.032} \approx -1.84,$$
$$\mu_2^{(1)} \approx \frac{0.008(-2.2) + 0.027(-1.8) + 0.957(2.1) + 0.968(2.4)}{0.008 + 0.027 + 0.957 + 0.968} \approx 2.15.$$

After just one iteration, the parameters move toward the visible cluster centers. The example is pedagogically important because every step can be interpreted: soft latent assignment simplifies the optimization of an otherwise nonconvex incomplete-data likelihood.



E-step: compute soft assignments r_{ik}
M-step: move components toward responsibility-weighted averages

Figure 5.10: A one-iteration EM walk-through for a two-component Gaussian mixture. The latent assignment variables are not observed, but the E-step estimates them and the M-step refits the component parameters.

Case study 2: VAE latent interpolation sketch

A variational autoencoder gives a different generative picture. Instead of assigning each point to one of finitely many latent components, it learns a continuous latent coordinate z together with a decoder $p_\theta(x \mid z)$. A simple way to visualize what the model has learned is to encode two data points, interpolate in latent space, and decode back.

Suppose two observations $x^{(a)}$ and $x^{(b)}$ are encoded to approximate posterior means

$$\mu_\phi(x^{(a)}) = -1.2, \qquad \mu_\phi(x^{(b)}) = 1.4,$$

in a one-dimensional latent space. For interpolation parameter $\lambda \in [0, 1]$, define

$$z_\lambda = (1 - \lambda)(-1.2) + \lambda(1.4).$$

If the decoder mean is approximately linear near this path, say

$$\mathbb{E}[x \mid z] \approx 2z,$$

then the decoded means along the path are

$$-2.4,\ -1.1,\ 0.2,\ 1.5,\ 2.8$$

for $\lambda = 0, 0.25, 0.5, 0.75, 1$. The exact numbers are not important. What matters is the geometric idea: the model is not switching among discrete components but moving continuously through a learned latent coordinate system.

This example also reveals a limitation. Interpolation is convincing only if the decoder behaves sensibly away from the encoded training points. If the latent path passes through a region of z that the approximate posterior rarely uses, decoded samples may be blurry, implausible, or semantically mixed. Thus latent interpolation is strong evidence that a VAE has learned some organization of variation, but it is not a theorem that every linear path in z-space corresponds to a meaningful path in data space.

Case study 3: GAN mode collapse on a toy bimodal target

A useful way to understand both the power and fragility of adversarial learning is to study a target distribution with two modes. Consider the one-dimensional target distribution

$$p_{\text{data}} = \tfrac{1}{2}\,\delta_{-2} + \tfrac{1}{2}\,\delta_2,$$

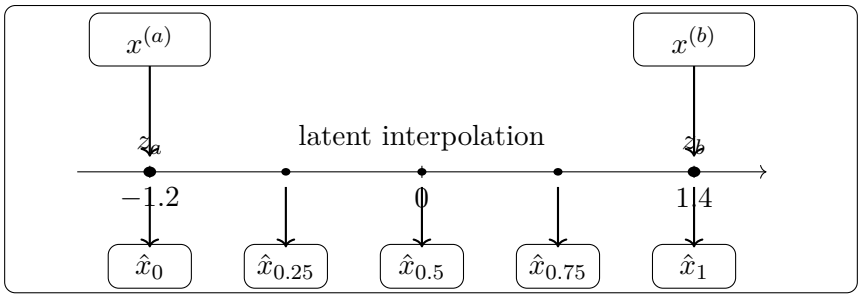


Figure 5.11: A VAE interpolation sketch. Two examples are encoded into latent points, a line segment is traversed in latent space, and the decoder maps that path back to data space. Smooth interpolation is evidence of learned latent organization, but not proof of semantic disentanglement.

which places equal mass near -2 and 2. Suppose the generator collapses to a single mode and produces only samples near 2. Then the model distribution is approximately

$$p_G \approx \delta_2.$$

This is a bad model of the true distribution, yet it may still produce individually plausible samples. That is the pedagogical point. Adversarial training optimizes distribution matching through a discriminator, but the generator is rewarded through whatever gradient information the current discriminator provides. If the discriminator is finite-capacity, trained imperfectly, or observed only through stochastic minibatches, the generator may keep exploiting the same successful region instead of covering all modes.

In this toy example, half the target support is missed completely. The failure is not that the generator cannot produce realistic points. The failure is that realism and coverage are different notions. A single sharp mode may look convincing sample by sample while still representing the data distribution poorly.

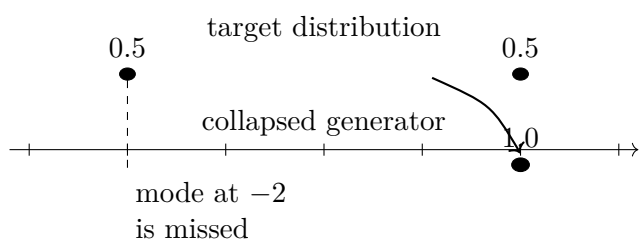


Figure 5.12: Mode collapse on a bimodal target. The generator produces realistic points near one mode but fails to cover the full data distribution. The example isolates the difference between realism and coverage.

Case study 4: A flow on a two-dimensional density

Normalizing flows are easiest to understand visually in two dimensions. Let the base variable be

$$u = (u_1, u_2) \sim \mathcal{N}(0, I_2),$$

and define the invertible map

$$T(u_1, u_2) = \big(u_1,\ u_2 + 0.8\sin(u_1)\big).$$

This is an affine coupling-style transformation: the first coordinate is left unchanged, and the second coordinate is shifted by a function of the first. The inverse is immediate:

$$T^{-1}(x_1, x_2) = \big(x_1,\, x_2 - 0.8\sin(x_1)\big).$$

Its Jacobian matrix is

$$J_T(u) = \begin{pmatrix} 1 & 0 \\ 0.8\cos(u_1) & 1 \end{pmatrix}, \qquad \det J_T(u) = 1.$$

Therefore the transformed density is

$$p_X(x_1, x_2) = p_\nu\big(x_1,\, x_2 - 0.8\sin(x_1)\big).$$

The determinant is trivial, but the geometry is not. Straight contours in the base space become sinusoidally warped contours in data space. This example shows exactly why flows appeal to mathematically inclined readers: the model is both visually intuitive and analytically exact.

At the same time, the example exposes the architectural constraint. The transformation must be invertible and must have a tractable determinant. These requirements are what make exact likelihood possible, but they also restrict the class of allowable maps.

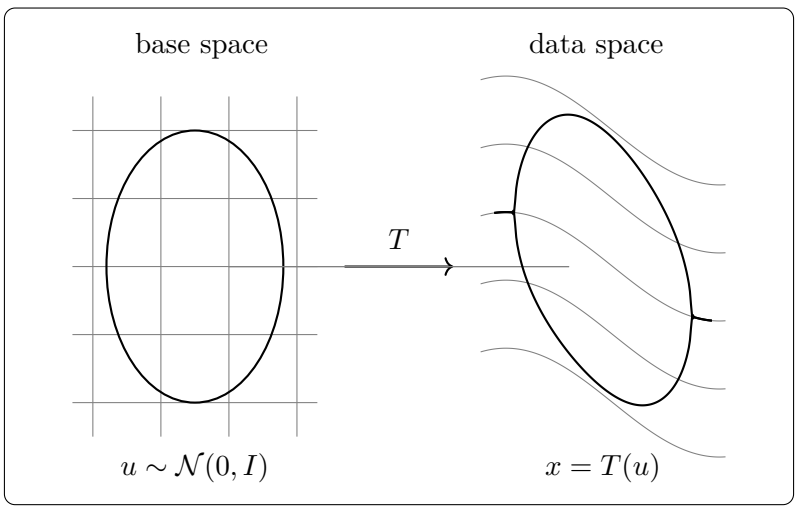


Figure 5.13: A two-dimensional flow warps a simple base density into a curved target density by an invertible transport map. Because the map is explicit and invertible, both sampling and density evaluation remain exact.

Case study 5: Diffusion denoising trajectory on a toy distribution

Diffusion models are easiest to grasp when viewed as progressive correction rather than single-shot sampling. Take a one-dimensional toy distribution concentrated near two modes, say near -2 and 2. A forward corruption process repeatedly adds Gaussian noise until the distribution becomes approximately normal around 0. The reverse process then learns to nudge a noisy point back toward one of the data-supported regions.

A tiny numerical sketch makes this concrete. Suppose the reverse model starts from $x_4 = 0.3$ and at each step predicts a denoised mean that moves toward the positive mode:

$$\hat{x}_3 = 0.9, \qquad \hat{x}_2 = 1.4, \qquad \hat{x}_1 = 1.8, \qquad \hat{x}_0 = 2.0.$$

This is not meant to mimic a real trained network exactly. It is a pedagogical summary of what the learned reverse chain does: each step removes some uncertainty while preserving enough randomness that different runs can end at different supported regions of the data distribution.

The crucial contrast with GANs is that diffusion does not attempt to jump directly from noise to data in one step. The crucial contrast with flows is that the path need not be invertible in one shot. The crucial contrast with VAEs is that the hidden variables are not summarized by one global latent code, but by a whole sequence of refinement states.

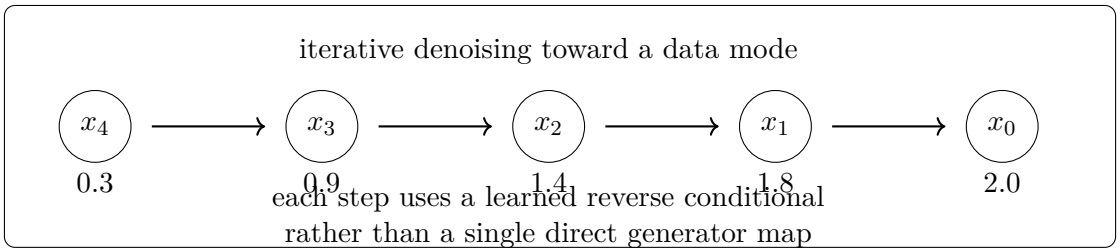


Figure 5.14: A toy diffusion denoising trajectory. Starting from a noisy sample, the reverse process repeatedly refines the state toward a data-supported region. The picture emphasizes iterative correction rather than exact numerical realism.

One family of toy problems, many modeling viewpoints

We can now compare the paradigms through the same pedagogical lens. The Gaussian mixture explains multimodality through an explicit latent assignment variable. The VAE explains it through a continuous latent code and a decoder. The GAN tries to match the target distribution through adversarial training, but can fail by concentrating on only one of several modes. The flow transports a simple base distribution into a more complex one through an invertible map. The diffusion model constructs generation as a sequence of denoising refinements.

This is the central lesson of the section. The paradigms are not merely different software stacks. They embody different answers to the question "how should a complicated data distribution be represented?" A mathematically inclined reader should notice that the hidden variable, the training objective, and the generative mechanism are coupled differently in each family.

What to remember

- Low-dimensional worked examples reveal structural differences among generative paradigms that are easy to miss in large-scale applications.
- EM explains multimodality through latent assignments; VAEs through continuous latent coordinates; GANs through adversarial distribution matching; flows through invertible transport; diffusion through iterative denoising.
- A visually convincing sample is not the same thing as good density coverage.
- Interpolation, transport, and denoising pictures are useful diagnostics, but they should be interpreted carefully rather than treated as proofs of deep understanding.

Common confusion

- A VAE interpolation path is not guaranteed to remain on the true data manifold.

Paradigm	**What the toy example highlights**	**Main caution revealed by the toy example**
Gaussian mixture / EM	Hidden assignments can turn a difficult marginal-likelihood problem into alternating weighted updates.	Local optima and component assumptions matter; the latent variable is discrete and model-specific.
VAE	Continuous latent coordinates can organize variation and support interpolation.	Interpolation quality depends on decoder behavior and actual latent usage.
GAN	High local realism does not guarantee full distributional coverage.	Mode collapse can hide behind plausible samples.
Flow	Exact density modeling can be visualized as invertible transport of probability mass.	Invertibility and tractable Jacobians constrain the architecture.
Diffusion	Generation can be understood as repeated denoising rather than one-shot synthesis.	Sampling is iterative and therefore computationally expensive.

Table 5.2: A comparative summary of what each worked example teaches.

- A GAN that produces sharp samples may still model the data distribution poorly if it misses modes.
- A flow picture may look simple only because the low-dimensional example hides the architectural burden of invertibility in higher dimensions.
- A diffusion trajectory is stochastic refinement, not deterministic decoding from a single latent code.

Exercises

1. Recompute the first EM iteration in the Gaussian-mixture example with initial means $\mu_1^{(0)} = -0.5$ and $\mu_2^{(0)} = 0.5$. How do the responsibilities and updated means change?
2. In the VAE interpolation example, suppose the decoder mean is $\mathbb{E}[x \mid z] = z^3$ instead of $2z$. Compute the decoded values along the same interpolation path and explain how nonlinearity changes the interpretation.
3. Construct a three-mode target distribution for which a GAN-like generator that covers only two modes would still produce visually plausible samples. Why is this failure hard to see from isolated samples alone?
4. For the flow example, verify explicitly that $\det J_T(u) = 1$ and compute $p_X(0, 1)$ in terms of the standard Gaussian density.
5. Consider a toy diffusion process with three reverse steps and predicted means $0.1 \to 0.7 \to 1.3 \to 2.0$. Explain which parts of this trajectory are deterministic and which parts would be stochastic in an actual diffusion sampler.
6. Take the same one-dimensional bimodal target and explain, in one paragraph each, how a

Gaussian mixture, a VAE, a GAN, a flow, and a diffusion model would represent it.

Notes and references

These worked examples are deliberately elementary, but they mirror the conceptual structure of the major paradigms. The EM walk-through reflects the classical latent-variable literature for Gaussian mixtures and incomplete-data likelihood. The interpolation sketch is the simplest visual diagnostic for latent-variable autoencoders. The mode-collapse example abstracts a major theme from the GAN literature: adversarial realism does not automatically imply full support coverage. The flow example reflects the transport viewpoint of invertible density models. The denoising trajectory is the simplest low-dimensional analogue of the iterative sampling picture used in modern diffusion models.

For a reader studying the literature, the right habit is to ask, in each case, what is mathematically exact, what is an approximation, and what is an empirical visualization meant only to build intuition.

5.10 What Is Established and What Remains Open

Generative modeling combines mature mathematics with rapidly evolving empirical practice. It is therefore especially important to distinguish settled ideas from open questions.

What is well established

Several core principles are by now firmly understood.

EM as lower-bound optimization. The EM framework provides a principled way to optimize latent-variable likelihoods through alternating inference and parameter updates.

ELBO logic in variational learning. The evidence lower bound gives a clean variational foundation for approximate latent-variable inference in deep models.

Change of variables in flows. The flow framework is mathematically exact once one specifies an invertible transformation with tractable Jacobian determinant.

Diffusion forward and reverse formalism. The Gaussian forward process, its closed-form marginals, and reverse-process approximation framework are well established.

What remains difficult

Other aspects are still less resolved.

GAN stability and mode coverage. Adversarial training remains delicate, and ensuring both realism and diversity is challenging.

Semantic controllability in latent spaces. Latent-variable models often learn useful structure, but guaranteeing interpretable and controllable semantics remains difficult.

Why diffusion scales so well perceptually. Diffusion models are empirically powerful, but the full explanation for their perceptual success across domains is still incomplete.

Representation versus generation tradeoffs. How sample quality, latent usefulness, controllability, and likelihood should be balanced remains an open modeling question.

Evaluation across objectives. Comparing generative models is inherently difficult because they optimize different goals: likelihood, realism, diversity, reconstruction, or denoising quality may not align.

Closing perspective

This chapter has organized generative modeling around four major routes:

- latent-variable inference,
- adversarial learning,
- invertible transport,

- iterative denoising.

Each route offers a different answer to the problem of modeling complex data distributions. No single paradigm dominates every criterion.

The broader lesson is that generative modeling is not a peripheral topic in modern AI. It is one of the main ways of learning structure, representation, and uncertainty in rich data.

The next chapter steps back from specific model classes and asks a more reflective question:

> what do we actually understand about deep learning, where does it remain unreliable, and what remains genuinely open?

Chapter 6

Understanding Deep Learning: Generalization, Reliability, and Open Questions

6.1 Why Deep Learning Still Puzzles Us

Deep learning has achieved striking empirical success across a wide range of domains: vision, language, speech, control, scientific modeling, and generative AI. Yet this success raises a scientific puzzle.

Classical learning intuition suggests that highly flexible models should be at serious risk of overfitting. Deep neural networks are often massively overparameterized, capable of fitting training data extremely well, and sometimes even capable of interpolating the training sample almost perfectly. Nevertheless, they frequently generalize far better than naive intuition would predict.

This creates a tension:

> deep learning works extremely well in practice, but our theory still does not provide a single complete explanation of why it works so well.

Overparameterization and interpolation

A modern neural network may contain far more parameters than training examples. From a purely classical counting perspective, this might appear dangerous. One could imagine many parameter settings that fit the training data while behaving very differently away from it.

And yet empirical practice shows something subtler. In many regimes, larger networks are not only more expressive, but also easier to optimize and sometimes better at generalizing.

This does not mean classical learning theory is wrong. Rather, it means that the interaction between model size, optimization, data structure, and inductive bias is more subtle than early intuition suggested.

Optimization and generalization are different questions

Two questions must be kept separate:

1. **Optimization:** can we find parameters that fit the training objective well?
2. **Generalization:** will the resulting solution perform well on new data?

Deep learning has complicated both questions. Overparameterization may make optimization easier in some regimes, while also changing the geometry of the space of solutions. The learned model is not just any interpolating solution; it is the solution found by a particular optimization process with a particular architecture and initialization.

Why theory trails practice

One reason theory trails practice is that modern systems combine many ingredients:

- compositional function classes,
- stochastic gradient-based optimization,
- normalization and residual pathways,
- architectural inductive bias,
- large-scale pretraining,
- self-supervised and generative objectives.

Each ingredient is partly understood in isolation. What is harder is to explain how they interact in realistic large systems.

The purpose of this chapter

This chapter is not meant to give a complete theory of deep learning. That theory does not yet exist in a fully unified form. Instead, the goal is more modest and more useful:

- to identify what is genuinely puzzling,
- to describe the main partial explanations,
- to explain why strong benchmark performance is not the same as reliability,
- to clarify what remains open.

6.2 Generalization Beyond Classical Expectations

The most famous theoretical puzzle of deep learning concerns generalization. Why do large networks often predict well on new data despite extreme flexibility?

Fitting versus predicting

The training objective is usually based on empirical risk:

$$\widehat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i).$$

The quantity we ultimately care about is the population risk:

$$R(f) = \mathbb{E}_{(x,y)\sim P}[\ell(f(x), y)].$$

Classical intuition emphasizes the danger that low empirical risk may not imply low population risk when the model class is too rich. That danger is real. But in modern deep learning, interpolation of the training data is often compatible with surprisingly good test performance.

Implicit bias of optimization

One important idea is that learning algorithms do not explore parameter space arbitrarily. Gradient-based optimization may prefer certain kinds of solutions over others, even when many parameter settings achieve nearly identical training error.

This preference is often called an *implicit bias* or *implicit regularization* of optimization.

Thus the learned predictor is not determined only by:

- the model class,
- the dataset,
- the explicit loss.

It is also shaped by:

- initialization,
- optimization dynamics,
- architecture,
- training schedule,
- normalization and regularization mechanisms.

This helps explain why overparameterized systems may still converge to useful solutions rather than arbitrary memorizing ones.

Norm and margin viewpoints

Another family of explanations emphasizes geometric quantities such as norm and margin.

Norm. A predictor may interpolate the training data while still being "simple" in an appropriate geometric sense, for example by having controlled parameter norm or controlled function complexity.

Margin. In classification, correct predictions made with large margin are often more stable than predictions made barely on the correct side of a boundary. Margin-based reasoning was central in classical methods such as support vector machines, and related ideas continue to matter in deep learning.

These viewpoints do not provide a complete explanation of generalization in all deep models, but they often provide useful partial structure.

Overparameterization as a changed regime

A striking modern lesson is that increasing model size does not always hurt test performance. In some settings, once the model becomes large enough to fit the data well, further increases in size may actually improve generalization again.

This suggests that the classical "more complex model means worse generalization" picture is too simplistic for highly overparameterized regimes.

Double descent

One influential conceptual picture is *double descent*. In a simplified description, as model size increases:

- test error may first decrease,
- then increase near an interpolation threshold,
- then decrease again in a larger overparameterized regime.

This phenomenon does not occur identically in all settings, but it captures a real and important departure from classical bias–variance intuition.

What appears established

Several broad points now seem plausible and robust:

- interpolation alone does not determine generalization quality,
- optimization dynamics matter,
- geometric notions such as norm and margin often remain relevant,
- overparameterization changes the effective learning regime rather than simply making it worse.

What remains unclear

At the same time, no single theory fully explains:

- why stochastic gradient methods often find good solutions so reliably,
- why overparameterized models can be simultaneously easy to optimize and strong at prediction,
- what depth contributes beyond width or linearized approximations,
- how architecture and data structure enter the generalization story in a unified way.

Thus deep-learning generalization is not mysterious in the sense of being completely opaque, but it remains incomplete in the sense of lacking a definitive unified account.

6.3 Reliability Under Real-World Conditions

Generalization on a held-out test set is important, but it is not the same as real-world reliability. A system may perform well on benchmark data and still fail in ways that matter greatly in deployment.

Distribution shift

Most learning theory assumes that training and test data come from the same distribution. In reality, this assumption often fails.

The environment may change because of:

- new populations,
- new sensors,
- new contexts,
- changing behavior,
- altered feedback loops,
- rare events not represented in the training set.

This is called *distribution shift*.

A model trained under one distribution P_{train} may behave quite differently under another distribution P_{test}.

Why shift is difficult

A powerful model can only generalize well to structure it has learned or can reasonably infer. When the environment changes, learned regularities may no longer be sufficient.

For example:

- visual models may fail under new lighting or viewpoint conditions,
- language models may behave differently under unfamiliar domains or instructions,
- medical predictors may degrade when moved between hospitals or populations.

Thus robustness to shift is a stronger requirement than ordinary in-distribution test accuracy.

Adversarial vulnerability

Another major reliability issue is adversarial perturbation. A small, carefully chosen change to an input can sometimes alter a model's prediction dramatically even when the perturbation is nearly imperceptible to a human observer.

This reveals that high average accuracy does not imply local stability in input space.

Adversarial examples are important not only as a security concern, but also as a scientific clue: they show that modern classifiers may rely on brittle decision boundaries or subtle non-robust features.

Uncertainty and calibration

A reliable system should not only predict; it should also express appropriate uncertainty.

Aleatoric uncertainty. This is uncertainty arising from inherent noise or ambiguity in the data.

Epistemic uncertainty. This is uncertainty arising from lack of knowledge, such as unfamiliar inputs or insufficient training coverage.

In practice, many models are overconfident. They may produce highly certain scores even when wrong or when facing unusual inputs.

A related concept is *calibration*. A predictor is well calibrated if, roughly speaking, events predicted with confidence 0.8 actually occur about 80% of the time.

Benchmark success versus trustworthy deployment

Benchmarks are useful because they provide shared evaluation tasks. But benchmark success is not the same as trustworthy deployment.

Real deployment often requires:

- robustness to shift,
- uncertainty awareness,
- calibration,
- monitoring,
- safety constraints,
- resistance to manipulation or failure cascades.

Thus reliability is a broader notion than accuracy.

The central lesson

A strong machine-learning system should be judged on at least two levels:

- how well it performs under benchmark-like conditions,
- how reliably it behaves under realistic and changing conditions.

These are related, but they are not the same.

6.4 Interpreting Learned Representations

Deep models often behave as high-dimensional distributed systems whose internal logic is not directly transparent. This motivates the study of interpretability.

Why interpretability matters

Interpretability is important for several reasons:

- debugging model failures,
- understanding what signals a model uses,
- evaluating trustworthiness,
- detecting shortcut learning or spurious correlations,
- gaining scientific insight into learned representations.

Interpretability is therefore not merely a philosophical issue. It is connected to reliability, validation, and model improvement.

Feature attribution

A common local question is: which parts of an input contributed most to a given prediction?

Feature-attribution methods try to assign importance scores to input components, for example pixels, tokens, or features. At a high level, they ask:

> if this input had changed, how might the prediction have changed?

These methods can be useful, but they also have limitations: different attribution methods may disagree, and a visually plausible explanation need not reflect the full causal structure of the model.

Probing internal representations

Another strategy is to analyze hidden states directly. If a model has intermediate representation h, one may ask whether h contains information about some property of interest, such as syntax, object identity, sentiment, or latent state.

This is often studied through *probing*: train a simple auxiliary model to predict a property from the representation. If the property is predictable, then the representation contains relevant information.

However, care is needed. A successful probe shows that the information is present, but not necessarily that the model uses it in the way the probe suggests.

Circuit-style and mechanistic viewpoints

A more ambitious approach aims to identify structured internal mechanisms: subnetworks, pathways, heads, or feature interactions that implement specific computations.

This mechanistic perspective seeks not merely to visualize importance, but to explain how a model computes.

Such analyses are promising, but they remain difficult. Large deep systems are distributed and context-dependent, and local components may participate in many overlapping functions.

What explanation methods can and cannot reveal

Interpretability tools can be genuinely informative. They can reveal:

- spurious cues,
- failure modes,
- internal clustering,
- saliency patterns,
- useful hypotheses about model behavior.

But they also have limits:

- explanations may be unstable,
- local attributions may oversimplify distributed computation,
- interpretability does not automatically imply causality,
- a model may remain opaque even if some of its parts are interpretable.

A modest conclusion

Interpretability is best understood neither as a solved problem nor as a hopeless one. It is a growing collection of tools for partial understanding.

A realistic stance is:

- deep models are not fully transparent,
- some useful explanatory tools exist,
- these tools must themselves be interpreted carefully.

6.5 Scaling, Efficiency, and Practical Constraints

Understanding deep learning is not only about theory and prediction. It is also about resources. Modern systems are shaped by data, memory, energy, and computation.

Data, compute, and memory as limiting resources

Large models often improve performance, but they demand:

- more training data,
- more computation,
- more memory,
- more engineering effort.

Thus the practical success of a method is never independent of the infrastructure required to support it.

Efficiency versus performance

A model that is slightly more accurate but vastly more expensive may not be preferable in many applications. Practical deployment depends on:

- latency,
- throughput,
- memory footprint,
- energy use,
- reliability under hardware constraints.

Hence there is often a tradeoff between raw predictive performance and usable efficiency.

Compression, distillation, and quantization

Several broad strategies aim to make models more efficient.

Compression. Reduce redundancy in parameters or activations.

Distillation. Train a smaller student model to imitate a larger teacher.

Quantization. Represent parameters or activations with reduced numerical precision.

These strategies are important because they show that modeling success is not only about scale, but also about how scale can be made usable.

Why this matters scientifically

Efficiency constraints are not merely engineering details. They influence which ideas become practically dominant. They also shape what kinds of models can be deployed safely and widely.

Thus the history of deep learning is driven not only by mathematical ideas, but also by the interaction between algorithms and computational resources.

A balanced view

Scaling has been extraordinarily powerful. But scaling alone is not a full scientific explanation of intelligence, nor is it a free resource. It operates under real economic, physical, and design constraints.

6.6 Open Problems and Future Directions

Deep learning is powerful and transformative, but it remains scientifically unfinished. This section highlights several durable open directions.

Generalization and feature learning

A deeper theory is still needed to explain:

- why gradient-based optimization often finds strong solutions,
- how learned features arise during training,
- what depth and architecture contribute beyond broad expressivity,
- how pretraining changes downstream generalization.

This is one of the most fundamental open areas.

Robustness and trustworthy AI

Modern systems still struggle with:

- adversarial fragility,
- uncertainty under shift,
- calibration,
- reliability in safety-critical deployment.

Making models not only accurate but trustworthy remains a central challenge.

Multimodal and interactive systems

Many important environments are:

- multimodal,
- partially observed,
- dynamic,
- interactive.

Future systems will likely need richer integration of perception, memory, action, and feedback. This requires not only larger models, but also better understanding of structure, control, and adaptation.

Reasoning, planning, and world models

A major open question is how predictive modeling, representation learning, and decision making fit together. Can learned representations support robust reasoning and long-horizon planning? What role should internal models of the environment play? How should uncertainty and simulation interact with action?

These questions sit near the boundary between machine learning, control, cognitive modeling, and AI more broadly.

Scientific humility

The field is evolving rapidly. Some explanations that seem natural today may later be revised. Some current paradigms may prove temporary, while deeper organizing principles emerge more clearly.

A good scientific attitude is therefore:

- to distinguish robust knowledge from fashion,
- to state uncertainty honestly,
- to recognize both the power and the incompleteness of current theory.

6.7 Final Synthesis

We can now step back and summarize the arc of the book.

From risk minimization to learned representations

The book began with learning as a mathematical problem:

- inputs, outputs, and distributions,
- loss functions and risk,
- Bayes-optimal prediction,
- empirical risk and generalization.

This established the core language of machine learning.

From linear prediction and fixed features to deep architectures

We then saw that classical machine learning often relied on fixed or engineered representations. Deep learning changed this by making representation itself learnable.

Neural networks introduced:

- compositional function classes,
- hierarchical representation,
- backpropagation-based learning,
- trainability through initialization, normalization, and residual design.

From architecture to reusable representation

Architectural ideas such as convolution, recurrence, gating, and attention showed that model structure embodies inductive bias. Pretraining and self-supervision then extended this picture: representations can be learned once and reused across many tasks.

This led naturally to foundation models and broad transfer.

From prediction to generation and interaction

The book also expanded beyond supervised prediction. We studied:

- self-supervised learning,
- reinforcement learning,
- generative modeling through latent variables, adversarial learning, flows, and diffusion.

These developments showed that modern AI is not one narrow technique, but a family of ways of learning structure from data and interaction.

From successful methods to open scientific questions

The final lesson is not that deep learning is fully understood. Rather, it is that deep learning is:

- mathematically rich,
- empirically powerful,
- practically transformative,
- still incomplete as a science.

The central theme

If one idea unifies the book, it is this:

> a central theme of modern AI is the learning of useful representations: representations for prediction, for generation, for transfer, for control, and for interaction with complex environments.

This does not answer every open question. But it provides a coherent intellectual thread through the field.

Closing note

Deep learning has changed how machine learning is practiced and how intelligence is modeled computationally. Yet many of its deepest principles remain only partially understood.

That is not a weakness of the field. It is a sign that the subject remains scientifically alive.

The appropriate conclusion is therefore neither triumphalism nor skepticism, but disciplined curiosity:

> to understand what has been achieved, to recognize what remains fragile, and to continue asking what deeper principles still wait to be discovered.